PROGRAMMING PROJECT

REVIVE

A CALORIE TRACKING MOBILE APPLICATION

DOMINIC STACHURA

Contents

Contents 1
Analysis 7
Problem Identification: 7
Computational Methods 7
Visualisation 8
Algorithms 8
Decomposition 8
Divide and Conquer 9
Iteration 9
Abstraction 9
Stakeholders 10
Personal trainers 10
Average person who works out / Newcomers to the nutrition field 10
Nutritionists 11
Doctors / Medical workers 11
Interview 11
Problem Research 13
App Research 13
MyFitnessPal: 14
Nutracheck: 17
Summary of both apps / what I could use in my own: 18
Features of my solution 18
Application 18
Tutorials 19
Creating objects as foods/meals -> Object oriented programming 19
Calorie Tracker 19
Notification Reminders 20
Data presented. 20
Limitations of my solution 20
Foods 20
Devices 21
Features 21

Minimum System Requirements 21
Success Criteria 22
Design 27
General Decomposition 27
Authorisation Module 28
Decomposition 28
Flowchart 29
Pseudocode 30
General Summary 31
Personal Details Module 32
Personal details input 32
Flowchart 33
Pseudocode 34
Recommendations 34
Decomposition 35
Flowchart 37
Pseudocode 38
General Justification 40
Calculations 41
Object creation system. 41
Decomposition 41
Flowchart 43
Pseudocode 44
General Justification 45
Calorie tracking system. 46
Decomposition 46
Flowchart 47
Pseudocode 48
General Justification 48
Tutorials 49
Explanation 49
Flowchart 50
Pseudocode 51
General Justification 52

Notifications 53
Explanation 53
Flowchart 1 54
Flowchart 2 55
General Justification 57
GUI 57
General Designs / Main usability features 58
Home screen 58
Profile and personal information viewing screens 59
Item addition and removal screen 61
Item and Meal creation 62
Notifications 65
Test Plan 65
Key Variables, Data structures and classes 66
Suitable Test Data 68
Further Test Data 71
Developing 73
Important information 73
General Items / Layouts 73
Background screens 73
Code for any Recycler Items and List Items that will be used throughout 74
Colours and Styles 76
Detail Changing Dialog 78
Dialog to display options 80
Authorisation Module 82
Login 82
XMLS 82
Login XML 83
Forgot password dialog XML 86
Logic 88
Main login 88
Forgot password dialog 89
Sign up 92

XMLS 92
Logic 95
Testing 97
Module Summary 101
Personal Information: Input Module 102
XMLS 103
Logic 107
Testing 112
Module Summary 117
Navigation Modules 117
Bottom Navigation Menu 118
XMLS 118
Logic 121
Side Navigation Menu 122
XMLS 123
Logic 126
Testing 128
Module Summary 131
Personal Information: Profile Module 132
XML 133
Logic 137
Testing 138
Module Summary 141
Personal Information: Personal Information Display 141
XMLS 142
Logic 149
Viewing and changing personal info data 149
Recalculating Calories 159
Testing 161
Module Summary 164
Personal Information: Setting and Changing Daily Goals 165
Daily Goals XML 166
Edit Goals XML 169
Logic 173

Main screen 173
Edit Goals Button 175
Edit Calories Button 180
Testing 183
Personal Information Module Summary. 185
Object Creation Modules 185
Food Item Creation Module 186
Item page XML 187
Item Creation Dialog XML 189
Item information XML 193
Logic 194
Creation of items 196
Search Feature 199
Item Interactivity 200
Edit Items 203
Delete Items 205
Testing 208
Module Summary 214
Meal Creation Module 214
XMLS 214
Navigation Change 214
Meal Fragment 215
Create Meal Screen 216
Dialog to enter amount in grams 219
Dialog to select food 222
Dialog to display meal details 223
Logic 224
Adapter and Meal Fragment Setup 225
Meal Creation Fragment 228
Add food items during creation 231
Remove food items during creation 238
Save food items 239
Searching meal items 241

Testing: Meal Creation 243
Meal Fragment 246
Loading Meals 247
Searching created meal items 248
Item Interactivity 251
Meal details dialog 254
Edit or Delete and its dialog 255
Edit Meals 258
Delete Meals 264
Testing: Meal Fragment 267
Module Summary 269
Updates to the Login Module 270
Logic 270
Testing 271
Calorie Tracking Module 272
XMLS 273
Logic 279
Daily Meals Class 279
Home Fragment 281
Show food items function 285
Item interactivity 294
Edit 298
Delete 300
Testing 300
Development Summary 306
Evaluation 306
Testing to inform evaluation 306
Success Criteria Comparison 312
Usability Features 320
Registration Feature 320
Input of Personal Information 320
Navigation Module 322
Personal Details Module 322

Object Creation Module 325
Calorie Tracking Module 327
Stakeholder Review 328
Maintenance 333
Limitations and improvements of the solution 334
Code Appendix 336

Analysis

Problem Identification:

Over time, the interest in nutrition and healthy eating has grown and grown, with it being at an all-time high right now, many people are trying to find ways to improve their eating habits. One of the solutions people find is to install a calorie tracking app, however I have seen that many of these apps require the person to have done research and have the knowledge of nutritional based eating beforehand, so that they just input their desired numbers, and the app will track it for them. Along with this, many nutritional based apps, such as MyFitnessPal, while it has an initial GUI that gives you a base look on the stats, as you go further into the app, features are almost thrown at you from everywhere making the apps extremely complex to work around. Most people would find most of the features unnecessary, and all the different ways stats are displayed are unneeded for a beginner, this means that there is a high chance that a beginner will be turned away from getting into nutrition, since everything looks too complicated for them to want to start

The features of a computer system that would be required for this problem is to make use of an algorithm that asks the user to input their daily habits, weight, rough desired goals, etc and give them a recommendation based on those inputs. The problem lends itself to a computational solution as an app like this would need to base this on the person's lifestyle, so that it is not just a base calorie number for everyone, so an algorithm would be used to consider every aspect of the person's life, to best suit the numbers to them. In addition to this, a UI that abstracts all the complicated features for a beginner and creates simplicity for ease of use would be highly beneficial, this means that everyone will be able to make use of an app like this, whether they are a beginner or advanced in the field, but there are a lot more people with an interest now so it encourages those who are newer to this area to not give up and continue, however to better suit those more advanced in nutrition, an option to increase the complexity of the UI, as well as display more ways of tracking such as charts or graphs.

Computational Methods

The problem is suited to a computation approach because the process of using an algorithm is inherently computation, calculating numbers and creating equations requires the problem to be

broken up into multiple individual components in the process of decomposition, as well as other steps listed below:

Visualisation

The core idea behind this project is the visualisation of a person's progress over time, so that day in day out they can understand the goals they need to hit. Visualisation will be used in aspects involving the calculations of numbers, displaying the recommendations for users, setting their daily goals and amounts of calories/grams they must eat, etc, this means that the user will be able to better understand the information they have collected on themselves to track their progress, making this a suitable method as it is better for humans to see rather than to try understand other forms of data representation.

Algorithms

There will need to be an algorithm that initially collects all data necessary about a person according to government requirements to work out healthy amounts for them to eat and what they should eat. Using their BMI (Body Mass Index), etc to match equations and use basic data for newer people, to solve a more complex equation that outputs everything an individual needs to live healthy, the results will need to be stored as a daily amount that resets each day, where the user can edit any of the details that impact the end output at any time. This is suitable to be used for an algorithm as it will compare multiple factors and turn them all into equations quickly to generate an outcome for the user. The output will be the daily calories they use, and then that will need to be further taken apart to separate it into individual meals, etc.

I will need to use search algorithms in order to go through each of the data choices entered and pick out the conditions of the person from all the options.

Decomposition

Decomposition is the process of breaking down a large complex task into smaller more manageable tasks to be individually worked on, making creating a solution to the whole problem a lot simpler. The main question in problem, “What is my calorie amount per day if my goal is x?” can be decomposed into individual components. Each component is a factor that plays into the final decision. This means factors such as the amount of exercise in a week is a problem that cannot be solved with one value such as x number of times in a week, factors of intensity as well as the actual exercise done such as lifting weights or playing a sport all have a different impact on the user, we can treat each of these different options as their own objects, each object can have an integer value, e.g. intensity raises from 1 to 5 depending on the option picked, this is justified because all these different factors can impact you very differently. This process can be continued until each main factor can be broken down into its little aspects, and the final calculation can be made by bringing all these values together.

Other factors that will come from the main question:

- Age
- Gender
- Preferred number of meals per day
- Goals (cut/maintain/Bulk)

- Height

These will all need to be individually solved and stored in different variables in order to work out more complex values such as BMI that an average person might not know, these create a solution to the bigger question, there are more factors not listed so I will need to use modular programming so that I can add them to the final equation later if the need arises.
Decomposition as a computational method when coding will be highly useful as it allows the breakdown of the problem to where each individual section can be treated as its own sub-module, which acts out its own purpose. For example, I could have a meal creation module, that contains all the code needed to group different food items together for easier use. This is justified as it is much easier to solve a solution through little individual solutions instead of tackling the large problem in one go.

Another suitable problem within my program suitable for decomposition is the actual method of displaying data, since many people require many different forms of visualisation, the problem would be broken down where each individual module is a different way of displaying the data for the user to be able to pick from within the app. This also means that I can later add more methods, but it will also be easier to code, and I can later reuse the same methods of displaying data for other aspects in case I decide to add other functionality than just calories later.

Divide and Conquer

Divide and conquer will be used in my program to solve all the individual steps and problems and then combining them towards the end in order to display recommendations and aspects based on everything the user entered. Using modular programming makes use of a divide and conquer method. It can be used to search for different items created in a database, such as a binary search with an O notation of O(logn). This approach is justified as it reduces the complexity of the problem, meaning the code will be easier to write as well as the finished solution will be more memory efficient.

Iteration

Iteration in computer science refers to the process of repeating a block of code through an increasing index count until a condition is met. Iteration would be used to add elements to different lists that can then be used to keep track of items over time, so the list would be extended as time goes on so that a person can track their progress. This means that variables are not just added constantly making the program a mess to work with, instead it is one continuous variable for one specific type of data that can be fetched from. Iteration is justified because it allows the repetition of multiple steps without having to rewrite code making the program more maintainable, efficient and reusable.

Abstraction

Abstraction is used to remove all unnecessary details to allow the developers to focus on the important key features. Since the plan is to make this a beginner friendly app, abstraction will need to be used on a lot of different aspects unless the user wishes to see them. A beginner would not understand the concept of BMI, etc and would just want to log their calories and their weight and

have the app calculate things for them. This is because a user would want a kind of experience that does not throw everything at them all at once like I have seen other applications do, but instead be slowly led into everything and be able to choose what they do not want and want to see. So, the level of abstraction will need to vary. Furthermore, functions and procedures will also be abstracted from the user and instead they will be given only the options to input values, and view their outputs.

This method will also be used for the overall aesthetic of the app, eliminating the need for complex designs, a study condoned by dieter ram when creating his 10 principles of good design shows that simple is often better than the complex. Abstraction would be used for all the complicated aesthetic features in order to keep everything simple, this means that the actual solution to the problem can be more focused on, instead of focusing on making the app look aesthetically pleasing.

Stakeholders

Personal trainers

Personal trainers represent a critical stakeholder group in the fitness tracking application, offering a sophisticated solution to client progress management. By providing an innovative platform that allows them to visualise client metrics over time, the application enables trainers to make strategic, data-driven decisions about nutrition and training interventions.

The traditional approach of manually tracking client progress is inherently time-consuming and inefficient, making this an appropriate stakeholder. A digital solution addresses this challenge by streamlining data collection and presentation, allowing personal trainers to return personalised insights to their clients according to their exact progress.

Average person who works out / Newcomers to the nutrition field

Fitness enthusiasts also represent a crucial stakeholder group for our application, targeting individuals who are transitioning from casual exercise to a more structured approach to health and wellness. Typically, these are people who currently engage in occasional workouts at home, in the gym, or through sports, but have not yet considered the critical role of nutrition in their fitness journey. The application serves as a bridge, providing an accessible pathway for those seeking to elevate their fitness approach beyond sporadic physical activity.

Recognizing that most users will lack specialized nutritional expertise, the application should be designed with an intuitive, beginner-friendly interface that demystifies nutrition and provides guided support making this group appropriate stakeholders. By offering an algorithmic approach to nutritional recommendations, the platform empowers users to make informed dietary choices without the intimidation of complex nutritional science. The key value proposition lies in providing personalized, low risk guidance that transforms nutrition from an overwhelming concept to a manageable, actionable component of personal fitness development.

Nutritionists

Much like personal trainers, nutritionists would also be a group of stakeholders for this project. Since nutritionists focus mainly on the food aspects of training, they can recommend this app to their clients along as part of their assistance. A nutritionist can very well see the differences between their clients from when they started to their current state, this also means that this app could serve as a form of motivation, since a person can see how they have progressed overtime, and if positive, could encourage them to continue and live a better, healthier life.

This is another appropriate stakeholder, a nutritionist would need to keep track of multiple clients, as well as directly monitor their progress to give them the best advice/service they require. So, an app that allows users to share their progress through data would be highly beneficial.

Doctors / Medical workers

A less likely, but still possible stakeholder could be those who work in the nutritional side of the medical field. This app would allow them to track the foods of their patients, to make sure they are eating healthy along with what they are eating to help with recovery. This means that the app would need to be professional and robust since it could be completely changing lives.

This is an appropriate stakeholder, since a doctor who specialises in the physical health field would need to keep track of many patients, so an app that has accounts for different people and automatically stores all data that is inputted will be very useful to them, since they can accurately and quickly track the data they need in order to better help the people under their care.

Interview

To conduct further research, I found 15 random people from a potential target market and asked them to fill out a questionnaire

1) *What platform do you primarily use day-to-day?*
   - Mobile (10)
   - PC (3)
   - Tablet (2)

2) *How often do you use your devices?*
   - 0-2 hours (1)
   - 2-5 hours (6)
   - 5-10 hours (6)
   - More than 10 hours (2)

3) *How often do you forget things?*
   - Very little (2)
   - Little (3)

- Often (7)
- Very Often (3)

4) *Are you better with visual aspects or written aspects?*
    - Visual (12)
    - Written (3)

5) *How much knowledge do you have in the field of nutrition?*
    - Little to none (8)
    - A little bit (3)
    - Lots (2)
    - Enough to do everything myself (2)

6) *Do you understand how much food matters in your personal fitness?*
    - I eat anything whenever and wherever (4)
    - I know little but try to eat better (7)
    - I have a diet in place by someone else or from the internet (3)
    - I'm on a strict diet where macros matter heavily (1)

7) *Which of the following features would you like to see added to a nutritional app?*
    - Graph trackers (4)
    - Notifications (5)
    - Integrated apps such as Fitbit (2)
    - Recipes for different foods (4)

From this questionnaire, I understand that implementing simple, easy to understand features will suit majority of the users, since many people do not have the appropriate knowledge to be presented with complex features. The fact that a dominant amount of people from my questionnaire have little to no knowledge about the field of nutrition cannot be overlooked, my application would then be justified to have a feature of some form of assistance in educating users on what they should and should not necessarily consume and help them lead a healthier life. The importance of a high forgettability rate among my interview implies that a lot of people would do well with reminders and a companion to keep track of things, therefore evoking the idea that some form of notification to their main device, a mobile phone, would be the most beneficial to keep them on track with eating and tracking.

Following, analysis of the frequency in which users allocate time to their mobile devices during the day makes a higher chance of them forgetting to eat / keep track of their fitness further justifying a notification feature. Furthermore, the strong preference to visual features will need to be kept key throughout my application, allowing the users to see rather than to read. This is justified as humans

have always been better visual learners, so showing the user rather than asking them to read will increase engagement with the app encouraging them to keep using them.

Overall, the users should feel the app as more of a companion along the way with them rather than just a tool that they are using for the time being before they get bored of it, this means that user retention will increase and hopefully the user's fitness will increase alongside this.

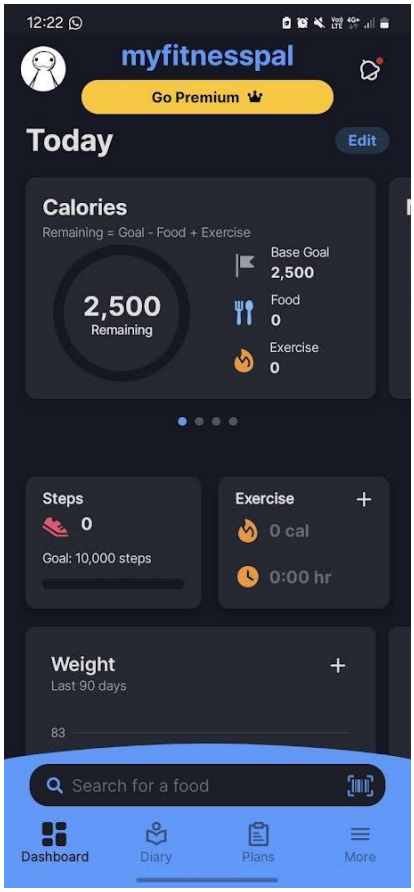
Problem Research

App Research

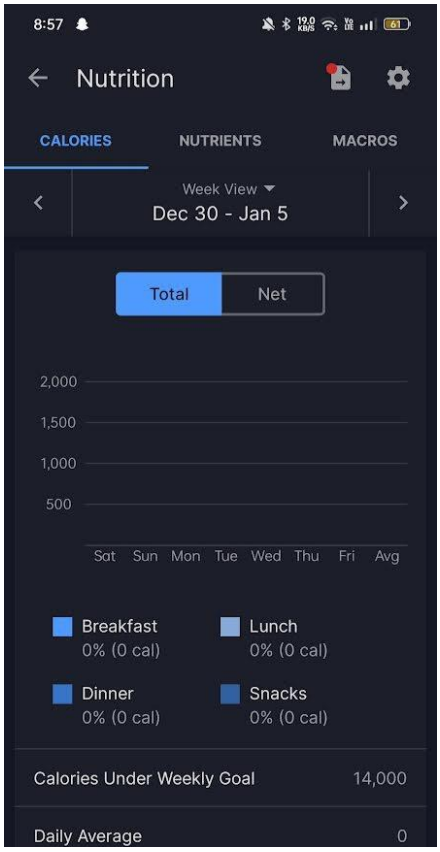
After using a few different apps for my own health benefits, it became clear that most apps built and designed for users who want to track their calories and improve their health are absolutely packed with lots and lots of different options and features, which allows very good tracking capability, since every detail can be monitored, however for those that are not as extreme and want to track casually, this means that they will have a lot of unnecessary clutter to their app, sometimes even to the point where it becomes uncomfortable to experience such a menu of features, since they completely overwhelm a new user.

I will use two different applications, one being MyFitnessPal which is a well-known health and calorie tracker app, and an application called Nutracheck:

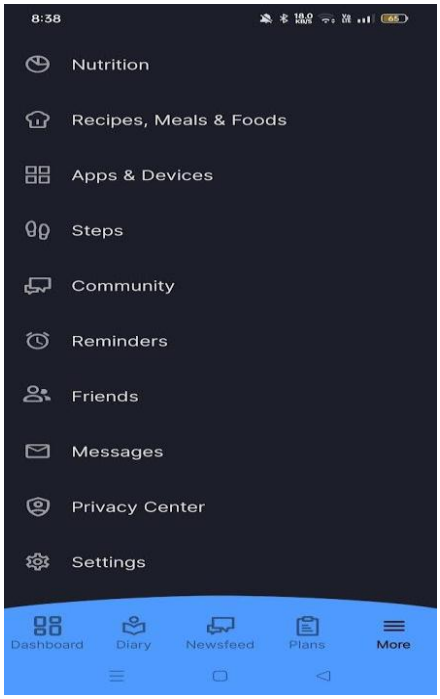
MyFitnessPal:



The home screen of MyFitnessPal is not necessarily cluttered with options. There are some simple icons and a rough outline of food that has been tracked, your steps if you add a step counter and an option for you to select your exercise. This is a high-end app so naturally it will have a good choice of layout and options to choose from. However, anything past the dashboard and there are many, many different options to a point where it completely overwhelms a user and can almost make them feel inadequate as it displays information, they have no clue they were tracking or should be tracking without any prior notice.



There are several unnecessary features and complications involved into, for example adding your daily calories, after pressing the main" calories" option you are taken to a screen which further breaks down what you already know, I feel like this entire screen could have been abstracted, or placed as an option if the user specifically clicked onto it, a graph to see your different mealtimes is not needed.



Clicking further into the app, the number of features available can just be scrolled until you get lost, with each one having several further features inside of it. Many of these will never be accessed by the user, and many will go completely unnoticed unless genuinely looked for, such as the" reminders" tab, I never knew such existed until taking the screenshots for this document. So, the user interface, although simple with icons, ends up being overly cluttered and many features are lost. Along with this, there is a huge paywall in front of many options, with not even a hint of a free trial for a user to experience the app's true capabilities, this means that a user is almost disinterested in wanting to further progress through the app and learn more about the different features are offered, since most are completely locked behind a paywall and cannot even be seen.

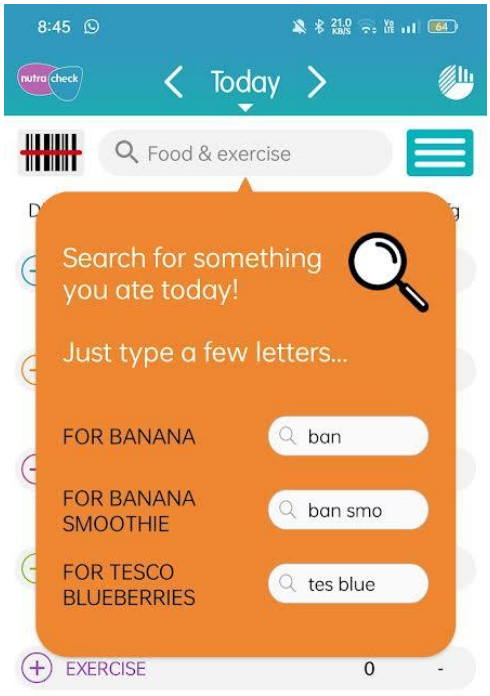
On the other hand, the app has an appealing first screen with a friendly-looking font, the number of calories remaining in a day are clearly displayed with a circle to help a person visualise at first glance, they also keep the first screen very basic so it is easy to understand and swap to what you need. This simple design is an idea I would take and like to keep constant throughout my application as well as helping users visualise certain aspects to greater increase the convenience of the user.

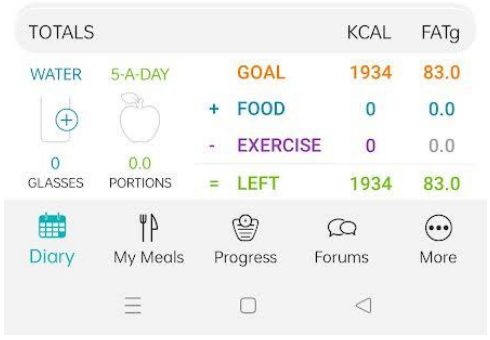
In summary,

Even though the app provides many functions, as well as has a sophisticated look, they almost appear to be aimed at those who have been doing nutrition and tracking their calories for years and are experts in the field, this means that the average user experience of the app will not be very positive, as even those who are experts will struggle to get around the app before they understand what all of the features do. Furthermore, the paywall that blocks most features almost feels like the company does not want you to track and help you with your help unless you pay them, this could discourage many users from wanting an app like this, since the second they do not have the extra money for a subscription, the app turns its back on them. Of course, there are quite a few features for a free experience, but the premium section of the app is heavily advertised across almost all screens other than the main menu.

Nutracheck:

Initially I had very positive views after downloading this app, they had a friendly beginning and explained that they needed certain details to help with tracking, however after inputting those details, I was not told **why** or **how** they have been used and just given a random output that felt nothing close to an achievable goal, and a list of dropdown menus for me to pick my own goals, almost as if there was zero help in recommendation other than a small “we recommend 10%” which took a while to figure out what the initial screen meant by that. This means that a user will feel as if they are immediately obligated to share many details about themselves without really being told why they need this or how it is being used to calculate their “recommendations,” which could spark concerns and distrust in the app straight from the beginning.





As soon as I was done with a log in, I was shown this screen, which very much lifted my hopes, thinking they had included somewhat of a ”Tutorial” or a guide to use the app, but as soon as I followed the instructions, this screen disappeared never to be seen again. I was thrown into an app with organisation that was not the best, everything felt everywhere with way too many different options. This app is another app that would do well when given to a user who knows how to operate it so that they can take full advantage of it, but for the average user, this will be a lot of clutter and unnecessary information. This means that they will not have the best initial experience, which could ultimately mean that they end up completely not wanting to track their calories. This app is another app where I found it difficult to find features I wanted.

Summary of both apps / what I could use in my own:

Both apps appear heavily cluttered, which is what I have seen in many modern fitness apps, where they attempt to beat each other on the number of features available, instead of fully embracing what a good user experience should be and look like. I will make my app with the average user experience in mind, and try abstract as many unnecessary features as possible, but instead make the appearance of the app and the functionality simple, and to the point. This means that the app will target a wider audience, since there are many more people who are not experts at nutrition and will be more willing to try a new app, compared to those who are already experienced in their applications and will not be willing to try something new.
In addition to this, after explaining why and how the information collected will be used in a friendly beginning experience, I will have a small introduction to the app where the user is taken around the initial features of the app and the functionality is mostly explained to them. This will also be a chance to recommend a few features, such as a notification to remind them to not forget to track their food (this was something I experienced as a user myself over a certain period), to them and make them known. This means that the average user will not just be thrown into the app and expected to make their way and will make them more likely to return to the app and use it as their main source of tracking their data.

On the other hand, using abstraction to eliminate many features may result in a limitation for those who come to the app wanting a complex and in-depth tracking experience, for a person like this it may be best to create the program with a modular nature, so that in future development I could add modules that are features themselves that can be toggled on and off for a person who wants more of an in-depth experience. This means that the app will have to be built through procedures so that later modules can be added, this will help as modules can be saved into libraries and re-used at later times, as well as save development and debug time since each module will use local variables so the entire program does not have to be run.

Features of my solution

Application

My solution will be a mobile application just like the other apps. This is a definite requirement because it allows users to track their meals on the go from a device they will always have on their person, their phone. I chose to develop this app on android OS as it is simpler to work with than IOS,

as well as being on many more devices compared to those just from apple. This decision will hopefully allow more users to access and use this app, as well as track and save data locally to their device which means the application will be able to run without the use for a connection, and users can always track their meals wherever they are.

Tutorials

When first launched and an account has been created that meets the requirements, a small tutorial will be displayed acting as a guide around the app to introduce the main features of the app, and how to add and track calories as well as foods. I have decided to include this as most other applications throw you straight in and expect you to work things out yourself / expect you to already know how to handle things, a user having the features lightly explained to them will be more willing to stay and use the app since it provides a chance for them to understand it better.

Another tutorial feature will be included within each smaller section outside of the main tutorial, since even they understand the main features, an app like this should eventually have more niche features that allow for a more precise tracking experience, so smaller tutorials explaining what each extra feature does within sections, or just a summary of what it does would be useful to a new user. This means that a user will never feel overwhelmed and not have a sense of giving up when exploring the app.

Creating objects as foods/meals -> Object oriented programming

After the user has completed the tutorial, they will be provided with an option to create a new food or to edit an old one. This is important as it allows them to set the calories and other nutrients exactly to the label of their food product, instead of relying on the “average” that many apps seem to use, abstracting the need to create or connect to a pre-existing database of food items. Of course, some foods that do not come with ratings such as fruits will still need an average rating and will come pre-created for the user.

These foods should then be able to be created into meals, so that the user with one press of a button can add certain meals instead of having to add every single option which may become tedious. This means that they will be more likely to keep tracking their foods since it is only a quick option, and they know everything is done for them once they have set it up.

Calorie Tracker

At the home page of the application, the users' daily calories as well as how many they have eaten/ have left should be displayed. This should also allow for an option for a user to see how many they have eaten over a certain period of days, to see an average and be able to adjust their requirements accordingly. This means that the first thing a user will see is the main thing they are downloading the app for and can quickly change certain settings and view other information.

Right next to the displayed calories, there should be a “+” feature that directly takes the user to a

page that allows them to add foods they have eaten. Many other apps have a complicated way of going about this and I often find myself searching for how to do it or forgetting how to. This means that having an obvious option to add food will allow the user to eventually turn this procedure into muscle memory and use the app subconsciously without thinking about it as it will not be complicated at all. This would also be followed by a "-"button or feature that allows them to remove foods they have added in case something was wrong or they added too much.

Notification Reminders

This will be a feature that allows the user to set themselves reminders that will be sent as priority notifications for their phone in case they forget to track, or worst case forget to eat in general. This means that the app will feel more like a helpful companion than a tedious task that they have set themselves up to do. This can be taken further will randomised messages of encouragement to the user which will further boost the companion feel as they will feel as if the app also cares for their progress. This is a justified feature due to the forgetful nature of humans and the ability to boost user retention by reminding the user to come back to the app at certain times, and by having the user set themselves the notification times they will not as feel as if the app is pushing itself onto them to leech on their time.

Data presented.

Possibly a feature that allows the user to select certain data they want to be presented, in a graph, table, etc so that they can have a visual aspect of things they are tracking, for example if they were to track weight overtime the best way to represent this would be a line graph of weight-time. This is because a visual approach is important as humans tend to rely and learn better from visual examples, as compared to just numbers and text on a screen.

Limitations of my solution

Foods

The biggest limitation to my solution is that I am unable to support different kinds of foods like other apps, such as barcode scanning and having every type of item from every store. To combat this, I will create my own item creation system that stores items as objects and allows users to input their own details exactly how they need it without having to search a database. To decide how this will be implemented I will survey the stakeholders and implement only the inputs that they deem relevant. This means that they will have the best experience catered towards them.

Devices

Another limitation to my solution is that I am unable to support all different kinds of devices, for example due to me developing the app for the androidOS, it automatically means that anyone with an IOS device will be unable to download and use this application as IOS has completely different software requirements to be able to be uploaded to their app store. I will also not be able to support all the android devices, since there are so many that will be made incompatible due to the sheer amount of devices available, so I will attempt to make this app available for the google play store, so any device that can run it will be able to download it, this means that I am catering to the widest possible audience since the google play store is the most likely to be on many android devices.

Features

Most fitness applications have many features that I do not understand yet, so naturally I will not be able to include many of them that may be helpful to people who come from other applications, a smart way to do this would be to include a "suggestions" feature so that through later development they can be added, this means that a person who uses the app may not have everything they need in one section, but however can feel as if they contributed to the development of the app by requesting useful features. Furthermore, although the app may not have such a wide variety of features, this is almost on purpose as my intent is to heavily abstract the features of other applications, to make this app easy to use and understand.

Other fitness apps have the ability to suggest a nutritional plan based on lots of data collected about the user by being trusted enough by them to collect data on their habits, etc. My application will not be able to do that so the limitation of it will be that it will only be able to suggest certain recommendations based on the limited amount of data it is able to collect but it will not be able to give a full plan.

My application will not be as popular and be able to hold servers for community posts and guides from other members in the app. This means the app will not have as much ability for user interaction as other apps to help keep them on and into the app. This is a big limitation as community is what can make or break an app; however, the app is supposed to be a personal companion instead of a chat service so I will refrain from attempting to add this section to the app. Furthermore, this also means that my app will not have an extensive library of recipes uploaded by different users, if I was to include recipes there would be a very limited amount as I would have to personally upload them so will most likely also refrain from including this.

Minimum System Requirements

1.2Ghz Processor

(Almost no phones that are still supported by their manufacturers run at this low of a speed anymore, they have either been discontinued or stopped being supported quite some time ago)

-> Justified as there will need to be quite a few calculations performed in such things as the recommendations, as well as animation and other things.

Snapdragon: Adreno 520

-> Minimum graphic specs from phones below the years of 2017, will be used mainly for small animations in the app and used for the aesthetic, not a lot of processing power required as this will not be a game or require heavy graphic processing

200mb ram

-> Mobile apps require hardly any ram, and this app at its most intensive probably will not even reach 300mb of ram. This also not a difficult requirement as most phones that haven't been discontinued have more than 4gb of ram, so a phone should be expected to be able to use more than 200mb on a single application.

720x1280 pixel display

-> A 720p display is a minimum for any images included to not be completely pixelated and be horrible to look at, a display with this minimum means that there are higher colour accuracies that can be used leading to a more appealing finished application.

The specs I am using on my mobile to run and test this software:

- Device Name: Oppo Find x2 Lite
- Ram: 8gb
- Storage: 128gb
- Processor: Qualcomm SDM765G 5G Octa-core (Snapdragon 765G 5G) -> 2.4-1.8Ghz cores
- Android version: 12
- 1080x1200 display

Success Criteria

#1 – Registration	Criteria	Testing Method
1.1	Create a log in screen that links to a sign in screen	Screenshot of how the GUI looks.

1.2	Integrating the firebase authentication to enter, validate and store the user details in order to use it in a future login and let the user access their saved account details, should also only give access to their own data.	Screenshot of the sign-in that pops up when integrated and a successful log in.
#2- User registers details	Criteria	Testing Method
2.1	Information screen explaining why data will be taken	GUI screenshot
2.2	Name and age input	Screenshot
2.3	Weight and Height	Screenshot
2.4	Activity levels drop down menu	Screenshot of input
2.5	Data is taken to the personal info	Data inputted sent to a test file so can see class working and being created.
#3 – App Navigation	Criteria	Testing Method
3.1	The user is able to navigate through different main screens of the app through a bar at the bottom with different buttons.	Video
3.2	The user is able to navigation through different, less important screens by showing a side menu with all the different options necessary.	Video
#4 - Calculations	Criteria	Testing Method
4.1	Calculation of BMR and storing it in a personal information class.	Screenshot of data outputted to a test file from a get method
4.2	Calculation of AMR from previously calculated BMR and stored in the personal information class.	Screenshot of data outputted to a test file from a get method
4.3	Calculation of the calories needed to be consumed to loose fat.	Screenshot of data outputted to a test file from a get method
4.4	Calculation of nutrients in grams a person needs to have daily.	Screenshot of data outputted to a test file from a get method
4.5	Ask if the user would like to set these calculations as their default.	Screenshot of the page that asks
4.6	Allow the user to input their own calories IF option no is chosen and recalculate the nutrients.	Screenshot of data outputted to a test file from a get method
4.7	Calculations performed to add or remove values from the daily progress for the user.	Video
#5 – Tutorials	Criteria	Testing Method

5.1	The screen is darkened, and a text box pops up at the bottom explaining a tutorial will now happen as the app is first launched	Screenshot
5.2	Further text boxes with explanations and highlights to different features of the main page	Screenshots
5.3.1	Highlight of the Foods tab at the bottom in the UI	Entire 4.3 Section will be done as a video
5.3.2	Highlighting the add name to object	
5.3.3	After name has been entered text box appears explaining that foods must be entered in 100g as the app calculates based on that	
5.3.4	Boxes for user to enter the grams of the food they have chosen	
5.4	Meal creation tutorial	Entire 4.4 Section will be done as a video
5.4.1	User encouraged to press the Foods tab again from main page	
5.4.2	Once user is in the food creation page encourage with highlight to swap to the meals page	
5.4.3	Create a new meal and enter food from food objects to be added to the meal. -> explain all foods must be selected here	
5.4.4	Enter option for each object amount in grams that is being added.	
5.4.5	Prompt user to press the create button and explain that they have completed the task	
5.5	Explaining the “+” button: Once a meal has been created tutorial to explain how to add food will start.	Entire 4.5 Section will be done as a video
5.5.1	“+” button gets highlighted from main page	
5.5.2	Screen darkens and text box appears that explains that meals and individual objects are separated. Then gets user to swap to the meal option by highlighting the tab	
5.5.3	Highlights the object that the user has created and gets them to press a secondary “+” button that will appear next to each meal object.	
5.6	Explaining the “-” button	Entire 4.6 Section will be done as a video
5.6.1	Screen darkens and the “-” button is highlighted from main page	
5.6.2	Text box explaining function of “-” button and how the screen the user has been taken to is a summary of all the foods the user has entered that day	
5.6.3	Highlight of the “edit” button at the top of the summary page and user encouraged to press	
5.6.4	User is prompted to press the trash icon next to meal object	
#6 – Object creation system	Criteria	Test Methods
6	User will enter the food creation tab and stay at the default individual food tab and presses the create button	Actual creation in app will be done through a video
6.1	User inputs name of object	

6.2	User will input calories and nutrients -> Will always have a reminder to be based on 100g	
6.3	Calculations to set the nutrients and calories of the object in grams to per 1g of the object	Data outputted to a test file from a get method in the class
6.4	Object will be available to be added and seen in the "+" button menu	
#7 – Meal creation system	Criteria	Test Method
7.1	User selects meal tab from the top menu whilst in the food creation tab and presses the create button	Actual creation in app will be shown through a video and outputs will be shown
7.2	A tick list is displayed of all the individual objects a user has created and allows the user to select all the foods that they want to enter	
7.3	Able to give the meal a name and enter each food amount in grams that has been entered	
#8 – Calorie Tracking system	Criteria	Test Method
8	"+" "-" and "Remaining calorie" visual displays on main page.	Screenshot of main page
8.1.1	"+" button option	Video of adding an object as a food and it is updating in the calorie menu \/
8.1.2	Taken to a separate tab that has another menu option for individual foods and meals	
8.1.3	Objects should be able to be selected, and another button appears underneath to finalise	
8.1.4	Calculations performed to remove from daily total left and add to daily grams of nutrients	Print to a test document as well as included in video
8.1.5	Display on main menu should be updated	
8.2	"-" button	Will be presented in video form \/
8.2.1	User taken to the summary page for food	
8.2.2	User able to press the edit button at the top of page	
8.2.3	Trash icons displayed next to objects and able to be pressed by user	
8.2.4	Calculations to add to daily total and remove from daily grams	Print to a test document as well as included in video
8.2.5	Display on main menu should be updated	
8.3	Main menu calorie display	Screenshots
8.3.1	Number display to show how many calories left	
8.3.2	Visual display in form of ,e.g. a bar slowly filling / emptying	

8.3.3	Visual display for nutrients	
#9 – Notification System	Criteria	Test Method
9	A separate tab from main menu that allows users to set themselves notifications	Notifications on phone screen at set times screenshots
9.1	Option to set every day, or which day of the week they want notifications	
9.2	Option to set what time notifications will arrive	
9.3	Notifications arrive at phone after set	
9.4	Message sent to phone is randomly selected from a list of potential messages	
#10 – Design	Criteria	Test Method
10.1	Nature theme, lots of shades of green and white	Screenshots and documented opinions of others
10.2	Main menu window	Screenshot of all 9.2
10.2.1	Contains company logo	
10.2.2	Contains daily calorie tracking system	
10.2.3	Sliding menu to travel to different tabs in app	
#11 – Ability to edit information	Criteria	Test Method
11	Settings window for editing information	Screenshot and change in information in app
11.1	Settings button to UI in sliding menu of app	
11.2	Separate buttons to edit each of the collected personal information	Screenshot of test document displayed before and after change
11.3	Button to edit the daily calories instead of relying on recommendations	
11.4	User is able to edit the individual details of each food item they have created.	Video
11.5	User is able to edit the content in, and amount of each food in a meal item.	Video
11.6	Ability to edit and delete any items added to the daily calorie tracking module	Video
11.7	Ability to edit the quantity of any items added to the daily calorie tracking module.	
#12 – Ability to search information	Criteria	Test Method
12.1	The user is able to search already created food items	Video
12.2	The user is able to search for individual food items during the creation of a meal	Video
12.3	The user is able to search for already created meal items	Video
12.4	The user is able to search for both food items and meal items when adding them to the daily progress amounts.	Video

Design

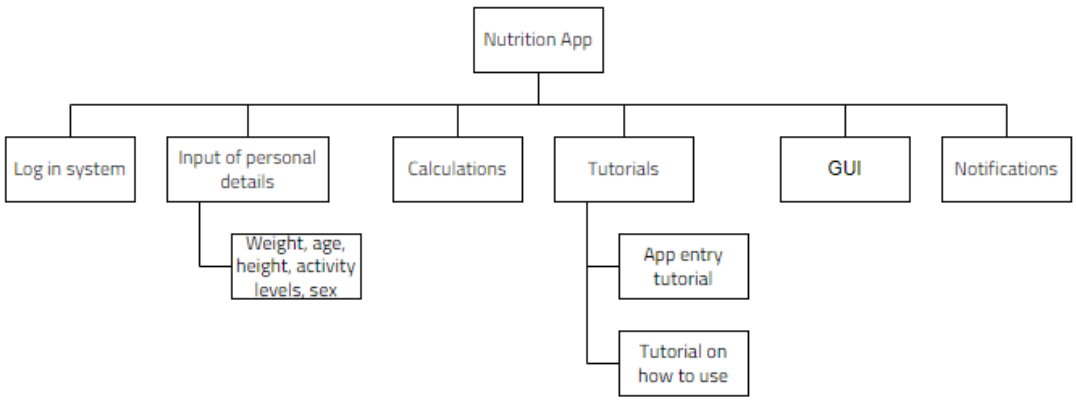
The solution will be coded using Android studio, which uses a Gradle-based build system that allows developers to customize, configure, and extend the build process. This allows users to create activities or fragments from activities which are individual screens that an application would have. The solution will consist of Kotlin activities or fragments on activities, and an XML design user interface layouts that link to each activity/fragment.

General Decomposition

Breaking down a complex problem into smaller, more precise components is a powerful approach to tackling a large-scale problem like this. By systematically dividing the main challenge into progressively smaller modules, the problem at hand transforms a large complex task into a series of manageable, individual yet connected problems that can each be addressed independently.

Working with modules will be beneficial as it will allow testing of the program to prove less of a hassle, each module can be isolated and tested individually. Furthermore, decomposing the solution allows the creation of activities or fragments specific to the functions they need to complete, lessening the hassle of coding the solution in one go, these activities can then be linked together in order to create a finished solution and allows for further development in the future through the creation of new activities / building fragments on activities.

Below is a decomposition diagram acting as a hierarchy chart to show how the problem will be decomposed into modules:

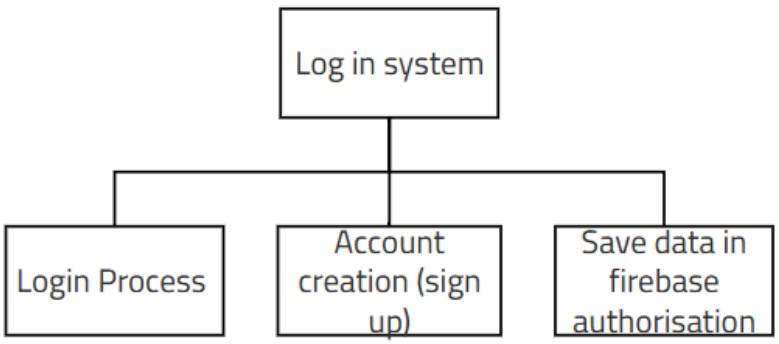


This breaks down the app into 6 main sections that will act as features in the app, starting with a log-in system that allows the users to keep their data under one account, to notifications that will send reminders to the user. These modules can be broken down and worked on separately and then connected together to form a solution, so future development can happen to improve the app and add features by developing new modules.

The app is warranted to be broken down into 6 main features (Authorization, Personal Details, Calculations, GUI, Tutorials, and Notifications) for several reasons. First, it keeps each major function separate and independent, making the code easier to maintain and update. For example, we can update how calculations work without touching the user interface or change the tutorial content without affecting core features. Second, this structure supports a natural flow of data - user authentication leads to personal details, which feeds into calculations, which then display in the GUI. This clear path prevents data inconsistencies and makes the system more reliable. Third, the modular design makes it easier to add new features in the future, it makes it easy to see the purpose of each module and the part that it will contribute to create the app, as each module will perform its own specific function separate from the rest so it can be coded individually, then each module will be able to be brought together and the modules can interact with one another to create a finished solution. The current modules will be further broken down to even smaller sections so that they can be easily worked on. These modules include:

Authorisation Module

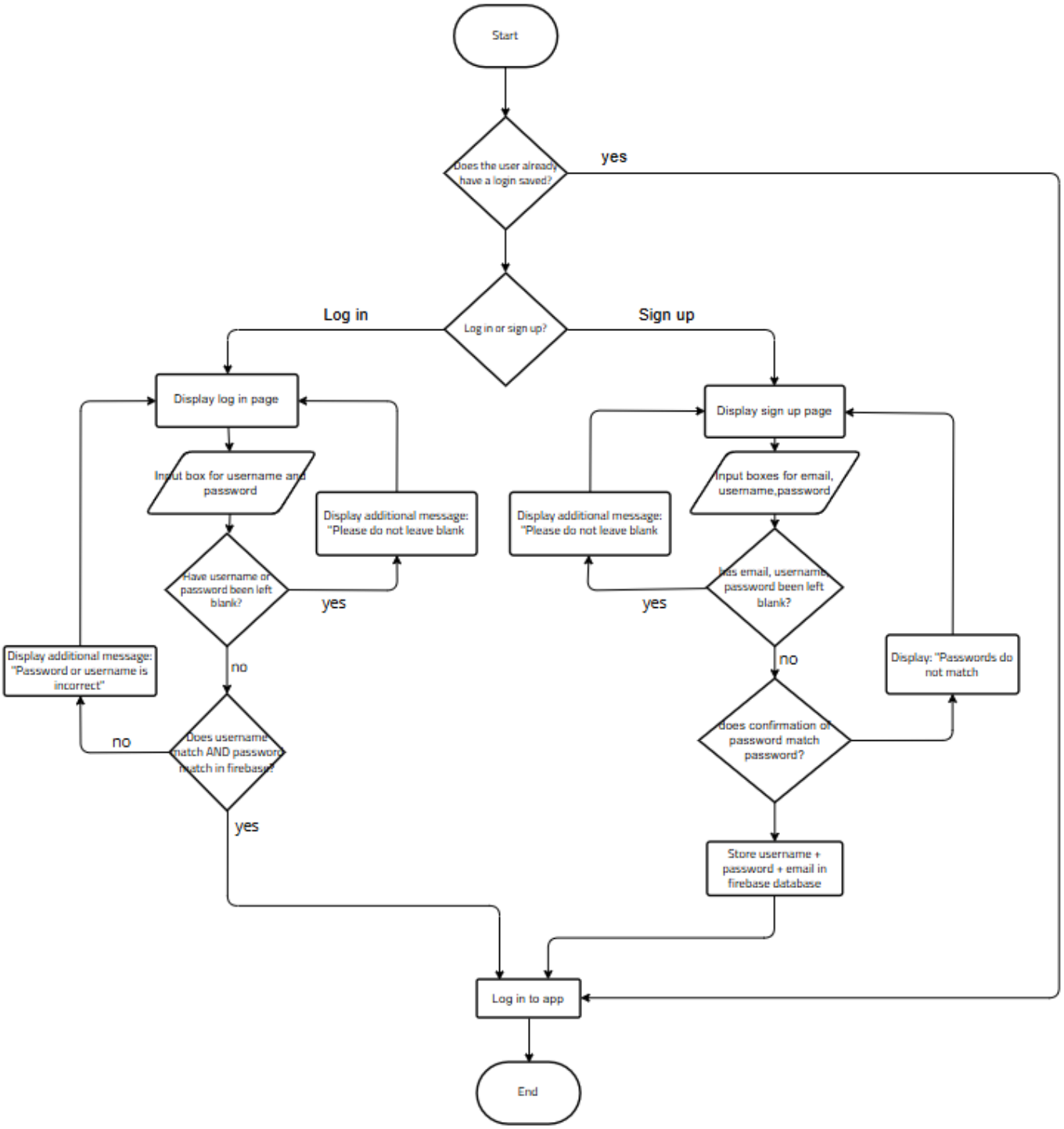
Decomposition



The authorisation in module will be responsible for allowing users to create an account that stores all their entered detailed and created objects. This information will be stored using firebase authentication, this is greatly important as the API provides encryption and data storage provided by a trusted company at their servers, storing information using firebase authentication generates a specific user id string, that can later be used to identify individual accounts so that the user will be able to log in on a separate device. The login-first approach was chosen over starting with signup as it prioritizes returning user convenience, which is crucial since users will access the app frequently to

track their daily intake. This follows established patterns from major applications where login is the default state, with signup as an auxiliary option, making the interface more intuitive for users who are already familiar with other apps. It is further justified to group this into one module since all the sub-modules need to be able to interact together to allow a user to create their account.

Flowchart



This is a flowchart to roughly explain the log in process that a user will have to go through whether they already have an account or not.

As the user is logging in to the app, the app will check whether or not a session has already been created and stored in the users device, if this session is created then the login screen is skipped and the user can proceed straight to the app without having to log in every time. This session-based approach was chosen over requiring login each time because nutrition tracking requires frequent daily access, and reducing friction in this process is crucial for user retention. If the user does not have a session previously opened the app will take them to either the login screen or signup screen, this will most likely be used by linking an activity from one page to another through a button or text. If the user does not have an account already created, they will move to the sign up page, where they will be asked to enter an email, password and confirm the password. The app will check whether the email is real or not before proceeding with the creation of an account, with specific validation requirements for passwords including minimum length and complexity. These validation requirements are justified as they provide security while remaining manageable for users who need to frequently access their nutritional data.

The structured flow of the login and authentication algorithm is justified by both security requirements and user experience optimization. The initial session check is warranted to prevent redundant authentication steps for returning users, making daily app access more efficient. The deliberate separation of login and signup flows is suitable for maintaining security standards, as it ensures proper email verification can occur. This design approach is further justified by its alignment with industry security practices while remaining accessible to users new to nutrition tracking.

Once the account has been created the user will be sent back to the login screen to re-enter their details and proceed into the app - this extra step of requiring login after signup was chosen to ensure the user remembers their credentials and can successfully authenticate, preventing future access issues.

Pseudocode

```
procedure check_existing_session()
        If user_logged_in equals true then
                // proceed to the main app
        Else
                 // proceed to login screen.
    endif
end procedure
```

This procedure is designed to handle the initial app launch, checking if there's an existing authenticated session before proceeding. The session check happens before any UI is shown to the user, making the app launch seamless for returning users while maintaining security by verifying the authentication state and data existence. This approach significantly reduces friction for regular app users who need frequent access for tracking their daily nutrition.

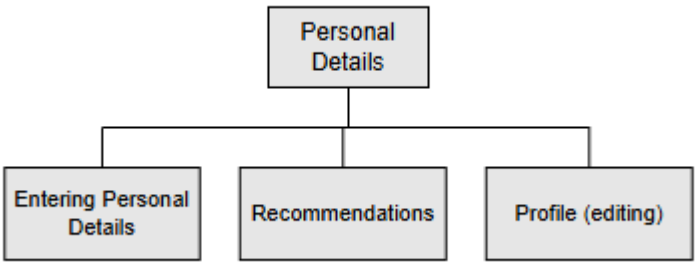
```
If login_button is pressed then
    If email and password are not empty then
        // find username and password stored in firebase and set to stored variables
        if username == storedusername and password == storedpassword then
            app.login
        else
            // display an authorisation error message
    else
        // display a message to fill in fields


If sign_up_redirect is pressed then
    // send the user to the sign up screen




If sign_up_button is pressed then
    if (username.isNotEmpty() and password == matchingpassword) (password == complexity) then
        // store data securely in database and create unique userid
    else
        // display message to fill in fields
```

Here is the basic pseudocode behind the logic of the login and sign up, there may be a complexity filter within the login feature where the user must enter a certain number of characters or a mixture of characters before they are allowed to properly create an account. The login button then handles the matching of the username to the password. The username will be treated as the user email instead of having the user think up a unique username, since emails are all unique in the first place it makes this a simpler solution, eliminating the need to store potentially hundreds of different usernames on top of hundreds of different emails when users download the app. The sign up redirect will be a function from the login screen so that the user can move to the sign up screen and create an account. This is also justified as many applications start with a login screen, assuming the user already has an account and then from that screen they will be redirected, so this adheres with what a user is used to helping lift the professionalism of the app.

General Summary

This module is a general module that is included in every application that stores user data, it means this module is justified as it turns the app into a piece of software that can match that of competitors by storing data seamlessly and allowing the user to access this same data from a different device should they need to, as well as protecting their data in a secure online database.

Personal Details Module



This is a rough breakdown of the personal details module, this module will be used to create the personalised details for a user and will consist of the entry, recommendations and later a feature to be able to edit this information in order to maintain accuracy in calculations.

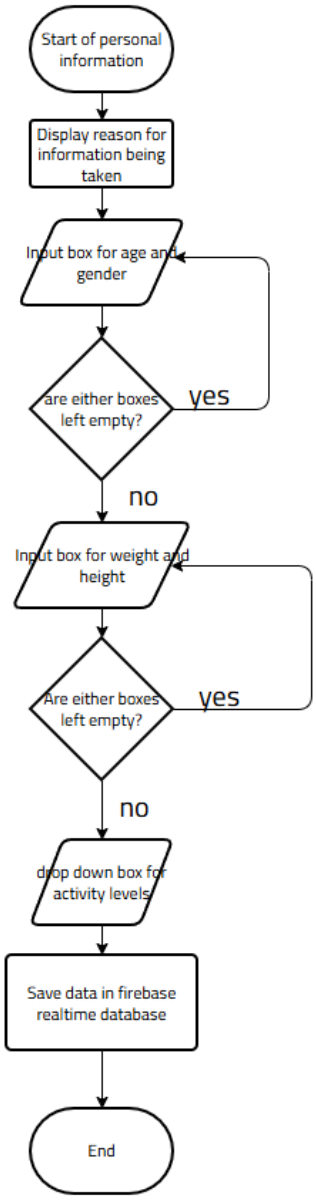
Personal details input

Upon first creating an account, the input of personal details module will be like a welcoming screen that requires the user to fill in a couple pieces of information about themselves. If the user has already filled in this information and is logging in, for example, a second time this screen will be skipped due to a session already created in the user's device, and the user will proceed straight into the app. Asking the user to input some details can be justified to my solution as it is basic information that the user would already know, but in turn it can act as a way of abstracting certain calculations for them and recommend numbers in case they lack knowledge, this is a feature most other applications do not have as most of the time they would set a very generic goal, not taking into account the users circumstances.

The design of this screen will use a background designed on Canva with different text inputs placed over the top using XML, the boxes will use their own validation, such as being set to only allow a specific input such as age, height and weight only allowing a number input to prevent errors through converting a string to an integer, this can be done in the xml during the edit box, where the input type is specified to allow only the number keyboard of the device to be used. For sex and activity levels which will use strings will include a dropdown option, which has specific strings available so that the user cannot enter information that cannot be used.

The personal details module is a crucial foundation for the application, capturing essential user information that directly influences nutritional and fitness recommendations. By capturing data using a dedicated module, the system creates a focused onboarding experience that introduces key information without overwhelming the user. Grouping this together as a module allows for a flush, small tutorial that does not overwhelm a user and generates a set of data that can be stored in an object/variable to be able to be passed into the calculation's module.

Flowchart



This is a flowchart that represents what would happen *after a first-time login* and account creation of a user, explaining the choices needed to be made to save all the variables, these variables will then be sent to a firebase database under the users id in a section for personal information so that they can be accessed from at a later date, from anywhere in the program and on other devices should the user choose to. There will also be an explanation at the start for what the information will be used for and how it will be calculated, so the user can be comfortable giving that information working as a tutorial for the user.

This stepwise creation for personal details is justified by its ability to prevent data errors through immediate validation. The structured approach is warranted because it ensures all BMR and AMR calculation requirements are met before proceeding. The decision to validate each input type separately (numbers for age/weight/height, dropdown selections for activity levels) is suitable for

preventing calculation errors downstream. This methodology is justified by its ability to guarantee accurate recommendations from the user's very first interaction with the app.

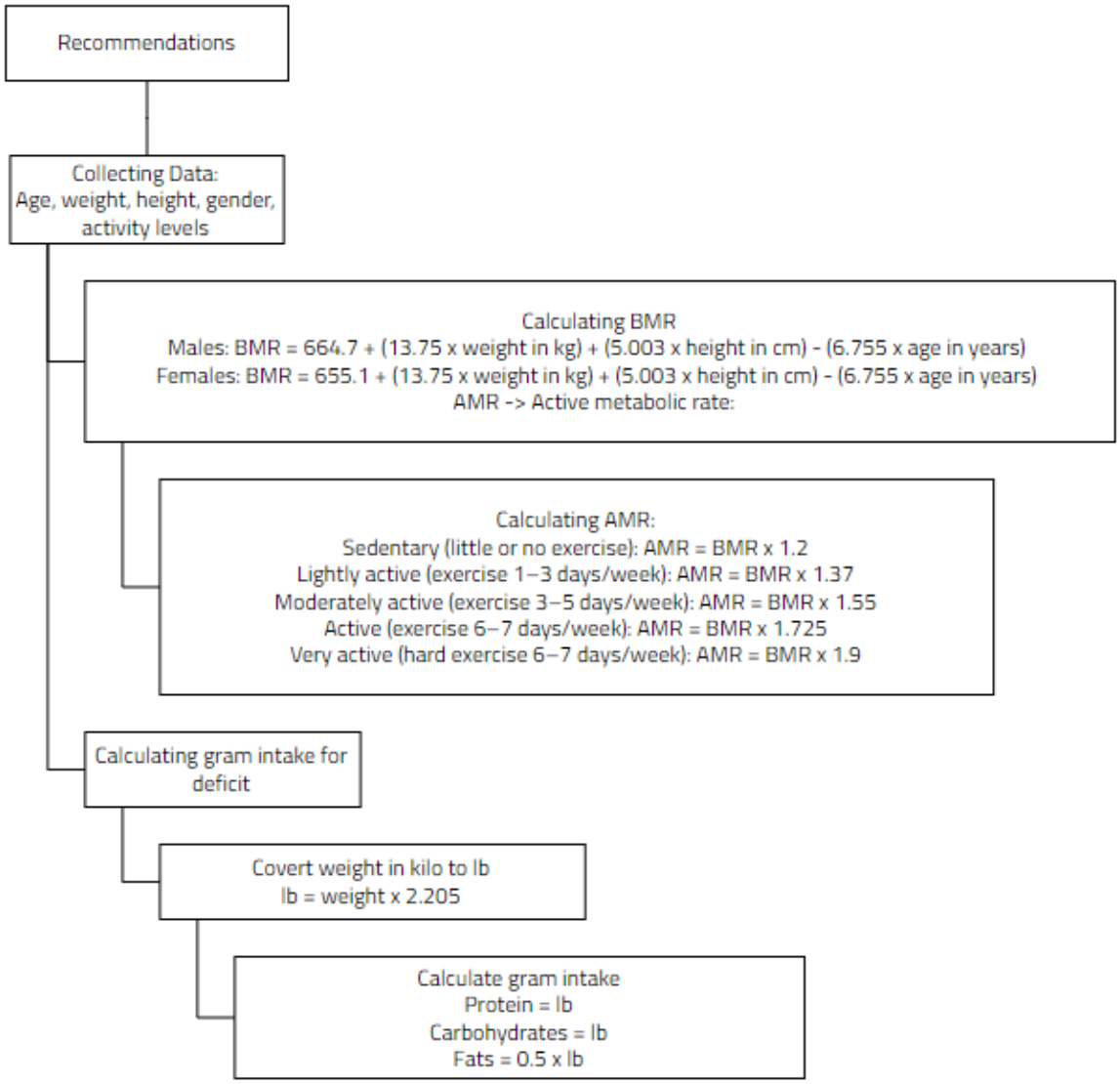
Pseudocode

```
class personalinfo()
      private age
      private weight
      private height
      private sex
      private activitylevels

      public procedure setinfo(a,w,h,s,al)
            age = a
            weight = w * 2.205 # Converts to lb
            height = h         # Entered as cm
            sex = s
            activitylevels = al
      endprocedure
```

This is pseudocode for the main class that will hold the personal information, this information will be passed into the class through a collection of input boxes displayed for the user, with a submit button that is linked to the setinfo call to the class. This is the best option for this as everything is inputted in one go and set at once, these will also be required boxes to prevent errors, meaning the user will not be able to proceed until they have inputted these details. It is justified for an object to be used here as it is a lot easier to upload a single object to firebase with multiple variables, rather than calling the same database upload multiple times for each and every variable, it also means that other procedures can be created inside of the class that will perform calculations when needed.

Recommendations

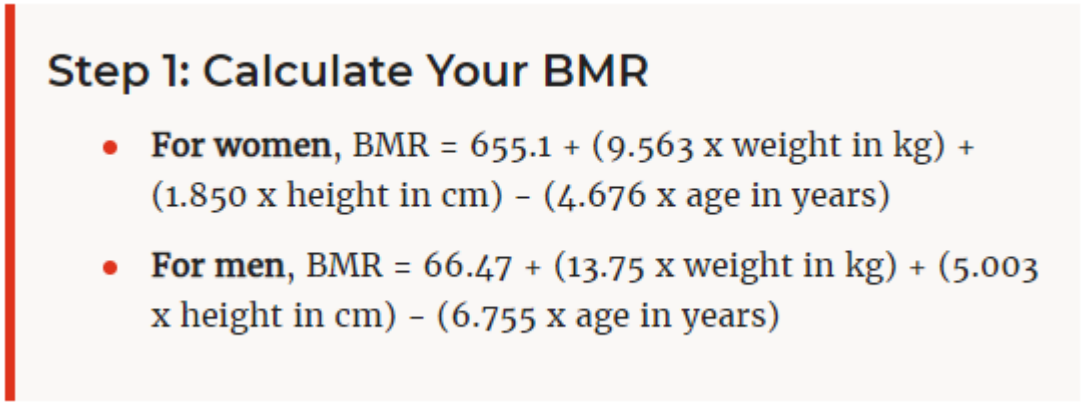
Decomposition



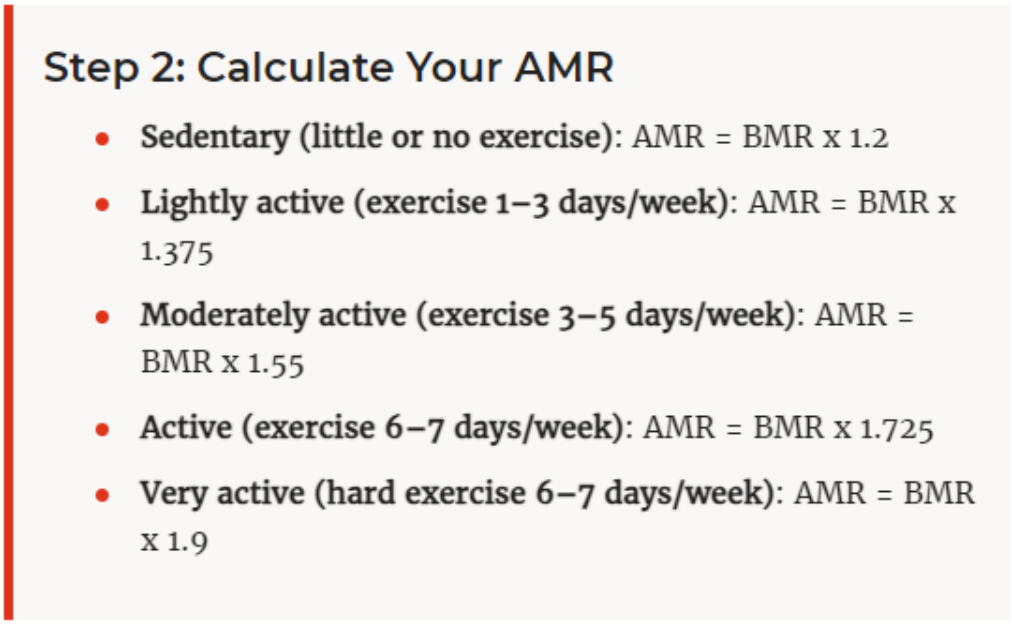
I think recommendations are essential to give users a starting point without having them need to learn lots of things in order to figure out their numbers. This means that the recommendations feature will be important as it will calculate numbers for the user from just a few details about themselves. As previously mentioned, increasing convenience and acting as a companion that is there to help them justifies this module because the user only needs to enter basic details, but not have a full in depth knowledge of nutrition.

This decomposition diagram breaks down the main calculations that will be performed during the app *outside of any specific modules*, to do with recommendations and targets into single functions that can be worked on individually. By using an external class to calculate these numbers means the calculation can be performed from anywhere in any activity, the information would have to be pulled from the firebase Realtime database and passed into the class methods as an object. The calculation method will take in the information of the user and the end result will be based on conditions such as which sex has been entered, and their activity levels to calculate AMR and BMR

It is recommended by almost any doctor/GP/etc that a person should exercise at least 3 times a week. This means that a person's calorie requirements for their day-to-day life may increase, depending on how intense and how much activity they do. One of the formulas created to work out the number of calories needed is called the "Harris-Benedict Formula" which uses a person's BMR (rate of metabolism -> conversion of calories and oxygen to energy) and AMR (active metabolic rate -> BMR to do with how active a person is): This tool works out a person's maintenance calories (the amount of calories to be consumed to maintain the same weight and activity levels) [1]
The formula:
BMR:

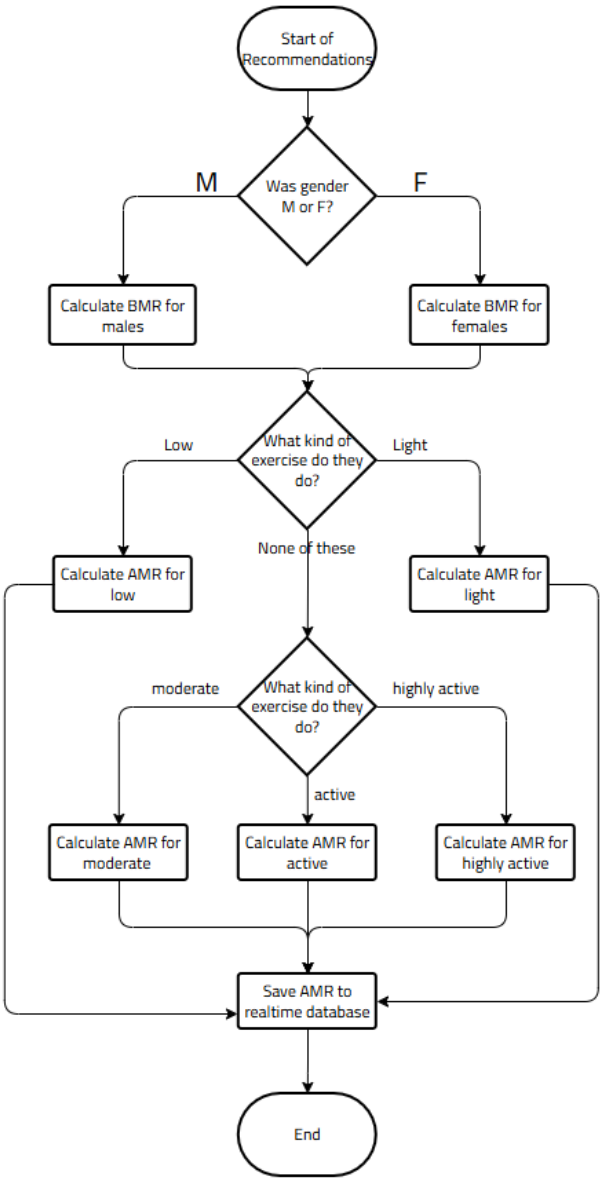


Which is then turned into AMR:



[1] https://www.verywellfit.com/how-many-calories-do-i-need-each-day-2506873 -> accessed (24/02/2024)

Flowchart



Here the flow chart simplifies the choices that need to be made by the algorithm depending on user choices to calculate some recommendations for them. This section will most likely be handled inside of the class containing the object with the user's personal information. The algorithm itself will be a procedure.

Meaning the following items should be measured to find out a person's required calories:
Age, Weight, Height, Gender, Activity Levels.

Pseudocode

I have created pseudocode for this section, the code builds on the personalinfo class justified above would look something roughly like this, with further extended set and get methods for each individual item when coding which have been abstracted from the pseudocode for ease of understanding. These would all be objects created through object-oriented programming, which allows me to store multiple data values in one object that can then be fetched or set from. Any variables would be set as private as to encourage encapsulation, so that they can only be accessed and changed through methods written in the class, this means that they will not be accidentally changed during runtime.

```
class personalinfo()
      private age
      private weight
      private height
      private sex
      private activitylevels

      public procedure setinfo(a,w,h,s,al)
            age = a
            weight = w * 2.205 # Converts to lb
            height = h         # Entered as cm
            sex = s
            activitylevels = al
      endprocedure

      public procedure calculatebmr()
            if sex == "F" then
                  bmr = 655.1 + (9.562 * weight) + (1.850 * height) - (4.676 * age)
            elif sex == "M" then
                  bmr = 664.7 + (13.75 * weight) + (5.003 * height) - (6.755 * age)
            endif
      endprocedure

      public procedure calculateamr()
            if activitylevels == "low" then
                  amr = bmr * 1.2
            elif activitylevels == "light" then
                  amr = bmr * 1.375
            elif activitylevels == "moderate" then
                  amr = bmr * 1.55
            elif activitylevels == "active" then
                  amr = bmr * 1.725
            elif activitylevels == "highly active" then
                  amr = bmr * 1.9
            endif
      endprocedure

      public function getage()
            return age
      end function

...
endclass
```

This code shows the creating and setting of private variables that act as the personal info entered by the user, these will be stored in an object to be accessed together and not get lost with any other variables. The procedure sets the variables through arguments that will be called in the main code after the user has entered the initial values after creating an app or gone into the app options to change their data later.

The calculatebmr and calculateamr procedures stem from the calculations listed above and will generate the value that will be used as the calorie limit for the user and will take already set variables from the personalinfo class to complete the calculations. There will be later added set and get methods if the user wishes to enter their own values.

This would be a suitable way to create the recommendations and requirements for the user, as an object can be used to directly save and retrieve details from a database. This is also effective due to the private variables being encapsulated, meaning that the values can only be accessed through these set and get methods, so it reduces the change to accidentally change their values at any point randomly through the app.

```
User = personalinfo.setInfo(age,weight,height,gender,activity levels)
# -> All will be set from input boxes when the user enters app for the first time
```

This line of code shows roughly how the user profile will be created when the app is first started and how we can later set information if it changes.

Functions to return weight and amr into a separate variable outside the class so that it can be used in other calculations.

```
new function getWeight()
        return weight
end function
new function getamr()
        return amr
end function
```

The daily intake of nutrients and calories that a person will need to achieve as a “goal” will be calculated either within the personalinfo class or its own separate data class specifically storing daily variables, this variable will not be changed so I may set them to private, but instead their values will be copied into a variable that “resets” back to the copy of the original value each day. This means that most calculations to do with a person's info will be stored in the same class instead of creating multiple, this means that it will be easier to develop since there is no need to remember all the different classes for each specific thing, and everything is stored together in one place that can be used as easy access.

```
    private Calories
    private Proteins
    private Carbs
    private Fats
```

```
public procedure CalculateNutrients()
    Calories = amr
    Proteins = weight #-> same number as lb but grams
    Carbs = weight
    Fats = 0.5 * weight
end procedure

public function getDailies(option)
    if option == "calories" then
        return Calories
    elif option == "protein" then
        return Proteins
    elif option == "carbs" then
        return Carbs
    elif option == "fats" then
        return Fats
    end if
end function
```

Followed by a function that returns the daily nutrients that are stored in the personalinfo class for use later in the *calorie tracker system*, other calculations or other classes, this means that the values inside the personalinfo class cannot be accidentally changed as they can only be accessed through the get and set methods.

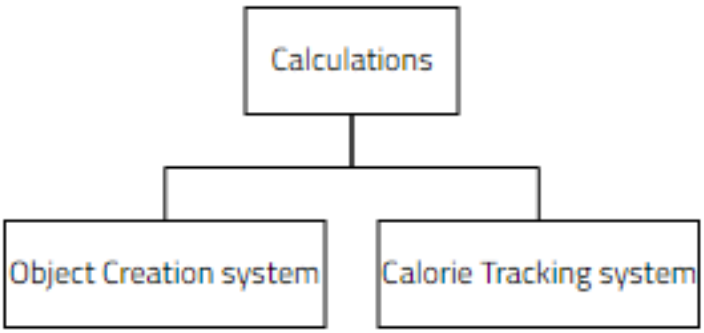
These algorithms are justified to be included to help form a complete solution as an object can be created from anywhere in the program and used to store and calculate these values, it also means that any other module that requires some sort of calculation based on user data or progress indicators based on the user goals can retrieve these values when needed by the required methods.

General Justification

The personal details module addresses a critical gap in existing nutritional applications by simplifying the complex process of nutritional planning for novice users. Many existing apps overwhelm users with technical information and complex calculations, creating a barrier for individuals new to nutrition. Therefore, the personal details module is justified to be a part of this application as it serves as an educational tool that builds user confidence and trust. By handling complex calculations and providing clear, personalized recommendations, the module immediately demonstrates the apps value. New users are more likely to adopt and continue using an application that feels accessible and supportive, rather than one that requires extensive nutritional knowledge. Essentially, the module acts as a bridge, transforming potentially overwhelming nutritional science into an approachable, user-friendly experience that empowers individuals to take control of their health and fitness journey.

Calculations

The calculations sub module can further be decomposed down to three main concepts for the module from the many that were previously presented, each with their own further submodules.

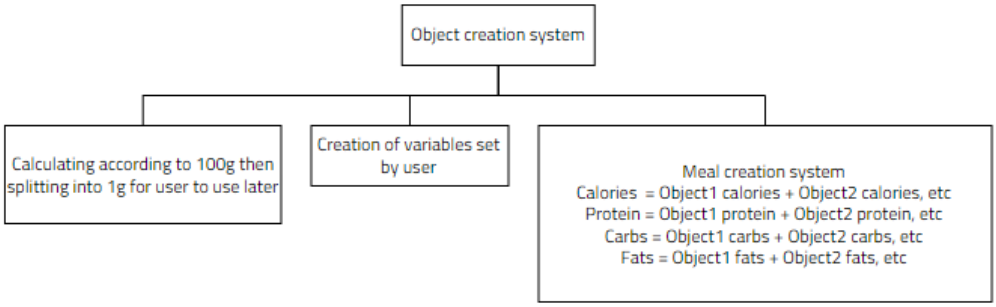


For calculating and figuring out the recommendations to give to the user, such as their calorie maintenance, how much calories they need to eat to lose weight, what a healthy way to lose that weight would be, etc. This means that the person will not have to work these numbers out themselves and will increase convenience for the user, as well as help those newer to the sector with potential research they could do to expand their own knowledge. Grouping all the calculations into one module will allow for an object to be created that stores all the numbers other modules will need to take to complete their requirements.

Unfortunately, due to the variety of functions that the individual systems need to complete there is not much available reusability within different sub-sections, but the modules themselves will need to be reused, for example every time the user wants to create a new food object or group together certain objects. This unfortunately means that the app will be less efficient than if I found a way to reuse modules for different tasks, but it should be relatively efficient since one module will be reused many times within the same task.

Object creation system.

Decomposition



The object creation system module will be responsible for the creation of any food items the user will create. Since this app is not as technically advanced as others it will not have functions such as the ability to scan barcodes / have data inputted by millions of users, the user will have to create their own. To solve this issue, the program will take an object-oriented programming approach that will have a class that can create objects of a food item and have the user give each object a name and the amount of calories, proteins, fats and carbohydrates that are listed on the food label and store it as an object that the user can then add to their daily calories. This is justified to be its own module as it means that each food will be stored as its own object that can later be manipulated to fit the users' needs whenever they please. This means that the user will not have to input the same data every time they want to use it, instead they will just select the object they want to add from the list of already created ones saving them time and ensuring accuracy.

This will be the method of food item creation for the user, with their first creation, a tutorial will be presented (not shown in modules as not a calculation). This will inform the user on where to find the information they are looking for (back of UK packs of food with the label containing nutritional information) so that the user can input their own details based on the foods they are eating. This is a hugely justified feature as it allows the abstraction of the need to create a huge database on the millions of different foods, brands and their data, furthermore this also allows for more personalisation as many times competitors apps use only a rough estimate and are sometimes off on the true data of food item, instead of the exact values like the ones on the packets which are required by law.

Nutrition

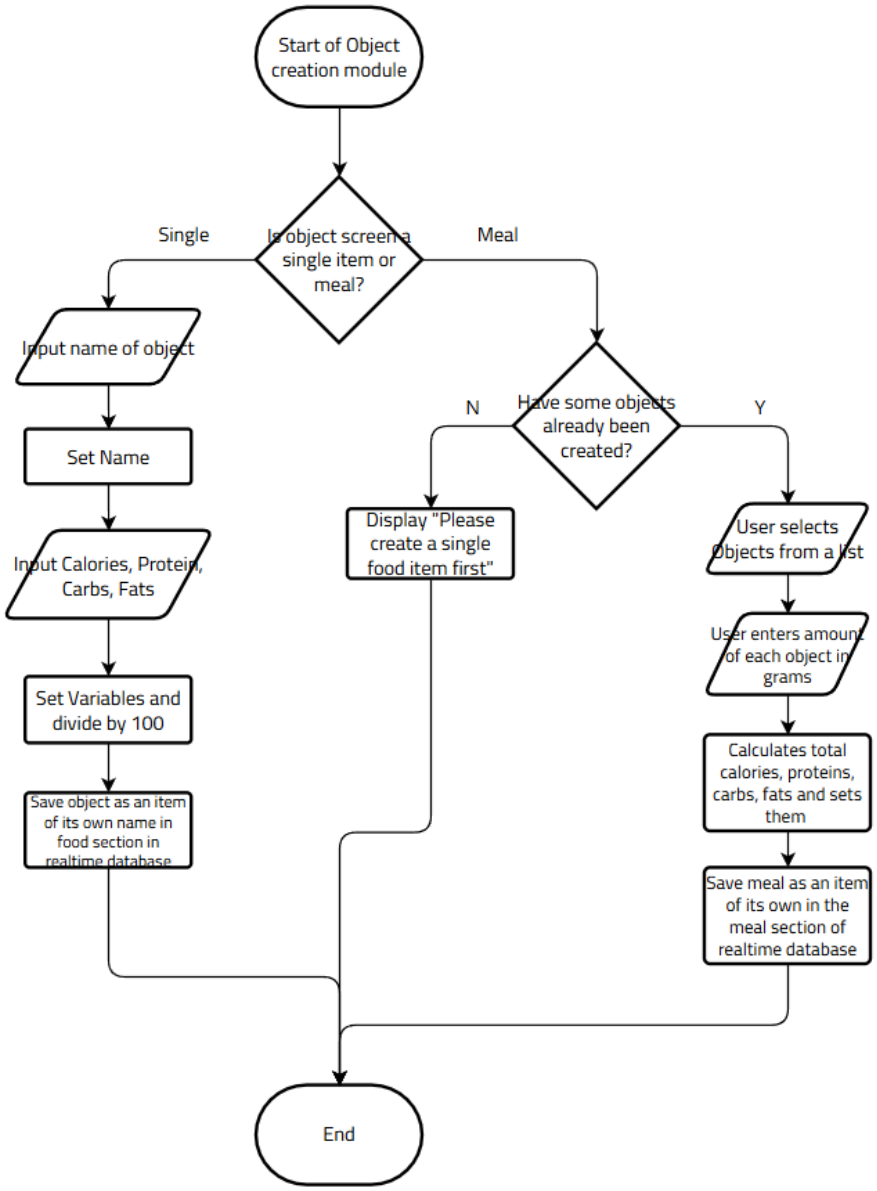
Typical values	100g contains	Each slice (typically 44g) contains	% RI*	RI* for an average adult
Energy	985kJ 235kcal	435kJ 105kcal	5%	8400kJ 2000kcal
Fat	1.5g	0.7g	1%	70g
of which saturates	0.3g	0.1g	1%	20g
Carbohydrate	45.5g	20.0g		
of which sugars	3.8g	1.7g	2%	90g
Fibre	2.8g	1.2g		
Protein	7.7g	3.4g		
Salt	1.0g	0.4g	7%	6g

This pack contains 16 servings

*Reference intake of an average adult (8400kJ / 2000kcal)

Since all consumable foods in the UK must have a label that gives information based on 100g or 100ml, this will be my main method of inputting data, the user will input information based on 100g or 100ml, then the app will divide these values by 100 in order to get 1g or 1ml that can then be multiplied by however many grams the user has when inputting to the daily calories. This will mean that all items will be able to be considered for, as the user will not have to sit there and work out what each item is in the required grams as foods will always have nutritional information of 100g.

Flowchart



This is a simplified version of my object creation system, there will be two separate screens on the object creation system, separating individual food items and meal items. These will both be their own classes and will most likely be handled with a third class to store all the different items in a list to be fetched from. The first choice handles which screen the user is on. If there is a single item there does not need to be any decision made further, an input screen can be displayed for a user to enter values. However if the user is on the meal creation system, if there are no objects already created then it is not possible to make a meal and therefore a message needs to be displayed, but if

there are objects then the algorithm needs to give the user the ability to select the objects they want, enter the amount of each object, and then total all the variables together.

The separation between individual food items and meal creation in this algorithm is justified by the need for data reusability while maintaining accuracy. The standardized approach to nutritional values (per 100g) is warranted as it ensures consistent data storage and reliable calculations. This design is suitable for allowing users to build their own food database while ensuring standardization. The approach is further justified by its ability to maintain data integrity while offering users flexibility in food tracking.

The objects created should transfer from device to device so that the user does not need to recreate the same objects over and over, this means the objects would also need to be saved to the Realtime database, these will also need to be placed under the user id created through authentication however they cannot interfere with the data created in the personal details module, therefore a new child node will need to be created specifically for foods and then another for meals, where each food is another child node under the foods child containing the name and information related to the food item.

Pseudocode

```
class FoodCreation()
        public Calories
        public Proteins
        public Carbs
        public Fats

        public procedure setNutrients(cal,p,c,f)
                Calories = cal / 100
                Proteins = p / 100   # -> Takes what was inputted as 100g and turns it into 1g
                Carbs = c / 100
                Fats = f / 100
        end function

        public function getNutrients(type)
                if type == "calories" then
                        return Calories
                elif type == "proteins" then
                        return Proteins
                elif type == "carbs" then
                        return Carbs
                elif type == "fats" then
                        return Fats
                endif
        end function
```

Creating a class to hold the food creation allows for the creation of a single object holding all the variables created by the user that can then be uploaded to firebase to be stored permanently in one go, as compared to having to constantly call an upload command to upload each and every variable

```
class MealCreation() inherits FoodCreation
    private Calories
    private Proteins
    private Carbs
    private Fats

    public procedure setMealNutrients(Foods,amounts) # All objects inputted will betaken
                                                     # as a list and a separate list for their amounts
        for x 0 to len(Foods)-1
            Calories = Calories + ( Foods[x].getNutrients("calories") ) * ( amounts[x] )
            Proteins = Proteins + ( Foods[x].getNutrients("proteins") ) * ( amounts[x] )
            Carbs = Carbs + ( Foods[x].getNutrients("carbs") ) * ( amounts[x] )
            Fats = Fats + Foods[x].getNutrients("fats") * ( amounts[x] )
        next x
    endprocedure

    public function getNutrients(type)
        super.getNutrients(type)
    endfunction

endclass
```

The meal creation system module is a submodule of the object creation system and will be responsible for the grouping of objects and giving a total amount based on what is added. The class for this can be the same to perform calculations as the details of each individual object will be fetched from the database, a total calculated locally and then saved onto the class to be sent as its own meal. This means the main class can be reused to store and fetch data. A possible feature is a drop-down menu of certain sections that can be interchanged at a user's will without having to delete/mess around with the meal object, for example if a user has 3 different types of bread, they can add each of the 3 into a drop-down menu that they can later quickly select according to which one they are having in that meal.

The return of the procedure is the total for all the foods that have been added to the list, so the meal will come up as one big object that the user can quickly enter whenever they need to. This means that development will be more efficient as some code is reusable. Since amounts will be entered upon the creation of the meal, they must be placed into the procedure as a separate list, and placing all the amounts corresponding to the item position simplifies the process and allows the calculations to be performed in one go.

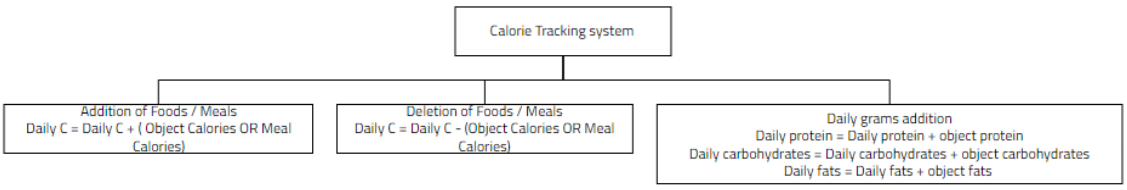
Then the object can be uploaded to firebase under a name entered by the user separately and can be retrieved when needed based on the values needed, making this a justified algorithm as it allows the creation of the entire object in one go, instead of having to save the same item multiple times and allows the objects to be easily grouped.

General Justification

Both the individual food item and the meal creation classes are justified to form a complete solution as they will allow other modules ,such as the tracking system, and any other goals to be calculated to make use of these methods and attributes meaning that these modules will be in constant use throughout the app outside of their respective creation sections. This also allows the app to perform just as well as other application, if not better than other applications because the user can enter the exact information, they need to about a food item instead on having to rely on the estimates based on potentially old data in other apps.

Calorie tracking system.

Decomposition



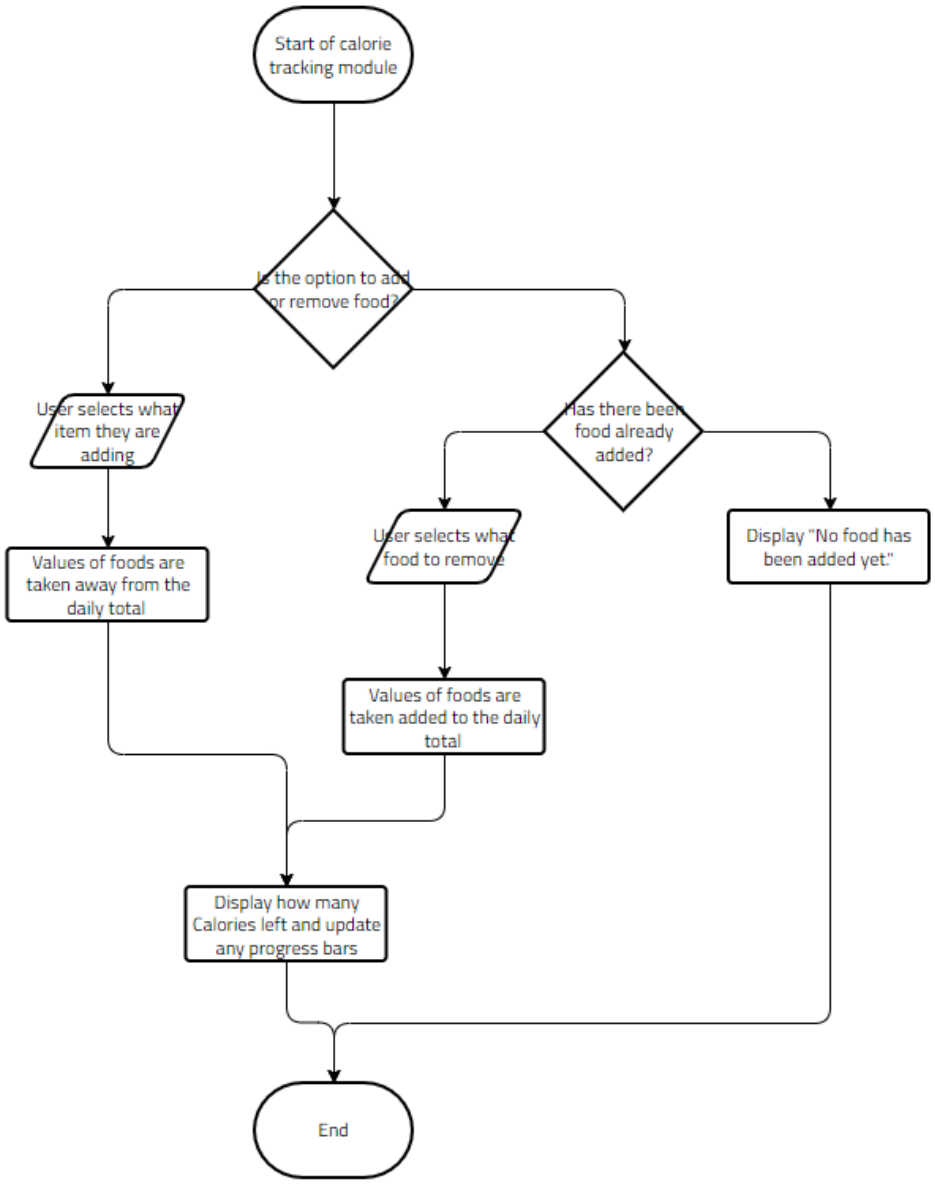
The calorie tracker module will allow a user to view the calories they have left for the day, how many they have consumed, what they have consumed, and their macro-nutrients (what their grams in fats, protein and carbohydrates add up to) as well as the module that actually allows the user to add their created objects together for each day. The calorie tracker will work as a giant progress bar that goes round in a circle, with smaller progress bars for the lesser macro goals The search sub-module will work within this to allow the users to search through their various created objects / meals for convenience. This means that a user will never get lost or forget how many they have consumed in a day as it will be easily readable information from the front page.

There will be a couple variables that reset with each day at a certain time (1am uk time) and will reset back down to 0 so that the user can input new objects and keep track of their calories for the new day. This will be done automatically meaning that it will be more convenient for the user. This is important as around 1am most people would be asleep meaning their digestive system can work at full power so that they can have calories again the next day, meaning the app would need to reset to encourage the user to refill their body.

These variables will be: Daily calories, Daily protein, Daily carbohydrates, Daily fats.
These are the most important factors for people attempting to track their foods, as they either base it on daily calories or a daily intake in grams for specific nutrients, this means that I will abstract the rest meaning that less calculations and processes will need to be completed on all the little niche information. These variables will be able to be incremented with foods at any point during the day, but I will also need to create an option to remove certain objects from these variables removing the number they have increased the daily by, in case of a mistake of an input from the user. This is important as humans tend to make mistakes often, so I do not want them to have to inconvenience themselves by remembering the fact they added an accidental input for the rest of the day and the number is too high over their daily limit.

This module can be broken down into three main sections and is justified to be done this way as it simplifies the process needed, and each section has its own separate functions and the process it completes, however when joined together all three make up the main process of the daily calorie tracking system

Flowchart



Here is a rough flowchart depicting what options would be taken whether the user chooses to add or remove food to their daily amount, this would handle all 4 key variables that have been entered, but the algorithm has to notice which screen the user has selected and proceed accordingly, if the user is on the remove screen with no food to remove then a message should be displayed. This means that they main process of adding and removing food, the key feature of the app is all in one module and can be worked on individually.

The simultaneous update mechanism in the tracking algorithm is justified by the need to maintain consistency across all nutritional metrics. The decision to preserve food objects while only manipulating daily totals is warranted as it supports both long-term data retention and daily tracking accuracy. This design approach is suitable for providing users with flexibility to correct entries while maintaining data integrity.

Pseudocode

```
function DailyVariables(user)
        Calories = user.getDailies("calories")
        Protein = user.getDailies("protein")
        Carbs = user.getDailies("carbs")
        Fats = user.getDailies("fats")
        return Calories, Proteins, Carbs, Fats

if time == (0100) then
        DCalories, DProtein, DCarbs, DFats = DailyVariables(user)
endif
```

To set the daily variables I will take the variables stored from the personalinfo class as mentioned before and use a function to return all the values I will need in one go. This means that space is saved and is more efficient in coding as multiple variables are set at once instead of writing out the call function multiple times.

```
function updateDaily(Calories, Proteins, Carbs, Fats, operation, object)

if operation == "add" then
    Calories = Calories + object.getNutrients("calories")
    Proteins = Proteins + object.getNutrients("proteins")
    Carbs = Carbs + object.getNutrients("carbs")
    Fats = Fats + object.getNutrients("fats")

elif operation == "remove" then
    Calories = Calories - object.getNutrients("calories")
    Proteins = Proteins - object.getNutrients("proteins")
    Carbs = Carbs - object.getNutrients("carbs")
    Fats = Fats - object.getNutrients("fats")


end if
return Calories, Proteins, Carbs, Fats
```

To update the daily tracker there is no need to store a completely new class and create objects since there will be only 1 per day and will be reset every day, so instead I have decided to just stick to a function that will add or remove based on an operation inputted by the user. Object will be the food / meal object that has been created in advance.

“add” -> this will **add** amounts from the daily variable when food is ADDED. Meaning these variables can be displayed as an amount the person has had in the day.
“remove” -> this will **remove** amounts to the daily variable when the user decides to REMOVE a food they have added. Meaning it will update the variable and the user can see that they have more left to eat in the day.

General Justification

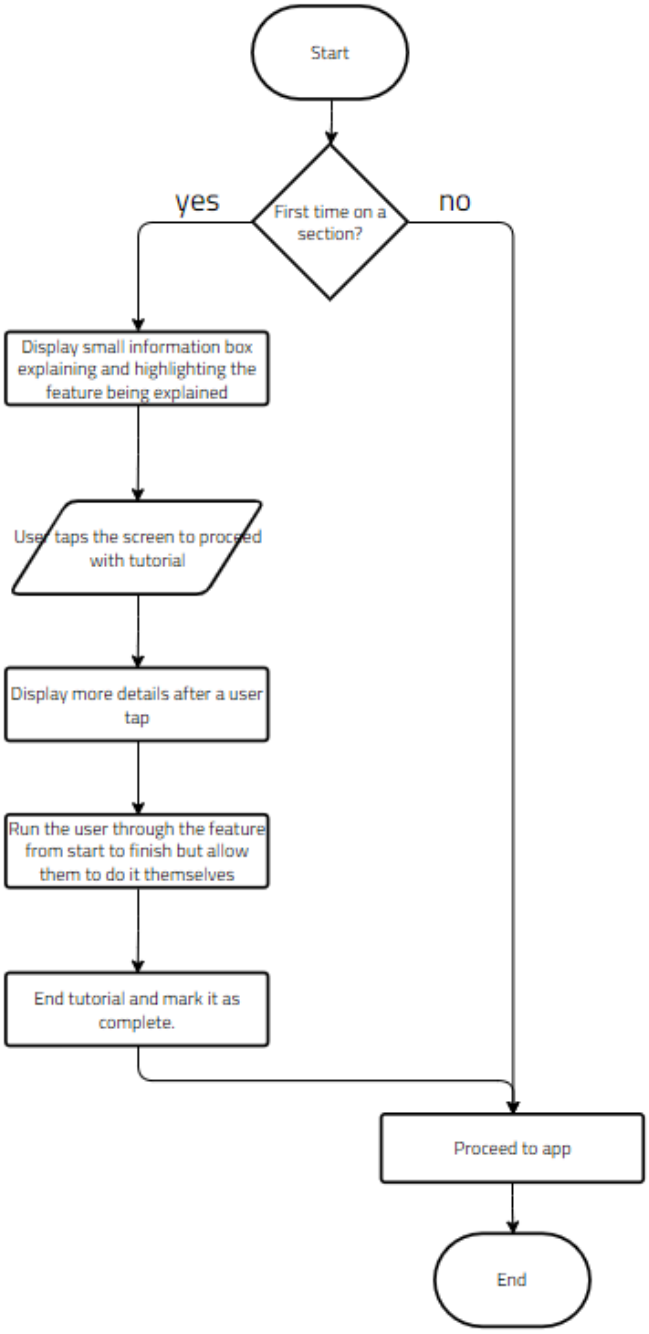
The calorie tracking module is justified as it is the module that connects all the modules created together, all foods items, meals, personal information entered, calculations performed are all joined together to create the core of the app that allows the user to track their foods that they have created, by the day, based on a daily goals that they would have had calculated and recommended to them in a simple matter easy for the user to understand what to do, making sure they do not get lost in the process.

Tutorials

Explanation

The tutorials module will be a main class for all the tutorials around the app. These will work individually from each other, but it is justified to place them under one parent-module so that they are all stored in the same place and the code does not get messy, since each section of the app that requires a tutorial upon first opening can request a sub-module from the parent-module of tutorials. This should mean that the code stays neat and tidy and in future developments only sub-modules need to be added and will already have a place to go. A tutorial is incredibly important for an application for new users as other apps tend to just throw their users in and they get overwhelmed, and it could lead to them not even wanting to attempt to continue their experience, so a tutorial can guide a user through the app and its features, giving a higher rate of retention and letting them learn at their own pace.

Flowchart



This is a very basic and generalised flowchart of what the tutorials in the app would run off. The tutorials would only appear if it was a user's first time entering that section / entering the app. This is to allow them to learn with guidance what each section does, but them to do it themselves with guidance in order to get a hang of how the app works. This will only run if it is a user's first time entering as it will be tedious going through the same tutorial over and over again if they already know how to use that section, meaning the convenience for the user is increased, as well as they do not feel as if they have just been thrown into the app and told to just do it like many of the other apps out there.

Pseudocode

```
class Tutorial()
    private TutorialID
    private TutorialName
    private CurrentStep
    private TotalSteps
    private IsCompleted

    public procedure initTutorial(id, name, totalSteps)
        TutorialID = id
        TutorialName = name
        TotalSteps = totalSteps
        CurrentStep = 0
        IsCompleted = false
    endprocedure
```

This pseudocode sets up a new tutorial to a class by assigning it a unique identifier, a descriptive name, and defining the total number of steps required for completion, meaning this is how many steps the user will need to go through to finish the tutorial. It also initializes the user's progress at the beginning and marks the tutorial as not yet completed. Creating a dedicated initialization function is justified because it establishes a consistent framework for all tutorials, ensuring nothing critical is overlooked during setup.

```
    public function getCurrentStep()
        return CurrentStep
    endfunction

    public function getTotalSteps()
        return TotalSteps
    endfunction

    public procedure nextStep()
        if CurrentStep < TotalSteps then
            CurrentStep = CurrentStep + 1
            if CurrentStep == TotalSteps then
                IsCompleted = true
            endif
        endif
    endprocedure
```

There will also be specific functions in the class in order to progress the user across the tutorial and get and methods to maintain the state of the tutorial, in case the user was to leave the app mid tutorial, etc, they will not have to restart it in its entirety. Using dedicated get and set methods is suitable as it creates a controlled interface that prevents other components from accidentally modifying the tutorial state.

```
public procedure resetTutorial()
    CurrentStep = 0
    IsCompleted = false
    HighlightedElement = null
endprocedure

public function isCompleted()
    return IsCompleted
endfunction
```

These procedures return the tutorial to its initial state. This would be used when a user explicitly requests to see a tutorial again or when the app detects that a user is struggling with a feature despite having gone through the tutorial previously. Having a dedicated reset function is warranted as it ensures all necessary values are properly reset in one operation, preventing partial or inconsistent resets. Furthermore, the function to make sure a tutorial is completed checks whether the user has successfully progressed through all steps of the tutorial. The app would use this to decide whether to show the tutorials again on future app launches, or to unlock more advanced features once a user demonstrates understanding of the basics. This could be used in a feature that tracks how long the user has been absent from the app, where they are able to replay a tutorial to refresh their memory on how to use it, or prevent tutorials from showing up again where they are not needed.

General Justification

The tutorials module is a crucial part of the complete solution as it directly addresses one of the main problems identified in the analysis - that many nutrition apps overwhelm new users by throwing too many features at them at once. By working together with every other module in the app, the tutorials create a guided learning experience that introduces features gradually and naturally. For example, when users first enter the object creation system, the tutorial shows them where to find nutritional information on food packaging and how to input it correctly, making the whole process less daunting. The tutorials also work alongside the calorie tracking module to show users how their daily goals work and how to add or remove foods, teaching them step by step rather than expecting them to figure it out alone. What makes this particularly effective is how it only appears the first time a user enters each section, storing this information under their account through the authorization module so they don't have to see the same tutorials repeatedly. This approach helps solve the main problem of making nutrition tracking more beginner-friendly, as users can learn at their own pace without feeling overwhelmed. The tutorials essentially act as a bridge between the user and the app's features, making sure that even complex features like meal creation become approachable and easy to understand.

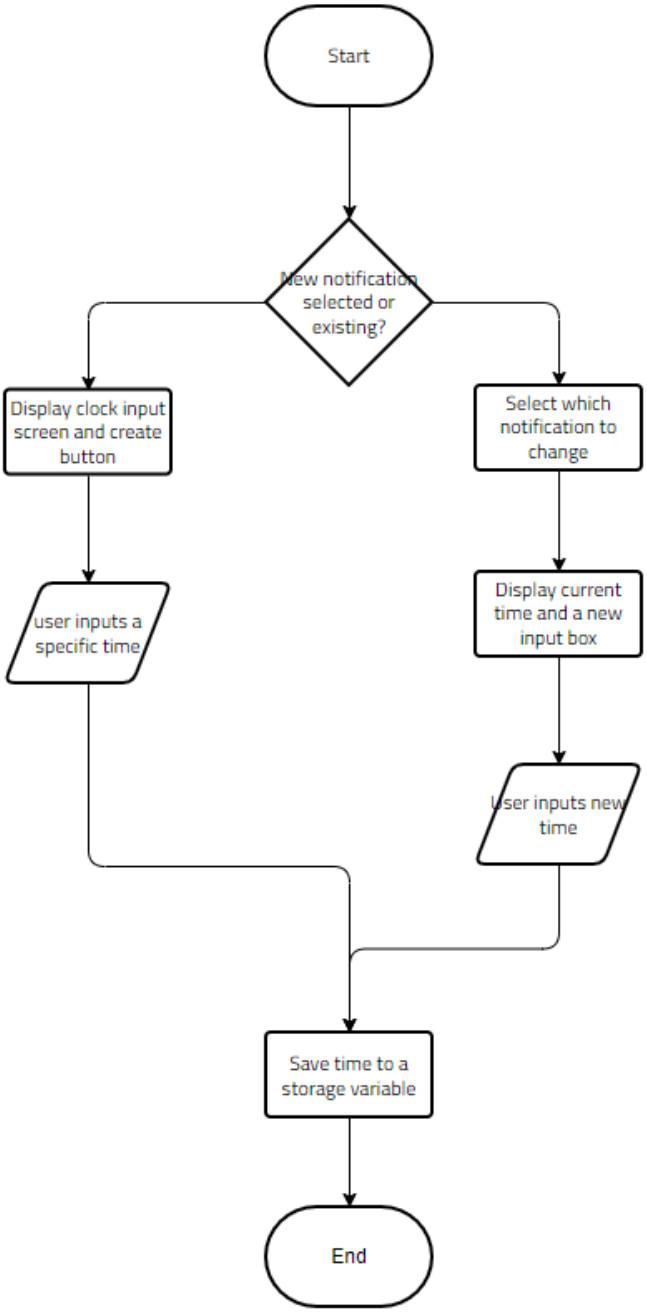
Notifications

Explanation

The notifications module will send notifications at a desired time set up by the user or at a time after 12pm if the user has not entered any food. 12pm is a good time as most people would be awake after that time, so if they had not eaten during that time or forgotten to log a notification reminding them would be useful so that they do not forget later on in the day what they had.

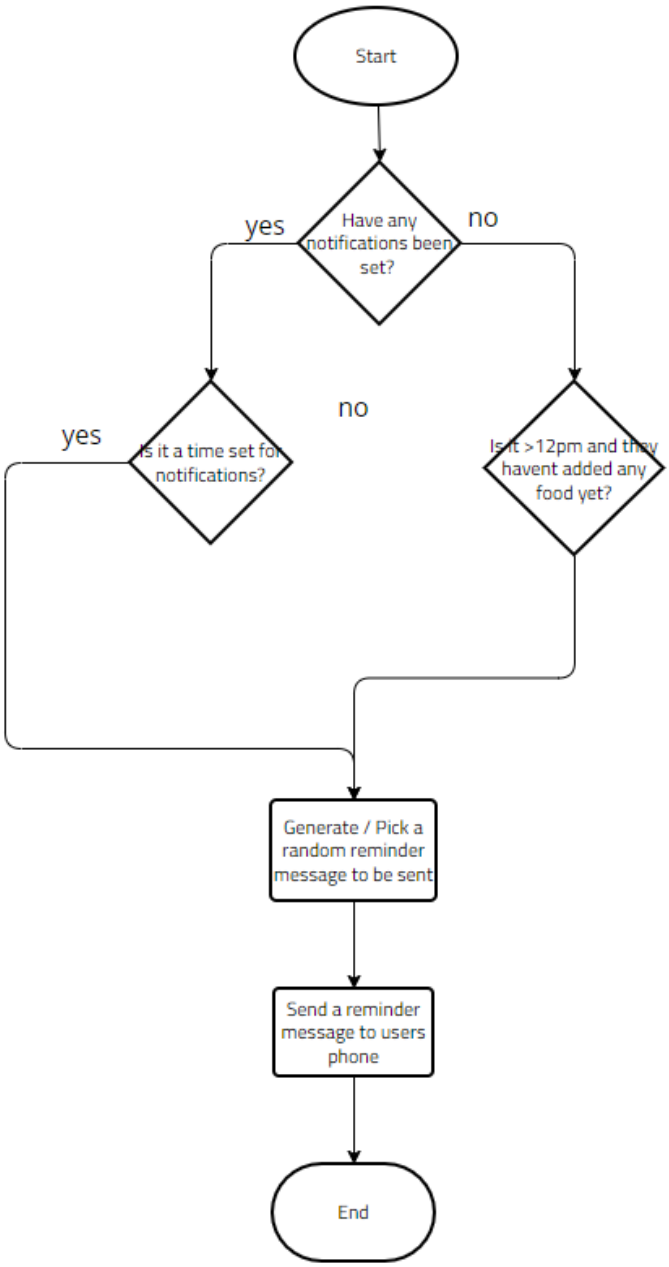
The notification module is essential as a computational solution for several key reasons. Firstly, it directly addresses a core problem faced by many users new to nutrition tracking - forgetting to log their meals or even forgetting to eat at regular intervals. By leveraging the device's ability to schedule and trigger notifications programmatically, the app can proactively remind users at specified times, making it more effective than non-computational alternatives like manual reminders. So the notification feature means that a user will not forget to update their food, and on a larger scale it may lead to them not forgetting to eat, since although it seems simple, it is quite a common fault in those who are underweight.

Flowchart 1



Here is the flowchart that will depict the decisions taken when the user is creating notifications for themselves, they will have the choice to either edit existing notification or create a new one, the new one will just have them enter a specific time they would like and press a create button, whereas the edit button will bring about the time they already have and have an option to change it. This is justified as a user may have a change in their schedule and not need the reminders at those times anymore, increasing convenience. These steps simplify the user journey, making it easy for new users to understand what to do as notifications are self-explanatory, prioritising efficiency to allow the users to quickly set themselves a time and move on.

Flowchart 2



This flowchart depicts roughly the decisions to be made by the program whether to send a notification to the phone depending on the time. First the system checks if any notifications have been set by the user. If they have been, then the system waits for the time when a notification should be sent before sending a randomised message that acts as a reminder. However, if there is no notification set, then the app will automatically send a message after 12pm if there has been nothing added to the app. This approach is justified as it reduces any unnecessary processing, getting straight to the point when the time is right, and further evokes the message that the app does care about its user, even if the user forgets themselves.

```
class Notification()
      private id            // Id for database
      private name         // Name set by user
      private time          // Time set by used
      private isEnabled    // Option to turn on off

      public function show(worldTime)
            if worldTime == time and isEnabled == True then
                  // command that will show the notification
                  if successful then
                        return true
                  else
                        return false
            endif
      end function

      public procedure schedule(userName, userTime)
            id = // function based on name and randomizer
            name = username
            time = userTime
            isEnabled = True
      endfunction

      public function NotificationSwitch()
            if isEnabled == True then
                  isEnabled = False
            else
                  isEnabled = True
      end function
```

This pseudocode depicts how a notification object would be formed by taking in a user input. The class would have four main variables, however there may be more if the time needs to be specific to the minutes or seconds.

The class will consist of three base features, which will start with the procedure schedule, which will take in the user input for the name and time of the notification and set the notification to be enabled, meaning the notification will be sent on the time required, lastly an id will be created based on a function of the name of the notification, so that if the user creates two notifications the id will not be confused. This is a suitable way to create a notification as it eliminates unnecessary processing in the background.

There will also be a function that shows the notification if it is the scheduled time and will return either true or false to the Ui so that the app will know whether the notification was sent successfully or not.

Lastly, a secondary function used to switch on and off the isEnabled Variable when the user does or does not want the notification. This function is justified as not everyone wants constant notifications

every day, but a user may not want to delete them in case they need to quickly turn them back on another day.

General Justification

The notification module works together with all the other modules to create a complete solution that helps users stay on track with their nutrition goals. When combined with the personal details and calorie tracking modules, it acts like a helpful companion that knows when to remind users about tracking their meals. For instance, if someone hasn't logged any food by lunchtime, the notification system works with the calorie tracking module to send them a friendly reminder. This is especially helpful for people who are new to tracking their nutrition, as mentioned in the stakeholder analysis, since they might forget to log their meals regularly. The notifications also connect with the meal creation system, so they can remind users about their usual mealtimes based on the meals they've created. By turning the app into more of a helpful friend than just a tracking tool, the notification system makes the whole app more welcoming and easier to use for beginners. This helps solve the main problem of making nutrition tracking less overwhelming for new users, as they have gentle reminders to keep them going rather than having to remember everything themselves.

GUI

The GUI module will hold all the styles and appearances of the application, so that in future development all styles can be changed in one place instead of having to change it multiple times throughout the code. This means that the theme will stay more consistent throughout and can be changed for events / other occasions.

The app will have an overall green theme towards it, many of the images and background will have either a lime green or darker green aspect towards it. This is because green is normally considered the nature's "healthy" colour so it will encourage the users to keep using it as it will make them feel as if they are being healthier. The image of leaves, greenery and sceneries will be repeated throughout the app to constantly remind the user that they must keep going to be healthy. This also links in with the name of the app, "revive" as to revive something often means to bring it back to life, which is another connotation of the colour green. In this case, bringing back to life will mean bringing a person back from being unhealthy to properly living a healthy lifestyle.

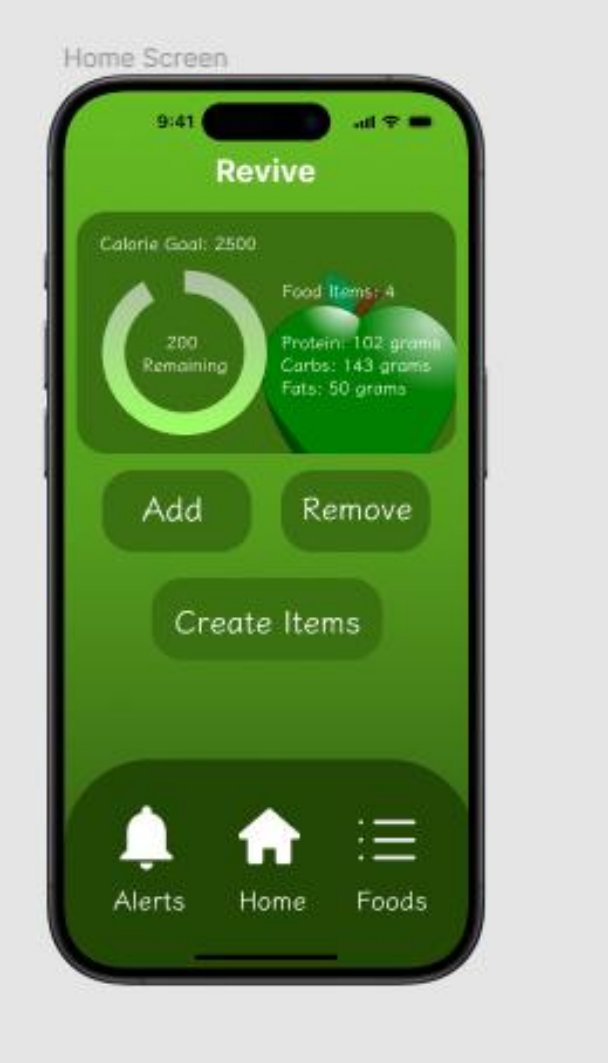
Using green gradients will be over just regular colours, this will make an app UI feel much more professional as compared to just something thrown together quickly. Along with this I would prefer to make the theme more of a nature vibe, instead of just basing it around one colour as this gives me a much more concrete theme meaning that it will feel consistent throughout the app. This is justified as the connotations mentioned earlier will help solve the problem of nutritional apps feeling like an expert game that other competitors have created, and instead help make the app feel like a companion that wants to help the user.

General Designs / Main usability features

These designs will act as a rough guide for the development, in order to help aide the consistency of the application for a more appealing user experience.

Home screen

These are some GUI designs for the application and have been created using Figma, a cloud-based UI design tool.



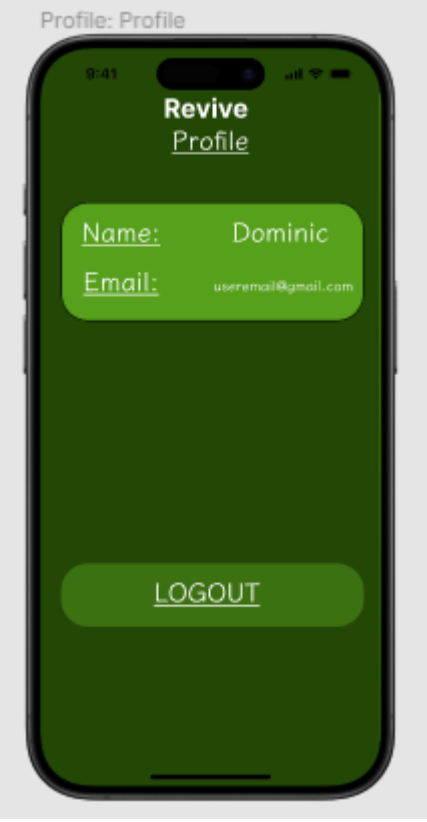
This is my initial idea for my home screen design, I believe the green and white match nicely and the two key buttons are brightly displayed. The bottom of the UI will be able to slide so that the user can access more features that will be added. Creating an initial canvas means that when coding the application, I already have designs and can just focus on how to create it, instead of trying to design and code it at the same time. The buttons will allow users to add or remove and I might have a button straight from the main screen to create an item, however this button may just be moved completely to the foods sections. I will then have the objects that have been added during the day appear under all the buttons and the user will be able to interact with them.

There will be a bottom navigation bar that houses the main features of the app, such as the meal and food creation and the notifications creation as well as the home menu. The bottom navigation will work as the main activity and will always be displayed, but when an option in the menu is pressed, it will lead to a fragment, which will change the screen under the navigation bar to the current option. I will also have a side navigation bar housing the more trivial details such as account information, personal information and to edit them and any other niche features that may be added in the future, this will also work on a fragment based display so that the rest of the app can still function, this means that development in the future is made easy as an option just needs to be added to the side menu bar, and coded as its own separate fragment.

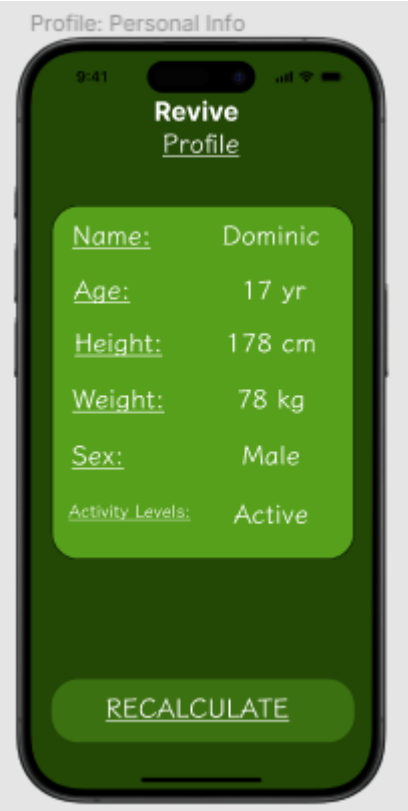
The home screen's bottom navigation design is justified by its adherence to modern mobile design patterns, keeping essential features within thumb reach. The prominent display of tracking data is warranted as it provides immediate feedback to users about their daily progress. The decision to use a side navigation for secondary features is suitable for decluttering the main interface while maintaining easy access to all functionality, justified by established UX research showing users prefer clean, organized interfaces.

Furthermore, the home screen will be the main calorie tracking module, so it will be the first activity that the user will access when they open the app after they have logged in and filled in their details. Meaning that this should be the most frequent screen for the user, therefore it has to appeal and match the apps aesthetic to convey the intended message of the app being a companion.

Profile and personal information viewing screens



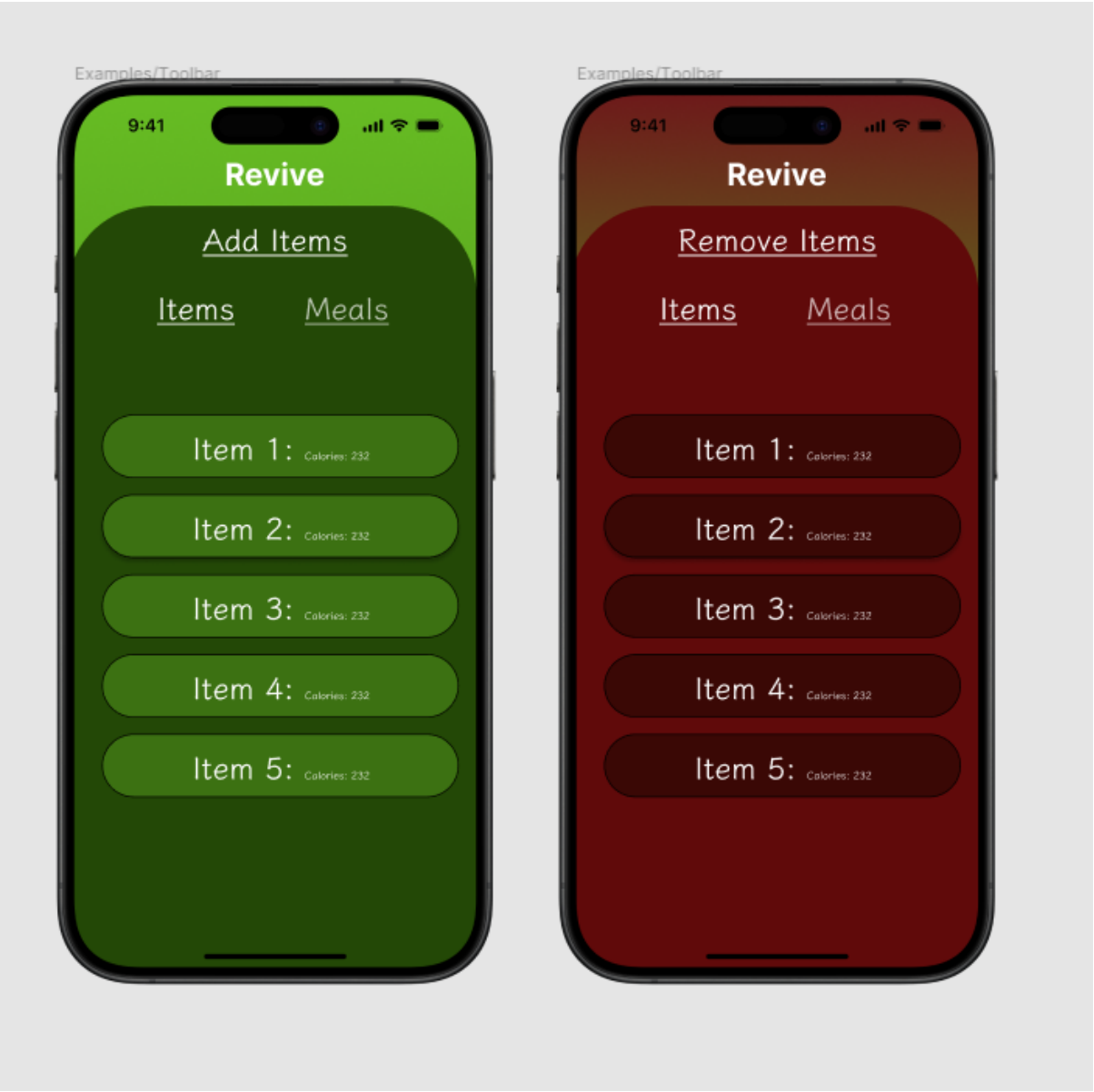
This will be a side screen that the user can enter that displays their information, I will probably only include the user name and email for basic information for the user, however this screen mainly acts as the logout button for the user to swap to a different account in case they want different information or to remove the app. This means that the screen does not need anything complex and is suitable to be done this way since almost all applications have their logout button by the users profile, so this is just adhering to the what the user will be used to.



This is a secondary profile screen which will display the information the user has entered, in case anything changes or was entered incorrectly, the user will be able to select which value they would like to change, and once it has been changed they can press the recalculate button which will perform the calculations from the personal info methods again so that all calories, macros, etc are up to date with the information provided. This means that the user will always have control over all the information that they have entered, and it is suitable to do it this way as the user will be able to see what their information is being used for when they get a confirmation that their calories have been calculated.

The two-screen profile system is justified by information hierarchy principles - separating basic account management from detailed personal metrics. The placement of the logout function in the profile area is warranted by user expectations, as this matches common application patterns. The inclusion of a recalculate button is suitable for providing immediate feedback when users modify their information, ensuring users trust the system's accuracy.

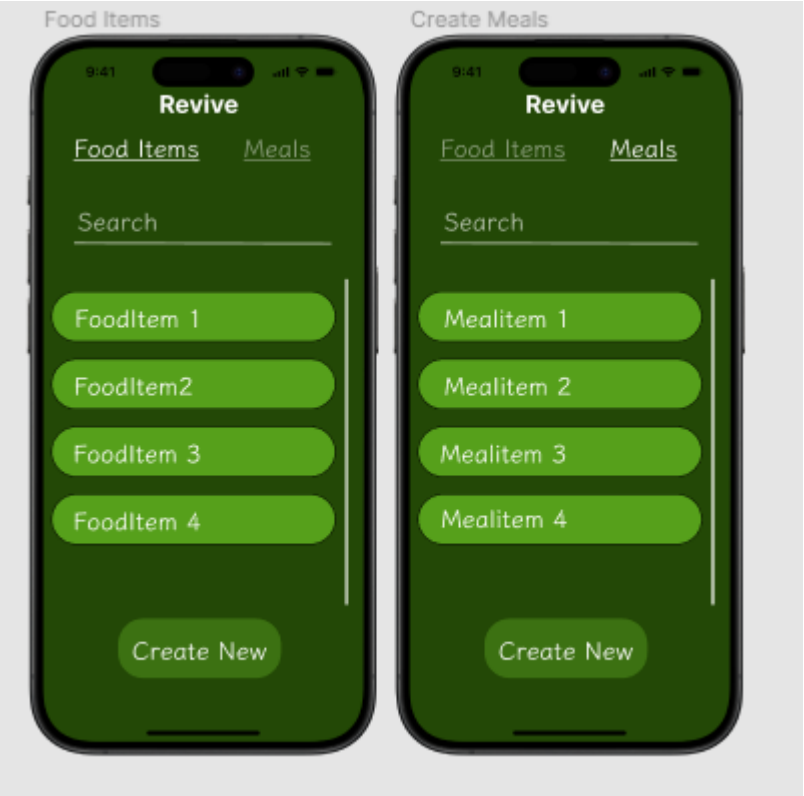
Item addition and removal screen



Green maintains theme however when removing objects the app can swap to a more reddish theme this means that the app will visually show the user that they are removing items so that they do not accidentally go on the wrong option. Since red is most known as the colour of destruction, the user will be able to see what they are doing without actually needing to read anything, which just adds to the appeal of the app. Otherwise the user will be able to select which item they want to add, and then I will add a dialog that asks the user how much of each item they would like to add. A colour change is justified as it further proves the message of simplicity to the user, they will always know exactly what they are doing and never have to remain confused, which again is a common part of other nutritional apps. The contrasting colour scheme between addition (green) and removal (red) is justified by universal colour coding principles, providing immediate visual feedback about the user's current action. The implementation of confirmation dialogs for quantities is warranted for preventing data entry errors. The display of meal components as individual foods while maintaining totals is suitable for balancing detailed tracking with user convenience.

When the user adds a meal to the daily foods all the foods in the meal object will be displayed as individual foods, but the total amount of them all will be added to the progress bars as saved in the meal.

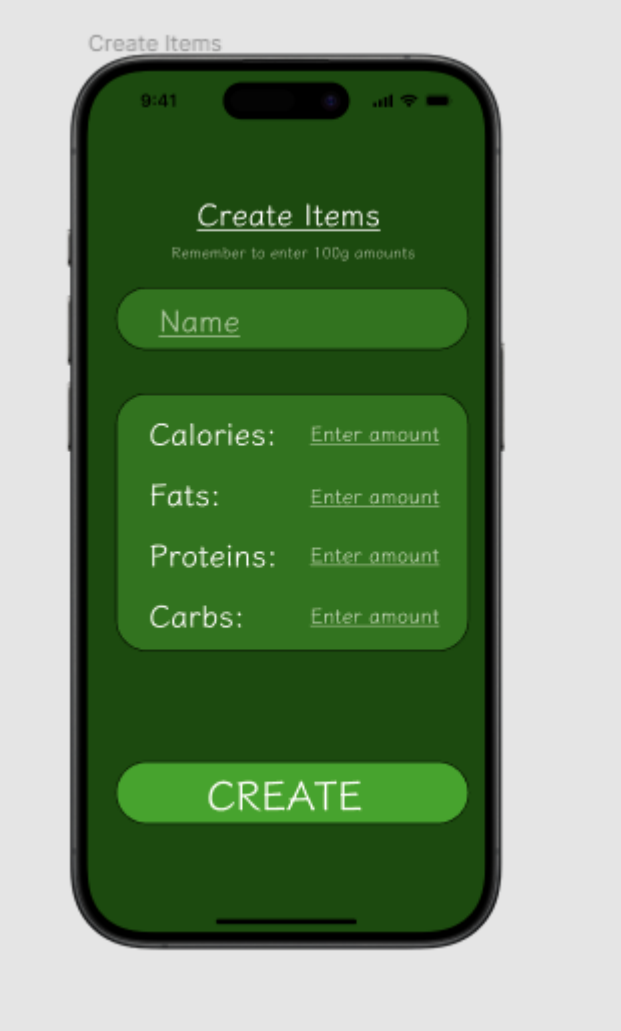
Item and Meal creation



This screen will be the “items” option from the home screen and will display a list of all the individual items a user has created, and on a separate screen the meals a user has created. Each item should be able to be individually clicked on and edited. Also consisting of an option to then create a new item.

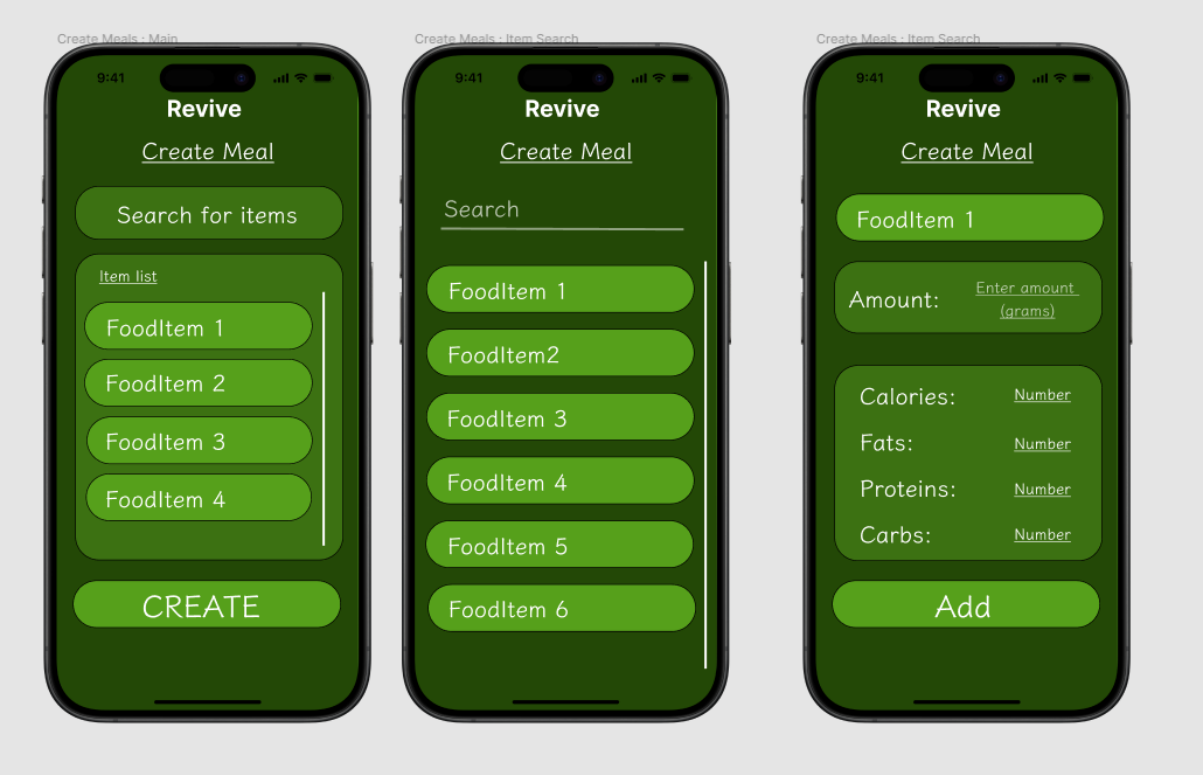
This screen will be formed using a recycler view in android studio, meaning objects can be displayed as a list and viewed individually, as well as been given options on top, so the user will be able to see each object and edit them individually, where the updates will then be stored in the database. There will be an edit button which pops up a dialog screen to edit the information of the fooditem and a delete button in case the user does not want it anymore, meals will then also be displayed using their own separate recycler view.

Using a recycler view for food items is justified by its efficient handling of large datasets. The popup dialog approach for data entry is warranted as it maintains user focus and reduces errors. The three-screen meal creation flow is suitable for breaking down complex tasks into manageable steps, justified by the need to reduce user cognitive load while maintaining data accuracy.



This is the screen that will appear when the user presses the “create new” button on the Food items section above allowing them to enter the name of the object and all the main sub information of the product with a reminder to always enter the 100g amounts they find on the packets. To create an item this way is justified as it makes it as simple as possible for the user as all the information will be readily available on the product they have bought so they just need to copy it into the correct slots. The information would then be sent to the class that creates individual food items and performs the calculations necessary to store in firebase.

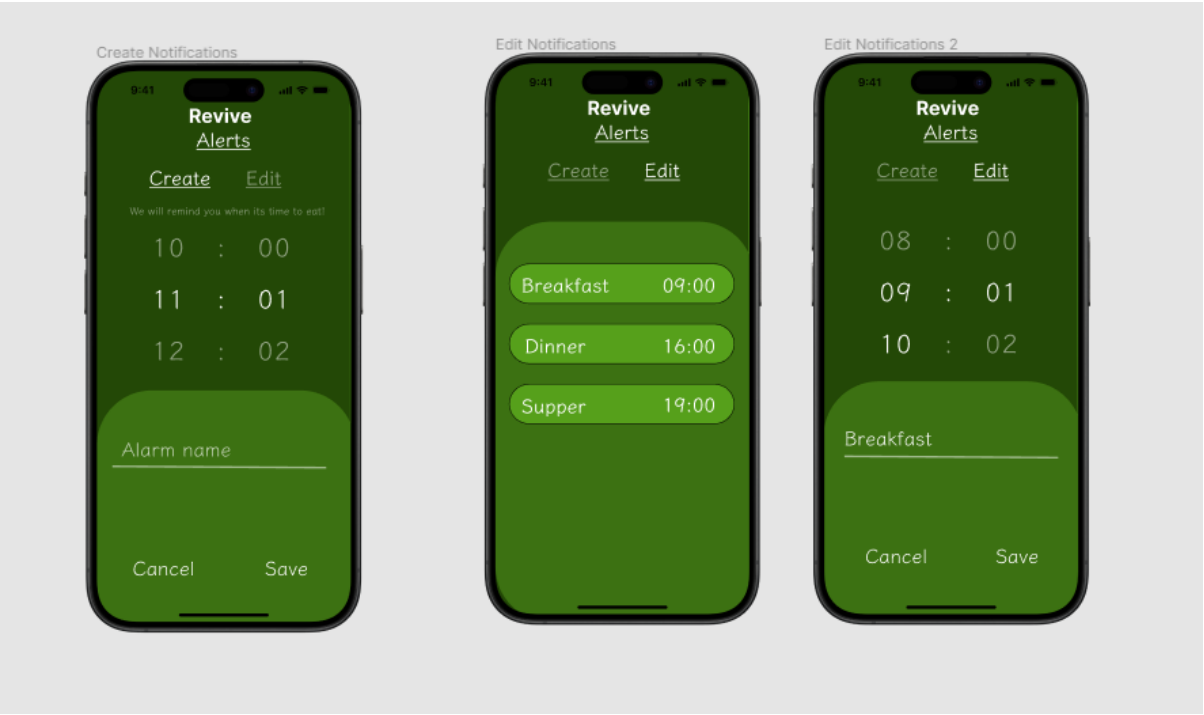
This screen will most likely be a dialog popup or its own activity that will upload data to a separate section in the firebase database under the user id, as it is just a singular screen that does not need to link to anything further once the create button has been pressed because only the data needs to pass.



These three screens depict the creation of a meal when the option is pressed, the leftmost screen will show all the added items of the user to the meal, and as they press ”Search for items” they will be taken to the second screen where they search for the food item they want to add where the objects will be stored in an ordered form and the search used will be a binary search. Lastly the third screen will appear when the user has pressed on the food item they want, to add an amount in grams, since everything is stored in 1-gram amounts as previously stated. After the” add” button is pressed the user will be taken back to the leftmost screen again to and the item they have added will appear under the list. This is a justified approach as it simplifies things for the user and helps keep them engaged with the app, understanding what is happening at each step.

This will most likely be also be presented as a recycler view, where when one item is selected it will lead to a screen that allows the user to input how much of an item they want in grams, this will then be calculated inside the activity to display the amount of calories, etc the meal will be using and displayed over blank text according to the id. Using a recycler view is justified as each object is able to be interacted with, for example if it is added to the meal but then clicked on, it can be removed from the meal before it is created.

Notifications



These three screens would depict the notification creation screen, with two options inside it, to either create a new notification, or edit an existing one. This is a design justified as it is commonly used for alarm creation so it will feel familiar to the user, meaning that they will feel even more comfortable on the app. The edit screen will give the user a list of their current notifications and allow them to edit both the name and the time they wish to receive it.

The alarm-style interface for notifications is justified as it leverages users' existing familiarity with similar systems. The separation of creation and editing functions is warranted for maintaining clear user paths for each task. This approach is suitable for both new and experienced users, making notification management intuitive and efficient.

Test Plan

Firstly, I will conduct alpha testing to ensure that the final solution will function as expected, this will be done throughout the development phase which means that each function created will be individually tested and work locally. I will then carry out integration testing once all modules have been created.

I will start with iterative testing, carrying out white box testing, setting watches on variables and ensuring all calculations are carried out correctly and all the correct data is being used. This is justified as my program will have a lot of calculations that need to be completed and variables that need to stay updated as the user changes data throughout the app. Then I will do black box testing by trying to use the app as a normal user would, to validate that all features are working correctly, such as notifications sending on the correct time. Furthermore, I will test the complete program using beta testing so that the stakeholders may identify any bugs that I may have missed to be patched before the solution is finished. This means that the stakeholders will have a project suited

more to their needs as well as this there will be a higher chance of spotting bugs or glitches that may not be found by testing it by myself making the software more robust.

Key Variables, Data structures and classes

Name	Type	Purpose and justification	Validation
UserID	Variable	User Id will store the string returned from the firebase authentication to know whether the user is logged in or not, this variable is very important as it will allow a logged in user to access all of their data, read and write to it as needed so rules can be set in the database where only users that can send this specific string to the database can change the database.	Whether or not the user has access to their data within the app.
Personal Info	Class	Will store the users google account information such as display name, id and so that the app can feel more personalised, justified as a companion app should feel as friendly and approachable as possible, so displaying things the user is familiar with will be key.	Information appears in map user variable and the user is logged in to the correct account
Name, Age, Weight, Height, Activity Levels	Attributes	Will store all the user's personal information so that it can be fetched from and changed and lead to any other calculations that may be needed. Justified as these are the base variables needed to perform calculations for the user in many different formulas.	Will need to be validated as weight and height can be a float, age would be an integer, name would be fetched from the google account and activity levels will be a drop-down option.
Personal Information	Class	To store all the variables of the user all in one place that can be transferred from activity to activity when needed. Justified as information should be all in one place that can be retrieved from at any time instead of having to store it in random places potentially loosing the information and wasting development time finding them again or validating them.	That the class has its methods set to public and can be stored and fetched to.
Tutorial	Class	To store any code needed for the tutorial as well as variables to make sure the tutorial only runs on a first-time login of the user (until variable is set as tutorial complete) and does not reset. Justified as no one wants to go through the same tutorial multiple times and also better to store things in one place instead of having a million different tutorial options.	Tutorial doesn't appear again after the user logs out / exits the app and all data taken in the tutorial is kept.

FoodItems	Class	Will be the class in which food objects will be created and stored to so that they can be used throughout the app at later times for quick creation. Justified as the user will create a lot of objects for different foods, so many different objects will need to be stored, therefore it is easier to store them using a class.	Make sure each object created is only stored once, each variable inside each object is correct and methods are public and can be stored and fetched from.
Food {name}	Object	Object that is created from the FoodItems class containing variables that will be used in calculations. Justified as storing many different variables all over the place will be inefficient and prove to be a hassle, so storing them in one place is mandatory.	Making sure all attributes in object are correctly stored and in the correct locations
Calories, Proteins, Fats, Carbohydrates	Attributes	Will be stored in each food object created by the user to be added to their daily amount, these will be added or removed from the goals. Justified as they are the 4 key items that must be displayed on all food packaging in the UK, the rest can necessarily be abstracted.	Whether or not other variables are impacted when these variables are called, for example the calorie counter, whether it increases or not.
MealItems	Class	Will be the class in which Food objects can be stored together in one big object for further ease of use. Justified as it creates a much easier experience for the user in their future use.	Make sure each object created is only stored once, each variable inside each object is correct and methods are public and can be stored and fetched from.
Meal {name}	Object	To store all the variables of the compiled food items, the methods and attributes can be inherited from the main FoodItems class but may be changed around in order to allow list creation. This is justified as each object already holds many items, so creating one large object out of all of them would be much more efficient instead of having to go through each individual object.	Making sure all attributes in object are correctly stored and in the correct locations
Calorie_Counter Protein_Counter Fats_Counter Carb_Counter	Variables / Integers	To store the daily amount of each of the nutrients and calorie to allow for a progression throughout the day, these will influence a visual aspect too. Justified as there needs to be a separate way to tally	Objects not being deleted upon reset, counters display correct amount when

		each food item that can safely reset without deleting any objects that the user ahs created.	objects entered and affect their visual counterpart in the correct way.
Notifications	Class or Method (undecided)	To hold the messages and methods of sending notification to the user upon the appropriate times. Justified as messages will either be randomised and need a method, or a lot of different ones will be stored, and a random one will be chosen.	Making sure time is saved according to the local time, and messages are not sent at wrong times / wrong messages sent.
Notification_Times	Objects or Variables	To hold the times created by the user to send them notifications of reminders to track their food. This is justified as sometimes when a person sets themselves a specific time it helps them get into routine, they'll know that they should have had their {meal} at this time, a routine helps with a person's general health.	Making sure the times are saved as a float value according to local clock of user.

Suitable Test Data

Test Number	Test description and Justification	Test Data	Expected Outcome
1	Entering the values to sign up to an account. Entering the values to log in to an account This is a justified test as the authorisation module needs to be the most robust feature so that a user cannot easily get into the account of another user, or create an account that is invalid and will not be able to have data saved in the database under it.	**Attempt to sign up:** Incorrect Email: *"reviveapplicationteam"* Non matching password strings: *"revive"* *"incorrect"* Correct Email: *reviveapplicationteam@gmail.com* Correct matching passwords: *Revive* *Revive* **Attempt to log in:** Incorrect Email: *"reviveapplicationteam"* Non existing Email: *crashsuper6@gmail.com* Existing email: *reviveapplicationteam@gmail.com* Incorrect Password: *incorrect* Correct Password *Revive*	After the sign up screen the user should be taken back to the initial log in screen. Then they should be given a message saying account creation was successful. Their account email should appear in the firebase authentication platform During the login any incorrect values should return their corresponding error message informing the user to change details. On a correct login the user should be given a userid and be allowed to proceed into the app, fetching any existing data if

			they already had an account.
2	Entering values of age, height and weight and making sure all values are saved to and appear in firebase This is justified as this information will need to be used in calculations for the user, if the values entered are incorrect or do not match the type they need to be it could cause the app to crash and become unusable.	Attempt to enter: (age) Integer: 17 Negative integer: -20 Float: 1.5 Character: “A” String: “Hello” Attempt to enter: (height) Integer: 178 Negative integer: -20 Float: 1.5 Character: “A” String: “Hello” Attempt to enter: (weight) Integer: 77 Negative integer: -20 Float: 1.5 Character: “A” String: “Hello”	Since the input should allow only for a number keyboard to come up there should be no option to input a character or string. An integer should be accepted but a float should be converted to an integer in order to not crash. Negative integers should be rejected. All the data should then appear in firebase in the form intended
3	Testing whether or not navigation to different screens works throughout the app. This is a highly justified test as without a method of navigation through the app the user will be stuck to one screen and every other module included will effectively become useless as it is unusable. This also means that the app will not just be one giant screen and can be simplified down to multiple.	n/a	As soon as options to swap windows are pressed a new activity / fragment should be placed on top of the existing one, whilst keeping the navigation menu visible at all times.
4	Entering values for individual Food Items: Name Calories, Proteins, Fats, Carbohydrates This is a justified test as this information will	Integer: 90 Negative integer: -20 Float: 90.6 Character: “A” String: “Apple”	Name -> Should accept a string or character but reject an integer Calories, etc -> Should be no option to enter a character as only number keyboard will

	need to be used in calculations for the user, if the values entered are incorrect or do not match the type they need to be it could cause the app to crash and become unusable.		show up, should reject negative integers and floats, only accept positive integers
5	Entering different foods for meal items	Food items: Apple Chicken Food Name: “Apple Chicken”	Name -> Should accept a string or character but reject an integer Food items -> Should only accept pre-existing food items and if there are none then there should be an error message given to the user, and displayed in a list with the total values of the entire meal available for the user to see.
6	Saving information to account This is a justified test as information for other modules will be used from firebase, so if the app is unable to save to the database correctly many modules may fail to work, and data will not be able to be saved across accounts	Information saved to firebase database	Will be able to open the same account on a different device with all data saved and available
7	Creating objects This is a justified test as food objects will be the base of what makes the calorie tracking usable, since otherwise a database of foods would have to be used.	An object is saved to a FoodItems class and then to database	Only one object is created and has its own value, this object should be able to be fetched from and stored to and updated.
8	Calorie tracking system changes when objects are added or removed, and a progress bar is updated	An object that was previously created is added to the count, only once	Progress bar should increase according to the increase of the counter Counter should display correct total

	This is a highly justified test as the app needs to be able to respond to changes made by the user based on the choices they have made, updating certain aspects dynamically.		amount as well as sub amount of fats, proteins, carbs.
9	Ability to edit personal information and complete calculations based on it. This is a justified test as the user may not always get their information correct the first time, and values such as their activity level or weight can change by the day so the user may want to always edit them. But since the app is targetted towards those newer to the field they may not know how to perform these new calculations so the app needs to be able to take the new values and do it for them.	Changing previously set information in a window of the app: Changing age, height, weight, activity levels Changing object food properties.	Able to accordingly update each section according to what the user wants to do. Display a screen that allows them to change details, and display a popup if they choose to recalculate their calories that informs them of what the calculations have completed.
10	Notifications send according to user input	Time: 10:00 am	Randomised message sent exactly at 10:00 AM from a list of messages to be sent.

Further Test Data

Post development tests will also need to be handled to make sure everything on the app works as intended.

Test description and Justification	**Test Data**	**Expected Outcome**
The user needs to be able to edit information they have entered in case of any mistakes or changes in information on their side, and values	New data entered by user different	Any data inputted by the user gets updated both in the app and firebase, and any changes made to this data also gets updated in an efficient manner so

that have been calculated from this information also need to be updated	from current data	that calculations from this data can still be performed.
When the user uses the search bar the objects they are actually searching for will appear and the other objects will not making it easier for the user to find them. This is justified as if the user creates lots of objects they may become lost trying to find the same items every time and the app will become tedious for them to use, therefore they will be able to quickly find what they need.	Entering values into search bars based on objects created	Anything the search bar is related to, for example a search bar when finding meal objects, will only show the objects that the user is intending to find based on keywords entered by the user in the bar, other objects will be hidden as they are not what the user is searching for.
Any values that are changed or affected should reflect in the UI so that the user is able to view these changes and see the outcome of their decisions, this includes any totals, displays, etc to create an all-round connected user experience.	Any data changes should show this data change.	Any values that are changed should have their corresponding UI value change, and any variables that are a result of calculations should also have their values changed and displayed.
All progress bars update and objects appear where they need to appear. Such as when the user creates a food item, it should show up under all the other food items they have created, and when they add that object to the daily list of what they have consumed the progress bars should all update to show the correct information. This is a highly justified test as if none of the objects show up where they need to the app is effectively useless and nothing else can happen on it as the user cannot actually see what they have created, even if the data is there in the database.	Taking a created object and inputting it into the daily lists.	Any and all progress bars should be updated and any lists containing the items should always show the items that are specific to what the user is doing.
Any data created on one device also shows up and can be retrieved on the same account on a different device. This is justified as the user may lose the device they normally use, etc so data should always be saved.	Any data created in the app.	All data entered on the computer simulator will also show up on a mobile device if the user logs in to the same account.
Tests performed to make sure that the daily variables reset each day so that the user can enter all new objects based on what they have eaten that day. This is justified as otherwise the user will have to remove the items themselves each day or always will be over the goal before the day has even started.	Food Objects: Apple Chicken	The food objects will be erased from the totals in the progress bars and lists in the daily food but not be completely erased as they will remain in the created items sections.

Developing

Important information

At the beginning of modules, there will be a small paragraph explaining how the module will work and what the module is about, this should explain most things about the module that may not be obvious.

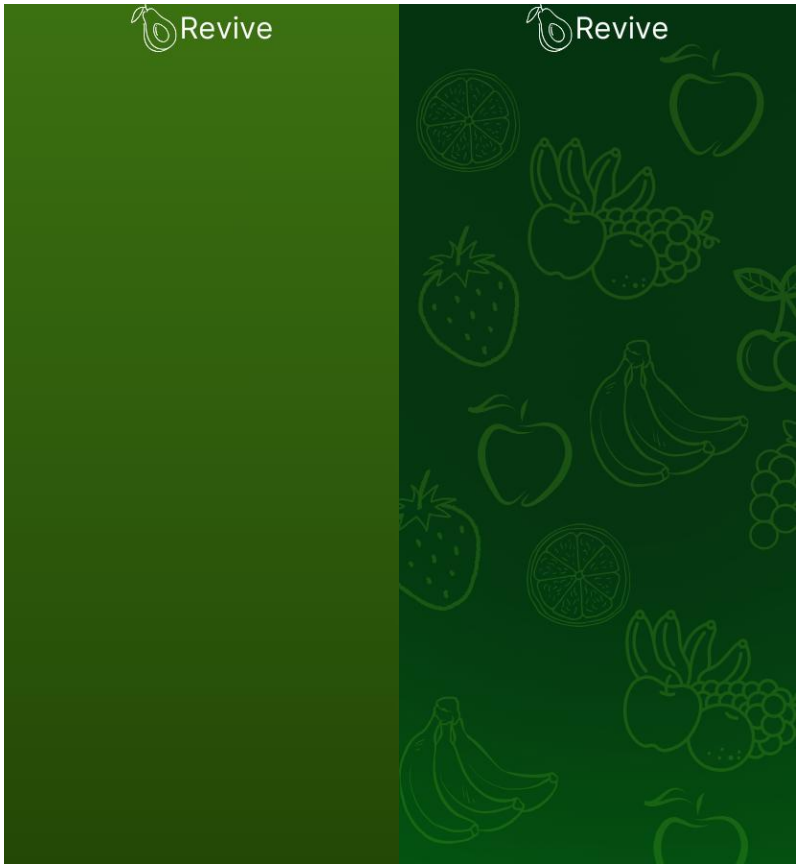
Furthermore, at the beginning of every module, all the XML files, which is the code that is used to stylise the pages in the application, will be presented, these modules do not contain logic code and therefore will not contain information about what the screen will do, this information will be explained in the logic part of the code, further explaining how and what the module is for.

General Items / Layouts

These next few points are not classes or modules however these depict items, layouts or designs that will be referred to throughout the program in order to form the completed solution, since they will be reused throughout, they cannot be grouped into one module, this means that the application should be more efficient as less memory will be used for the same types of item.

Background screens

These are initial screens created in Canva design that will work as a background image for sections of an application instead of having to code every single little piece:



The leftmost screen allows for a simple backdrop that allows anything to be placed on top of it and will be used as the main screen inside the actual application keeping the theme of the app constant, whilst the rightmost screen will be used as welcoming screens and any screens that will be mostly covered up as a “professional” backdrop.

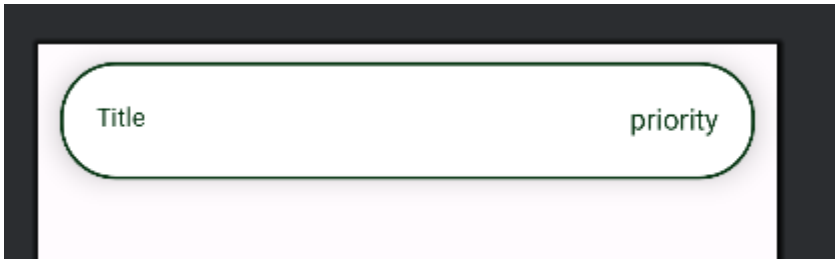
Code for any Recycler Items and List Items that will be used throughout

recycler_item.xml

This is a layout style created in xml that will be used throughout the program to give any lists or item selections a custom style so that the app can remain consistent, also allows for one section of code to be reused instead of having to be coded over and over. A recycler view is used to display this xml, which is a way to list a group of objects that can be interacted with in various ways, such as scrolled through, tapped, long tapped, etc.

recycler_item.xml

```
<androidx.cardview.widget.CardView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/recCard"
    android:layout_marginStart="12dp"
    android:layout_marginEnd="12dp"
    android:layout_marginHorizontal="10dp"
    android:layout_marginVertical="10dp"
    app:cardCornerRadius="31dp"
    app:cardElevation="8dp">

    <RelativeLayout
        android:layout_width="match_parent"
        android:layout_height="65dp"
        android:background="@drawable/green_border">

        <TextView
            android:id="@+id/recTitle"
            android:layout_width="wrap_content"
            android:layout_height="30dp"
            android:layout_marginStart="20dp"
            android:layout_marginTop="20dp"
            android:layout_alignParentStart="true"
            android:layout_toStartOf="@+id/recPriority"
            android:ellipsize="end"
            android:maxLines="1"
            android:text="Title"
            android:textColor="@color/darkGreen"/>

        <TextView
            android:id="@+id/recPriority"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_alignParentEnd="true"
            android:layout_marginEnd="20dp"
            android:layout_marginTop="20dp"
            android:text="priority"
            android:textAlignment="center"
            android:textColor="@color/darkGreen"
            android:textSize="16sp" />
    </RelativeLayout>
```



Where the title is the name of the item in the list and the priority will be the most important value within that object that would be useful to the user. This means that the title is an id that will be replaced by the name of the item when I want to use a list, and the priority text will be replaced by the value that I want displayed, for example if I was displaying a food item, the Title would be replaced by the name of the item, and the priority would be replaced by the calorie amount of the item.

List_item.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<TextView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:padding="8dp"
    android:textSize="16sp"
    android:textColor="@color/darkGreen"
    />
```

Any items where its a dropdown options for when specific values need to be chosen and the user input cannot be any random input.

Colours and Styles

These next styles will be used in the xml files for future designs, so they may not be referred to directly in the logic code, but will be referred to within the xml files to create consistency throughout the app.

```xml
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>
    <color name="darkGreen">#053411</color>
    <color name="grey">#ff424242</color>
    <color name="grey_green">#A8E063</color>
    <color name="light_green">#56AB2F</color>
</resources>
```

These give variables to certain hex colour values so that the values can be used throughout the program and keep a consistent look instead of having to remember each hex code every time it is used, this also means that the apps entire theme can be changed from one colour to another simply by changing the hex values in these variables

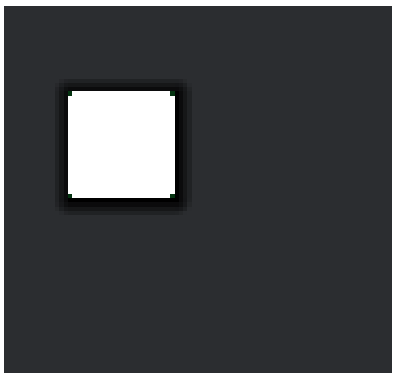
```xml
    <style name="NumberPickerStyle">
        <item name="android:textColorPrimary">@color/white</item>
        <item name="android:colorControlNormal">@color/white</item>
        <item name="android:textSize">22sp</item>
    </style>

</resources>
```

A style to any sliders / pickers that may occur to keep things consistent.

green_border.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<shape
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="rectangle">
    <corners
        android:radius="30dp"/>
    <stroke
        android:color="@color/darkGreen"
        android:width="2dp"/>

    <solid
        android:color="@color/white"/>


</shape>
```

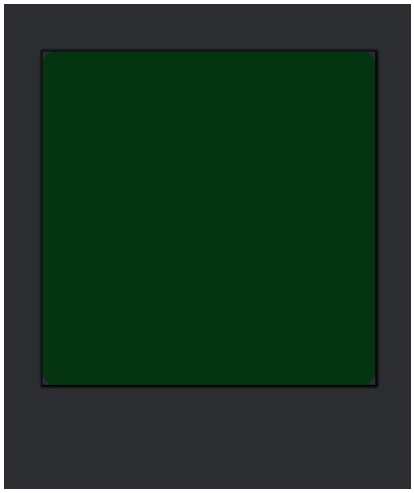
This is a style that simply makes a shape with a white background and green border, this will be used often as a drawable object in xmls so that a consistent radius is used and the same shape, and if there are any changes to colours / looks ever made all items will be updated the same from one place.



darkgreen_round.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android" android:shape="rectangle">
    <solid android:color="@color/darkGreen"/>

    <corners
        android:topLeftRadius="20dp"
        android:topRightRadius="20dp"
        android:bottomLeftRadius="20dp"
        android:bottomRightRadius="20dp"/>

</shape>
```

Makes a shape with a the dark Green colour created earlier and a radius all around to be used throughout.



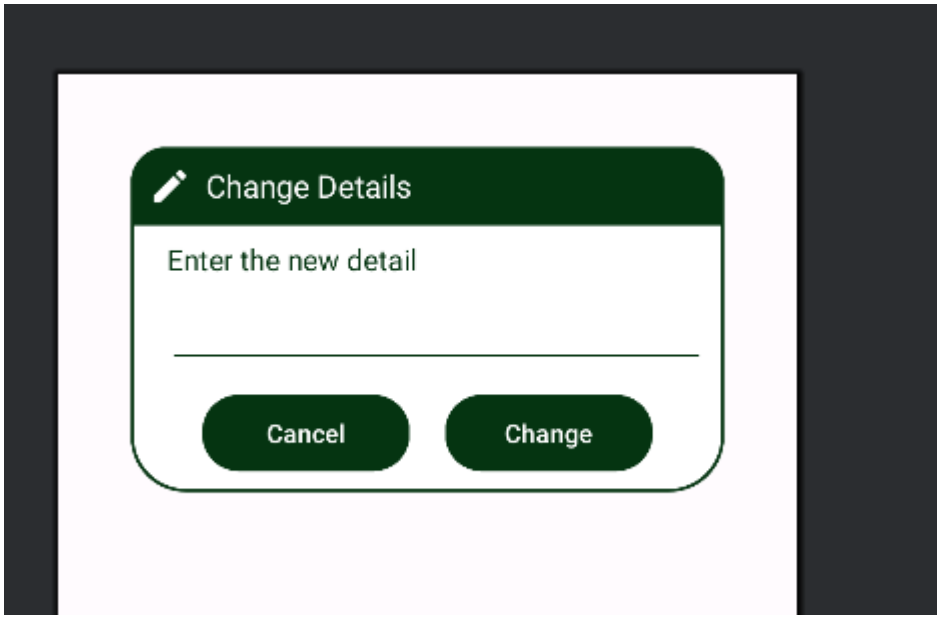
Detail Changing Dialog

dialog_changeinfo.xml

This will be a dialog that will be very generic so that the text can be modified so that it can be used to enter a single value that will update an existing value. Creating this dialog means that one style can be used to change multiple values which adds to the efficiency of the program and saves time in coding new dialogs. This is also a base dialog where new ones can be created copying this code, that can easily be changed around as everything has its own id to fit a new dialog if it is needed, such as changing the edit text box to a dropdown option in case there are specific items needed, or adding different buttons to perform a function. This also means that there will be a consistency of aesthetic throughout the program.

PersonalInfoFragment.kt | dialog_changeinfo.xml | PersonalInfo.kt

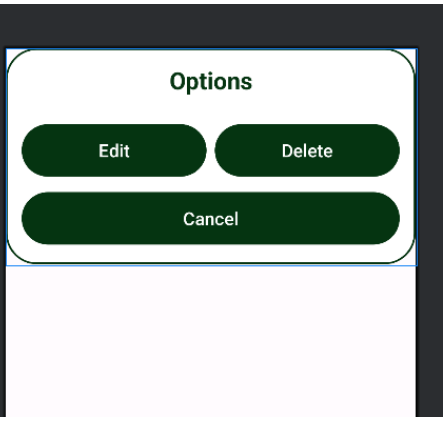
```
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="
    <androidx.constraintlayout.widget.ConstraintLayout
        android:layout_height="wrap_content"
        android:id="@+id/dialogBox"
        app:layout_constraintTop_toTopOf="parent"
        android:background="@drawable/green_border">
        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:id="@+id/ChangeTitle"
            android:background="@drawable/darkgreen_round_flat_bottom"
            android:padding="10dp"
            android:text="Change Details"
            android:textColor="@color/white"
            android:textSize="17sp"
            android:drawableLeft="@drawable/baseline_edit_24"
            android:drawablePadding="8dp"
            app:layout_constraintTop_toTopOf="parent"/>

        <TextView
            android:id="@+id/desc"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="8dp"
            android:layout_marginStart="20dp"
            android:text="Enter the new detail"
            android:textColor="@color/darkGreen"
            android:textSize="16sp"
            app:layout_constraintBottom_toTopOf="@id/editBoxDetails"
            app:layout_constraintTop_toBottomOf="@id/ChangeTitle"
            app:layout_constraintVertical_bias="0.0"
            tools:layout_editor_absoluteX="-8dp" />

        <EditText
            android:layout_width="300dp"
            android:layout_height="wrap_content"
            android:id="@+id/editBoxDetails"
            android:textColor="@color/darkGreen"
            android:textSize="16sp"
            android:layout_marginStart="20sp"
            android:backgroundTint="@color/darkGreen"
            android:layout_marginBottom="30dp"
            android:maxLines="1"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintTop_toBottomOf="@id/ChangeTitle"
            android:inputType="number"/>
        <Button
            android:layout_width="0dp"
            android:layout_height="50dp"
            android:id="@+id/btnCancelDetails"
            android:layout_marginStart="40dp"
            android:layout_marginEnd="10dp"
            android:textColor="@color/white"
            android:textSize="14sp"
            android:text="Cancel"
            android:layout_marginBottom="8dp"
            app:cornerRadius = "20dp"
            android:layout_marginTop="10dp"
            app:layout_constraintTop_toBottomOf="@id/editBoxDetails"
            app:layout_constraintBottom_toBottomOf="@id/dialogBox"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintEnd_toStartOf="@id/btnChangeDetail"
            android:backgroundTint="@color/darkGreen"/>
        <Button
            android:layout_width="0dp"
            android:layout_height="50dp"
            android:id="@+id/btnChangeDetail"
            android:layout_marginStart="10dp"
            android:layout_marginEnd="40dp"
            android:textColor="@color/white"
            android:textSize="14sp"
            android:layout_marginBottom="8dp"
            android:layout_marginTop="10dp"
            android:text="Change"
            app:cornerRadius = "20dp"
            app:layout_constraintBottom_toBottomOf="@id/dialogBox"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toEndOf="@id/btnCancelDetails"
            android:backgroundTint="@color/darkGreen"/>
    </androidx.constraintlayout.widget.ConstraintLayout>
</androidx.constraintlayout.widget.ConstraintLayout>
```



Dialog to display options

dialog_edit_options.xml

```xml
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="16dp"
    android:background="@drawable/green_border">

    <TextView
        android:id="@+id/dialogTitle"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Options"
        android:textSize="24sp"
        android:textStyle="bold"
        android:textColor="@color/darkGreen"
        android:gravity="center"
        android:layout_marginBottom="24dp"
        />

    <!-- Horizontal LinearLayout for Edit and Delete buttons -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:layout_marginBottom="8dp">

        <Button
            android:id="@+id/editButton"
            android:layout_width="0dp"
            android:layout_height="60dp"
            android:layout_weight="1"
            android:text="Edit"
            android:textSize="18sp"
            android:backgroundTint="@color/darkGreen"
            android:textColor="@color/white"
            android:layout_marginEnd="4dp"/>
```




```xml
        <Button
            android:id="@+id/deleteButton"
            android:layout_width="0dp"
            android:layout_height="60dp"
            android:layout_weight="1"
            android:text="Delete"
            android:textSize="18sp"
            android:backgroundTint="@color/darkGreen"
            android:textColor="@color/white"
            android:layout_marginStart="4dp"/>
    </LinearLayout>

    <Button
        android:id="@+id/cancelButton"
        android:layout_width="match_parent"
        android:layout_height="60dp"
        android:text="Cancel"
        android:textSize="18sp"
        android:backgroundTint="@color/darkGreen"
        android:textColor="@color/white"/>

</LinearLayout>
```

This XML layout defines a vertical LinearLayout that contains a title and several buttons, all styled with a green theme. The layout starts with a TextView displaying the title "Options" in bold, dark green text, centered at the top. Below the title is a horizontal LinearLayout that holds two buttons, "Edit" and "Delete," which are equally spaced and styled with a green background and white text. Finally, a "Cancel" button is placed below the buttons with the same styling. The entire layout has padding and a green border background.

Authorisation Module

EXPLANATION: I have decided to first develop the login feature before the sign up feature as I would like to have the first screen that is launched when the application is ran to be the login screen, since there may be returning users who do not need to sign up and log straight in, this is the way many other professional applications work where there is then an option to swap to a sign up screen and create an account, however since both the sign up screen and the login screen are both needed for either one to work in this case, I will run tests for the authorization module as a whole, rather than individually for the sign up and login module.

Login

XMLS

Initial screen with added text in canva to be put into the background of the XML when creating the login_layout and the signup_layout.

Login XML

activity_login.xml AndroidManifest.xml

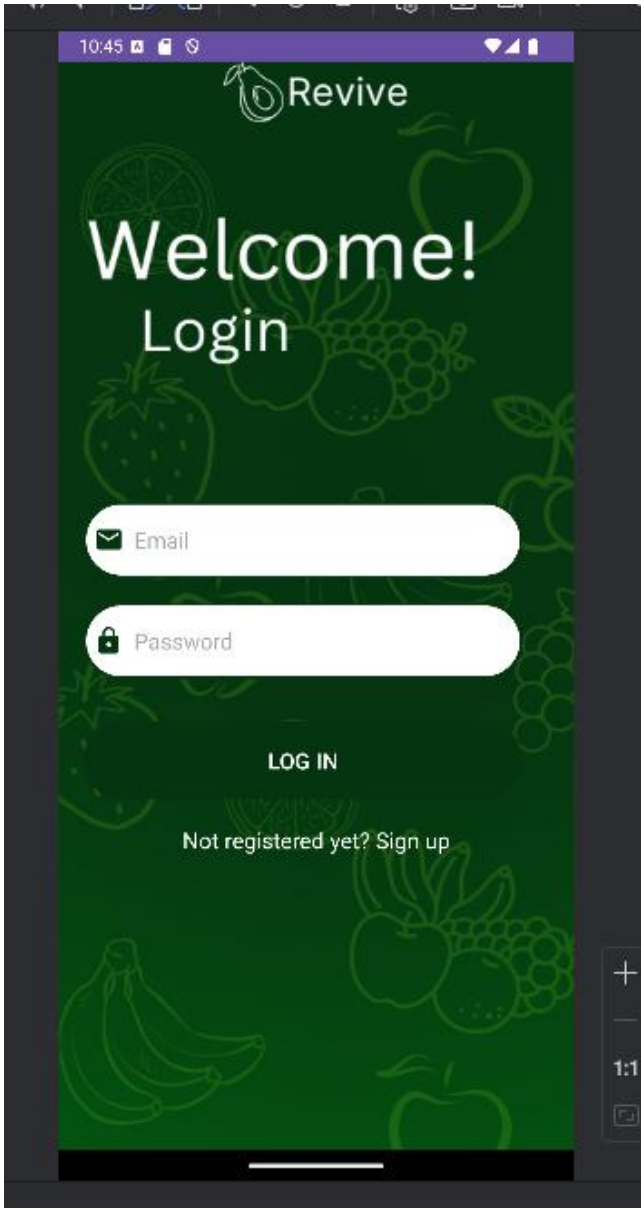
```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
    <uses-permission android:name="android.permission.INTERNET"/>

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="Revive"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.ReviveApp"
        tools:targetApi="31">
        <activity
            android:name=".PersonalDetailsActivity"
            android:exported="false" />
        <activity
            android:name=".LoginActivity"
            android:exported="true"
            android:theme="@style/Theme.App.SplashScreen">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <activity
            android:name=".SignupActivity"
            android:exported="false" />
        <activity
            android:name=".MainActivity"
            android:exported="false"
            android:theme="@style/Base.Theme.ReviveApp"></activity>
    </application>

</manifest>
```

This is the application manifest, this is just a small iteration to the creation of the application to make sure that the loginactivity is the activity launched upon the start up of the app, meaning authentication cannot be bypassed and the user has to log in.

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@drawable/loginscreen"
    tools:context=".LoginActivity">

    <ScrollView
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">
        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:layout_marginTop="300dp"
            android:padding="20dp">

            <EditText
                android:layout_width="350dp"
                android:layout_height="60dp"
                android:id="@+id/login_email"
                android:layout_marginTop="30dp"
                android:background="@drawable/green_border"
                android:padding="8dp"
                android:hint="Email"
                android:drawableLeft="@drawable/baseline_email_24"
                android:drawablePadding="8dp"
                android:textColor="@color/grey"
                />

            <EditText
                android:layout_width="350dp"
                android:layout_height="60dp"
                android:id="@+id/login_password"
                android:layout_marginTop="20dp"
                android:background="@drawable/green_border"
                android:padding="8dp"
                android:hint="Password"
                android:inputType="textPassword"
                android:drawableLeft="@drawable/baseline_lock_24"
                android:drawablePadding="8dp"
                android:textColor="@color/grey"
                />


            <Button
                android:layout_width="350dp"
                android:layout_height="70dp"
                android:layout_marginTop="30dp"
                android:text="LOG IN"
                android:id="@+id/login_button"
                android:textSize="18dp"
                android:backgroundTint="@color/darkGreen"/>


            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:id="@+id/forgot_password"
                android:layout_gravity="center"
                android:layout_marginTop="6dp"
                android:padding="8dp"
                android:text="Forgot Password?"
                android:textColor="@color/white"
                android:textSize="16sp"/>

            <TextView
                android:layout_width="wrap_content"
```

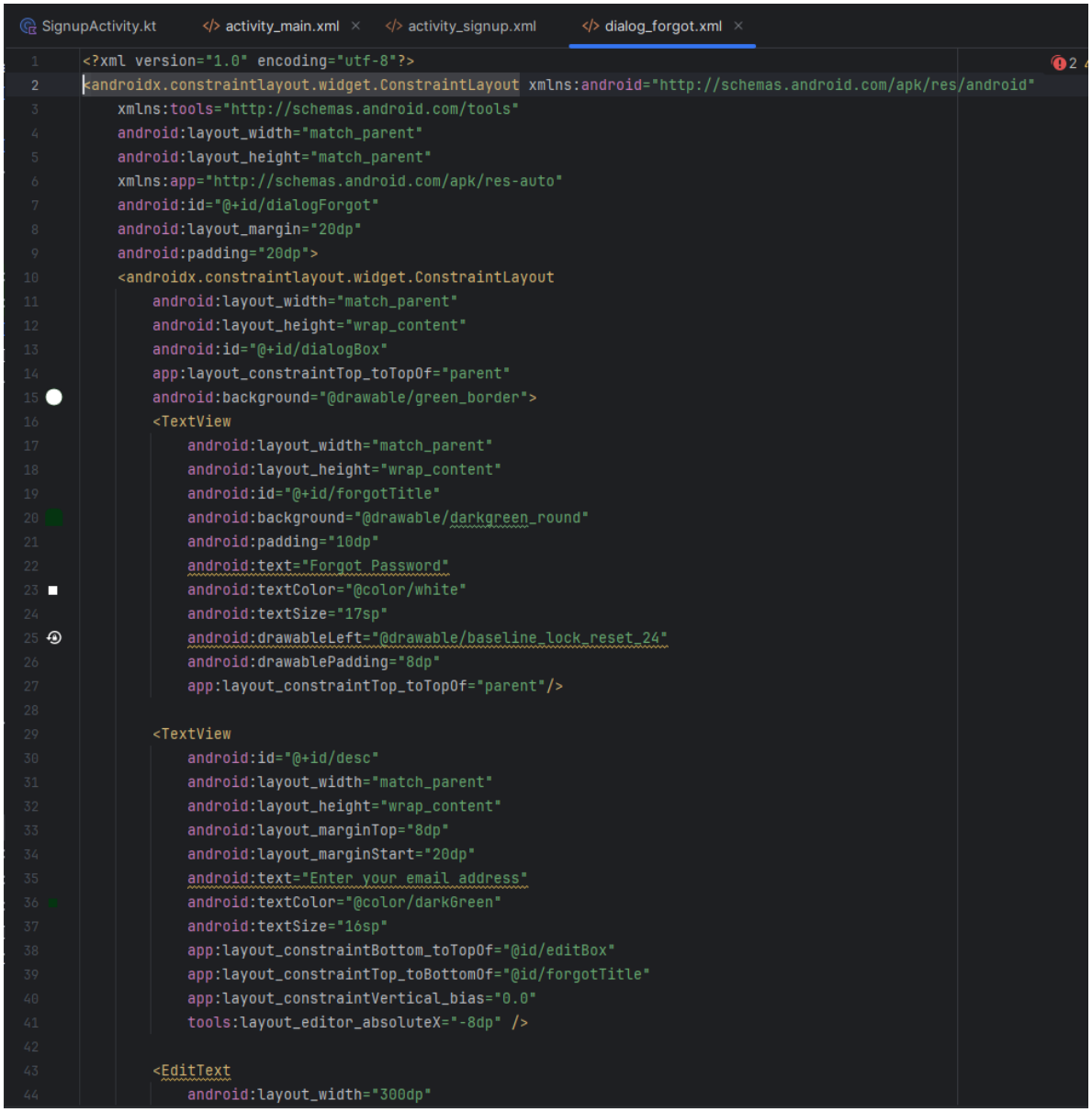
```
                android:textSize="18dp"
                android:textColor="@color/white"/>

        </LinearLayout>


    </LinearLayout>
    </ScrollView>

</LinearLayout>
```

This creates a screen on top of the background. The email entry does not have a specified input type, therefore the user can enter any character which may be part of their email, with an icon set to the left hand side. The password edit box has a specified password input type, meaning that the full keyboard is once again available, but when the user enters the password in the text box it will get hidden for them in case anyone else is looking.



Forgot password dialog XML

dialog_forgot.xml

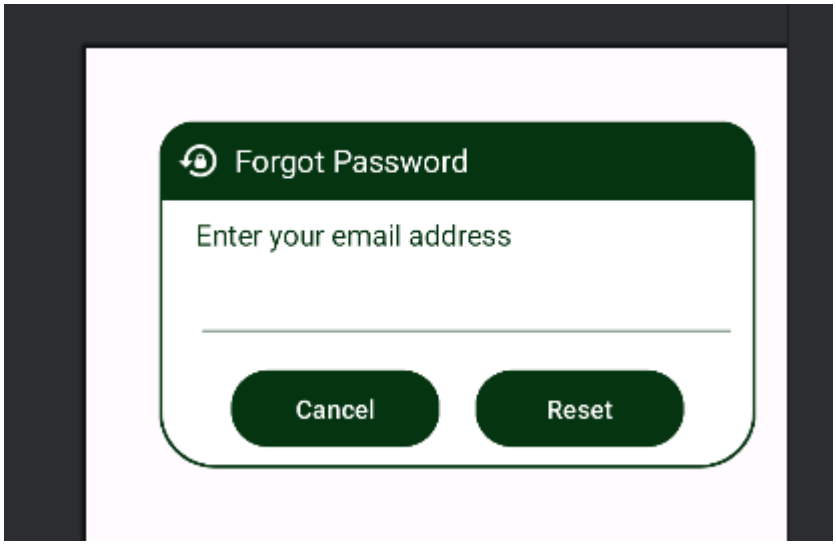


```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/dialogForgot"
    android:layout_margin="20dp"
    android:padding="20dp">
    <androidx.constraintlayout.widget.ConstraintLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:id="@+id/dialogBox"
        app:layout_constraintTop_toTopOf="parent"
        android:background="@drawable/green_border">
        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:id="@+id/forgotTitle"
            android:background="@drawable/darkgreen_round"
            android:padding="10dp"
            android:text="Forgot Password"
            android:textColor="@color/white"
            android:textSize="17sp"
            android:drawableLeft="@drawable/baseline_lock_reset_24"
            android:drawablePadding="8dp"
            app:layout_constraintTop_toTopOf="parent"/>

        <TextView
            android:id="@+id/desc"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="8dp"
            android:layout_marginStart="20dp"
            android:text="Enter your email address"
            android:textColor="@color/darkGreen"
            android:textSize="16sp"
            app:layout_constraintBottom_toTopOf="@id/editBox"
            app:layout_constraintTop_toBottomOf="@id/forgotTitle"
            app:layout_constraintVertical_bias="0.0"
            tools:layout_editor_absoluteX="-8dp" />

        <EditText
            android:layout_width="300dp"
```

```
            android:layout_height="wrap_content"
            android:id="@+id/editBox"
            android:textColor="@color/darkGreen"
            android:textSize="16sp"
            android:layout_marginStart="20sp"
            android:backgroundTint="@color/darkGreen"
            android:layout_marginBottom="30dp"
            android:maxLines="1"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintTop_toBottomOf="@id/forgotTitle"/>
        <Button
            android:layout_width="0dp"
            android:layout_height="50dp"
            android:id="@+id/btnCancel"
            android:layout_marginStart="40dp"
            android:layout_marginEnd="10dp"
            android:textColor="@color/white"
            android:textSize="14sp"
            android:text="Cancel"
            android:layout_marginBottom="8dp"
            app:cornerRadius = "20dp"
            android:layout_marginTop="10dp"
            app:layout_constraintTop_toBottomOf="@id/editBox"
            app:layout_constraintBottom_toBottomOf="@id/dialogBox"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintEnd_toStartOf="@id/btnReset"
            android:backgroundTint="@color/darkGreen"/>
        <Button
            android:layout_width="0dp"
            android:layout_height="50dp"
            android:id="@+id/btnReset"
            android:layout_marginStart="10dp"
            android:layout_marginEnd="40dp"
            android:textColor="@color/white"
            android:textSize="14sp"
            android:layout_marginBottom="8dp"
            android:layout_marginTop="10dp"
            android:text="Reset"
            app:cornerRadius = "20dp"
```

app › src › main › res › layout › dialog_forgot.xml

```
            app:layout_constraintBottom_toBottomOf="@id/dialogBox"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toEndOf="@id/btnCancel"
            android:backgroundTint="@color/darkGreen"/>
    </androidx.constraintlayout.widget.ConstraintLayout>
</androidx.constraintlayout.widget.ConstraintLayout>
```



This dialog will work on the login screen, which will be a pop up when called in the code. This is a window that makes use of an input box that will allow the user to enter an email, and two buttons with the green_border.xml style which will act as a confirmation or cancel button.

Logic

Main login

```
class LoginActivity : AppCompatActivity() {
    private lateinit var binding: ActivityLoginBinding
    private lateinit var firebaseAuth: FirebaseAuth

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityLoginBinding.inflate(layoutInflater)
        setContentView(binding.root)

        firebaseAuth = FirebaseAuth.getInstance()  // sets the application for the firebase importer procedure
```

To start off with I created a binding that allows the Kotlin code to link to the xml file created earlier, so that the code can look into the xml and any user inputs, clicks, etc can be seen by the code and reactions to those actions will be able to be handled accordingly. I also created a firebase auth variable to hold a user instance.

```
        binding.loginButton.setOnClickListener {  // waits for the login button to be clicked
            val email = binding.loginEmail.text.toString()
            val password = binding.loginPassword.text.toString()

            if (email.isNotEmpty() && password.isNotEmpty()) {  // makes sure the two email and password fields are not left empty
                firebaseAuth.signInWithEmailAndPassword(email, password).addOnCompleteListener {
                    if (it.isSuccessful) {  // opens to the next screen if password and email are correct
                        val intent = Intent(this, MainActivity::class.java)
                        startActivity(intent)
                        finish()
                    } else {
                        Toast.makeText(this, "Authentication Failed", Toast.LENGTH_SHORT).show() // returns an error if wrong
                    }
                }
            } else {
                Toast.makeText(this, "Fields cannot be empty", Toast.LENGTH_SHORT).show() // returns an error if fields are empty
            }
        }
    }
}
```

This binding works by setting the code to an onclicklistener where the code is waiting for the user to press the login button, where as soon as the button is pressed, The code logs in a user by checking if the email and password fields are filled, then using Firebase Authentication's signInWithEmailAndPassword method to verify the credentials. If successful, it retrieves the user's unique ID and checks Firebase Realtime Database for existing data. If data is found, the user is redirected to MainActivity; otherwise, they are directed to PersonalDetailsActivity to complete their profile. Errors like incorrect credentials or empty fields are handled with Toast messages for feedback. Testing Codes 1.6, 1.7, 1.8, 1.13

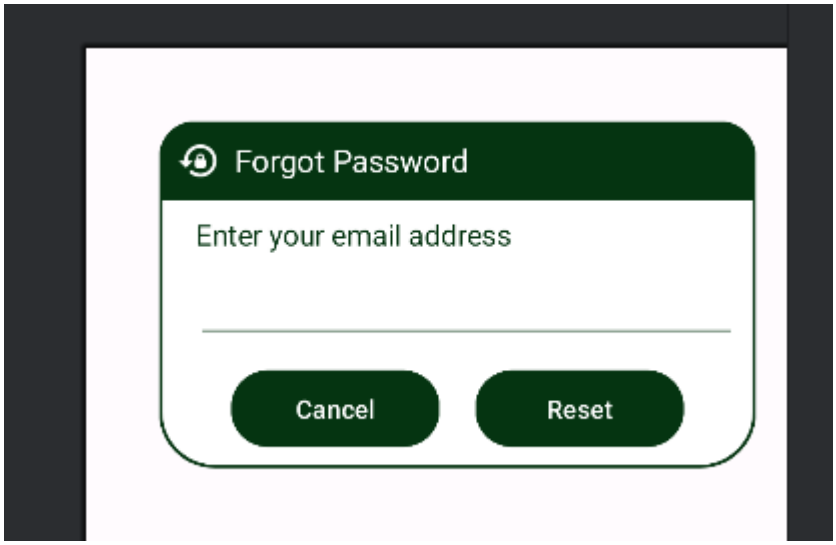
Using binding over the traditional view by id is justified as it reduces boilerplate code (code with repetitive segments) , prevents null pointers from invalid IDs and makes sure no views are missing. Furthermore it does not need to traverse the view hierarchy during the runtime to locate views, binding directly links the two, making the program more efficient. For this reason I will use binding throughout all the apps activities and fragments.

```
binding.signupRedirectText.setOnClickListener{  // listens for someone to click the "sign up now" text
    val signupIntent = Intent(this, SignupActivity::class.java)
    startActivity(signupIntent)
}
```

Then a second binding listens to when the user pressed the ”sign up now” text mentioned earlier, which will take the user to the sign up activity so that they can create an account. This means that the text has now effectively turned into a button, but keeps a cleaner look which will be more pleasing for the user. Testing Code 1.2

Forgot password dialog

Additionally, I have added a forgot password textview, which works in the same way the other textview works, however this time instead of leading to a new activity a custom dialog is style to appear as a separate popup with an input for an email, which is validated before sending a password reset request via firebase.

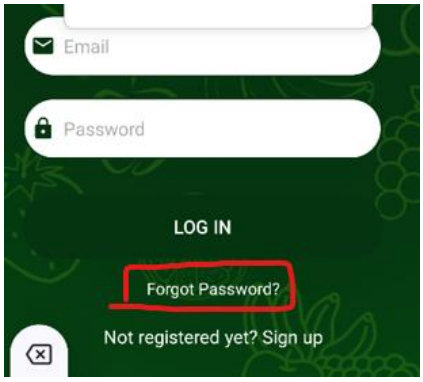


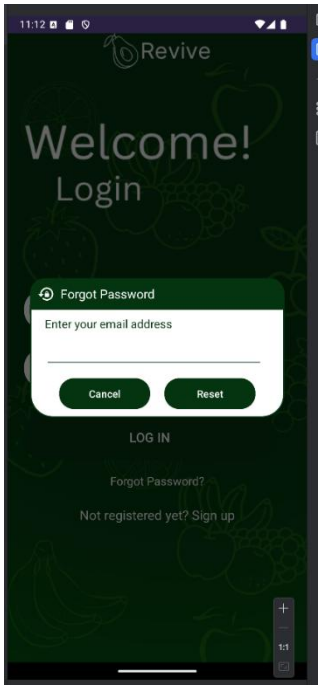

```
private fun compareEmail(email: EditText){
        if (email.text.toString().isEmpty()){
            Toast.makeText(this, "Please enter your email", Toast.LENGTH_SHORT).show()
            return
        }
        if (!Patterns.EMAIL_ADDRESS.matcher(email.text.toString()).matches()){
            Toast.makeText(this, "Please enter a valid email", Toast.LENGTH_SHORT).show()
            return
        }
        firebaseAuth.sendPasswordResetEmail(email.text.toString())
            .addOnCompleteListener { task ->
                if (task.isSuccessful) {
                    Toast.makeText(this, "Check your email", Toast.LENGTH_SHORT).show()
                } else {
                    Toast.makeText(this, "Email does not exist", Toast.LENGTH_SHORT).show()
                }
            }
    }
}
```

I then created a function that would take an email that will be entered by the user in the future and compares it with the firebase database. But first the input box by the user is checked to see if the box is empty, or if a valid email has been entered. There is then further feedback for the user, if they have entered an email that does not exist in the database, they will be given a message and an email will not be sent, however if they have entered a valid email, they will be told to check their email for a firebase link.

```
binding.forgotPassword.setOnClickListener{  // listens to see if forgot password text is clicked
    val builder = AlertDialog.Builder(this)
    val view = layoutInflater.inflate(R.layout.dialog_forgot, null) // calls dialog_forgot xml
    val userEmail = view.findViewById<EditText>(R.id.editBox)
    builder.setView(view)
    val dialog = builder.create()
    view.findViewById<Button>(R.id.btnReset).setOnClickListener {
        compareEmail(userEmail) // makes sure email is real
        dialog.dismiss()
    }
    view.findViewById<Button>(R.id.btnCancel).setOnClickListener {
        dialog.dismiss() // cancels operation
    }
    if (dialog.window != null){
        dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
    }
    dialog.show()
}
```

This is the code that created a dialog that inflates the custom layout created earlier , called dialog_forgot, which contains an EditText for entering the email and two buttons: "Reset" and "Cancel." When the "Reset" button is clicked, it calls the compareEmail function to validate the email and initiate the password reset process via Firebase. The dialog is then dismissed. If the "Cancel" button is clicked, the dialog is simply dismissed without any action. The dialog's background is styled to be transparent for a cleaner look. Testing code 1.10





Sign up

EXPLANATION: This module will be accessed through the login module as a secondary screen and a new activity and will allow the user to create an account using their email and a password, which they will need to confirm by typing it in twice to make sure they remember it. This is a standard procedure for login systems and is used universally in professional applications, I am abstracting the need to create a username as this would require multiple layers of validation to make sure the emails are unique, as well as usernames are unique.

XMLS

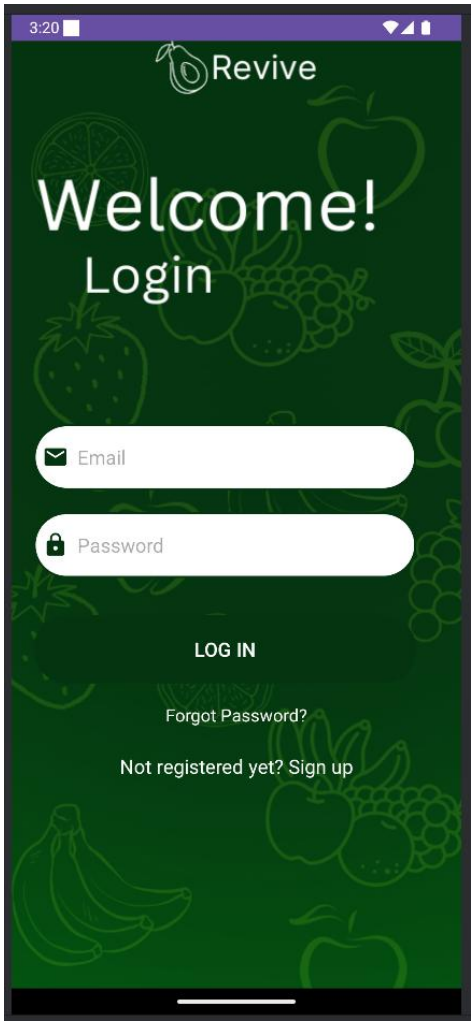
After the textview is clicked, a new intent is started creating a new activity redirecting users to the SignupActivity. This activity provides a few input boxes for a user to create an account which is saved to the firebase authorisation. The sign up module is placed as a linking screen from the login module as the login screen is the default page that the app opens to, assuming the user already has an account for convenience for already existing users, most professional sign up screens are done this way

SignupActivity.kt activity_main.xml activity_signup.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<ScrollView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@drawable/signupscreen"
    tools:context=".SignupActivity">
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">
        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:layout_marginTop="250dp"
            android:padding="20dp">

            <EditText
                android:layout_width="350dp"
                android:layout_height="60dp"
                android:id="@+id/signup_email"
                android:layout_marginTop="30dp"
                android:background="@drawable/green_border"
                android:padding="8dp"
                android:hint="Email"
                android:drawableLeft="@drawable/baseline_email_24"
                android:drawablePadding="8dp"
                android:textColor="@color/grey"
                />

            <EditText
                android:layout_width="350dp"
                android:layout_height="60dp"
                android:id="@+id/signup_password"
                android:layout_marginTop="20dp"
                android:background="@drawable/green_border"
                android:padding="8dp"
                android:hint="Password"
                android:inputType="textPassword"
                android:drawableLeft="@drawable/baseline_lock_24"
```

```
            android:drawablePadding="8dp"
            android:textColor="@color/grey"
            />

        <EditText
            android:layout_width="350dp"
            android:layout_height="60dp"
            android:id="@+id/signup_confirm"
            android:layout_marginTop="20dp"
            android:background="@drawable/green_border"
            android:padding="8dp"
            android:hint="Password"
            android:inputType="textPassword"
            android:drawableLeft="@drawable/baseline_password_24"
            android:drawablePadding="8dp"
            android:textColor="@color/grey"
            />

        <Button
            android:layout_width="350dp"
            android:layout_height="70dp"
            android:layout_marginTop="30dp"
            android:text="SIGN UP"
            android:id="@+id/signup_button"
            android:textSize="18dp"
            android:backgroundTint="@color/darkGreen"/>

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:id="@+id/loginRedirectText"
            android:text="Already a user? Login"
            android:layout_gravity="center"
            android:padding="8dp"
            android:layout_marginTop="8dp"
            android:textSize="18dp"
            android:textColor="@color/white"/>
```

Once again, two linear layouts are used for the same purpose as before inside a scrollview for in case of any viewing issues. The same input types are used as before, a regular type for email without needing to be specified and a password input for the password and confirmation of password as shown there to hide the password when the account is being created.



Logic

```
SignupActivity : AppCompatActivity() {

    // Declaring binding for accessing views and FirebaseAuth for user authentication
    private lateinit var binding: ActivitySignupBinding
    private lateinit var firebaseAuth: FirebaseAuth

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge() // Enables edge-to-edge UI support
        binding = ActivitySignupBinding.inflate(layoutInflater) // Initializing binding
        setContentView(binding.root) // Setting the content view to the binding's root layout

        firebaseAuth = FirebaseAuth.getInstance() // Initializing FirebaseAuth instance
```

First I set up binding to link the sign up activity to the sign up xml created.

```
// Listener for the "Sign Up" button
binding.signupButton.setOnClickListener {
    val email = binding.signupEmail.text.toString() // Get email from input
    val password = binding.signupPassword.text.toString() // Get password from input
    val confirmPassword = binding.signupConfirm.text.toString() // Get confirm password from input

    // Check if all fields are filled
    if (email.isNotEmpty() && password.isNotEmpty() && confirmPassword.isNotEmpty()) {
        // Check if password and confirm password match
        if (password == confirmPassword) {
            // Attempt to create a new user in Firebase
            firebaseAuth.createUserWithEmailAndPassword(email, password).addOnCompleteListener {
                if (it.isSuccessful) {
                    // Redirect to login activity on successful sign-up
                    val intent = Intent(this, LoginActivity::class.java)
                    startActivity(intent)
                } else {
                    // Show error message if sign-up fails
                    Toast.makeText(this, "Incorrect Input", Toast.LENGTH_SHORT).show()
                }
            }
        } else {
            // Show error if passwords do not match
            Toast.makeText(this, "Password does not match", Toast.LENGTH_SHORT).show()
        }
    } else {
        // Show error if any field is empty
        Toast.makeText(this, "Fields cannot be empty", Toast.LENGTH_SHORT).show()
    }
}
```

When the "Sign Up" button is clicked, the app checks that the email, password, and confirm password fields are filled. If the passwords match, it attempts to create the user with Firebase using the provided email and password. On success, the user is redirected back to the login screen LoginActivity to proceed with a login to the account. If the passwords don't match or there's an error during registration, it displays a message using Toast. Testing Code: 1.3, 1.4, 1.5

```
// Listener for "Log In" text to redirect users to the login screen
binding.loginRedirectText.setOnClickListener {
    val loginIntent = Intent(this, LoginActivity::class.java)
    startActivity(loginIntent)
}
```

Afterwards I also added a "Log In" textview that works the same way as before being turned into a button that redirects users back to the login screen in case they accidentally clicked onto the sign in screen. Again, there is a use of view binding to access layout views and adjusts padding for better compatibility with modern screen layouts. Testing Code 1.2

Testing

When testing the forgot password section, when I entered the incorrect information, the app responded as it should until it got to entering an email that is a real email but does not exist in the firebase authentication as an account has not been created on it. Testing Code: 1.9

```
private fun compareEmail(email: EditText){
        if (email.text.toString().isEmpty()){
            Toast.makeText(this, "Please enter your email", Toast.LENGTH_SHORT).show()
            return
        }
        if (!Patterns.EMAIL_ADDRESS.matcher(email.text.toString()).matches()){
            Toast.makeText(this, "Please enter a valid email", Toast.LENGTH_SHORT).show()
            return
        }
        firebaseAuth.sendPasswordResetEmail(email.text.toString())
            .addOnCompleteListener { task ->
                if (task.isSuccessful) {
                    Toast.makeText(this, "Check your email", Toast.LENGTH_SHORT).show()
                } else {
                    Toast.makeText(this, "Email does not exist", Toast.LENGTH_SHORT).show()
                }
            }
    }
}
```

This was because the code I had written would call the function to send an email based on the user function, however the code did not return whether the task had actually failed or not, since the task would still be marked as complete just an email would not be sent, the program would always display the ”check your email” message instead of the error message.

```
        firebaseAuth.fetchSignInMethodsForEmail(email.text.toString())
            .addOnCompleteListener { task ->
                if (task.isSuccessful) {
                    val signInMethods = task.result?.signInMethods
                    if (signInMethods.isNullOrEmpty()) {
                        // Email doesn't exist in Firebase
                        Toast.makeText(this, "Email does not exist", Toast.LENGTH_SHORT).show()
                    } else {
                        // Email exists, send reset email
                        firebaseAuth.sendPasswordResetEmail(email.text.toString())
                            .addOnCompleteListener { resetTask ->
                                if (resetTask.isSuccessful) {
                                    Toast.makeText(this, "Check your email", Toast.LENGTH_SHORT).show()
                                } else {
                                    Toast.makeText(this, "Failed to send reset email", Toast.LENGTH_SHORT).show()
                                }
                            }
                    }
                } else {
                    Toast.makeText(this, "Error checking email", Toast.LENGTH_SHORT).show()
                }
            }
    }
}
```

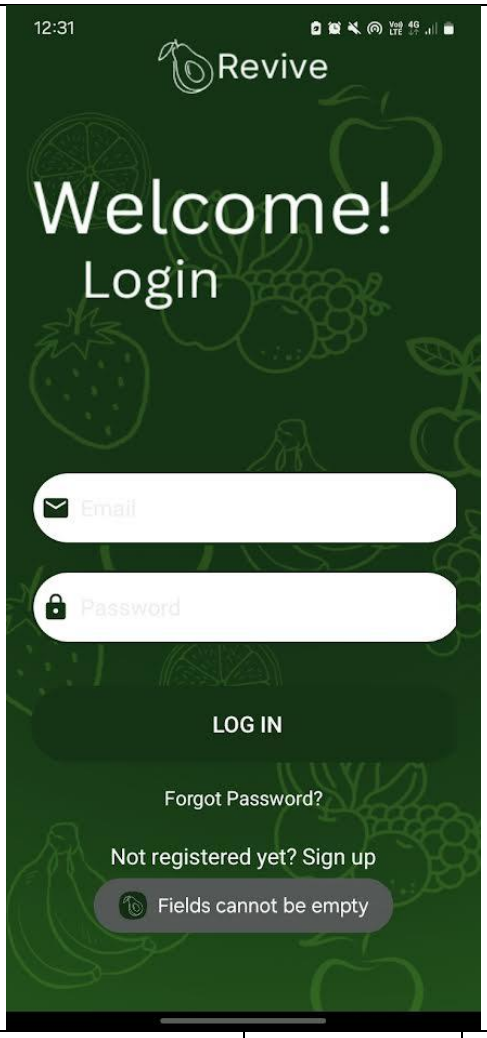
Therefore the code for the sign in method was changed so that instead of first sending the email, the method checks whether or not the email is registered with firebase. Now if the email does not exist no message will be sent, and instead a message will be displayed. If the email does exist then the message is sent. However just for safety measures I have added a failure message in case the email does exist, but somehow during the check the internet was cutoff, etc so the application was not able to send the email. Then another error message in case the user tries sending the email, again without internet connection and the app is not even able to check whether the email exists or not.
Testing code: 1.10

Login and Sign up testing video

This video showcases a couple the process of signing into the application, and follows a couple intentionally incorrect inputs to present how the app would respond to these incorrect inputs, additionally it runs through resetting a password with a wrong email, and then a correct one, and reveals how the correct information input leads to a desired outcome.

Test ID 1	Timestamp	PURPOSE	DATA	EXPECTED RESULTS	ACTUAL RESULTS
# Sign Up					
1.1	00:00	First loadup shows a splashscreen and then moves to the login screen		The splashscreen is displayed for a few seconds and then the login screen comes up	As expected
1.2	0:10	The text “Already a user?” works as a		A new activity is started which	As expected

		button and can be clicked		is the sign up screen	
1.3	0:20	To make sure that the app does not proceed with account creation if there is an invalid email.	Incorrect Email: *"reviveapplicationteam"*	A message pops up on the screen telling the user they need to input a valid email.	A message is displayed.
1.4	0:31	To make sure the app does not allow the creation of an account if the two passwords entered are not mathing	Non matching password strings: *"revive"* *"incorrect"*	A message pops up informing the user their passwords do not match	A message is displayed.
1.5	0:45	To try the correct details to sign up to an account and see if the account shows up in firebase authentication	Correct Email: *reviveapplicationteam@gmail.com* Correct matching passwords: *Revive* *Revive*	User is sent back to the login screen and their account appears in firebase.	As expected
# Login					
1.6 Timestamp: n/a Purpose: To test whether the application will allow the user to pass if there are no details inputted into the text boxes in the screen Data: No data Expected: A message is popped up informing the user they need to fill in the fields			Result:		

					
1.7	1:00	To enter an incorrect email in the login section, a non existing email in the database.	Incorrect Email: *"reviveapplication team"*	A message is displayed to the user, telling them authentication has failed	Message is displayed
1.8	1:15	To enter an incorrect password to the account	Incorrect Password: *"incorrect"*	A message is displayed to the user, telling them authentication has failed	Message is displayed
1.9	1:24	To enter an invalid email into the forgot password dialog box	Incorrect email: *"reviveapplication team"*	A message to be displayed to the user telling them to please enter a valid email	As expected.
1.10	1:36	To try and change the password to an account that does not exist in the	An incorrect non real email: (email hidden in testing as	The forgot password screen should be dismissed	Failed, the message that an email was

		firebase database by entering an email that does not exist.	personal email is used)	and the user should not have an email sent to their account.	sent was displayed, even though no email was sent.
1.11	1:53	To try and change the password to an account that does not exist in the firebase database by entering an email that does not exist.	An incorrect non real email: (email hidden in testing as personal email is used)	The forgot password screen should be dismissed and the user should not have an email sent to their account.	As expected, no email is sent and message is displayed.
1.12	2:14	To try and change the password to an account that does exist in the firebase database in case the user forgets their password.	Correct Email: *reviveapplicationteam@gmail.com*	The forgot password screen should be dismissed and the user should have an email sent to their account that allows them to input a new password	As expected
1.13	2:45	To enter valid login details with a correct email and password, where the password is the most recent one saved.	Correct Email: *reviveapplicationteam@gmail.com* New correct password: *password*	The user is logged in, a new activity is started (the main activity) and the old login activity is closed.	As expected

Module Summary

This module is the fundamental login into the app, the user cannot proceed further into the app without making or logging in to an account meaning everything is validated, all of this data is linked to the firebase authentication process, improving security.

By putting everything under an account the user will be able to store all their data and be able to regain access to it from other devices should they need to. This means that further on in the program all data can be saved under the users specific userid generated by firebase, and the account can be given edit permissions in the database for specifically their own data meaning all data for all users can be saved in one space, but nobody else can access anyone elses data.

The user can regain access to their account should they forget their sign in details, if they have access to, and know what email they saved their account under. These functions have been tested

throughout and the current prototype of my app so far holds a window to hold the creation of an account, and to proceed to an empty screen should the details be correct.

Personal Information: Input Module

EXPLANATION: Immediately after the user has first created an account and has logged into the app, they will be taken to the PersonalDetailsActivity which will have the user input any details required for later calculations in the app. This happens in the login module as explained earlier.

XMLS

activity_personal_details.xml

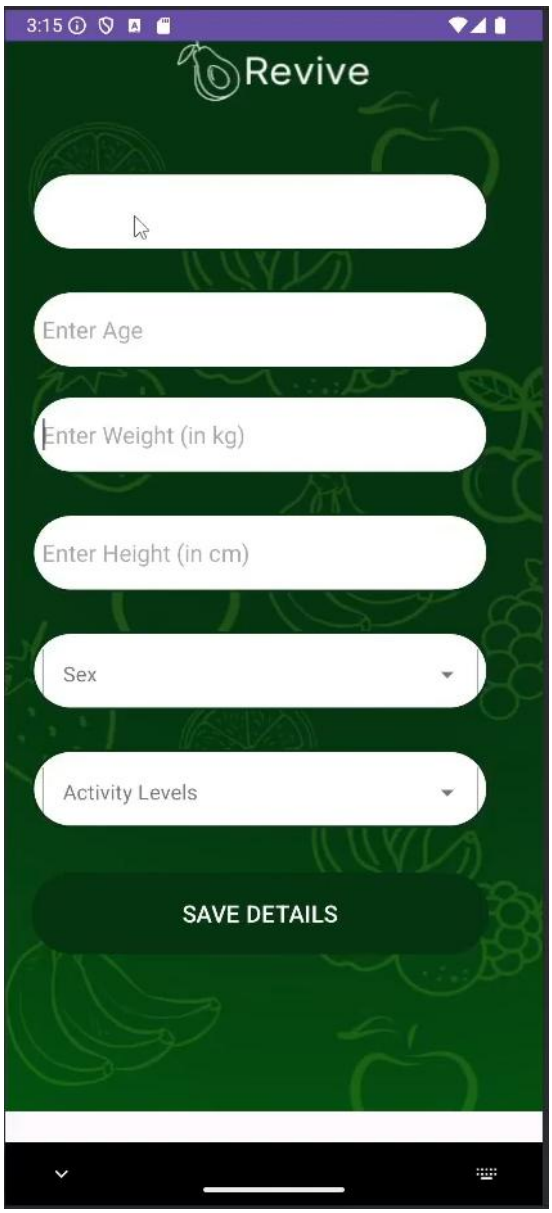
```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@drawable/initialscreen"
    tools:context=".LoginActivity">

    <ScrollView
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical">
            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="vertical"
                android:layout_marginTop="50dp"
                android:padding="20dp">


                <EditText
                    android:inputType="text"
                    android:layout_width="350dp"
                    android:layout_height="60dp"
                    android:id="@+id/name_input"
                    android:layout_marginTop="30dp"
                    android:background="@drawable/green_border"
                    android:padding="8dp"
                    android:hint="Enter Name (cannot change this la
                    android:drawablePadding="8dp"
                    android:textColor="@color/grey"
                    />

                <EditText
                    android:inputType="number"
                    android:layout_width="350dp"
                    android:layout_height="60dp"
                    android:id="@+id/age_input"
                    android:layout_marginTop="30dp"
                    android:background="@drawable/green_border"
                    android:padding="8dp"
                    android:hint="Enter Age"
                    android:drawablePadding="8dp"
                    android:textColor="@color/grey"
                    />

                <EditText
                    android:inputType="number"
                    android:layout_width="350dp"
                    android:layout_height="60dp"
                    android:id="@+id/weight_input"
                    android:layout_marginTop="20dp"
                    android:background="@drawable/green_border"
                    android:padding="8dp"
                    android:hint="Enter Weight (in kg)"
                    android:drawablePadding="8dp"
```

```
            <com.google.android.material.textfield.TextInputLayout
            android:layout_marginTop="30dp"
            android:padding="8dp"
            android:background="@drawable/green_border"
            style="@style/Widget.MaterialComponents.TextInputLayout.Outlined
            android:layout_width="350dp"
            android:layout_height="60dp">

            <AutoCompleteTextView
                android:id="@+id/sex_input"
                android:layout_width="match_parent"
                android:layout_height="50dp"
                android:layout_weight="1"
                android:hint="Sex"
                android:inputType="none"
                android:drawablePadding="8dp"
                android:textColor="@color/grey"
                />

        </com.google.android.material.textfield.TextInputLayout>

            <com.google.android.material.textfield.TextInputLayout
                android:layout_marginTop="30dp"
                android:padding="8dp"
                android:background="@drawable/green_border"
                style="@style/Widget.MaterialComponents.TextInputLayout.Outl
                android:layout_width="350dp"
                android:layout_height="60dp">

                <AutoCompleteTextView
                    android:id="@+id/fitness_input"
                    android:layout_width="match_parent"
                    android:layout_height="50dp"
                    android:layout_weight="1"
                    android:hint="Activity Levels"
                    android:inputType="none"
                    android:drawablePadding="8dp"
                    android:textColor="@color/grey"
                    />

            </com.google.android.material.textfield.TextInputLayout>

            <Button
                android:layout_width="350dp"
                android:layout_height="70dp"
                android:layout_marginTop="30dp"
                android:text="SAVE DETAILS"
                android:id="@+id/personalinfo_save_button"
                android:textSize="18dp"
                android:backgroundTint="@color/darkGreen"
                android:textColor="@color/white"/>


        </LinearLayout>


    </LinearLayout>
</ScrollView>

</LinearLayout>
```

This XML defines a user interface layout for a personal information input screen in an Android app.

It is important to note that in the age, weight, and height edit text boxes the input type is set to only allow numbers to be inputted, this means that no other letter can be added preventing any errors of the code not being able to perform calculations.

```
<EditText
    android:inputType="number"
```



Logic

```
class PersonalDetailsActivity : AppCompatActivity() {

    private lateinit var binding: ActivityPersonalDetailsBinding
    private lateinit var databaseReference: DatabaseReference

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityPersonalDetailsBinding.inflate(layoutInflater) // Layout binding to be able to see buttons ,etc
        setContentView(binding.root)
```

The first thing was to set up binding once again for this activity and generate a variable to hold the database reference that will be used later in order to reference the firebase database where all the user data will be stored.

```
    binding.personalinfoSaveButton.setOnClickListener{  // waits until "SAVE DETAILS" is pressed
        val name = binding.nameInput.text.toString() // assigns values to variables
        val age = binding.ageInput.text.toString().toIntOrNull() ?: 0
        val height = binding.heightInput.text.toString().toIntOrNull() ?: 0
        val weight = binding.weightInput.text.toString().toIntOrNull() ?: 0
        val sex = binding.sexInput.text.toString()
        val activitylevel = binding.fitnessInput.text.toString()

        if (name.isEmpty() || age == 0 || height == 0 || weight == 0 || sex.isEmpty() || activitylevel.isEmpty()) {
            Toast.makeText(this, "Please fill in all fields", Toast.LENGTH_SHORT).show()
            return@setOnClickListener // makes sure fields arent empty
        }
```

The very next thing was to setup the onClickListener so that when the user clicks the button to save all their information, the very first thing the code does is validate if even one of the boxes is empty. If it is, the code will display a text that informs the user to fill in all fields and the return ends the function in order to not proceed with the next steps. Testing code: 2.4

This means that the user can currently input a letters into the name box, a value into the age, height and weight (Test code 2.1, 2.2) but cannot do anything with the sex and activity level.

Therefore I created an array that will hold string values for both sex and activity levels with the corresponding items, this was stored in the strings tab which is a separate resources tab in android

studio.

```
<string-array name="Sexes">
    <item>Male</item>
    <item>Female</item>
</string-array>

<string-array name="Activity_Levels">
    <item>Little</item>
    <item>Light</item>
    <item>Moderate</item>
    <item>Active</item>
    <item>Very Active</item>

</string-array>
```

```
<?xml version="1.0" encoding="utf-8"?>
<TextView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:id="@+id/dropdown"
    android:padding="14dp"
    android:text="TextView"
    android:textColor="@color/black"
    android:textStyle="bold"
    android:textSize="16sp"
    android:backgroundTint="@color/white"
    android:background="@color/white">

</TextView>
```

Then I simply created an xml, dropdown_item.xml for the items that would appear in the dropdown list, to keep things more consistent by keeping them the same size and making sure the text will be black with a white background behind it, as without this the usual android dropdown will be a grey and black option which does not look nice.

```
    val sexes = resources.getStringArray(R.array.Sexes)  // sets up drop down for sexes option
    val sexadapter = ArrayAdapter(this,R.layout.dropdown_item,sexes)
    binding.sexInput.setAdapter(sexadapter)

    val activitylevels = resources.getStringArray(R.array.Activity_Levels)  // sets up drop down for activity levels option
    val activitylevelsadapter = ArrayAdapter(this,R.layout.dropdown_item,activitylevels)
    binding.fitnessInput.setAdapter(activitylevelsadapter)


    binding.personalinfoSaveButton.setOnClickListener{  // waits until "SAVE DETAILS" is pressed
        val name = binding.nameInput.text.toString() // assigns values to variables
```

Then above the binding created earlier, this code section handles the creation of two dropdown menus in the Android app's interface. Both dropdowns populate their fields based on the strings created earlier, an adapter is then created using the layout for each item created earlier and whatever the user selects as their items is put into their respective variables Testing code 2.3

```
    binding.personalinfoSaveButton.setOnClickListener{  // waits until "SAVE DETAILS" is pressed
        val name = binding.nameInput.text.toString() // assigns values to variables
        val age = binding.ageInput.text.toString().toIntOrNull() ?: 0
        val height = binding.heightInput.text.toString().toIntOrNull() ?: 0
        val weight = binding.weightInput.text.toString().toIntOrNull() ?: 0
        val sex = binding.sexInput.text.toString()
        val activitylevel = binding.fitnessInput.text.toString()

        if (name.isEmpty() || age == 0 || height == 0 || weight == 0 || sex.isEmpty() || activitylevel.isEmpty()) {
            Toast.makeText(this, "Please fill in all fields", Toast.LENGTH_SHORT).show()
            return@setOnClickListener // makes sure fields arent empty
        }
```

```
val userId = FirebaseAuth.getInstance().currentUser?.uid // makes sure current user id is not null
if (userId != null){
    databaseReference = FirebaseDatabase.getInstance().getReference("Information")
```

Within the binding created I generated a specific userId based on the current session of the user, this means that a specific string will be generated for the user if they are logged in. Since firebase required the user to be logged in for this check, a null check is required to make sure a value has actually been generated before proceeding. **This check will happen before any code that will require the code to add, update, delete, read, etc from the database later in the project.**

```
data class PersonalInfo(
    var name: String = "",
    var age: Int = 0,
    var height: Int = 0,
    var weight: Int = 0,
    var sex: String = "",
    var activityLevel: String = "",
    var calories: Double = 0.0,
    var deficitcalories: Double = 0.0,
```

I then created a separate data class that will hold any and all information regarding user info so that calculations etc can happen in this class using an object, instead of creating lots of other variables specific to the personalDetailActivity. This means I could create methods that could manipulate what was stored in the object and reuse this class and its objects in another activity when needed.

```
){
    fun calculatecalories(){
        if (sex == "Male"){
            calories = 66.5 + (13.75 * weight) + (5.003 * height) - (6.75 * age)
        }else{
            calories = 655.1 + (9.563* weight) + (1.85 * height) - (4.676 * age)
        }

        if (activityLevel == "Little"){
            calories = 1.2 * calories
        } else if (activityLevel == "Light"){
            calories = 1.375 * calories
        } else if (activityLevel == "Moderate"){
            calories = 1.55 * calories
        } else if (activityLevel == "Active"){
            calories = 1.725 * calories
        } else{
            calories = 1.9 * calories
        }

        // rounds
        calories = kotlin.math.round(calories)
        deficitcalories = kotlin.math.round(calories - 500)
    }
```

This is the method that calculates the users' calories based on the options that will be saved to the firebase that the user has inputted. This is the Harris-Benedict Formula. From this simply reducing the number of calories by 500 sets the user into a calorie deficit, this deficit will be what is presented to the user as their goal. They will be able to change this calorie number at a future stage should

they not want to take the apps calculations.

```
data class PersonalInfo(
    var name: String = "",
    var age: Int = 0,
    var height: Int = 0,
    var weight: Int = 0,
    var sex: String = "",
    var activityLevel: String = "",
    var calories: Double = 0.0,
    var deficitcalories: Double = 0.0,
    var protein: Int = 0,
    var fats: Int = 0,
    var carbs: Int = 0,
    var proteinpercent: Double = 0.3,
    var fatspercent: Double = 0.3,
    var carbspercent: Double = 0.4
){
```

I then added a couple more variables that will depict the protein, carbs and fats values also saved under the user's personal information. These values will be their separate own goals for the user to reach calculated from the user calories.

A common split for a person is 30% for protein, 30% for fat and 40% therefore percentage variables are initialised with these percentages as decimals, instead of asking the user to fill it out at this moment in time and allows all the calculations to be completed as they would usually.

```
    fun calculatemacros(){
        if (calories != 0.0){
            protein = ((deficitcalories * proteinpercent) / 4).toInt()
            carbs = ((deficitcalories * carbspercent) / 4).toInt()
            fats = ((deficitcalories * fatspercent) / 9).toInt()
        }
    }

}
```

To calculate the variables based on the calories calculated through the user input, a multiplication of the deficit calories by the percentage. These values are then divided by 4 for protein and carbs, as a gram of protein or a gram of carbs will produce 4 calories, and divided by 9 for fats, as a gram of fat will produce 9 calories.

Back in the fragment:

```
val userId = FirebaseAuth.getInstance().currentUser?.uid // makes sure current user id is not null
if (userId != null){
    databaseReference = FirebaseDatabase.getInstance().getReference("Information")
```

```
val personalinformation = PersonalInfo(name,age,height,weight,sex,activitylevel,0.0,0.0)
personalinformation.calculatecalories()
personalinformation.calculatemacros()
```

After a check for the database reference, the code then sends the specific variables to the class and saved the object under the personalinformation variable, then calls the two functions to fill the rest of the fields so that all values are ready to send to firebase.

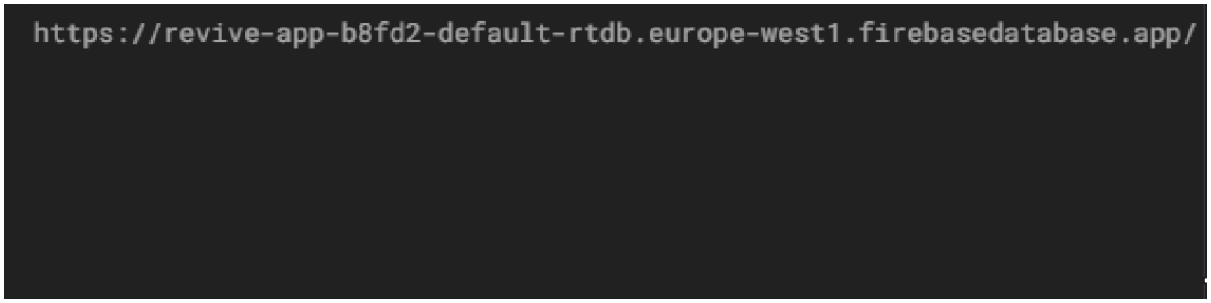
```
databaseReference.child(name).setValue(personalinformation).addOnSuccessListener{  // saves the data to firebase realtime database
    binding.nameInput.text.clear()
    binding.ageInput.text.clear() // clears values from variables as they are not needed anymore
    binding.heightInput.text.clear()
    binding.weightInput.text.clear()
    binding.sexInput.text.clear()
    binding.fitnessInput.text.clear()

    Toast.makeText(this,"Saved",Toast.LENGTH_SHORT).show() // shows a success message
    val intent = Intent(this, MainActivity::class.java)
    startActivity(intent)
    finish()
}.addOnFailureListener{
    Toast.makeText(this,"Failed",Toast.LENGTH_SHORT).show() // in case of failiure shows a message
}
}else {
    // Handle the case if user is not authenticated (userId is null)
    Toast.makeText(this, "User not authenticated", Toast.LENGTH_SHORT).show()
}
```

This code snippet sends the data to the database referenced earlier under the child based on the user inputted name. If this is successful then all the values in the current input boxes are cleared, the user gets a confirmation message “Saved” and the next activity is started, which leads to the main app.

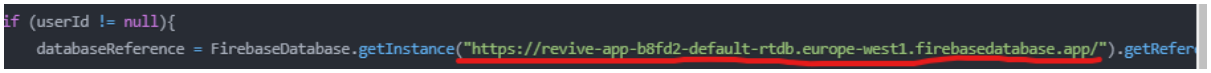
This activity can then have the finish() command as the values have been saved, and stop extra processing needed for it in the background, making the app slightly more efficient. Testing code 2.5

Testing

Upon testing whether the data saved to the database, when pressing the send button, the “Saved” message was being displayed, but the database remained empty. Testing code 2.5



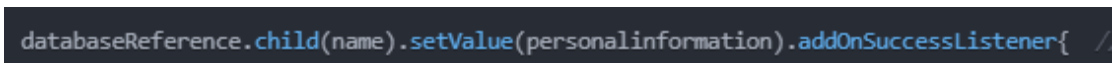
This was because I had set the database reference to the default one by firebase. When creating the database initially I had selected to use a European server for my database as it was closer to the default US one so I assumed it would help a little with efficiency, since the default call looks for the US server specifically, it was not finding my database based on the logged in user.



To bypass this I inputted the link to my specific database so there was no way the app would not have to search for the database, and instead it would be able to go straight to the required database, solving this error actually helps with efficiency overall since without this every time the reference is needed the code would need to search for the correct database, whereas here it can go straight to it.

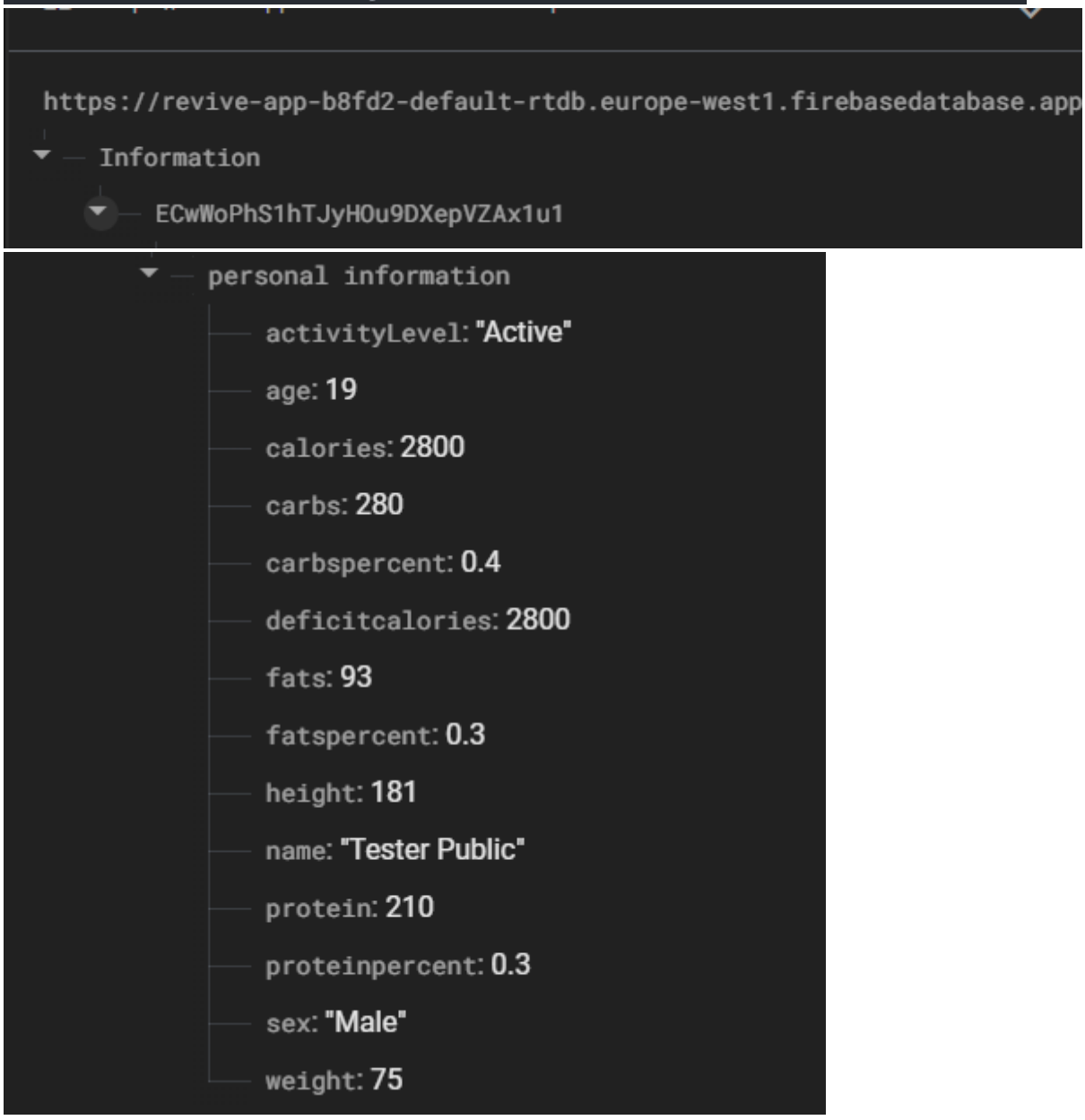
Fortunately, this meant that my app could find the correct database and upload the data correctly, Unfortunately there was another error, which saved the information under a subclass of the users name, instead of under the specific userID i had wanted it to. Testing code 2.6

This meant that if another user had created an account and also given themselves the name ”Dominic” they would override the data stored in this section with their own data that was meant to be specific to an account.



This was caused by the call of the database reference being set to .child(name), saving the data under the name instead of the user id, to fix this I changed the name variable to the userID variable used to check for a firebase authentication earlier and it worked perfectly. Testing code 2.7

```
databaseReference.child(userId).child("personal information")
    .setValue(personalInfo)
    .addOnSuccessListener {
```



(name was changed during testing from dominic to tester public)

For the other tests for this section, another video has been uploaded:
Personal Info | Details Input testing video

Test ID 2	Times tamp	PURPOSE	DATA	EXPECTED RESULTS	ACTUAL RESULTS
2.01	0:00	To test the name input.	String: “Dominic”	Any integers that are entered are converted to	All data values entered are converted to a string and saved

				strings regardless of whether or not it is an actual name as a name can be anything the user wishes. Then the data is saved to the personal information section in firebase.	to the personal information section name: "30" name: "30.1"
2.02	0:05	The user is able to enter their information for age, height and weight to be saved.	Attempt to enter: (age) Integer: 17 Negative integer: -20 Float: 1.5 Character: “A” String: “Hello” Attempt to enter: (height) Integer: 178 Negative integer: -20 Float: 1.5 Character: “A” String: “Hello” Attempt to enter: (weight) Integer: 77 Negative integer: -20 Float: 1.5 Character: “A” String: “Hello”	The user should not even have the option to enter the any kind of string or character, they should not have the option to enter a negative number. The number keyboard should also prevent them from entering float values.	As expected, only the number keyboard shows and the user can only enter integers.
2.03	0:15	The user can enter their details for their sex and activity levels, tests whether or not the user is able to enter their own data, or only	Access to only given data: Sex: “male” “female” Activity Levels: “Low”	Only a dropdown option should be displayed for the user to pick from, and the options in the dropdown	As expected.

		choose from the given options	“Light” “Moderate” “Active” “Very Active”	should correspond to what the user is trying to input	
2.04	0:20	To test whether the correct messages are displayed when the user does not fill out all the fields to enter their personal information. Both for the drop down options and the input boxes	Purposely leaving sections blank, starting with the dropdown options to the edit boxes.	A message telling the user that all fields need to be entered should be displayed	As expected
2.05	n/a	Testing whether or not the data appears in firebase.	n/a	All data will appear in firebase that was entered, as well as correctly calculated values for calories, deficit calories, and macros.	Fail, no data appeared
2.06	n/a	Testing whether or not the data appears in firebase correctly and under the correct children nodes, within the userid.	n/a	All data will appear in firebase that was entered, as well as correctly calculated values for calories, deficit calories, and macros.	Fail, data appeared under the wrong id under the information tab.
2.07	0:50	Testing whether or not the data appears in firebase correctly and under the correct children nodes, within the userid.	n/a	All data will appear in firebase that was entered, as well as correctly calculated values for	As expected

				calories, deficit calories, and macros.	

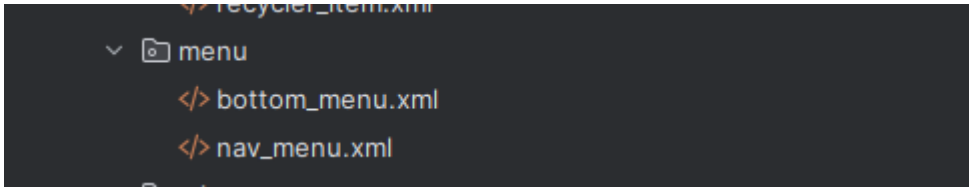
Module Summary

So far, the personal details module will be able to take a user input based on information about themselves, use this information to perform calculations and save those calculations and the entered details to firebase so that they can be retrieved later from anywhere in the program, as well as from other devices. It has specified inputs that only allow the intended types of values such as dropdown boxes, and the use of the number keyboard, and also does not allow the user to proceed without entering in all the required information, this is to make sure the code stays robust and does not crash during runtime when procedures are called with arguments that do not have data. A couple failed tests managed to be solved well and led to the creation of a more robust system.

Navigation Modules

PURPOSE: These modules will be the two that define how the user travels across the app and the different screens of the app.

EXPLANATION: These modules are basically the essentially backplate for the rest of the project, this part will work with the main navigation and its tools will all be handled within the main activity on the app, with fragments built on top of the main activity almost like child nodes of the main activity. This will consist of two types of navigation:

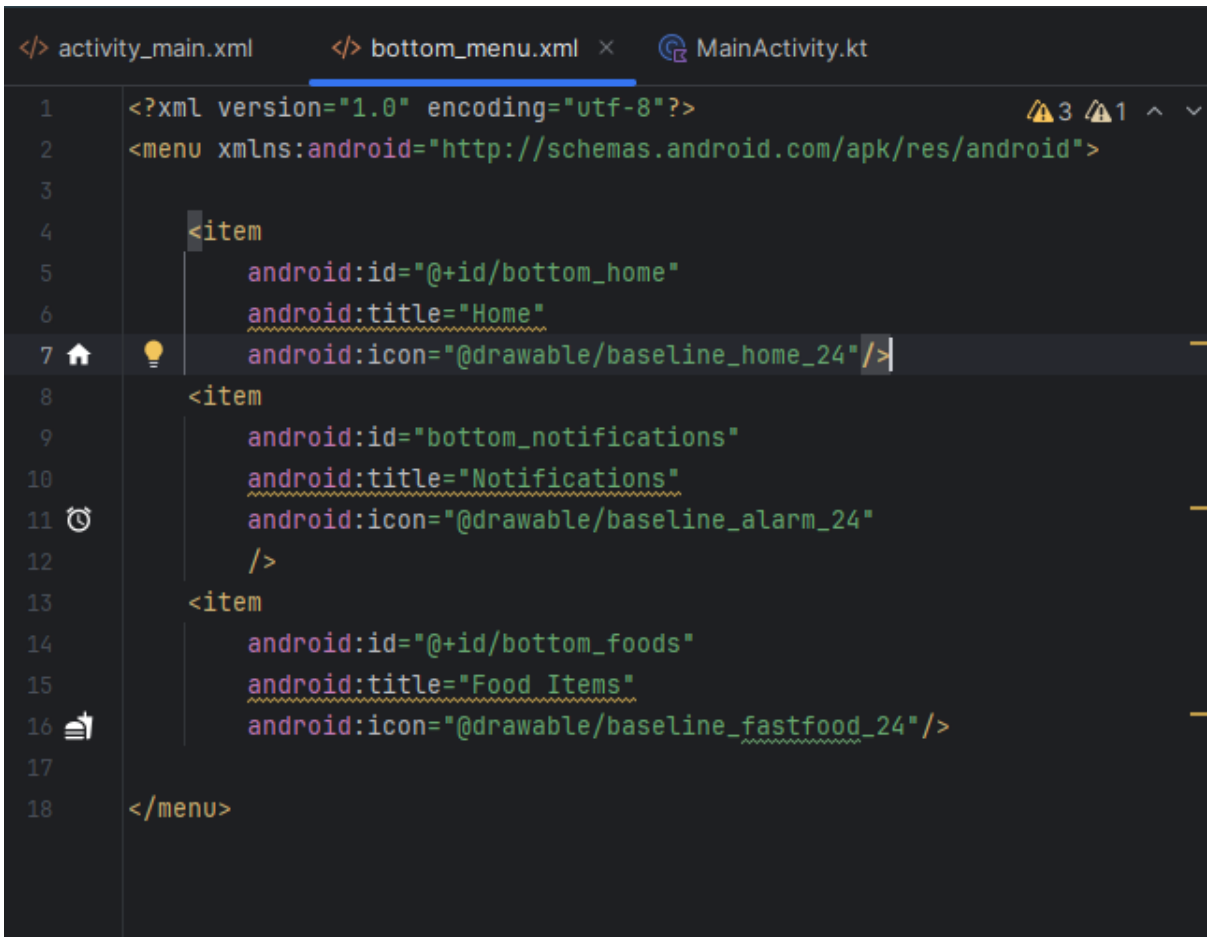


A bottom menu, which will always have options along the bottom of the screen for the user to choose from, these will always be displayed no matter where the user is on the app housing the more important modules.

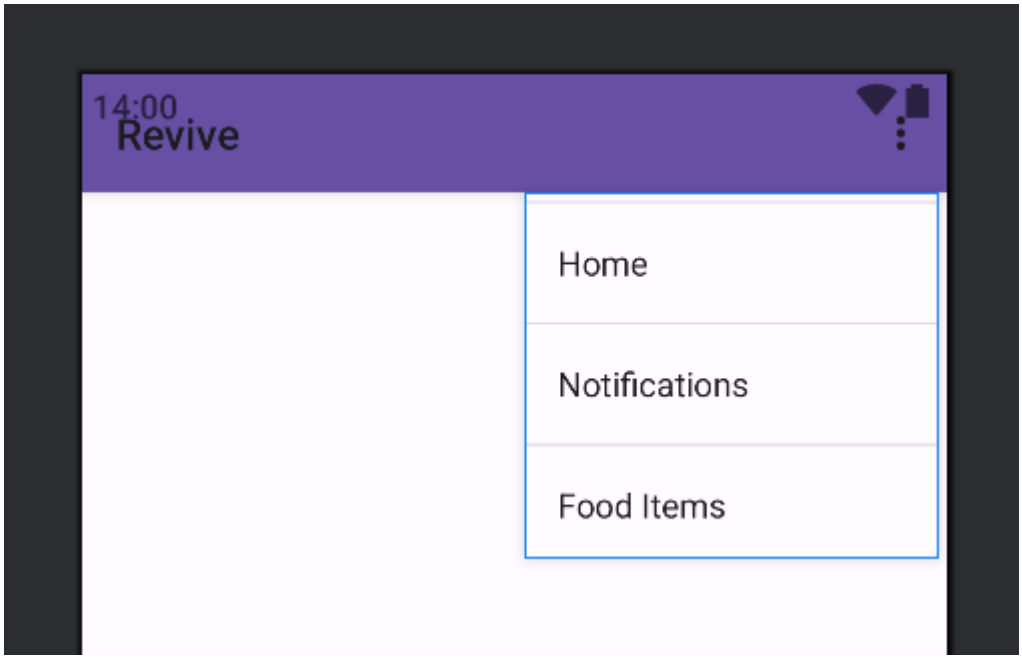
A navigational menu, which will be opened with a side button to house further fragments / less important ones, this navigational menu will be where any future modules could be developed as there will always be space for newer modules.

Bottom Navigation Menu

XMLS



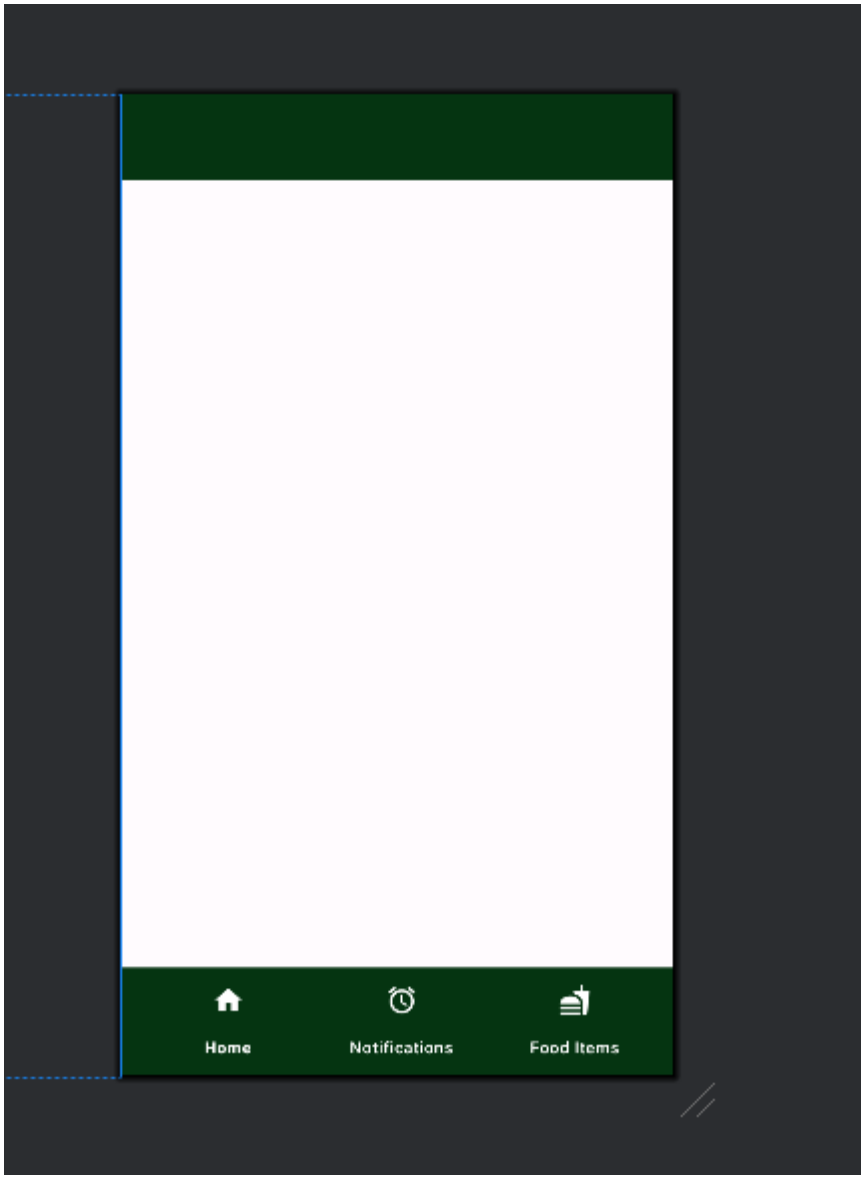
The bottom_menu.xml defines the items that will be shown in the bottom menu, where they can be given an id, an icon and a name, this means that all the options for the navigation will be stored in one place, and if a module is ever to be added to the bottom navigation, it just needs to be added to this list to be displayed and able to be selected.


14:00
Revive
Home
Notifications
Food Items

bottom_menu.xml activity_main.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<androidx.drawerlayout.widget.DrawerLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/drawer_layout"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity"
    >


    <RelativeLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent">

        <androidx.appcompat.widget.Toolbar
            android:layout_width="match_parent"
            android:layout_height="?attr/actionBarSize"
            android:id="@+id/toolbar"
            android:background="@color/darkGreen"
            android:elevation="4dp"
            android:theme="@style/ThemeOverlay.AppCompat.Dark.ActionBar"
            app:popupTheme="@style/ThemeOverlay.AppCompat.Light"/>

        <FrameLayout
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:id="@+id/fragment_container"/>

        <androidx.coordinatorlayout.widget.CoordinatorLayout
            android:layout_width="match_parent"
            android:layout_height="match_parent">

            <com.google.android.material.bottomappbar.BottomAppBar
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:id="@+id/bottomAppBar"
                android:layout_gravity="bottom"
                android:backgroundTint="@color/darkGreen">

                <com.google.android.material.bottomnavigation.BottomNavigationView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:id="@+id/bottom_navigation"
                    app:labelVisibilityMode="labeled"
                    app:menu="@menu/bottom_menu"
                    android:background="@android:color/transparent"
                    app:itemIconTint="@color/white"
                    app:itemTextColor="@color/white" />
            </com.google.android.material.bottomappbar.BottomAppBar>

        </androidx.coordinatorlayout.widget.CoordinatorLayout>
    </RelativeLayout>


</androidx.drawerlayout.widget.DrawerLayout>
```

This xml defines the bottom navigation view along with a top action bar that will house the future option to load the side navigation view. The bottom app bar is simply a window to house the items that have been created in the bottom navigation bar as individual items that can be selected.



Logic

```
class MainActivity : AppCompatActivity() {
```

```
    private fun openFragment(fragment: Fragment){
        val fragmentTransaction: FragmentTransaction = fragmentManager.beginTransaction()
        fragmentTransaction.replace(R.id.fragment_container,fragment)
        fragmentTransaction.commit()
    }
}
```

The very first thing I created in the main activity is the fragment navigation function, meaning that I can reuse this function for both the bottom navigation bar and the side navigation bar as it just replaces the current fragment with the one to be opened.

```
    private lateinit var fragmentManager: FragmentManager
    private lateinit var binding: ActivityMainBinding
```

```
    binding.bottomNavigation.background = null
    binding.bottomNavigation.setOnItemReselectedListener{ item ->
        when(item.itemId){
            R.id.bottom_home -> openFragment(HomeFragment())
            R.id.bottom_foods -> openFragment(ItemsFragment())
            R.id.bottom_notifications -> openFragment(NotificationsFragment())
        }
```

I then created the binding that allows the main activity to listen to user clicks. If the user presses on one of the three bottom menu options, the code will call the openFragment function created earlier with the fragment it wants to to open as the argument.

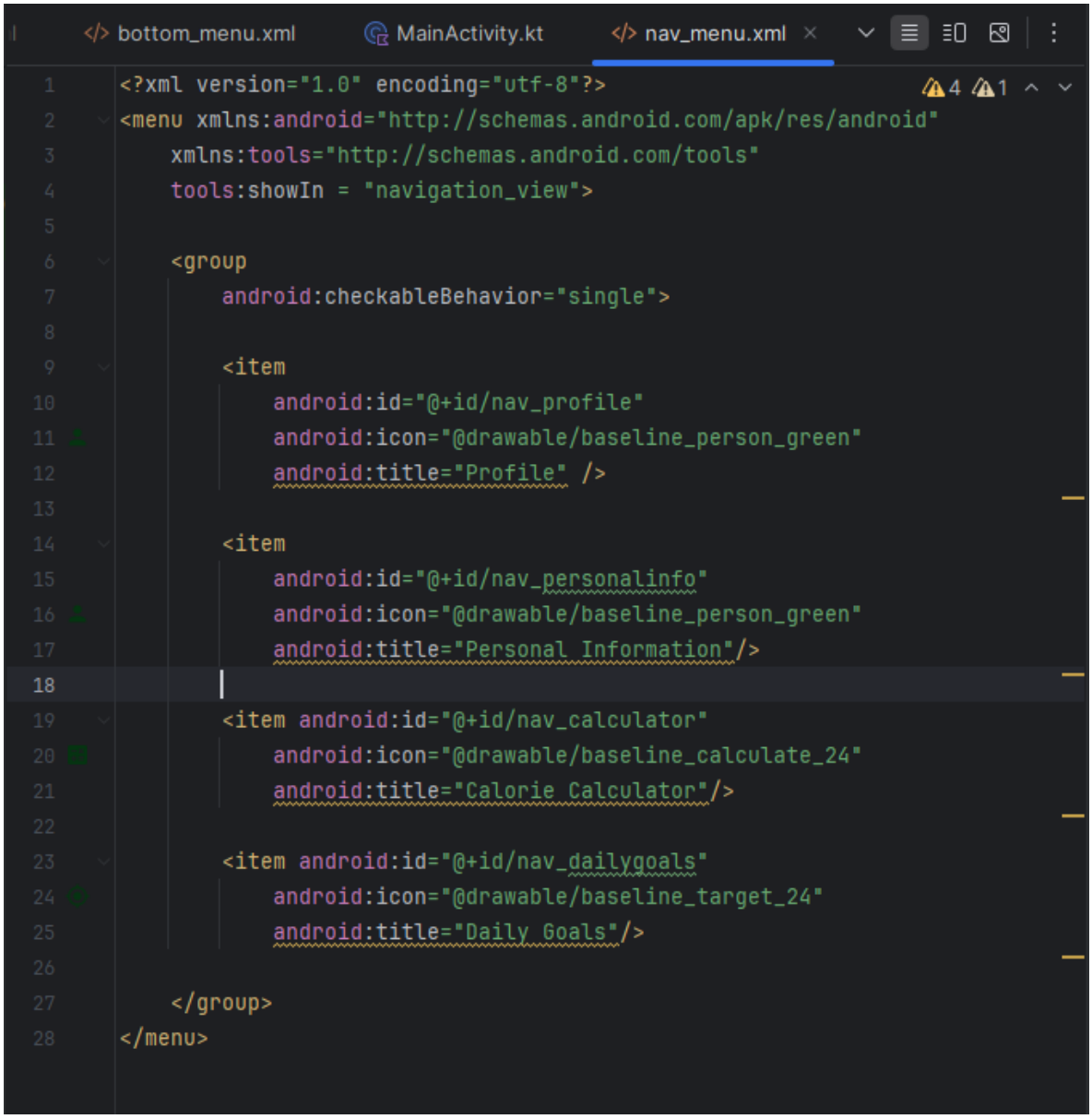
```
            true
        }
        fragmentManager = supportFragmentManager
        openFragment(HomeFragment())
    }
```

Afterwards the openFragment(HomeFragment()) means that the first screen the user is taken to upon app start is the home screen (calorie tracking module later). Testing Code: 3.1

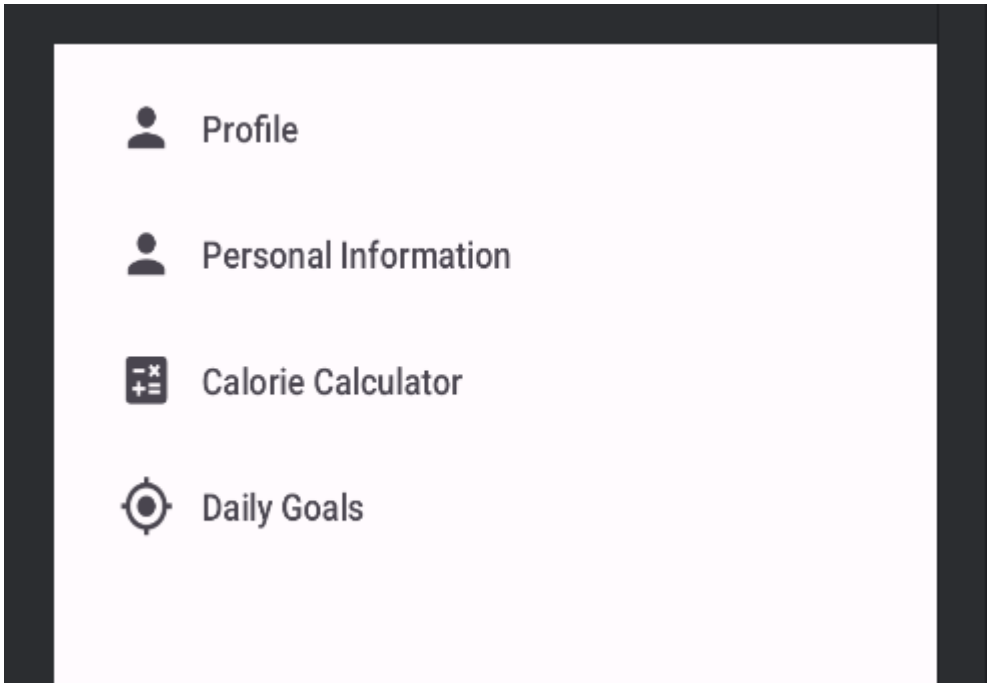
Side Navigation Menu

This navigation menu will appear as an openable window in the top right of the screen for the user that will hold all of the less important fragments / modules for the app so that they are not seen when not needed.

XMLS

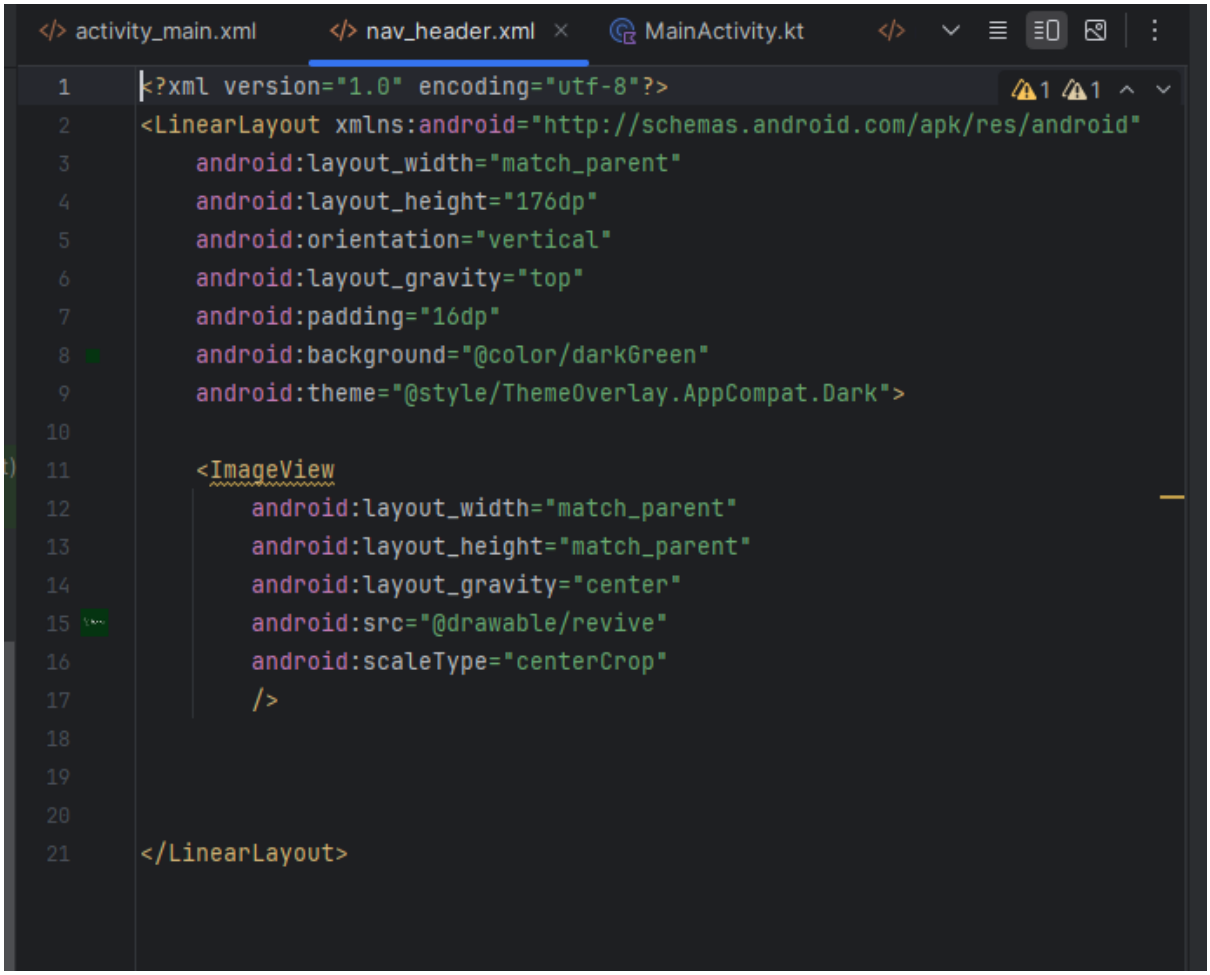


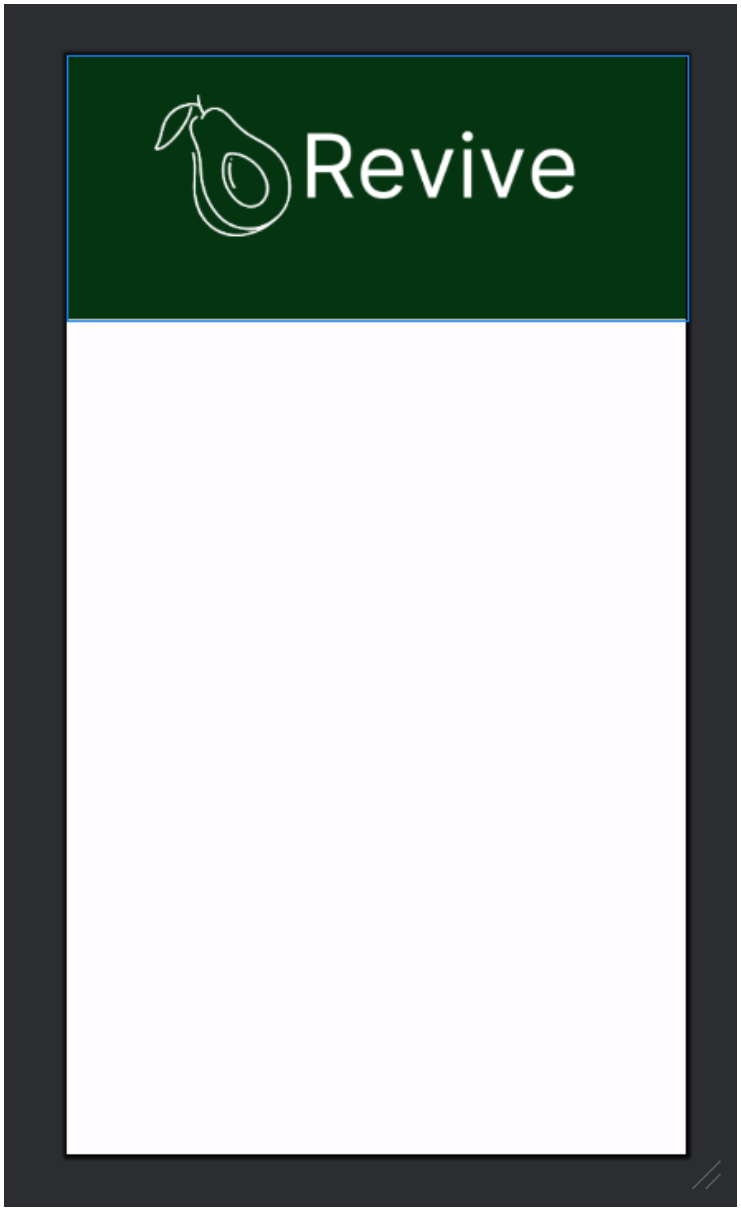
This xml works in the same way as the bottom navigation view, where items are listed that define them an id, a name to be displayed and an icon for appearance all grouped together. However there is no need for a drawer layout to be used as the layout will be defined in the actual side navigation.




```
        </androidx.coordinatorlayout.widget.CoordinatorLayout>
    </RelativeLayout>

    <com.google.android.material.navigation.NavigationView
        android:layout_width="wrap_content"
        android:layout_height="match_parent"
        android:id="@+id/navigation_drawer"
        android:layout_gravity="start"
        app:itemIconTint="@color/darkGreen"
        app:itemTextColor="@color/darkGreen"
        app:headerLayout="@layout/nav_header"
        app:menu="@menu/nav_menu"
        android:background="@color/white"/>



</androidx.drawerlayout.widget.DrawerLayout>
```

For the main activity, under the last relative layout I added a navigation view, which is basically the card that will display when the user pressed the button to access the side view, this will consist of an image defined in the nav_header.xml:


activity_main.xml
nav_header.xml
MainActivity.kt
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="176dp"
android:orientation="vertical"
android:layout_gravity="top"
android:padding="16dp"
android:background="@color/darkGreen"
android:theme="@style/ThemeOverlay.AppCompat.Dark">
<ImageView
android:layout_width="match_parent"
android:layout_height="match_parent"
android:layout_gravity="center"
android:src="@drawable/revive"
android:scaleType="centerCrop"
/>
</LinearLayout>



And allows me to set the height and width of the header so that the side navigation view does not cover the entire screen, instead only covers a section like a pop up window. The items mentioned earlier will appear under the header “Revive”.

Logic

```
class MainActivity : AppCompatActivity(), NavigationView.OnNavigationItemSelectedListener
```

```
setSupportActionBar(binding.toolbar)
val toggle = ActionBarDrawerToggle(this,binding.drawerLayout,binding.toolbar,R.string.nav_open, R.string.nav_close)
binding.drawerLayout.addDrawerListener(toggle)
toggle.syncState()

binding.navigationDrawer.setNavigationItemSelectedListener(this)
```

This sets up the action bar and binds to the toggle at the top right of the screen so that when it is pressed, the procedure to toggle the side navigation view is called and a listener is then called to see

what the user will press. The procedure is also called with two strings, R.string.nav_open and R.string.nav_close (default animation strings)

```
override fun onNavigationItemSelected(item: MenuItem): Boolean {
    when(item.itemId){
        R.id.nav_profile -> openFragment(ProfileFragment())
        R.id.nav_personalinfo -> openFragment(PersonalInfoFragment())
        R.id.nav_notifications -> openFragment(NotificationsFragment())
        R.id.nav_calculator -> openFragment(CalorieCalculatorFragment())
        R.id.nav_dailygoals -> openFragment(DailyGoalsFragment())
    }
    binding.drawerLayout.closeDrawer(GravityCompat.START)
    return true
}
```

This function works the same way as the bottom navigation, where depending on the item that is pressed, the corresponding fragment will be opened over the top, so the same openFragment() function can be used from earlier. Testing Code: 3.3

```
override fun onBackPressed(){
    if (binding.drawerLayout.isDrawerOpen(GravityCompat.START)){
        binding.drawerLayout.closeDrawer(GravityCompat.START)
    } else{
        super.getOnBackPressedDispatcher().onBackPressed()
    }
}
```

This function allows the user to press the back button on their device to exit out the side menu, but if the side menu is not open then the function calls what normally would happen if the back button was pressed.

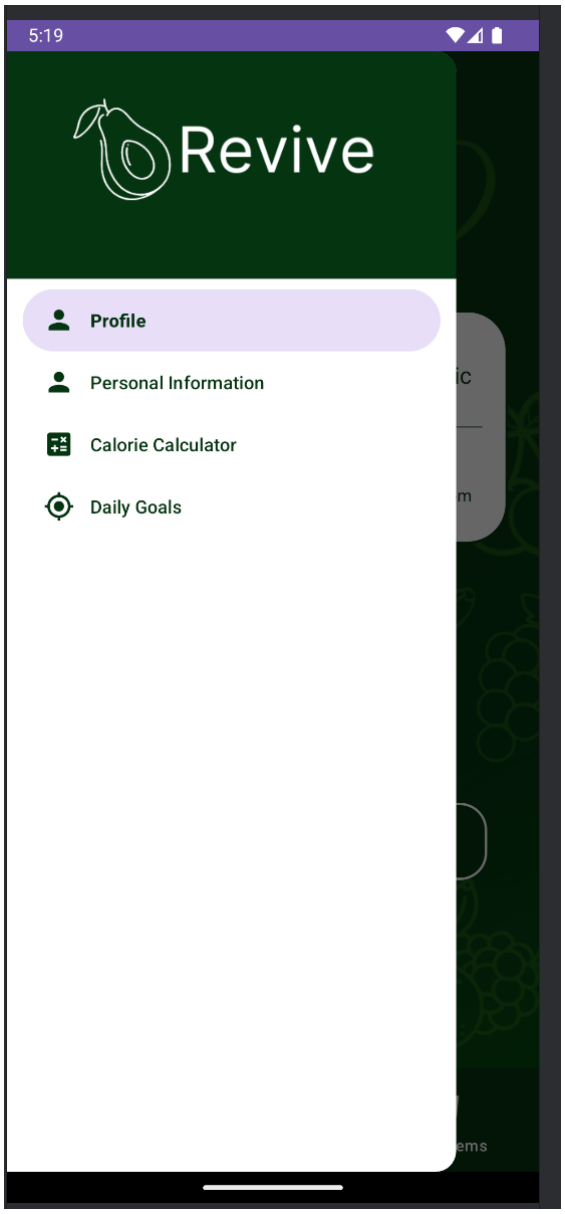
A problem I encountered was that the app crashed as soon as the code was run, this was because there was an incorrect statement in the code. Which i found to be this line of code

```
        super.getOnBackPressedDispatcher().onBackPressed()
```

Where the Logcat told me that the super class onBackPressed() did not have a function like this, even though when searching the default android files, the procedure was there. After searching i discovered that this call was simply depreciated, and it had to be called a different way.

```
} else{
    onBackPressedDispatcher.onBackPressed()
}
```

After I had changed to this method instead of calling from the super class the function worked as intended and the code was able to be run.



After the issue was sorted the side bar worked and all the options were displayed as intended

Testing

Whilst I was testing how the module works I discovered a logic error which involved the selection of fragments from the bottom navigation view, where instead of being able to press each option and have the fragment swap, each button had to be pressed twice for the screens to swap. Test ID 3.1

```
binding.bottomNavigation.background = null
binding.bottomNavigation.setOnItemReselectedListener{ item ->
    when(item.itemId){
        R.id.bottom_home -> openFragment(HomeFragment())
        R.id.bottom_foods -> openFragment(ItemsFragment())
        R.id.bottom_notifications -> openFragment(NotificationsFragment())
    }
```

This was caused by the navigation being set to on the item reselected instead of selected meaning that each item needed to be selected again to have the fragment code be swapped. This most likely happened due to using the IDE autofill feature.

```
binding.bottomNavigation.background = null
binding.bottomNavigation.setOnItemSelectedListener{ item ->
    when(item.itemId){
        R.id.bottom_home -> openFragment(HomeFragment())
        R.id.bottom_foods -> openFragment(ItemsFragment())
        R.id.bottom_notifications -> openFragment(NotificationsFragment())
    }
```

This error was fixed by changing the setOnItemReselectedListener to setOnItemSelectedListener so that as soon as each item is pressed the code is called. Meaning that the code now worked as intended. Testing Code: 3.2

Bottom Navigation and Side Navigation Testing Video

Test ID 3	Timestamp	PURPOSE	DATA	EXPECTED RESULTS	ACTUAL RESULTS
# - Bottom Navigation					
3.1	0:00	To test whether the bottom navigation view changes the visible fragment by replacing the current fragment with the intended fragment, terminating the process of the old fragment.	n/a	Each screen has a different fragment and is opened when clicked.	Fail, Each item in the menu needed to be pressed twice to swap the screen instead of swapping as soon as the button was pressed.
3.2	0:11	To test whether the bottom navigation view changes the visible fragment by replacing the current fragment with the intended fragment, terminating the process of the old fragment.	n/a	Each screen has a different fragment and is opened when clicked.	As Expected

# - Side Navigation					
3.3	0:17	To test whether the side navigation view changes the visible fragment by replacing the current fragment with the intended fragment, terminating the process of the old fragment.	n/a	Each screen has a different fragment and is opened when clicked.	As Expected

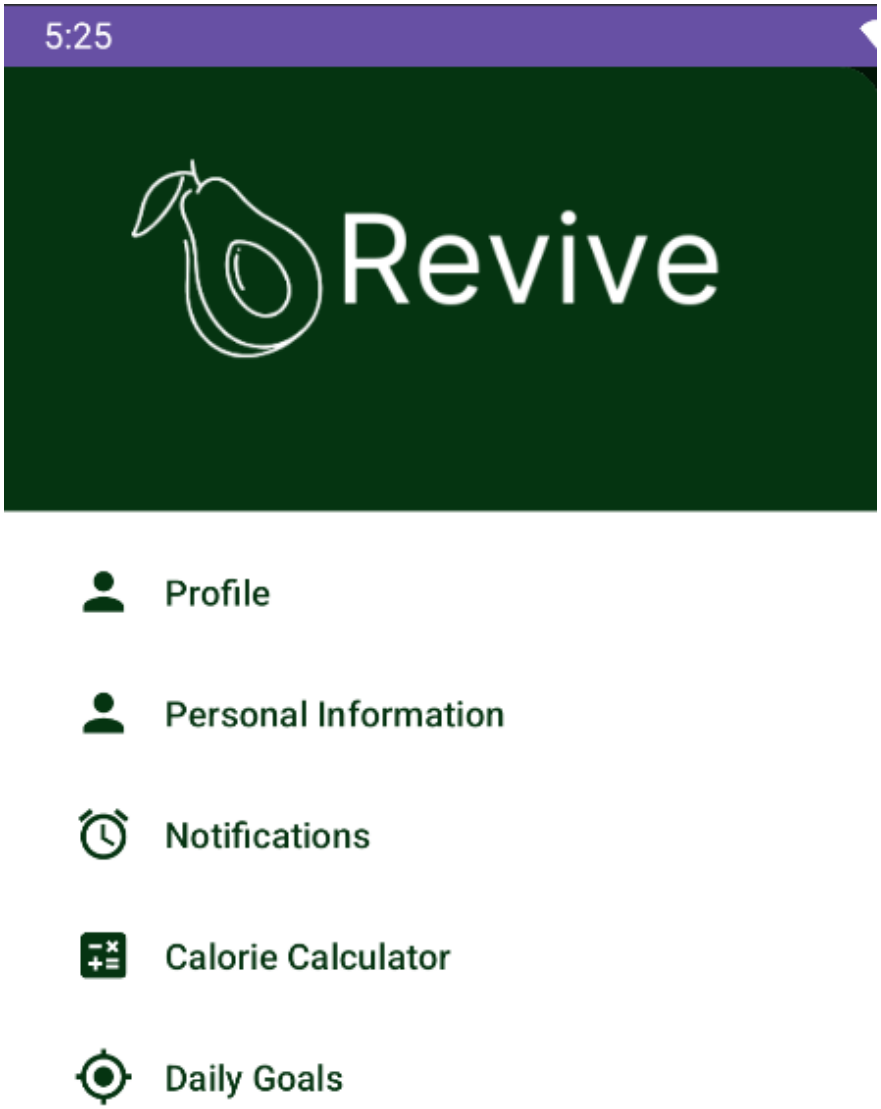
Small side note

After testing the app I decided to move the notifications section to the side navigation bar as I felt the navigation was not a core part of the application, but more like a separate feature that the user does not really need to use if they do not need to. This was mainly a change in the xml so that the item now had its id in the nav_menu (side menu bar) instead of bottom_menu (bottom navigation bar).

```
override fun onNavigationItemSelected(item: MenuItem): Boolean {
    when(item.itemId){
        // selects which fragment to open over the top
        R.id.nav_profile -> openFragment(ProfileFragment())
        R.id.nav_personalinfo -> openFragment(PersonalInfoFragment())
        R.id.nav_notifications -> openFragment(NotificationsFragment())
        R.id.nav_calculator -> openFragment(CalorieCalculatorFragment())
        R.id.nav_dailygoals -> openFragment(DailyGoalsFragment())
    }
    binding.drawerLayout.closeDrawer(GravityCompat.START)
    return true

}
```

The only code to do with logic that was changed was moving the navigation from being called in the bottom menu, and moving it to the side navigation view.

However doing this also demonstrates how modules can be added to the navigation in the future, they just need to be created as a fragment, and have an icon, id and name added to the list of items in either nav_menu or bottom_menu, then a singular function call implements them into the apps navigation.



The new sidebar looks a little like this

Module Summary

Adding this module means that the app has become fully modular from this moment onwards, meaning that any features added will have their own module that can be added to the navigation system to be accessed through the app where the key features to the app will be found towards the bottom as they will be quickly accessed by the user, any other features can be found through the side navigation. This approach is suitable as it greatly helps with future maintenance and development of the app, making it simply and easy to add features. With the current prototype, the user can login to the app, fill in the details required for calculations and navigate around the different empty screens that will later house different features.

Stakeholder Review –1

How intuitive is the navigation and data entry so far?

A: The app uses consistent layouts and interaction patterns throughout. Edit buttons trigger similar-looking dialogs, search bars work the same way across modules, and confirmation messages provide clear feedback. The navigation is compartmentalized - personal information, profile management, and goal setting each have their own dedicated sections to avoid confusion which is a good way of creating the application.

Personal Information: Profile Module

PURPOSE: This is a small module where the user can view information on a screen, such as their name and the email address that was used to create an account. This will mainly act as a profile module where the user will be given the ability to logout of the app if they wish.

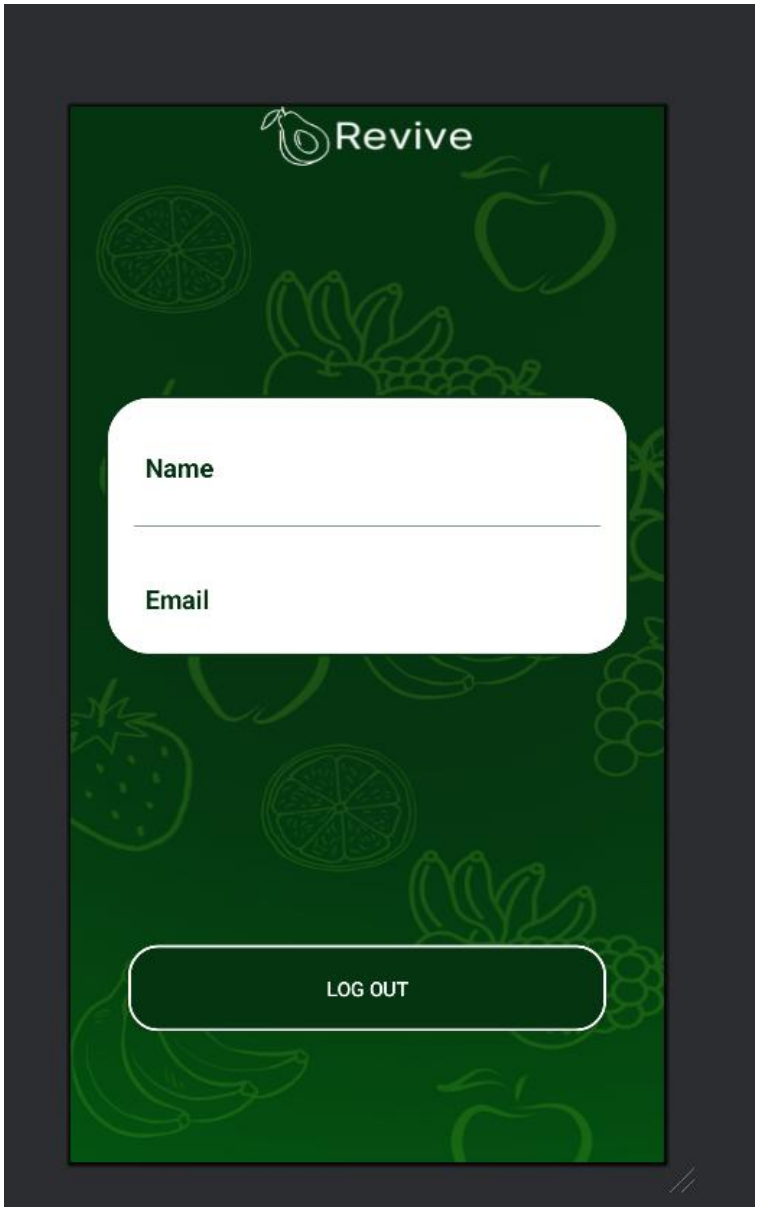
XML

ProfileFragment.kt fragment_profile.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http:/
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".ProfileFragment"
    android:background="@drawable/initialscreen">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:padding="10dp"
            android:layout_gravity="center"
            android:layout_marginTop="200dp"
            android:layout_marginStart="24dp"
            android:layout_marginEnd="24dp"
            android:background="@drawable/green_border">

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="vertical">

                <LinearLayout
                    android:layout_width="match_parent"
                    android:layout_height="match_parent"
                    android:padding="8dp"
                    android:orientation="horizontal"
                    android:layout_marginTop="20dp"
                    android:layout_marginEnd="10dp"
                    android:layout_marginStart="10dp"
                    android:layout_marginBottom="10dp">

                    <TextView
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:id="@+id/name"
                        android:layout_weight="1"
                        android:text="Name"
                        android:textStyle="bold"
                        android:textColor="@color/darkGreen"
                        android:textSize="18sp" />
                    <TextView
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:id="@+id/readName"
                        android:text=""
                        android:layout_weight="1"
                        android:textAlignment="viewEnd"
                        android:textColor="@color/darkGreen"
                        android:textSize="18sp"/>

                </LinearLayout>

                <androidx.appcompat.widget.AppCompatImageView
                android:layout_width="match_parent"
                android:layout_height="1dp"
                android:layout_margin="10dp"
                android:alpha="0.5"
                android:background="@color/darkGreen"/>

                <LinearLayout
                    android:layout_width="match_parent"
                    android:layout_height="match_parent"
                    android:padding="8dp"
                    android:orientation="horizontal"
                    android:layout_marginTop="20dp"
                    android:layout_marginEnd="10dp"
                    android:layout_marginStart="10dp"
                    android:layout_marginBottom="10dp">
```

```
                    <TextView
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:id="@+id/email"
                        android:layout_weight="1"
                        android:text="Email"
                        android:textStyle="bold"
                        android:textColor="@color/darkGreen"
                        android:textSize="18sp" />
                    <TextView
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:id="@+id/readEmail"
                        android:text=""
                        android:layout_weight="1"
                        android:textAlignment="viewEnd"
                        android:textColor="@color/darkGreen"
                        android:textSize="13sp" />

                </LinearLayout>

            </LinearLayout>


        </LinearLayout>

        <androidx.appcompat.widget.AppCompatButton
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:layout_marginTop="200dp"
            android:layout_marginEnd="40dp"
            android:layout_marginStart="40dp"
            android:text="LOG OUT"
            android:padding="20dp"
            android:textColor="@color/white"
            android:background="@drawable/darkgreen_round_white_outline"
            android:id="@+id/logout_button" />


    </LinearLayout>


</androidx.constraintlayout.widget.ConstraintLayout>
```

This XML defines a card that will consist of information about the user, a gap and then a button with a style set in the drawables to have a dark green rounded background, with a while outline:



The name will appear at the end of the card when the information is found in firebase, as well as the email that was used.

Logic

```
class ProfileFragment : Fragment() {
    private var _binding: FragmentProfileBinding? = null
    private val binding get() = _binding!! // safe access to binding
    private lateinit var databaseReference: DatabaseReference
    private lateinit var firebaseAuth: FirebaseAuth

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        // Inflate the layout for this fragment
        _binding = FragmentProfileBinding.inflate(inflater, container, false)
        readData()

        binding.logoutButton.setOnClickListener{
            Firebase.auth.signOut()
            val intent = Intent(requireContext(), LoginActivity::class.java)
            startActivity(intent)

            Toast.makeText(requireContext(),"Logout Successful", Toast.LENGTH_SHORT).show()
        }
        return binding.root
    }
}
```

This code defines the start of the fragment, where once again variable for auth and database are setup for future use as well as binding to match the code to the xml created. A click listener listens for when the user pressed the log out button and then signs them out of the app, returning to the login screen with a success message. Testing Code: 2.09

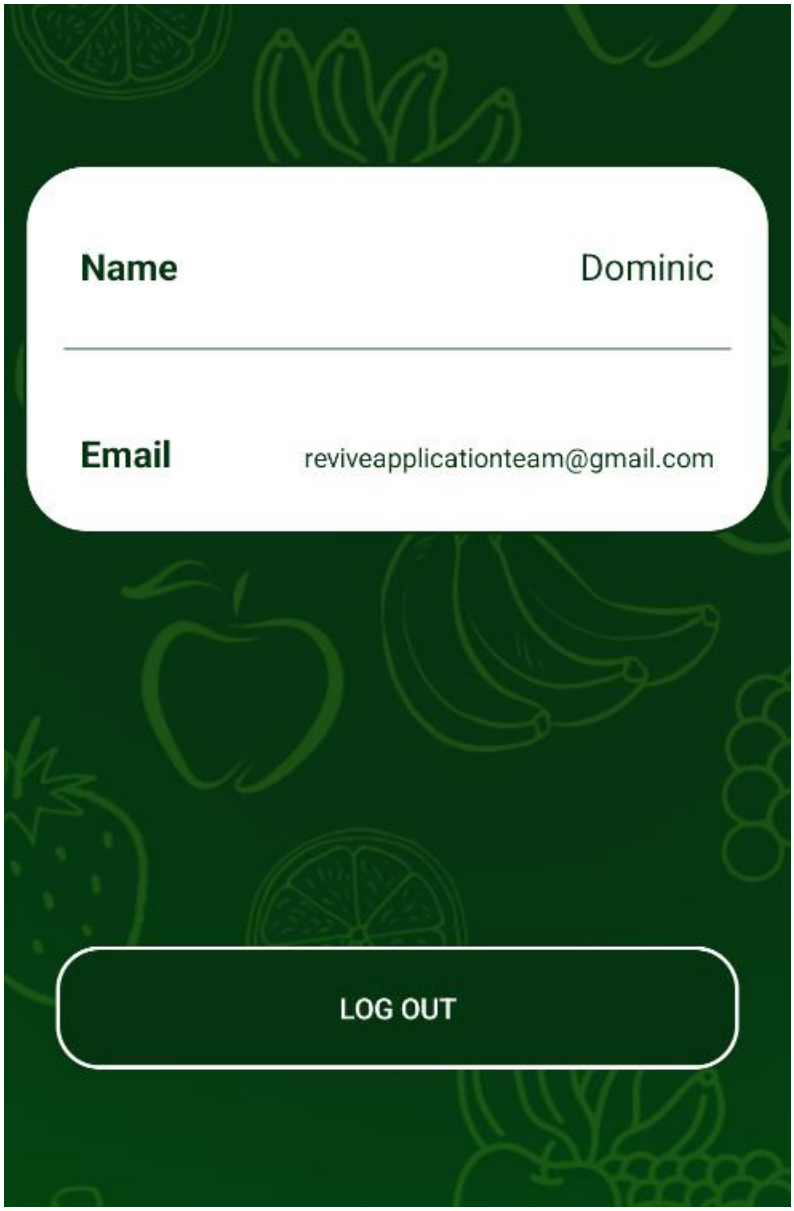
```
private fun readData(){
    val user = FirebaseAuth.getInstance().currentUser
    val userid = FirebaseAuth.getInstance().currentUser?.uid
    if (userid != null){
        databaseReference = FirebaseDatabase
            .getInstance("https://revive-app-b8fd2-default-rtdb.europe-west1.firebasedatabase.app/")
            .getReference("Information/${userid}/personal information")
        databaseReference.get().addOnSuccessListener { snapshot ->
            if (snapshot.exists()) {
                val name = snapshot.child("name").value
                val email = user?.email
                binding.readName.text = name.toString()
                binding.readEmail.text = email.toString()

            } else {
                Toast.makeText(requireContext(),"No data found", Toast.LENGTH_SHORT).show()
            }
        }.addOnFailureListener{
            Toast.makeText(requireContext(),"Something went wrong", Toast.LENGTH_SHORT).show()
        }
    }
}
```

This code is the ReadData() function referenced in the first screenshot, it taught me how to read data from firebase depending on data that is already in the database.

It works by setting the database reference directly to the child node that the information is stored under. Then setting a snapshot listener that looks at the information existing in firebase, if it exists then the data can be read, if the data somehow does not exist (which is not possible as you cannot proceed further into the app without filling in this information) to make the code more robust there will be an error message instead of continuing with the read so that the app does not crash. Testing Code: 2.08

Testing

Upon first testing the app displayed images according to how I wanted, but to make sure that there will be no complications to how the text appeared I made sure to set the text size in the xml to a small one and its width to wrap_content.



When the logout button was pressed, the app almost worked completely as intended and logged the user out, but instead of loading back the login screen, app just crashed and the user was exited out of the app. Testing Code: 2.09

```
        binding.loginButton.setOnClickListener {  // waits for the login button to be clicked
            val email = binding.loginEmail.text.toString()
            val password = binding.loginPassword.text.toString()

            if (email.isNotEmpty() && password.isNotEmpty()) {  // makes sure the two email and password fields are not left empty
                firebaseAuth.signInWithEmailAndPassword(email, password).addOnCompleteListener {
                    if (it.isSuccessful) {  // opens to the next screen if password and email are correct
                        val intent = Intent(this, MainActivity::class.java)
                        startActivity(intent)
                        finish()
                    } else {
                        Toast.makeText(this, "Authentication Failed", Toast.LENGTH_SHORT).show() // returns an error if wrong
                    }
                }
            } else {
                Toast.makeText(this, "Fields cannot be empty", Toast.LENGTH_SHORT).show() // returns an error if fields are empty
            }
        }
    }
```

This was because at the end of my login activity I used the finish() command which effectively destroys the instance of the activity, meaning that it cannot be reopened after this has been called, this meant that the app would crash because it would be told to go somewhere which was not possible. Therefore, even though it may use a little more memory keeping the login screen active, I removed this command and the logout button worked as intended. Testing Code: 2.10

Profile Display and Logout Button Testing Video

Test ID 2	Timestamp	PURPOSE	DATA	EXPECTED RESULTS	ACTUAL RESULTS
2.08	0:03	To test whether the data from firebase will be read by the code and displayed where needed	n/a as data comes from firebase	Data is displayed accordingly	Pass
2.09	0:04	To test whether the login button works by logging the user out of their session and returning to the initial log in screen	n/a	The user is logged out, a message is displayed and they are sent back to the login screen	Failed, app crashed after the user was logged out
2.10	0:13	To test whether the login button works by logging the user out of their session and returning to the initial log in screen	n/a	The user is logged out, a message is displayed and they are sent back to the login screen	Pass

Module Summary

The Profile Module serves as a streamlined interface for users to view and manage their basic account information. It provides a clean display of the user's name and email address retrieved from Firebase, along with essential account management functionality through a logout button. This minimalist approach creates a clear separation between basic account management and the personal information features elsewhere in the application, ensuring users can quickly access essential account functions when needed. Meaning the first module of the prototype is accessible and allows for the user to change their accounts at will.

Personal Information: Personal Information Display

This section of the personal details' module will be the screen that allows the user to view and change their details, depending on situations, etc such as having lost weight or in case of accidentally having put an incorrect detail when signing up. This will also be a screen that will allow the user to recalculate their calories based on the details they have entered.

XMLS

fragment_personal_info.xml

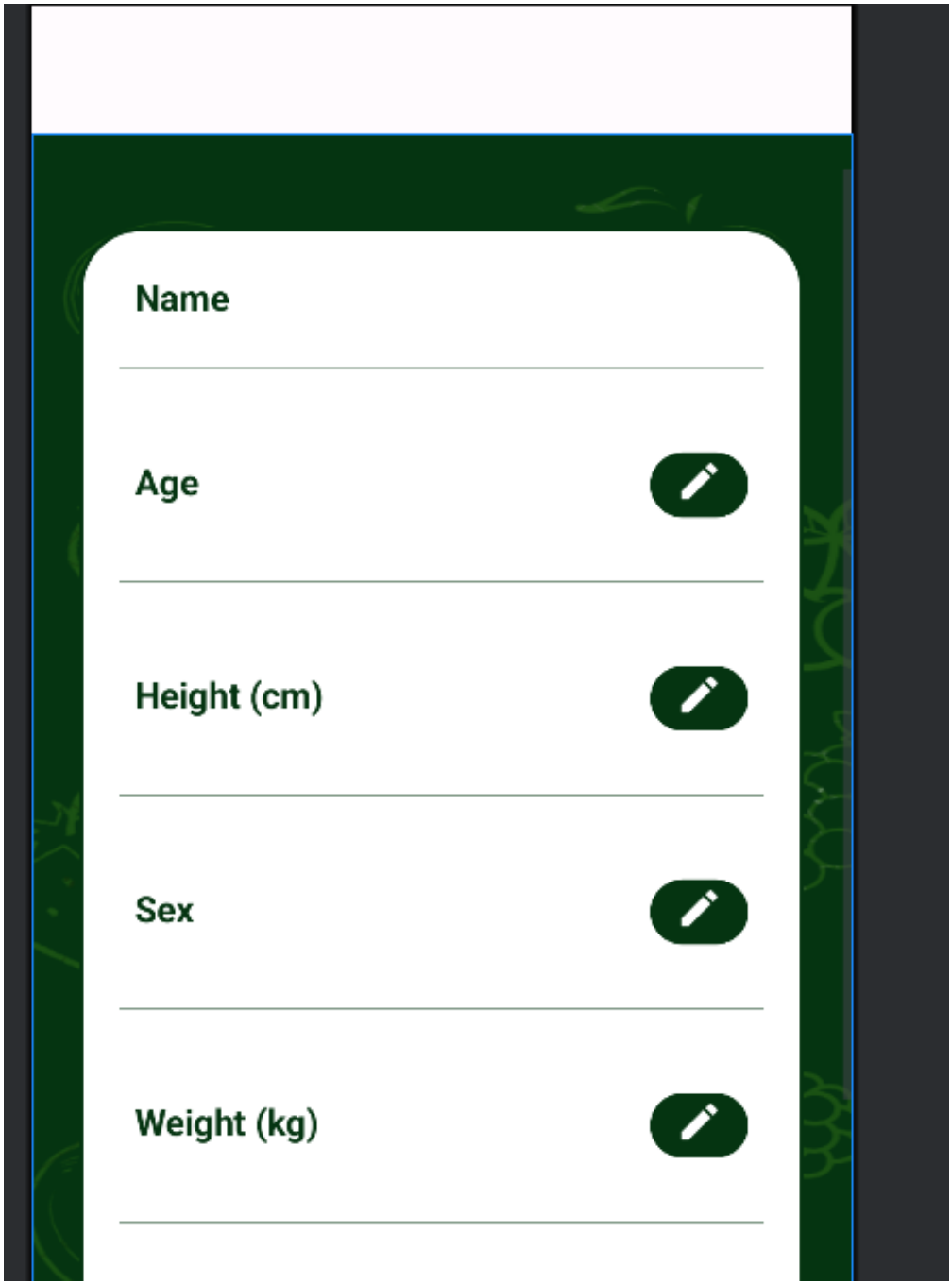
```xml
<?xml version="1.0" encoding="utf-8"?>
<ScrollView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true"
    android:scrollbars="vertical"
    android:layout_marginTop="?attr/actionBarSize"
    android:layout_marginBottom="70dp"
    android:background="@drawable/initialscreen">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:padding="10dp"
            android:layout_gravity="center"
            android:layout_marginTop="70dp"
            android:layout_marginStart="24dp"
            android:layout_marginEnd="24dp"
            android:background="@drawable/green_border">

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="vertical">

                <LinearLayout
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:padding="8dp"
                    android:orientation="horizontal"
                    android:layout_marginTop="5dp"
                    android:layout_marginEnd="10dp"
                    android:layout_marginStart="10dp"
                    android:layout_marginBottom="5dp">

                    <TextView
                        android:layout_width="0dp"
                        android:layout_height="wrap_content"
                        android:layout_weight="1"
                        android:text="Name"
                        android:textStyle="bold"
                        android:textColor="@color/darkGreen"
                        android:textSize="18sp" />
                    <TextView
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:id="@+id/readName"
                        android:text=""
                        android:layout_weight="1"
                        android:textAlignment="viewEnd"
                        android:textColor="@color/darkGreen"
                        android:textSize="18sp"/>

                </LinearLayout>

                <androidx.appcompat.widget.AppCompatImageView
                    android:layout_width="match_parent"
                    android:layout_height="1dp"
                    android:layout_margin="10dp"
                    android:alpha="0.5"
                    android:background="@color/darkGreen"/>

                <LinearLayout
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:padding="8dp"
                    android:orientation="horizontal"
                    android:layout_marginTop="20dp"
                    android:layout_marginEnd="10dp"
                    android:layout_marginStart="10dp"
                    android:layout_marginBottom="10dp">
```

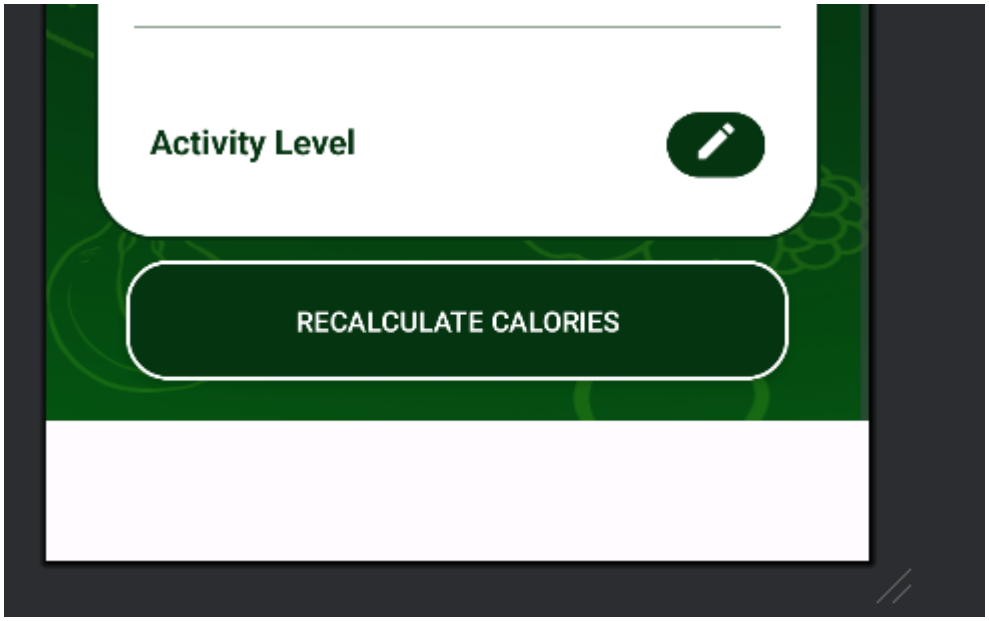
```
        <TextView
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Age"
            android:textStyle="bold"
            android:textColor="@color/darkGreen"
            android:textSize="18sp" />
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:id="@+id/readAge"
            android:text=""
            android:layout_weight="1"
            android:textAlignment="viewEnd"
            android:layout_marginEnd="20dp"
            android:textColor="@color/darkGreen"
            android:textSize="18sp" />

        <Button
            android:layout_width="49dp"
            android:layout_height="40dp"
            android:backgroundTint="@color/darkGreen"
            android:drawableBottom="@drawable/baseline_edit_24"
            android:id="@+id/btnEditAge"/>

    </LinearLayout>

    <androidx.appcompat.widget.AppCompatImageView
        android:layout_width="match_parent"
        android:layout_height="1dp"
        android:layout_margin="10dp"
        android:alpha="0.5"
        android:background="@color/darkGreen"/>

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:padding="8dp"
        android:orientation="horizontal"
        android:layout_marginTop="20dp"
        android:layout_marginEnd="10dp"
        android:layout_marginStart="10dp"
        android:layout_marginBottom="10dp">

        <TextView
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Height (cm)"
            android:textStyle="bold"
            android:textColor="@color/darkGreen"
            android:textSize="18sp" />
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:id="@+id/readHeight"
            android:text=""
            android:layout_weight="1"
            android:textAlignment="viewEnd"
            android:layout_marginEnd="20dp"
            android:textColor="@color/darkGreen"
            android:textSize="18sp" />

        <Button
            android:layout_width="49dp"
            android:layout_height="40dp"
            android:backgroundTint="@color/darkGreen"
            android:drawableBottom="@drawable/baseline_edit_24"
            android:id="@+id/btnEditHeight"/>
```

```
151
152             </LinearLayout>
153
154             <androidx.appcompat.widget.AppCompatImageView
155                 android:layout_width="match_parent"
156                 android:layout_height="1dp"
157                 android:layout_margin="10dp"
158                 android:alpha="0.5"
159                 android:background="@color/darkGreen"/>
160
161             <LinearLayout
162                 android:layout_width="match_parent"
163                 android:layout_height="wrap_content"
164                 android:padding="8dp"
165                 android:orientation="horizontal"
166                 android:layout_marginTop="20dp"
167                 android:layout_marginEnd="10dp"
168                 android:layout_marginStart="10dp"
169                 android:layout_marginBottom="10dp">
170
171                 <TextView
172                     android:layout_width="0dp"
173                     android:layout_height="wrap_content"
174                     android:layout_weight="1"
175                     android:text="Sex"
176                     android:textStyle="bold"
177                     android:textColor="@color/darkGreen"
178                     android:textSize="18sp" />
179                 <TextView
180                     android:layout_width="wrap_content"
181                     android:layout_height="wrap_content"
182                     android:id="@+id/readSex"
183                     android:text=""
184                     android:layout_weight="1"
185                     android:textAlignment="viewEnd"
186                     android:layout_marginEnd="20dp"
187                     android:textColor="@color/darkGreen"
188                     android:textSize="18sp" />
189
190                 <Button
191                     android:layout_width="49dp"
192                     android:layout_height="40dp"
193                     android:backgroundTint="@color/darkGreen"
194                     android:drawableBottom="@drawable/baseline_edit_24"
195                     android:id="@+id/btnEditSex"/>
196
197             </LinearLayout>
198
199             <androidx.appcompat.widget.AppCompatImageView
200                 android:layout_width="match_parent"
201                 android:layout_height="1dp"
202                 android:layout_margin="10dp"
203                 android:alpha="0.5"
204                 android:background="@color/darkGreen"/>
205
206             <LinearLayout
207                 android:layout_width="match_parent"
208                 android:layout_height="wrap_content"
209                 android:padding="8dp"
210                 android:orientation="horizontal"
211                 android:layout_marginTop="20dp"
212                 android:layout_marginEnd="10dp"
213                 android:layout_marginStart="10dp"
214                 android:layout_marginBottom="10dp">
215
216                 <TextView
217                     android:layout_width="0dp"
218                     android:layout_height="wrap_content"
219                     android:layout_weight="1"
220                     android:text="Weight (kg)"
221                     android:textStyle="bold"
222                     android:textColor="@color/darkGreen"
223                     android:textSize="18sp" />
224                 <TextView
225                     android:layout_width="wrap_content"
226                     android:layout_height="wrap_content"
```

```
                    android:textAlignment="viewEnd"
                    android:layout_marginEnd="20dp"
                    android:textColor="@color/darkGreen"
                    android:textSize="18sp" />

                <Button
                    android:layout_width="49dp"
                    android:layout_height="40dp"
                    android:backgroundTint="@color/darkGreen"
                    android:drawableBottom="@drawable/baseline_edit_24"
                    android:id="@+id/btnEditWeight"/>

            </LinearLayout>

            <androidx.appcompat.widget.AppCompatImageView
                android:layout_width="match_parent"
                android:layout_height="1dp"
                android:layout_margin="10dp"
                android:alpha="0.5"
                android:background="@color/darkGreen"/>

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:padding="8dp"
                android:orientation="horizontal"
                android:layout_marginTop="20dp"
                android:layout_marginEnd="10dp"
                android:layout_marginStart="10dp"
                android:layout_marginBottom="10dp">

                <TextView
                    android:layout_width="0dp"
                    android:layout_height="wrap_content"
                    android:layout_weight="1"
                    android:text="Activity Level"
                    android:textStyle="bold"
                    android:textColor="@color/darkGreen"
                    android:textSize="17sp" />
                <TextView
                    android:layout_width="0dp"
                    android:layout_height="wrap_content"
                    android:id="@+id/readActivityLevels"
                    android:text=""
                    android:layout_weight="1"
                    android:textAlignment="viewEnd"
                    android:layout_marginEnd="20dp"
                    android:textColor="@color/darkGreen"
                    android:textSize="18sp" />

                <Button
                    android:layout_width="49dp"
                    android:layout_height="40dp"
                    android:backgroundTint="@color/darkGreen"
                    android:drawableBottom="@drawable/baseline_edit_24"
                    android:id="@+id/btnEditActivityLevel"/>

            </LinearLayout>
        </LinearLayout>
    </LinearLayout>

    <androidx.appcompat.widget.AppCompatButton
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="10dp"
        android:layout_marginEnd="40dp"
        android:layout_marginStart="40dp"
        android:layout_marginBottom="20dp"
        android:text="RECALCULATE CALORIES"
        android:padding="20dp"
        android:textColor="@color/white"
        android:background="@drawable/darkgreen_round_white_outline"
        android:id="@+id/btnRecalculateCalories"/>
</LinearLayout>
</ScrollView>
```

This xml defines the look of the personal details module, each item will have a corresponding id text view which is currently blank as there is no text to display, and there is a scrollview used so that no matter what phone the user uses they will be able to access the whole page, regardless of size.





The user will be able to edit each option by the use of the buttons created and able to recalculate everything using the button below

Logic

Viewing and changing personal info data

```
class PersonalInfoFragment : Fragment() {

    private var _binding: FragmentPersonalInfoBinding? = null
    private val binding get() = _binding!! // safe access to binding
    private lateinit var databaseReference: DatabaseReference
    private lateinit var firebaseAuth: FirebaseAuth

    private var name = ""
    private var age = 0
    private var height = 0
    private var weight = 0
    private var sex = ""
    private var alevels = ""

    private var calories = 0.0
    private var proteinpercent = 0.0
    private var fatspercent = 0.0
    private var carbspercent = 0.0


    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        // Inflate the layout for this fragment
        _binding = FragmentPersonalInfoBinding.inflate(inflater, container, false)
        val userid = FirebaseAuth.getInstance().currentUser?.uid
```

Then once again I set up view binding to link the xml created to the class and set up the firebase instance to be able to fetch from firebase.

```
userid = FirebaseAuth.getInstance().currentUser?.uid

if (userid != null) {
    databaseReference =
        FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.europe-west1.firebasedatabase.app/")
            .getReference("Information").child(userid)
    databaseReference.get().addOnSuccessListener { snapshot ->
        if (snapshot.exists()) {
            name = snapshot.child("personal information").child("name").getValue()
            age = snapshot.child("personal information").child("age").getValue()
            height = snapshot.child("personal information").child("height").getValue()
            sex = snapshot.child("personal information").child("sex").getValue()
            weight = snapshot.child("personal information").child("weight").getValue()
            alevels = snapshot.child("personal information").child("activityLevel").getValue()

            binding.readName.text = name.toString()
            binding.readAge.text = age.toString()
            binding.readHeight.text = height.toString()
            binding.readSex.text = sex.toString()
            binding.readWeight.text = weight.toString()
            binding.readActivityLevels.text = alevels.toString()
        } else {
            Toast.makeText(requireContext(), "No data found", Toast.LENGTH_SHORT).show()
        }
    }.addOnFailureListener {
        Toast.makeText(requireContext(), "Something went wrong", Toast.LENGTH_SHORT).show()
    }
```

Next, I fetched the database reference and updated most of the variables created earlier to be ready to use further on in the code, Then I updated all the text views with the respective values based on their id to appear in the app for the user. This means that this will be the first code ran when the function is opened, so the data should load very quickly. Testing Code: 2.10

Again, the failure messages are simply in case of an accident to make the code more robust, such as the user being logged in, and suddenly disconnecting from the internet and trying to load the data.

```
private fun updateUI(option: String, newValue: String) {
    when (option) {
        "name" -> binding.readName.text = newValue
        "age" -> binding.readAge.text = newValue
        "height" -> binding.readHeight.text = newValue
        "sex" -> binding.readSex.text = newValue
        "weight" -> binding.readWeight.text = newValue
        "activityLevel" -> binding.readActivityLevels.text = newValue
    }
}
```

This function updates the UI so the user can see the changes they have made as soon as they have made them.

This also means that the variables created earlier will need to change to incorporate any changed values from the user, so that the user does not perform calculations using old values.

```
private fun updatevariableints(option: String, newValue: Int) {
    when (option) {
        "age" -> age = newValue
        "height" -> height = newValue
        "weight" -> weight = newValue
    }
}

private fun updatevariablestrings(option: String, newValue: String) {
    when (option) {
        "sex" -> sex = newValue
        "activityLevel" -> alevels = newValue
    }
}
```

Therefore, two more separate functions update the variables in the activity using the same option and new value as before, they are separate as when attempting to create a singular function, the logcat kept calling an error where one variable expects a string and another expects an integer, but the function can only take in the same value as an integer or a string, not both. Testing Code: 2.13

```
private fun call_DetailChange(option: String) { // listens to see if forgot password text is clicked
    val builder = AlertDialog.Builder(requireContext())
    val view = layoutInflater.inflate(R.layout.dialog_changeinfo, null) // calls dialog_changeinfo xml
    val changedetail = view.findViewById<EditText>(R.id.editBoxDetails)
    builder.setView(view)
    val dialog = builder.create()
    view.findViewById<Button>(R.id.btnChangeDetail).setOnClickListener {
        val userid = FirebaseAuth.getInstance().currentUser?.uid

        if (userid != null){
            val detailText = changedetail.text.toString()
```

I then created a function that will take in a string that will determine the detail that is changed. This code inflates the *dialog_changeinfo.xml* so that the user can enter a new detail that will overwrite an existing one. Creating the code in a function is justified because when the user presses a button, the function will be called with the corresponding string attached, whereas without the function the code for each button would have to be rewritten several times, wasting both memory and development time.

```
        if (userid != null){
            val detailText = changedetail.text.toString()
            updateDetail(option,detailText)
            dialog.dismiss()
        } else{
            Toast.makeText(requireContext(),"Failed",Toast.LENGTH_SHORT).show()
            dialog.dismiss()
        }

    }
    view.findViewById<Button>(R.id.btnCancelDetails).setOnClickListener {
        dialog.dismiss() // cancels operation
    }
    if (dialog.window != null) {
        dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
    }
    dialog.show()
}
```

Since I knew that the code will need to update variables for a different function too, which will be created after, for the sex and activity levels, I wanted to have another function that the dialog would call, *updateDetail* that would change the details for both functions, depending on what was needed. Since the values for detailText here would need to be changed into an integer to be placed into firebase for the age, height or weight this proved a problem

```
private fun updatedetail(option: String, detailText: String){
        if (detailText.isNotEmpty()) {
            try {
                val detailInt = detailText.toInt()

                // Update the database with the integer value
                databaseReference.child("personal information").child(option).setValue(detailText)
                    .addOnSuccessListener {
                        updateUI(option,detailText)
                        if (option == "age" || option == "height" || option == "weight"){
                            updatevariableints(option,detailInt)
                        } else {
                            updatevariablestrings(option, detailText)
                        }


                    }.addOnFailureListener {
                        Toast.makeText(requireContext(), "Failed to save", Toast.LENGTH_SHORT).show()
                    }
            } catch (e: NumberFormatException) {
                // Handle invalid number input
                Toast.makeText(requireContext(), "Please enter a valid number", Toast.LENGTH_SHORT).show()
            }
        } else {
            Toast.makeText(requireContext(), "Field cannot be empty", Toast.LENGTH_SHORT).show()
        }
    }
}
```

This was the function I created to change the details for hopefully all the values, however since I knew that sex and activity levels would have to be strings and cannot be sent to the database as integers, I could not figure out how I would use the database call with two different variables without using if statements, however this would require me to call the database and all the other code twice regardless, so doing it in a separate function would actually use more code than just calling the database reference twice in the two functions I would create. Testing Code 2.13

```
if (option == "age" || option == "height" || option == "weight"){
    detailText = detailText.toInt()
}

// Update the database with the integer value
databaseReference.child("personal information").child(option).setValue(detailText)
    .addOnSuccessListener {
        updateUI(option,detailText)
        if (option == "age" || option == "height" || option == "weight"){
            updatevariableints(option,detailText)
        } else {
            updatevariablestrings(option, detailText)
        }
```

I also tried changing the detailText value to an integer during runtime so that the detailText value would upload an integer if the option needed to, however once an variable of a certain type is assigned, it cannot be changed to a different type during runtime in kotlin. Testing Code: 2.13

```
private fun call_DetailChange(option: String) { // listens to see if forgot password text is clicked
    val builder = AlertDialog.Builder(requireContext())
    val view = layoutInflater.inflate(R.layout.dialog_changeinfo, null) // calls dialog_changeinfo xml
    val changedetail = view.findViewById<EditText>(R.id.editBoxDetails)
    builder.setView(view)
    val dialog = builder.create()
    view.findViewById<Button>(R.id.btnChangeDetail).setOnClickListener {
        val userid = FirebaseAuth.getInstance().currentUser?.uid

        if (userid != null){
            val detailText = changedetail.text.toString()
            if (detailText.isNotEmpty()) {
                try {
                    // Convert to integer
                    val detailValue = detailText.toInt()

                    // Update the database with the integer value
                    databaseReference.child("personal information").child(option).setValue(detailValue)
                        .addOnSuccessListener {
                            changedetail.text.clear()
                            updateUI(option,detailText)
                            updatevariableints(option, detailValue)
                            dialog.dismiss()
                        }.addOnFailureListener {
                            Toast.makeText(requireContext(), "Failed to save", Toast.LENGTH_SHORT).show()
                        }
                } catch (e: NumberFormatException) {
                    // Handle invalid number input
                    Toast.makeText(requireContext(), "Please enter a valid number", Toast.LENGTH_SHORT).show()
                }
            } else {
                Toast.makeText(requireContext(), "Field cannot be empty", Toast.LENGTH_SHORT).show()
            }
        } else{
            Toast.makeText(requireContext(),"Failed",Toast.LENGTH_SHORT).show()
            dialog.dismiss()
        }

    }
```

```
    view.findViewById<Button>(R.id.btnCancelDetails).setOnClickListener {
        dialog.dismiss() // cancels operation
    }
    if (dialog.window != null) {
        dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
    }
    dialog.show()
}
```

Therefore I opted to place the database code and its validation into the call_changedetail function to be specific for what is being changed (integer variables)

This validates if the user has entered a new detail, and pressed the “change” button, if they have the database changes the value based on the value the user has inputted, changing it from a string to an integer to be uploaded to firebase and then the *updateUI* and *updatevariablesints* functions are called so that the activity has a changed UI as well as changed variables.

If the user has not entered a correct value, they will be asked to input a correct value.

This means that the user cannot change a string value, therefore I will be making the name they entered at the start permanent. If the user does not wish to change the value, they have the option to press the “cancel” button and the dialog will be dismissed. Testing Code: also 2.13

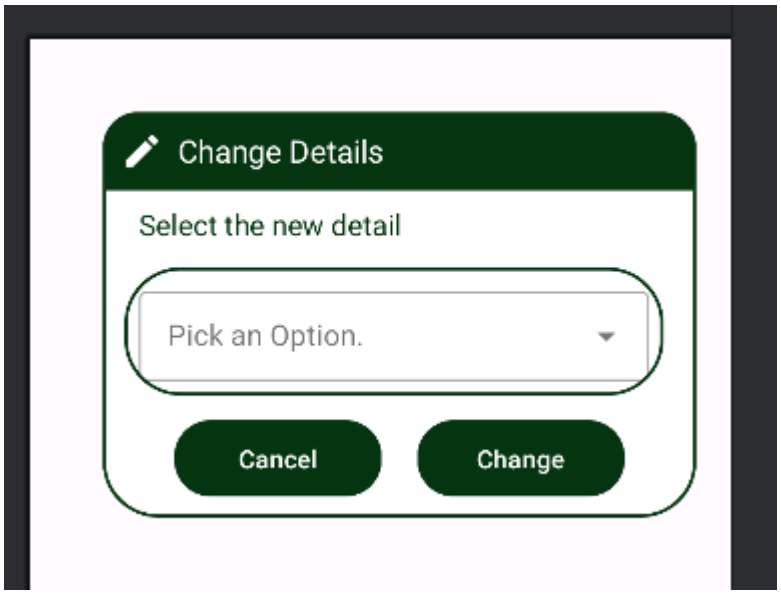
```
binding.btnEditAge.setOnClickListener{
    call_DetailChange("age")
}

binding.btnEditHeight.setOnClickListener{
    call_DetailChange("height")
}
binding.btnEditWeight.setOnClickListener{
    call_DetailChange("weight")
}
```

This function can then be called from the main fragment code, taking care that the strings entered match the name of the strings in the database.

However the user may still want to change their activity levels or their sex in case they entered the wrong one, or their activity levels have changed, this means that the *dialog_changeinfo.xml* will have to have a second dialog created called *dialog_changeinfo_dropdown.xml* where the edit text box is changed to a dropdown option input.

So the code in the new xml is copied from the dialog before, and the edit text box code is removed and replaced with the following xml code:

```
<com.google.android.material.textfield.TextInputLayout
    android:id="@+id/editDetailsDropdown"
    style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox.ExposedDropdownMenu"
    android:layout_width="300dp"
    android:layout_height="wrap_content"
    android:layout_marginStart="12dp"
    android:layout_marginTop="30dp"
    android:layout_marginBottom="30dp"
    android:background="@drawable/green_border"
    android:padding="8dp"
    android:textSize="16sp"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/ChangeTitle"
    app:layout_constraintVertical_bias="0.25">

    <AutoCompleteTextView
        android:id="@+id/Detail_input_dropdown"
        android:layout_width="match_parent"
        android:layout_height="50dp"
        android:layout_weight="1"
        android:drawablePadding="8dp"
        android:hint="Pick an Option."
        android:inputType="none"
        android:textColor="@color/grey" />

</com.google.android.material.textfield.TextInputLayout>
```



```
private fun call_detailChange_dropdown(option: String){
    val builder = AlertDialog.Builder(requireContext())
    val view = layoutInflater.inflate(R.layout.dialog_changeinfo_dropdown, null) // calls dialog_forgot xml
    val changedetail = view.findViewById<AutoCompleteTextView>(R.id.Detail_input_dropdown)

    if (option == "sex"){
        val sexes = resources.getStringArray(R.array.Sexes)  // sets up drop down for sexes option
        val sexadapter = ArrayAdapter(requireContext(),R.layout.dropdown_item,sexes)
        changedetail.setAdapter(sexadapter)
    }else{
        val activitylevels = resources.getStringArray(R.array.Activity_Levels)  // sets up drop down for activity levels option
        val activityleveladapter = ArrayAdapter(requireContext(),R.layout.dropdown_item,activitylevels)
        changedetail.setAdapter(activityleveladapter)
    }

    builder.setView(view)
```

The start of the function to change the sex and activity options starts the same way by inflating a dialog, this time it is the *dialog_changeinfodropdown.xml*, however this time the code to create the dropdown options using the same method as the one in the personal details input.

```
    view.findViewById<Button>(R.id.btnChangeDetailDropdown).setOnClickListener {
        val userid = FirebaseAuth.getInstance().currentUser?.uid

        if (userid != null){

            var detailText = changedetail.text.toString()
            if (detailText.isNotEmpty()) {
                try {

                    // Update the database with the integer value
                    databaseReference.child("personal information").child(option).setValue(detailText)
                        .addOnSuccessListener {
                            updateUI(option,detailText)
                            updatevariablestrings(option, detailText)

                        }.addOnFailureListener {
                            Toast.makeText(requireContext(), "Failed to save", Toast.LENGTH_SHORT).show()
                        }
                } catch (e: NumberFormatException) {
                    // Handle invalid number input
                    Toast.makeText(requireContext(), "Please enter a valid number", Toast.LENGTH_SHORT).show()
                }
            } else {
                Toast.makeText(requireContext(), "Field cannot be empty", Toast.LENGTH_SHORT).show()
            }

        }else {
            Toast.makeText(requireContext(), "Failed", Toast.LENGTH_SHORT).show()
            dialog.dismiss()
        }

        dialog.dismiss()
    }
    view.findViewById<Button>(R.id.btnCancelDetailDropdown).setOnClickListener {
        dialog.dismiss() // cancels operation
    }
    if (dialog.window != null){
        dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
    }
    dialog.show()
```

Otherwise, the code follows the exact same steps and validation as the code to change the details

before, with the removed conversion of the detail to be changed to an integer, since only a string is needed, and instead of *updatevariableints* being called, *updatevariablestrings* is called.

```
binding.btnEditSex.setOnClickListener{
    call_detailChange_dropdown("sex")
}

binding.btnEditActivityLevel.setOnClickListener(){
    call_detailChange_dropdown("activityLevel")
}
```

Therefore two more bindings are created to listen for the specific buttons pressed, and the function is called with the corresponding string. Testing Code for up to this point: also 2.13

Recalculating Calories

The last part of the code was to create a way for the user to calculate their calories based on any changes they have made, which is the RECALCULATE CALORIES button.

Once again to keep things simple I will have the binding call a function so that the fragment code is not overcrowded

```
binding.btnRecalculateCalories.setOnClickListener{
    recalculatecalories()
}
```

```
private fun recalculatecalories() {

    val userid = FirebaseAuth.getInstance().currentUser?.uid

    if (userid != null) {
        databaseReference =
            FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.europe-west1.firebasedatabase.app/")
                .getReference("Information").child(userid)
        databaseReference.get().addOnSuccessListener { snapshot ->
            proteinpercent = snapshot.child("personal information").child("proteinpercent")
                .getValue(Double::class.java) ?: 0.0
            fatspercent = snapshot.child("personal information").child("fatspercent")
                .getValue(Double::class.java) ?: 0.0
            carbspercent = snapshot.child("personal information").child("carbspercent")
                .getValue(Double::class.java) ?: 0.0
            calories = snapshot.child("personal information").child("calories").getValue(Double::class.java) ?: 0.0
            val personalInfo = PersonalInfo(
                age = age,
                height = height,
                weight = weight,
                sex = sex,
                activityLevel = alevels,
                proteinpercent = proteinpercent, // this ensures that if the user has changed their %, then it wont be reset here
                fatspercent = fatspercent,
                carbspercent = carbspercent
            )

            personalInfo.calculatecalories()
```

This function recalculates a user's caloric needs and macronutrient targets based on their current personal information.

It first retrieves the user's stored macro percentages from Firebase to preserve any custom splits they've set (daily goals variable created later) and updates the variables created earlier with these values.

Then it creates a new PersonalInfo object with the user's current metrics and calls the calculatecalories() function in the personal info class from earlier to calculate the calories and deficit calories.

```
            if (calories == personalInfo.calories){
                Toast.makeText(requireContext(),"No need to recalculate", Toast.LENGTH_SHORT).show()
            } else{
                personalInfo.calculatemacros()

                val updates = mapOf(
                    "personal information/calories" to personalInfo.calories,
                    "personal information/deficitcalories" to personalInfo.deficitcalories,
                    "personal information/protein" to personalInfo.protein,
                    "personal information/fats" to personalInfo.fats,
                    "personal information/carbs" to personalInfo.carbs,
                )
                databaseReference.updateChildren(updates).addOnSuccessListener {
                    showConfirmationDialog(personalInfo.calories, personalInfo.deficitcalories)
                }.addOnFailureListener {
                    Toast.makeText(requireContext(), "Failed to ReCalculate", Toast.LENGTH_SHORT).show()
                }.addOnFailureListener {
                    Toast.makeText(
                        requireContext(),
                        "Failed to load macronutrient percentages",
                        Toast.LENGTH_SHORT
                    ).show()
                }
            }
        }
    }
```

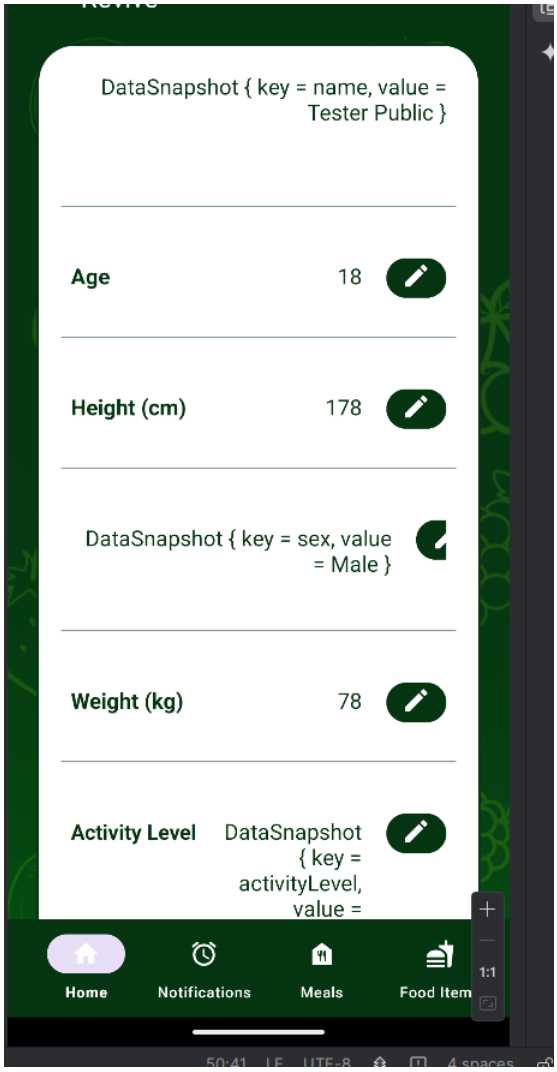
After calculating the new calorie needs, it checks if they're different from the current stored calories. If there's no change, it notifies the user there's no need to recalculate and does not perform any changes.

If the calories have changed, it calculates new macro targets and updates Firebase with the new calories, deficit calories, and macro amounts, this is done by using a mapOf function which allows me to select certain values from the object created and send only those to update firebase, making sure that no other values are changed on accident.

Upon successful update, it shows a confirmation dialog with the new calorie values, or displays error messages if the update fails. Testing Code: 2.15

Testing

During testing, when the app retrieved the data from firebase it had retrieved it in a form unintended and unreadable for the user and the screen looked a little like this, Testing Code: 2.11



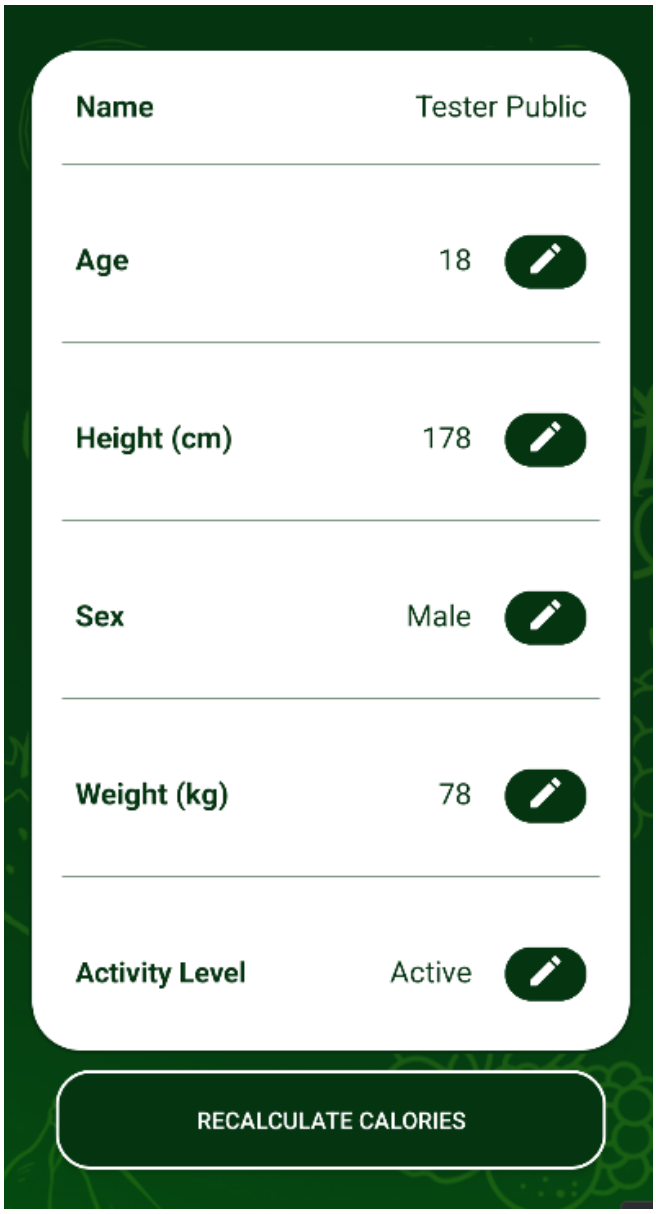

```
if (snapshot.exists()) {
    name = snapshot.child("personal information").child("name").getValue()
    age = snapshot.child("personal information").child("age").getValue()
    height = snapshot.child("personal information").child("height").getValue()
    sex = snapshot.child("personal information").child("sex").getValue()
    weight = snapshot.child("personal information").child("weight").getValue()
    alevels = snapshot.child("personal information").child("activityLevel").getValue()
```

I figured this issue was due to the code where I had retrieved the values from firebase because getValue retrieves a datasnapshot of the values that are stored there.

```
if (snapshot.exists()) {
    name = snapshot.child("personal information").child("name").getValue(String::class.java) ?: ""
    age = snapshot.child("personal information").child("age").getValue(Int::class.java) ?:0
    height = snapshot.child("personal information").child("height").getValue(Int::class.java) ?:0
    sex = snapshot.child("personal information").child("sex").getValue(String::class.java) ?: ""
    weight = snapshot.child("personal information").child("weight").getValue(Int::class.java) ?:0
    alevels = snapshot.child("personal information").child("activityLevel").getValue(String::class.java) ?: ""
```

So to solve this issue I specified the exact value that I want to retrieve and in what form I want it to

be retrieved, so that it can be updated to the activity as intended, and this caused the screen to display values just as intended. Testing Code: 2.12



Personal Info Display and Change testing video

Test ID 2	Timestamp	PURPOSE	DATA	EXPECTED RESULTS	ACTUAL RESULTS
2.11	n/a	To test whether the information shows up in the app when the user is on the personal information screen.	n/a data already in firebase	All data shows up in the correct form and is displayed as intended	Fail, incorrect strings and snapshots show up
2.12	0:00	To test whether the information shows up in the app when the user is	n/a data already in firebase	All data shows up in the correct form	As expected

		on the personal information screen.		and is displayed as intended	
2.13	0:03	To test whether the user is able to change the information, and if the value they have entered also gets updated to firebase. Then to test whether the UI will be updated afterwards	A new value for each item: Age: 18 Height: 180 Weight: 80 Sex: Female Activity Levels: Very High	As soon as the user pressed the "change" button the firebase value changes, and the value shown in the app also changes to the new number. Then UI is updated.	As expected.
2.14	0:29	To test whether the user is able to enter other information other than the information specific to what the program will allow.	Attempting to enter: String value Float value Comma Value	The app should not even give an option to enter a string, and the float and comma should not appear in the text box.	As expected.
2.15	0:37	Testing whether the data is recalculated when the recalculate calories button is pressed and then reupdated back in firebase.	New values for data: Age: 18 Height: 180 Sex: Female Weight: 80 Activity Level: Very Active	Calories should be recalculated and a dialog should pop up informing the user of their new calories, then data should be saved to firebase.	As expected

Module Summary

The Personal Information Display module allows users to view and modify their personal fitness metrics after initial setup. The prototype thus far displays the user's age, height, weight, sex, and activity level, along with their calculated target calories and macronutrient goals. Users can edit each value through dedicated buttons that trigger custom dialogs, with changes automatically recalculating their caloric needs using the Harris-Benedict Formula. The module validates all user input and updates both the UI and Firebase database to maintain data consistency. This design approach is justified as it gives users full control over their fitness parameters while maintaining data accuracy and providing a clear understanding of how changes to their personal information affect

their nutritional targets. All modifications are seamlessly synchronized with Firebase, ensuring data persistence across sessions and devices.

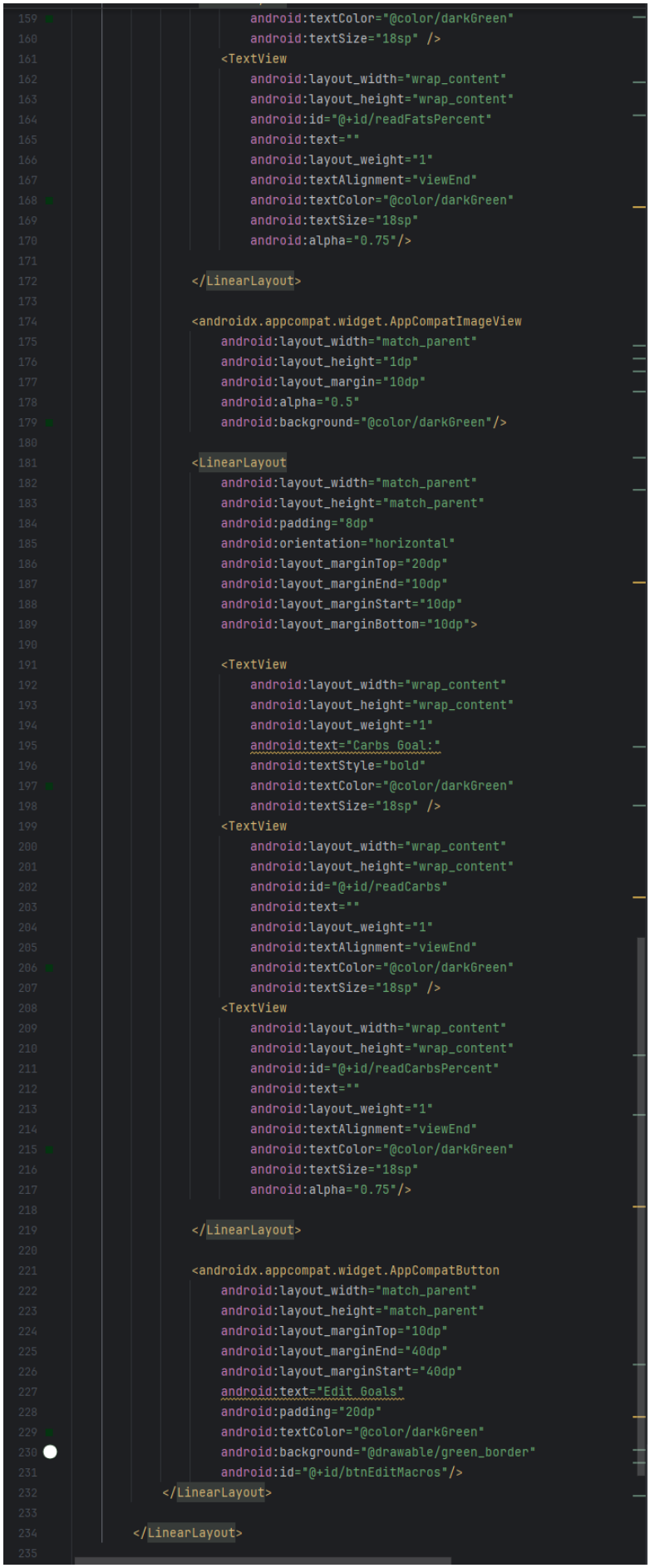
Personal Information: Setting and Changing Daily Goals

This will be the part of the personal information module that will allow the user to view and change their calorie goal if they do not want the same goal as the app has recommended for them, as well as change their percentage split for the macros they would like to have in a diet, this was previously set by default to be a 30-40-30 split for proteins-carbs-fats when the user entered their information into the personal information input section as a recommendation. But of course, not everyone may want the default selection, and they may want to change this, therefore the DailyGoals module will let them do that, as well as see their current split.

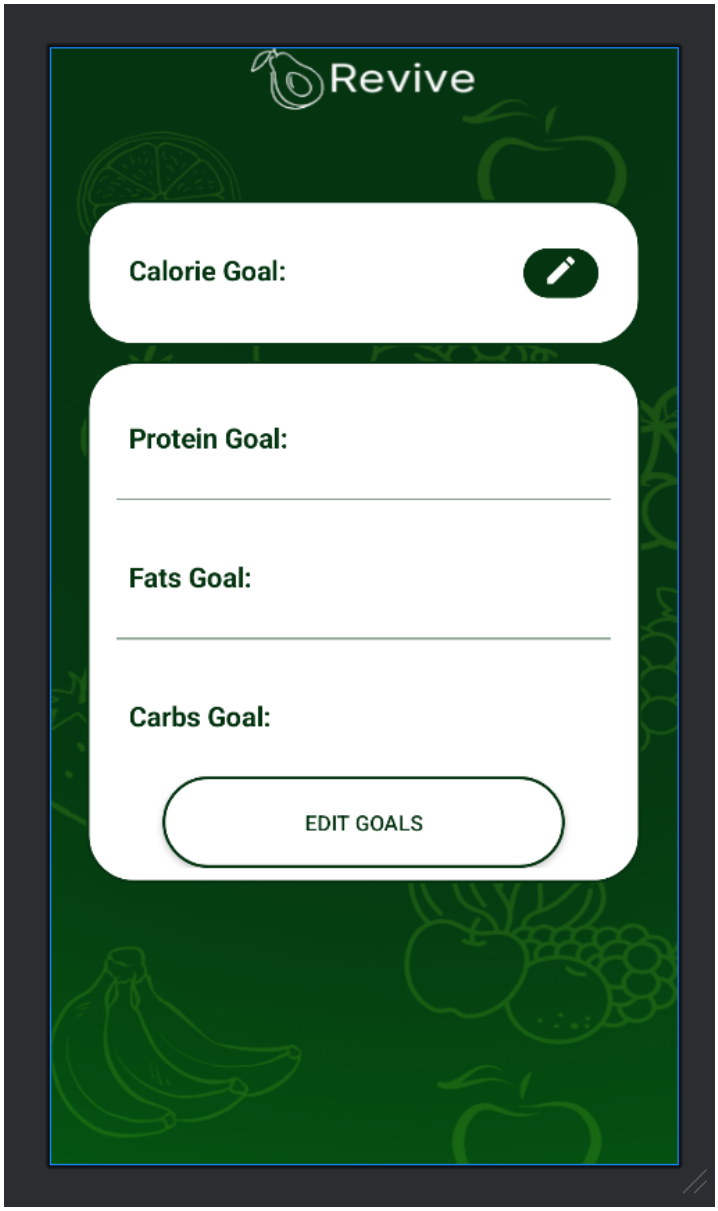
Daily Goals XML

```
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schem
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".ProfileFragment"
    android:background="@drawable/initialscreen">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:padding="10dp"
            android:layout_gravity="center"
            android:layout_marginTop="100dp"
            android:layout_marginStart="24dp"
            android:layout_marginEnd="24dp"
            android:background="@drawable/green_border">

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="vertical">

                <LinearLayout
                    android:layout_width="match_parent"
                    android:layout_height="match_parent"
                    android:padding="8dp"
                    android:orientation="horizontal"
                    android:layout_marginTop="10dp"
                    android:layout_marginEnd="10dp"
                    android:layout_marginStart="10dp"
                    android:layout_marginBottom="10dp">

                    <TextView
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:layout_weight="1"
                        android:text="Calorie Goal:"
                        android:textStyle="bold"
                        android:textColor="@color/darkGreen"
                        android:textSize="18sp" />
                    <TextView
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:id="@+id/readCalories"
                        android:text=""
                        android:layout_weight="1"
                        android:textAlignment="viewStart"
                        android:textColor="@color/darkGreen"
                        android:textSize="18sp"/>

                    <Button
                        android:layout_width="49dp"
                        android:layout_height="40dp"
                        android:backgroundTint="@color/darkGreen"
                        android:drawableBottom="@drawable/baseline_edit_24"
                        android:id="@+id/btnEditCalories"/>

                </LinearLayout>

            </LinearLayout>

        </LinearLayout>


        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:padding="10dp"
            android:layout_gravity="center"
            android:layout_marginTop="10dp"
            android:layout_marginStart="24dp"
            android:layout_marginEnd="24dp"
            android:background="@drawable/green_border">
```

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:padding="8dp"
        android:orientation="horizontal"
        android:layout_marginTop="20dp"
        android:layout_marginEnd="10dp"
        android:layout_marginStart="10dp"
        android:layout_marginBottom="10dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Protein Goal:"
            android:textStyle="bold"
            android:textColor="@color/darkGreen"
            android:textSize="18sp" />
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:id="@+id/readProtein"
            android:text=""
            android:layout_weight="1"
            android:textAlignment="viewEnd"
            android:textColor="@color/darkGreen"
            android:textSize="18sp"/>

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:id="@+id/readProteinPercent"
            android:text=""
            android:layout_weight="1"
            android:textAlignment="viewEnd"
            android:textColor="@color/darkGreen"
            android:textSize="18sp"
            android:alpha="0.75"/>

    </LinearLayout>

    <androidx.appcompat.widget.AppCompatImageView
        android:layout_width="match_parent"
        android:layout_height="1dp"
        android:layout_margin="10dp"
        android:alpha="0.5"
        android:background="@color/darkGreen"/>

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:padding="8dp"
        android:orientation="horizontal"
        android:layout_marginTop="20dp"
        android:layout_marginEnd="10dp"
        android:layout_marginStart="10dp"
        android:layout_marginBottom="10dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Fats Goal:"
            android:textStyle="bold"
            android:textColor="@color/darkGreen"
            android:textSize="18sp" />
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:id="@+id/readFats"
            android:text=""
            android:layout_weight="1"
```



```
                    android:textColor="@color/darkGreen"
                    android:textSize="18sp" />
                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:id="@+id/readFatsPercent"
                    android:text=""
                    android:layout_weight="1"
                    android:textAlignment="viewEnd"
                    android:textColor="@color/darkGreen"
                    android:textSize="18sp"
                    android:alpha="0.75"/>

            </LinearLayout>

            <androidx.appcompat.widget.AppCompatImageView
                android:layout_width="match_parent"
                android:layout_height="1dp"
                android:layout_margin="10dp"
                android:alpha="0.5"
                android:background="@color/darkGreen"/>

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="match_parent"
                android:padding="8dp"
                android:orientation="horizontal"
                android:layout_marginTop="20dp"
                android:layout_marginEnd="10dp"
                android:layout_marginStart="10dp"
                android:layout_marginBottom="10dp">

                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:layout_weight="1"
                    android:text="Carbs Goal:"
                    android:textStyle="bold"
                    android:textColor="@color/darkGreen"
                    android:textSize="18sp" />
                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:id="@+id/readCarbs"
                    android:text=""
                    android:layout_weight="1"
                    android:textAlignment="viewEnd"
                    android:textColor="@color/darkGreen"
                    android:textSize="18sp" />
                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:id="@+id/readCarbsPercent"
                    android:text=""
                    android:layout_weight="1"
                    android:textAlignment="viewEnd"
                    android:textColor="@color/darkGreen"
                    android:textSize="18sp"
                    android:alpha="0.75"/>

            </LinearLayout>

            <androidx.appcompat.widget.AppCompatButton
                android:layout_width="match_parent"
                android:layout_height="match_parent"
                android:layout_marginTop="10dp"
                android:layout_marginEnd="40dp"
                android:layout_marginStart="40dp"
                android:text="Edit Goals"
                android:padding="20dp"
                android:textColor="@color/darkGreen"
                android:background="@drawable/green_border"
                android:id="@+id/btnEditMacros"/>
        </LinearLayout>

    </LinearLayout>

```



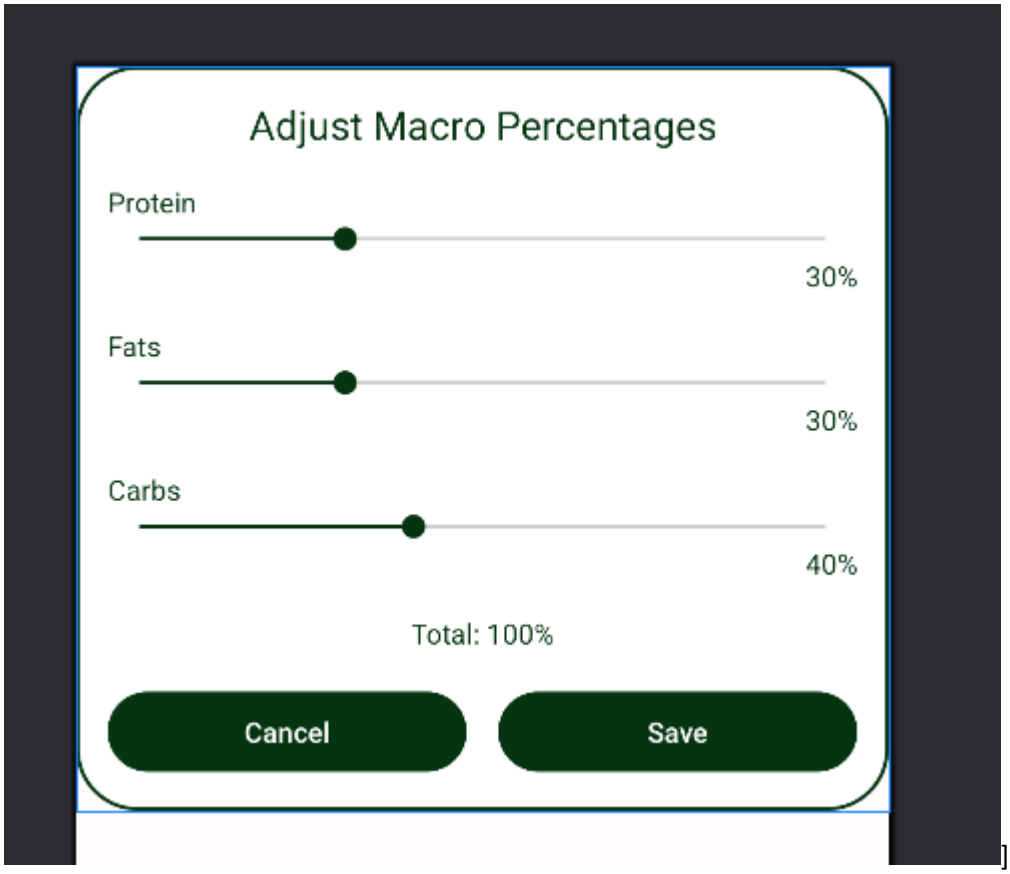
This style defines the look of the Daily goals section, there will be a button that will reuse the already created change_info.xml and then edit goals will lead to a different dialog. There is another current hidden text to the right of protein fats and carbs that will display the percentage taken from what is saved in firebase.

Edit Goals XML

dialog_macros_adjustement.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/androi
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="16dp"
    android:background="@drawable/green_border">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Adjust Macro Percentages"
        android:textSize="20sp"
        android:textColor="@color/darkGreen"
        android:gravity="center"
        android:layout_marginBottom="16dp"/>

    <!-- Protein Section -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:layout_marginBottom="16dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Protein"
            android:textColor="@color/darkGreen"/>

        <SeekBar
            android:id="@+id/proteinSeekBar"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:max="100"
            android:progress="30"
            android:progressTint="@color/darkGreen"
            android:thumbTint="@color/darkGreen"/>

        <TextView
            android:id="@+id/proteinPercentage"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="30%"
            android:textColor="@color/darkGreen"
            android:layout_gravity="end"/>
    </LinearLayout>

    <!-- Fats Section -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:layout_marginBottom="16dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Fats"
            android:textColor="@color/darkGreen"/>

        <SeekBar
            android:id="@+id/fatsSeekBar"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:max="100"
            android:progress="30"
            android:progressTint="@color/darkGreen"
            android:thumbTint="@color/darkGreen"/>

        <TextView
            android:id="@+id/fatsPercentage"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="30%"
            android:textColor="@color/darkGreen"
            android:layout_gravity="end"/>
    </LinearLayout>
```

```
    <!-- Carbs Section -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:layout_marginBottom="16dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Carbs"
            android:textColor="@color/darkGreen"/>

        <SeekBar
            android:id="@+id/carbsSeekBar"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:max="100"
            android:progress="40"
            android:progressTint="@color/darkGreen"
            android:thumbTint="@color/darkGreen"/>

        <TextView
            android:id="@+id/carbsPercentage"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="40%"
            android:textColor="@color/darkGreen"
            android:layout_gravity="end"/>
    </LinearLayout>

    <TextView
        android:id="@+id/totalPercentage"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Total: 100%"
        android:textColor="@color/darkGreen"
        android:gravity="center"
        android:layout_marginBottom="16dp"/>

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">

        <Button
            android:id="@+id/cancelButton"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Cancel"
            android:backgroundTint="@color/darkGreen"
            android:textColor="@color/white"
            android:layout_marginEnd="8dp"/>

        <Button
            android:id="@+id/saveButton"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Save"
            android:backgroundTint="@color/darkGreen"
            android:textColor="@color/white"
            android:layout_marginStart="8dp"/>
    </LinearLayout>

</LinearLayout>
```



]

This code represents an XML layout for a macro nutrient adjustment interface, likely part of a fitness or nutrition app. It creates a dialog or screen with three slider controls (SeekBars) for adjusting the percentages of protein, fats, and carbs in a diet plan. Each macro section includes a label, a slider bar styled in dark green, and a percentage display. The interface maintains a running total of the percentages (shown at the bottom) and includes "Cancel" and "Save" buttons for confirming or discarding changes. The layout is organized vertically with proper spacing and padding, and uses a consistent green color scheme throughout.

Logic

Main screen

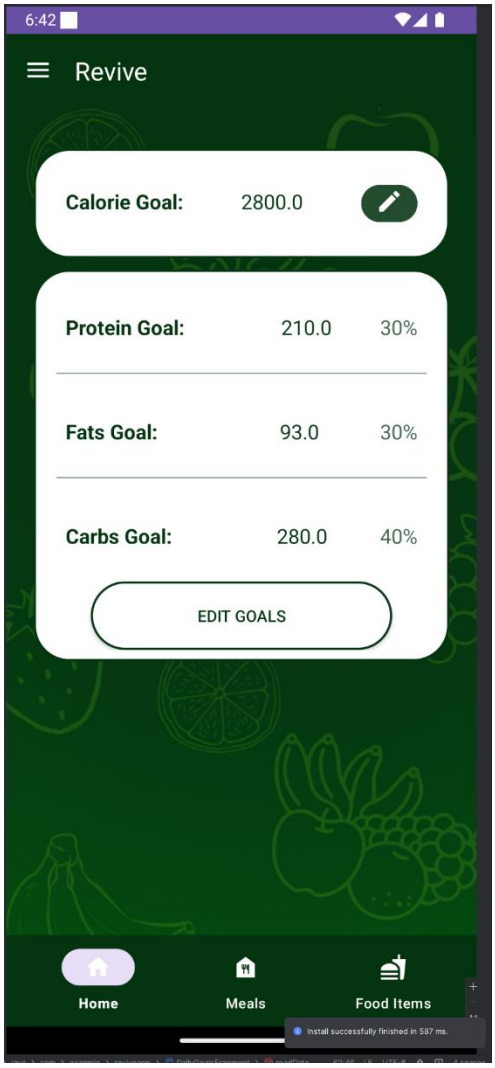
```
class DailyGoalsFragment : Fragment() {

    private var _binding: FragmentDailygoalsBinding? = null
    private val binding get() = _binding!! // safe access to binding
    private lateinit var databaseReference: DatabaseReference

    private var calorieG = 0.0
    private var proteinG = 0.0
    private var fatG = 0.0
    private var carbG = 0.0

    private var proteinpercent = 0.0
    private var fatspercent = 0.0
    private var carbpercent = 0.0

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        _binding = FragmentDailygoalsBinding.inflate(inflater,container,false)
```

```
databaseReference = FirebaseDatabase.getInstance(
    "https://revive-app-b8fd2-default-rtdb.europe-west1.firebasedatabase.app/"
).getReference("Information")
    .child(userid ?: "")
    .child("personal information")
```

To start off with, I created a few variables that would temporarily hold the percentages and the values taken from firebase and set up the binding to link the code to the xml file so that it can detect user inputs, as well as setup a general reference to the database to be used later on in the code.

```
private fun readData() {
    val userid = FirebaseAuth.getInstance().currentUser?.uid
    if (userid != null) {
        databaseReference.get().addOnSuccessListener { snapshot ->
            calorieG = snapshot.child("deficitcalories").getValue(Double::class.java) ?: 0.0
            proteinG = snapshot.child("protein").getValue(Double::class.java) ?: 0.0
            fatG = snapshot.child("fats").getValue(Double::class.java) ?: 0.0
            carbG = snapshot.child("carbs").getValue(Double::class.java) ?: 0.0

            proteinpercent = snapshot.child("proteinpercent").getValue(Double::class.java) ?: 0.0
            fatspercent = snapshot.child("fatspercent").getValue(Double::class.java) ?: 0.0
            carbpercent = snapshot.child("carbspercent").getValue(Double::class.java) ?: 0.0

            binding.readCalories.text = calorieG.toString()
            binding.readProtein.text = proteinG.toString()
            binding.readFats.text = fatG.toString()
            binding.readCarbs.text = carbG.toString()

            binding.readProteinPercent.text = "${(proteinpercent * 100).toInt()}%"
            binding.readFatsPercent.text = "${(fatspercent * 100).toInt()}%"
            binding.readCarbsPercent.text = "${(carbpercent * 100).toInt()}%"

        }.addOnFailureListener {
            Toast.makeText(requireContext(), "Something went wrong...", Toast.LENGTH_SHORT).show()
        }
    }
}
```

The readData() function is responsible for fetching a user's nutritional information from Firebase Realtime Database and displaying it in the app interface. When called, it first gets the current user's ID from Firebase Authentication to ensure its accessing the correct user's data. If a valid user ID exists, it retrieves several values from the database including calories, protein, fats, carbs, and their corresponding percentages. The function uses null-safe operators meaning it will default to 0.0 if any values are missing in the database. Once the data is retrieved successfully, it updates various text views in the app's interface to display these values, with the macro percentages being converted to a percentage format. If anything goes wrong during this process, it shows a simple error toast message to the user. Testing ID: 2.16



Edit Goals Button

```
private fun showMacrosAdjustmentDialog() {
```

```
val builder = AlertDialog.Builder(requireContext())
val view = layoutInflater.inflate(R.layout.dialog_macros_adjustment, null)

// Initialize views
val proteinSeekBar = view.findViewById<SeekBar>(R.id.proteinSeekBar)
val fatsSeekBar = view.findViewById<SeekBar>(R.id.fatsSeekBar)
val carbsSeekBar = view.findViewById<SeekBar>(R.id.carbsSeekBar)

val proteinPercentageText = view.findViewById<TextView>(R.id.proteinPercentage)
val fatsPercentageText = view.findViewById<TextView>(R.id.fatsPercentage)
val carbsPercentageText = view.findViewById<TextView>(R.id.carbsPercentage)
val totalPercentageText = view.findViewById<TextView>(R.id.totalPercentage)
```

The function starts by creating an AlertDialog and inflating the custom layout *dialog_macros_adjustement.xml* created earlier for adjusting macronutrients. It finds all the necessary UI elements (SeekBars and TextViews) from the inflated layout using findViewById. This sets up the foundation for the macro adjustment dialog that will be shown to the user.

```
// Set initial values rounded to nearest 5%
proteinSeekBar.progress = (((proteinpercent * 100) + 2.5) / 5).toInt() * 5
fatsSeekBar.progress = (((fatspercent * 100) + 2.5) / 5).toInt() * 5
carbsSeekBar.progress = (((carbpercent * 100) + 2.5) / 5).toInt() * 5
```

The code sets initial values for the protein, fats, and carbs SeekBars based on existing percentages stored in the app. It includes a rounding mechanism that rounds the values to the nearest 5% for better usability. This ensures that when users open the dialog, they see their current macro distribution as a starting point and when they want to adjust the sliders created the sliders will only move by 5% at a time rather than individual percentages. Testing Code: 2.17

```
fun updatePercentageTexts() {
    proteinPercentageText.text = "${proteinSeekBar.progress}%"
    fatsPercentageText.text = "${fatsSeekBar.progress}%"
    carbsPercentageText.text = "${carbsSeekBar.progress}%"
    val total = proteinSeekBar.progress + fatsSeekBar.progress + carbsSeekBar.progress
    totalPercentageText.text = "Total: $total%"
    totalPercentageText.setTextColor(if (total == 100)
        resources.getColor(R.color.darkGreen)
    else resources.getColor(android.R.color.holo_red_dark))
}
```

I then created a helper function updatePercentageTexts() is to handle updating all the percentage displays. It updates the individual percentage texts for each macro and calculates the total. The total percentage text changes color to green when it equals 100% and red when it doesn't, providing immediate visual feedback to the user. Testing Code: 2.18

```
// Set listeners for seek bars
val seekBarChangeListener = object : SeekBar.OnSeekBarChangeListener {
    override fun onProgressChanged(seekBar: SeekBar?, progress: Int, fromUser: Boolean) {
        if (fromUser) {
            // Round to nearest 5%
            val roundedProgress = ((progress + 2.5) / 5).toInt() * 5
            seekBar?.progress = roundedProgress
        }
        updatePercentageTexts()
    }
    override fun onStartTrackingTouch(seekBar: SeekBar?) {}
    override fun onStopTrackingTouch(seekBar: SeekBar?) {}
}

proteinSeekBar.setOnSeekBarChangeListener(seekBarChangeListener)
fatsSeekBar.setOnSeekBarChangeListener(seekBarChangeListener)
carbsSeekBar.setOnSeekBarChangeListener(seekBarChangeListener)

updatePercentageTexts()
```

The code implements a SeekBar change listener that's shared among all three SeekBars. This listener rounds the progress to the nearest 5% whenever the user adjusts a slider, ensuring consistent and clean values. It also triggers the percentage text updates whenever any slider is moved, keeping the display in sync with user input. The function to updatePercentages is then called so that all values are loaded into the correct text ids and the user can see their percentages. Testing Code: 2.16

```
builder.setView(view)
val dialog = builder.create()

// Cancel button setup
view.findViewById<Button>(R.id.cancelButton).setOnClickListener {
    dialog.dismiss()
}

if (dialog.window != null) {
    dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
}

dialog.show()
```

The code sets up the AlertDialog with the custom view mentioned earlier (this is the part where it actually causes the dialog to show) and implements two buttons: Save and Cancel. The Cancel button simply dismisses the dialog so it can be coded right away. The dialog's background is also set to transparent for better visual appearance.

```
view.findViewById<Button>(R.id.saveButton).setOnClickListener {
    val total = proteinSeekBar.progress + fatsSeekBar.progress + carbsSeekBar.progress
    if (total != 100) {
        Toast.makeText(context, "Total percentage must equal 100%", Toast.LENGTH_SHORT).show()
        return@setOnClickListener
    }

    // Convert to decimals for storage
    val newProteinPercent = proteinSeekBar.progress / 100.0
    val newFatsPercent = fatsSeekBar.progress / 100.0
    val newCarbsPercent = carbsSeekBar.progress / 100.0

    // Calculate new macro values based on current calories
    val newProtein = ((calorieG * newProteinPercent) / 4).toInt()
    val newFats = ((calorieG * newFatsPercent) / 9).toInt()
    val newCarbs = ((calorieG * newCarbsPercent) / 4).toInt()
```

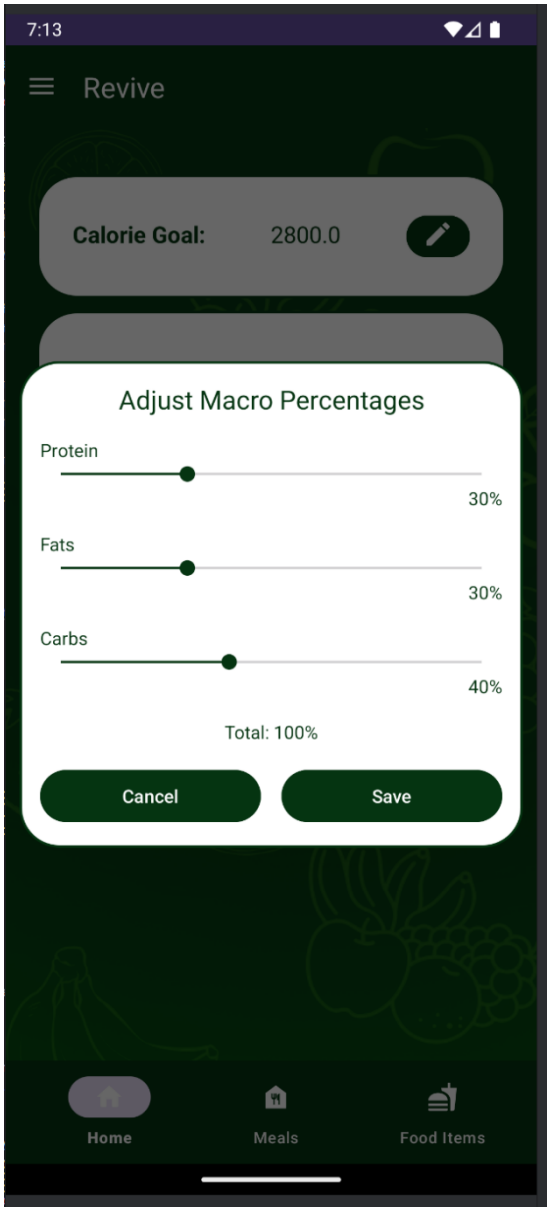
The save button's logic first validates that the total equals 100%. If valid, it converts the percentages to decimals and calculates new macro values based on the current calorie goal. The protein and carbs are calculated using a factor of 4 calories per gram, while fats use 9 calories per gram, following standard nutritional science, which is the same as what was used before when the macros were first calculated. However if the total does not equal 100%, the function is returned with a fail and a message is stated that the total must equal 100%, but the dialog is not closed. Testing Code for message: 2.19, 2.20

```
val updates = hashMapOf<String, Any>(
    "proteinpercent" to newProteinPercent,
    "fatspercent" to newFatsPercent,
    "carbspercent" to newCarbsPercent,
    "protein" to newProtein,
    "fats" to newFats,
    "carbs" to newCarbs
)

databaseReference.updateChildren(updates)
    .addOnSuccessListener {
        // Update local variables and UI
        proteinpercent = newProteinPercent
        fatspercent = newFatsPercent
        carbpercent = newCarbsPercent
        proteinG = newProtein.toDouble()
        fatG = newFats.toDouble()
        carbG = newCarbs.toDouble()

        // Update UI
        binding.readProtein.text = newProtein.toString()
        binding.readFats.text = newFats.toString()
        binding.readCarbs.text = newCarbs.toString()
        binding.readProteinPercent.text = "${(newProteinPercent * 100).toInt()}%"
        binding.readFatsPercent.text = "${(newFatsPercent * 100).toInt()}%"
        binding.readCarbsPercent.text = "${(newCarbsPercent * 100).toInt()}%"

        Toast.makeText(context, "Macros updated successfully", Toast.LENGTH_SHORT).show()
        dialog.dismiss()
    }
    .addOnFailureListener {
        Toast.makeText(context, "Failed to update macros", Toast.LENGTH_SHORT).show()
    }
```

Finally, the code handles the database update by creating a HashMap of all changes and sending them to Firebase. Upon successful update, it refreshes all local variables and updates the UI to reflect the new values. Success and failure are communicated to the user via toast messages. This ensures data consistency between the app, the UI, and the backend database. Testing Code: 2.21

```
binding.btnEditMacros.setOnClickListener{
    showMacrosAdjustmentDialog()
}
```

Then to make sure the code can actually react to the button, the binding sets a listener for the button being pressed and then the function can be called.



Edit Calories Button

```
private fun call_DetailChangeCalories() {
    val builder = AlertDialog.Builder(requireContext())
    val view = layoutInflater.inflate(R.layout.dialog_changeinfo, null) // calls dialog_ch

    val changedetail = view.findViewById<EditText>(R.id.editBoxDetails)

    changedetail.setText(calorieG.toString())

    builder.setView(view)
    val dialog = builder.create()
```

This section initializes the dialog for changing calorie goals. It creates an dialog with the style *dialog_changeinfo.xml* which is the same style used in the personal info screen. It then finds the

EditText view for calorie input, and sets its initial value to the current calories. This sets up the basic structure of the dialog that the user will interact with.

```
view.findViewById<Button>(R.id.btnChangeDetail).setOnClickListener{
        val userid = FirebaseAuth.getInstance().currentUser?.uid
        if (userid != null){
            val detailText = changedetail.text.toString()
            if (detailText.isNotEmpty()) {
                val detailValue = detailText.toInt()

                val newProtein = ((detailValue * proteinpercent) / 4).toInt()
                val newFats = ((detailValue * fatspercent) / 9).toInt()
                val newCarbs = ((detailValue * carbpercent) / 4).toInt()

                val updates = hashMapOf<String, Any>(
                    "calories" to detailValue,
                    "deficitcalories" to detailValue,
                    "protein" to newProtein,
                    "fats" to newFats,
                    "carbs" to newCarbs
                )
```

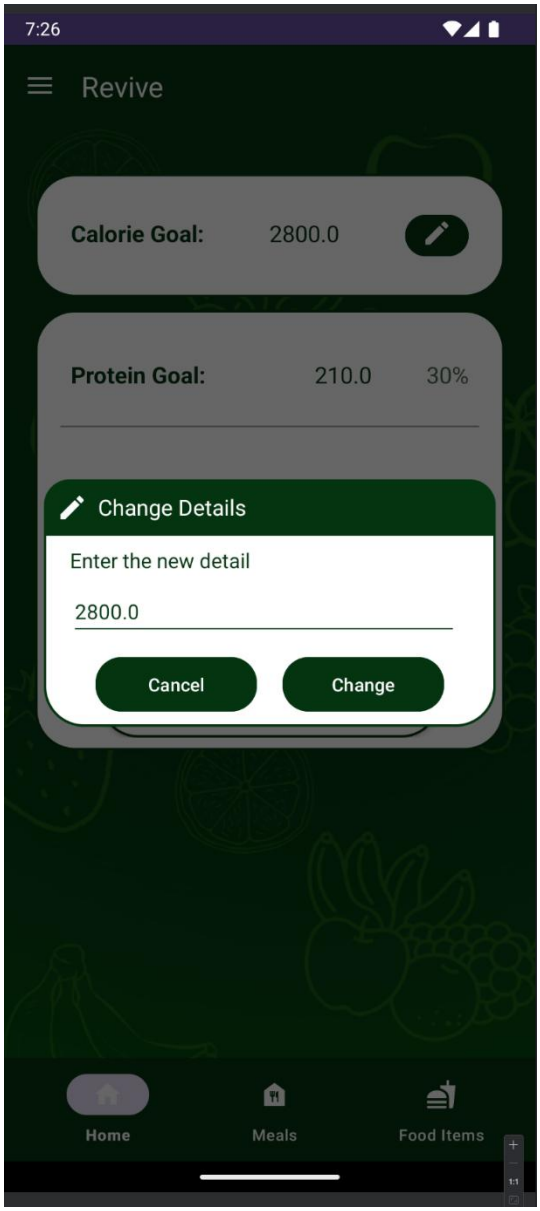
This section handles the save button logic. When the user enters a new calorie value, it recalculates all macro nutrients based on the existing percentages (using the standard 4 calories per gram for protein and carbs, 9 for fats).

```
            databaseReference.updateChildren(updates)
                .addOnSuccessListener {
                    calorieG = detailValue.toDouble()
                    proteinG = newProtein.toDouble()
                    fatG = newFats.toDouble()
                    carbG = newCarbs.toDouble()

                    binding.readCalories.text = detailValue.toString()
                    binding.readProtein.text = newProtein.toString()
                    binding.readFats.text = newFats.toString()
                    binding.readCarbs.text = newCarbs.toString()

                    dialog.dismiss()
                    Toast.makeText(context, "Goals updated successfully", Toast.LENGTH_SHORT).show()
                }
                .addOnFailureListener {
                    Toast.makeText(requireContext(), "Failed to save", Toast.LENGTH_SHORT).show()
                }

        } else {
            Toast.makeText(requireContext(), "Fields cannot be empty", Toast.LENGTH_SHORT).show()
        }

    } else {
        Toast.makeText(requireContext(), "Failed", Toast.LENGTH_SHORT).show()
        dialog.dismiss()
    }
}
```

The code then saves this new value to firebase along with the newly calculated macros and updates the UI so that the user can see the new changes made. The code has small amounts of validation in case anything were to go wrong with the connection, but the main validation is to make sure that the field cannot be left empty, so the user cannot enter an empty string to the database as this could cause the program to crash. Testing Code: 2.22, 2.23

```
view.findViewById<Button>(R.id.btnCancelDetails).setOnClickListener {
    dialog.dismiss()
}
if (dialog.window != null) {
    dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
}
dialog.show()
```

Then in the same manner as before the cancel button is set up to close the dialog, and the dialog is given a clear background so that the rest of the app can still be seen behind it.



Testing

Calorie Change ======= - Calorie change button message when empty - Calorie change when correct value

Daily Goals Testing Video

Test ID 2	Timestamp	PURPOSE	DATA	EXPECTED RESULTS	ACTUAL RESULTS
2.16	0:00	All calories and percentages are initially displayed in the screen from firebase according to what was set by default / what user has set them to.	n/a, data from firebase	All data in firebase under the child nodes for macros and macropercent shows up.	As expected.

2.17	0:04	Testing that the edit goals slider only moves by 5% up and down at each interval.	n/a	The user can drag as slow or as fast as they want and the slider will only jump 5%	As expected.
2.18	0:06	To test the UI whether it is updated when the user changes the sliders, if all three equal to 100% total the UI shows green text, otherwise the text is red	n/a	The colour of the text changes according to whether the total is 100% or not	As expected
2.19	0:14	To inform the user that the sliders have to equal 100% if they want to save the new macro split when the total is over 100%.	Values for sliders that do not equal 100% but are over	A message stating “Total percentage must equal 100%” should be displayed.	As expected.
2.20	0:19	To inform the user that the sliders have to equal 100% if they want to save the new macro split when the total is under 100%.	Values for sliders that do not equal 100% but are under	A message stating “Total percentage must equal 100%” should be displayed.	As expected
2.21	0:27	To test whether the app saves to firebase correctly when the correct data has been entered and if the macros are updated as a result, as well as UI is updated.	New marco split: Protein: 25% Fats: 30% Carbs: 45%	A message telling the user macros have been updated, then all values are saved and calculated to firebase and updated in the UI	As expected.
2.22	0:41	To test the calorie changing feature, whether the app will allow the user to save data that does not exist to the database, by having the user enter an empty string.	Data: Empty box	The data should not be saved and the user should have a message returned telling them the fields cannot be empty	As expected.
2.23	0:45	To test the calorie changing feature, whether the app will allow the user to save the correct data to firebase	Data: Calories: 2700 2800	The data should be saved to firebase and calculations performed for the calories and deficit calories, and the macros relating to those new calories based on the	As expected.

				percentage split in firebase.	

Personal Information Module Summary.

The Daily Goals module enables users to view and customize their nutritional targets beyond the automatically calculated values. Users can modify their daily calorie goal if they want to deviate from the app's recommendation, and adjust the macronutrient distribution from the default 30-40-30 (protein-carbs-fats) split.

The prototype thus far encompasses a complete personal information system: from initial user data collection at signup, to viewing and editing personal metrics, to fine-tuning nutritional goals. The flow begins with the Personal Details Input module collecting essential metrics, which feeds into the Profile module for account management and the Personal Information Display for metric modifications. This culminates in the Daily Goals module where users can customize their nutritional targets. This layered approach is justified as it progressively gives users more control over their fitness data - starting with basic metrics, then allowing modifications of those metrics, and finally enabling customization of the calculated nutritional targets. The modular structure also maintains clear separation of concerns while ensuring all components work together cohesively through Firebase integration.

Stakeholder review – 2

Q: What's your assessment of the data persistence approach?

A: Using Firebase for data storage enables seamless synchronization across devices while maintaining user-specific data segregation. The app handles network errors gracefully and includes fallback values where needed. One suggestion would be to add local caching to improve performance and offline functionality.

Q: The specification emphasized making calculations clear and transparent. How has this been achieved?

A: Each screen clearly displays current values and calculations. When users modify their personal details, they can immediately see how changes affect their calorie and macro targets. The app uses rounded values and clean progress bars to make numbers easily digestible. The 30-40-30 default macro split provides a sensible starting point while still allowing customization as users become more experienced.

Object Creation Modules

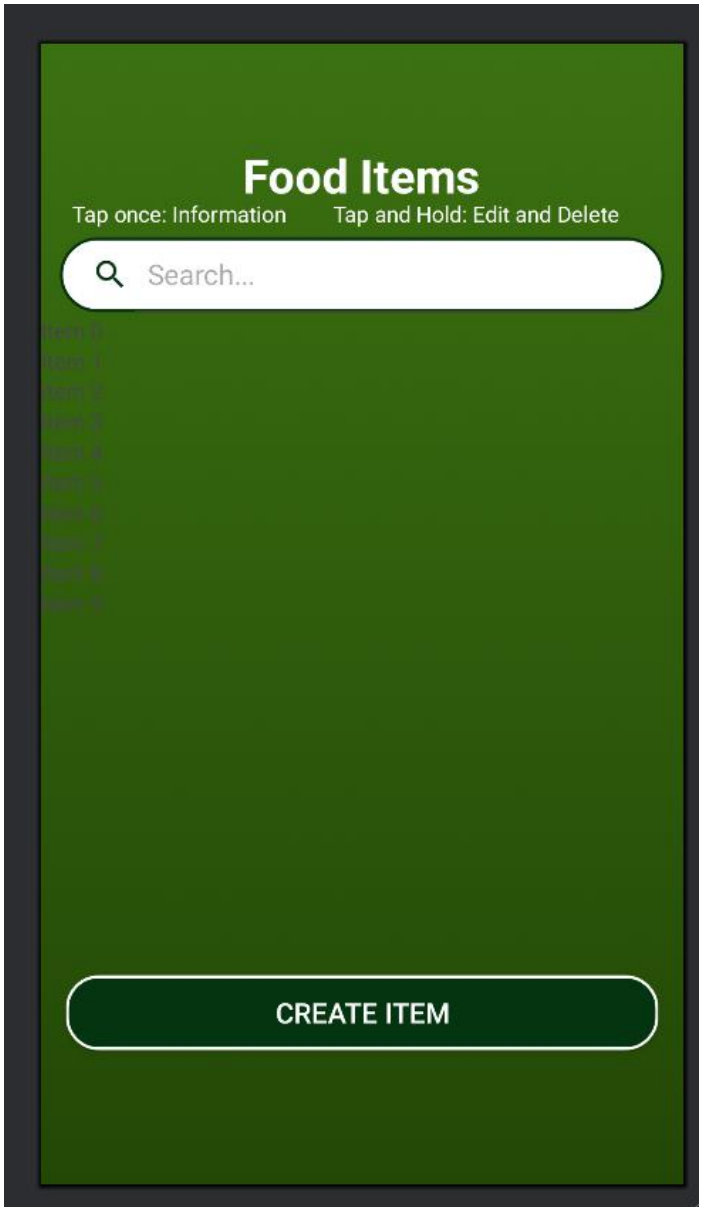
Food Item Creation Module

This section of the Object Creation Module will allow the user to create individual food items based on their exact product from the shop, since all foods in the UK as well as other countries are required to have labels on them by law, the user will be able to look at this label and enter values based on 100g of the product to create an object that will be used for later calculations in meals or their daily tracking.

Item page XML

FoodItem.kt ItemFragment.kt fragment_item.xml dialog_uploaditem.

```xml
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity"
    android:background="@drawable/default_screen_gradient">

    <FrameLayout
        android:id="@+id/item_content"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:layout_marginTop="?attr/actionBarSize"
        android:layout_marginBottom="70dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Food Items"
            android:layout_gravity="center|top"
            android:textSize="30dp"
            android:textColor="@color/white"
            android:textStyle="bold"/>

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="horizontal"
            android:layout_marginTop="35dp">
            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Tap once: Information"
                android:textColor="@color/white"
                android:layout_marginEnd="30dp"
                android:layout_marginStart="20dp"/>

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Tap and Hold: Edit and Delete"
                android:textColor="@color/white" />
        </LinearLayout>

        <androidx.appcompat.widget.SearchView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="60dp"
            android:layout_marginStart="12dp"
            android:layout_marginEnd="12dp"
            android:id="@+id/search"
            app:iconifiedByDefault="false"
            app:searchHintIcon="@null"
            app:queryHint="Search..."
            android:focusable="false"
            app:closeIcon="@drawable/baseline_clear_green"
            app:searchIcon="@drawable/baseline_search_24"
            android:background="@drawable/green_border"/>

        <androidx.recyclerview.widget.RecyclerView
            android:id="@+id/recyclerItemView"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="110dp"
            android:layout_marginBottom="80dp"
            android:paddingBottom="16dp"
            android:clipToPadding="false"
            android:scrollbars="vertical"/>

        <androidx.appcompat.widget.AppCompatButton
            android:id="@+id/item_upload_button"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_gravity="bottom"
            android:layout_margin="16dp"
            android:drawablePadding="8dp"
            android:text="Create Item"
            android:textSize="18sp"
            android:background="@drawable/darkgreen_round_white_outline"
            android:textColor="@color/white"/>
```



This xml defines the basic screen that will hold all the individual items that the user will create so that they can then later use them to form a meal, or add them specifically to their total amounts in the calorie tracking module later on, a recycler view will be made use of to display information from firebase in a list, where each object is its own list item that can be interacted with and used.

Item Creation Dialog XML

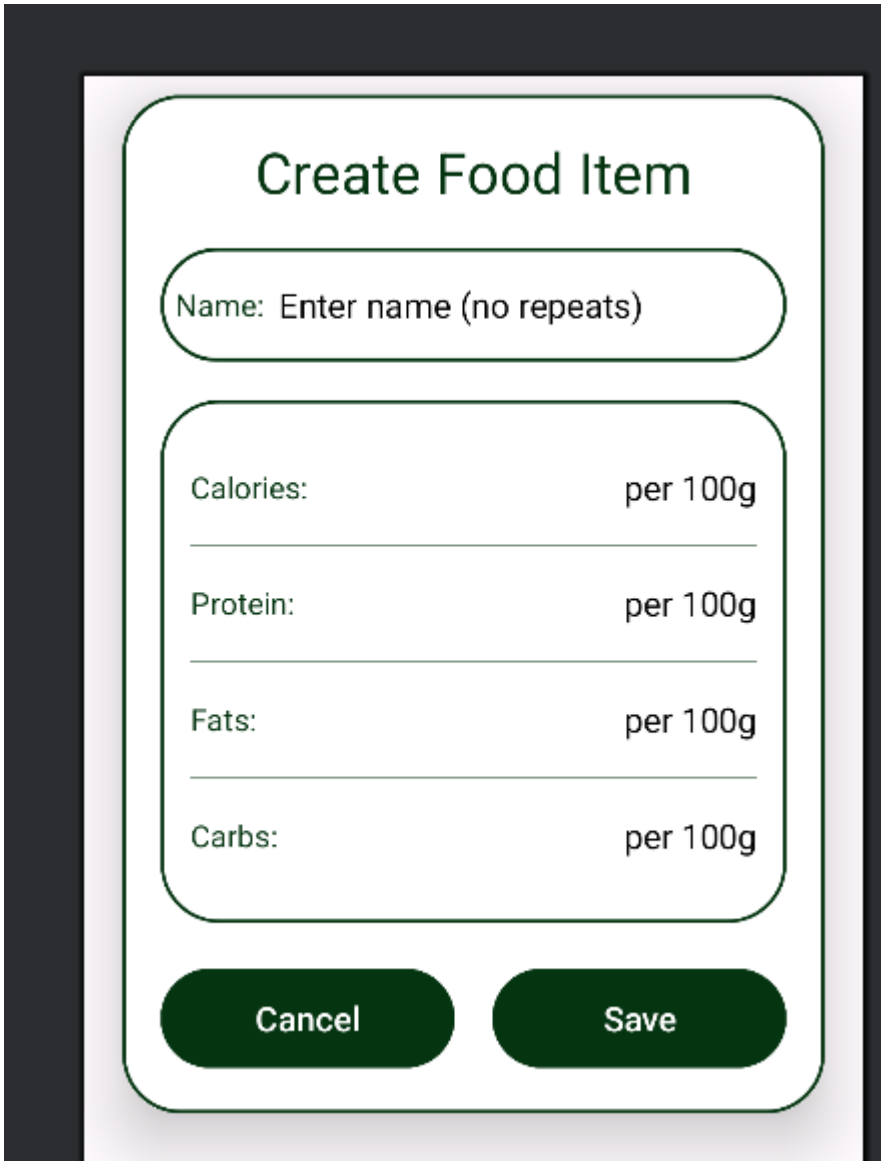
dialog_uploaditem.xml

Instead of starting a new activity to create an object, I will have a dialog pop up that allows the user to enter the necessary information.

```xml
<?xml version="1.0" encoding="utf-8"?>
<ScrollView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <androidx.cardview.widget.CardView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="10dp"
        android:layout_marginEnd="20dp"
        android:layout_marginStart="20dp"
        app:cardCornerRadius="30dp"
        app:cardElevation="20dp">
        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="20dp"
            android:background="@drawable/green_border">
            <!-- Header -->
            <TextView
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:text="Create Food Item"
                android:textSize="30sp"
                android:textAlignment="center"
                android:textColor="@color/darkGreen"/>

            <!-- Name Input -->
            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="60dp"
                android:layout_marginTop="20dp"
                android:background="@drawable/green_border"
                android:gravity="center_vertical"
                android:padding="8dp">

                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="Name:"
                    android:textColor="@color/darkGreen"
                    android:textSize="16sp"
                    android:layout_marginEnd="8dp"/>

                <EditText
                    android:id="@+id/uploadItemName"
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:background="@null"
                    android:hint="Enter name (no repeats)"
                    android:textColor="@color/darkGreen"
                    android:textColorHint="@color/black"/>
            </LinearLayout>

            <!-- Nutritional Info Container -->
            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="vertical"
                android:layout_marginTop="20dp"
                android:padding="16dp"
                android:background="@drawable/green_border">

                <!-- Calories -->
                <LinearLayout
                    android:layout_width="match_parent"
                    android:layout_height="60dp"
                    android:gravity="center_vertical">

                    <TextView
                        android:layout_width="100dp"
                        android:layout_height="wrap_content"
                        android:text="Calories:"
                        android:textColor="@color/darkGreen"
                        android:textSize="16sp"/>
```

```
        <EditText
            android:id="@+id/uploadCalories"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:background="@null"
            android:textAlignment="viewEnd"
            android:hint="per 100g"
            android:inputType="numberDecimal"
            android:textColor="@color/darkGreen"
            android:textColorHint="@color/black"/>
    </LinearLayout>

    <View
        android:layout_width="match_parent"
        android:layout_height="1dp"
        android:background="@color/darkGreen"
        android:alpha="0.5"/>

    <!-- Protein -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="60dp"
        android:gravity="center_vertical">

        <TextView
            android:layout_width="100dp"
            android:layout_height="wrap_content"
            android:text="Protein:"
            android:textColor="@color/darkGreen"
            android:textSize="16sp"/>

        <EditText
            android:id="@+id/uploadProtein"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:textAlignment="viewEnd"
            android:background="@null"
            android:hint="per 100g"
            android:inputType="numberDecimal"
            android:textColor="@color/darkGreen"
            android:textColorHint="@color/black"/>
    </LinearLayout>

    <View
        android:layout_width="match_parent"
        android:layout_height="1dp"
        android:background="@color/darkGreen"
        android:alpha="0.5"/>

    <!-- Fats -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="60dp"
        android:gravity="center_vertical">

        <TextView
            android:layout_width="100dp"
            android:layout_height="wrap_content"
            android:text="Fats:"
            android:textColor="@color/darkGreen"
            android:textSize="16sp"/>

        <EditText
            android:id="@+id/uploadFats"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:textAlignment="viewEnd"
            android:background="@null"
            android:hint="per 100g"
            android:inputType="numberDecimal"
            android:textColor="@color/darkGreen"
            android:textColorHint="@color/black"/>
    </LinearLayout>
```

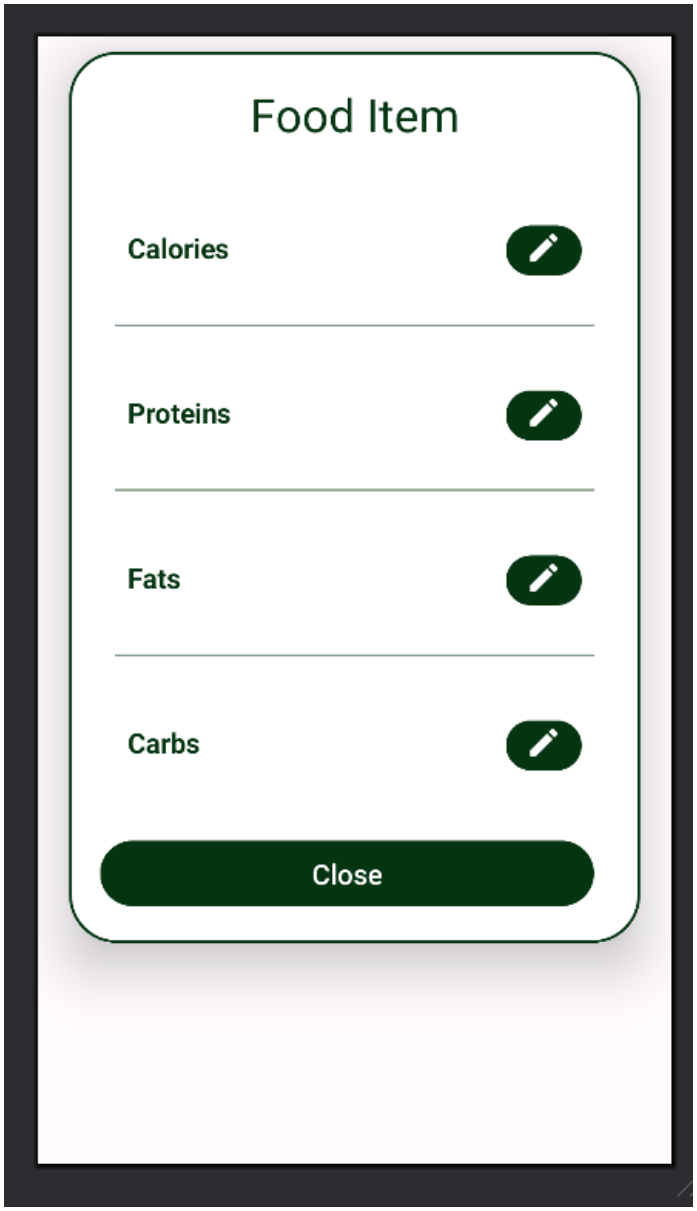
```
152
153                 <View
154                     android:layout_width="match_parent"
155                     android:layout_height="1dp"
156                     android:background="@color/darkGreen"
157                     android:alpha="0.5"/>
158
159                 <!-- Carbs -->
160                 <LinearLayout
161                     android:layout_width="match_parent"
162                     android:layout_height="60dp"
163                     android:gravity="center_vertical">
164
165                     <TextView
166                         android:layout_width="100dp"
167                         android:layout_height="wrap_content"
168                         android:text="Carbs:"
169                         android:textColor="@color/darkGreen"
170                         android:textSize="16sp"/>
171
172                     <EditText
173                         android:id="@+id/uploadCarbs"
174                         android:layout_width="match_parent"
175                         android:layout_height="wrap_content"
176                         android:background="@null"
177                         android:textAlignment="viewEnd"
178                         android:hint="per 100g"
179                         android:inputType="numberDecimal"
180                         android:textColor="@color/darkGreen"
181                         android:textColorHint="@color/black"/>
182                 </LinearLayout>
183             </LinearLayout>
184
185             <!-- Buttons -->
186             <LinearLayout
187                 android:layout_width="match_parent"
188                 android:layout_height="wrap_content"
189                 android:orientation="horizontal"
190                 android:layout_marginTop="20dp">
191
192                 <Button
193                     android:layout_width="0dp"
194                     android:layout_height="60dp"
195                     android:layout_weight="1"
196                     android:text="Cancel"
197                     android:id="@+id/itemCancelButton"
198                     android:textSize="18sp"
199                     android:backgroundTint="@color/darkGreen"
200                     android:layout_marginEnd="10dp"
201                     android:textColor="@color/white"/>
202
203                 <Button
204                     android:layout_width="0dp"
205                     android:layout_height="60dp"
206                     android:layout_weight="1"
207                     android:text="Save"
208                     android:id="@+id/itemsaveButton"
209                     android:textSize="18sp"
210                     android:backgroundTint="@color/darkGreen"
211                     android:layout_marginStart="10dp"
212                     android:textColor="@color/white"/>
213             </LinearLayout>
214         </LinearLayout>
215     </androidx.cardview.widget.CardView>
216 </ScrollView>
```



The user will be able to enter the meal name, and all the values they need to enter where the displayed text is, for the name it is where the “enter name” appears and for values it is the “per 100g”, the user will replace this text with the values they need. On a mobile device this text will appear as a hint, so it will be obvious to the user where they need to put their information.

Item information XML

dialog_infoitem.xml



XML code to display an individual food item and its details, with edit buttons so that the user can directly choose which details to change

Logic

The first thing I created was a data class, where just like in the personal information module, which will be used to upload all the food item data in one go as a singular object, and can be later used to retrieve the data from firebase into an object.

```
data class FoodItem(
    var name: String = "",
    var calories: Double = 0.0,
    var proteins: Double = 0.0,
    var fats: Double = 0.0,
    var carbs: Double = 0.0
) {
    // Calculates 1g amounts from 100g
```

```
fun convertto1gram() {
    calories = ((calories / 100.0))
    proteins = ((proteins / 100.0))
    fats = ((fats / 100.0))
    carbs = ((carbs / 100.0))
}

fun convertto100gram() {
    calories = ((calories * 100.0))
    proteins = ((proteins * 100.0))
    fats = ((fats * 100.0))
    carbs = ((carbs * 100.0))
}
```

Then within this class I gave it two methods that will be used to store the food items in database, the two methods will convert the values entered by the user back and forth when needed, where converting to 1 gram will be used to store the values as a gram so that when the user enters how much of an item they want to add, for example to a meal, the values are already ready to be multiplied by that amount, instead of having to convert it to 1g first and then perform calculations. The convert to 1g will be used for viewing purposes.

```
class ItemFragment : Fragment() {

    private var _binding: FragmentItemBinding? = null
    private val binding get() = _binding!! // safe access to binding
    private lateinit var databaseReference: DatabaseReference
    private lateinit var firebaseAuth: FirebaseAuth

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        // Inflate the layout for this fragment
        _binding = FragmentItemBinding.inflate(inflater, container, false)
        searchView = binding.search

        databaseReference = FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.europe-west1.firebasedatabase.app/").getReference("Information")
        val userId = FirebaseAuth.getInstance().currentUser?.uid
    }
}
```

I started by initialising the fragment, setting up the binding to link the fragment code to the xml created earlier, and set up a database reference and user id for reference later.

Creation of items

```
binding.itemUploadButton.setOnClickListener{
        val builder = AlertDialog.Builder(requireContext())
        val view = layoutInflater.inflate(R.layout.dialog_uploaditem, null) // calls dialog_uploaditem xml

        val name = view.findViewById<EditText>(R.id.uploadItemName)
        val calories = view.findViewById<EditText>(R.id.uploadCalories)
        val protein = view.findViewById<EditText>(R.id.uploadProtein)
        val fats = view.findViewById<EditText>(R.id.uploadFats)
        val carbs = view.findViewById<EditText>(R.id.uploadCarbs)

        builder.setView(view)
        val dialog = builder.create()

        view.findViewById<Button>(R.id.itemsaveButton).setOnClickListener {

            val itemname = name.text.toString()
            val intcalories = calories.text.toString().toDoubleOrNull() ?: 0.0
            val intprotein = protein.text.toString().toDoubleOrNull() ?: 0.0
            val intfats = fats.text.toString().toDoubleOrNull() ?: 0.0
            val intcarbs = carbs.text.toString().toDoubleOrNull() ?: 0.0
```

Afterwards, I set up an onClickListener to wait for the user to press the create item button at the bottom of the screen, which will display the dialog from the xml created earlier and setup variables to link to the input boxes for the user. Then when the user has entered values into the save button further variables capture those variables.

```
        if (itemname.isEmpty() || intcalories == 0.0 || intprotein == 0.0 || intfats == 0.0 || intcarbs == 0.0) {
            Toast.makeText(requireContext(), "Please fill in all fields", Toast.LENGTH_SHORT).show()
        } else {

            val itemupload = FoodItem(name = itemname, calories = intcalories, proteins = intprotein, fats = intfats, carbs = intcarbs)
            itemupload.converttogram()

            val userid = FirebaseAuth.getInstance().currentUser?.uid
            if (userid != null){
                databaseReference.child(userid).child("Food items").child(itemname).setValue(itemupload).addOnSuccessListener{

                    name.text.clear()
                    calories.text.clear()
                    protein.text.clear()
                    fats.text.clear()
                    carbs.text.clear()
                    Toast.makeText(requireContext(),"Saved",Toast.LENGTH_SHORT).show() // shows a success message
                }

            } else{
                Toast.makeText(requireContext(),"Failed",Toast.LENGTH_SHORT).show()
                dialog.dismiss()
            }

            dialog.dismiss()
        }
    }
    view.findViewById<Button>(R.id.itemCancelButton).setOnClickListener {
        dialog.dismiss() // cancels operation
    }
    if (dialog.window != null){
        dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
    }
    dialog.show()
}
```

Then the code performs validation to check whether the user has entered values or not, if they have not then a message is displayed telling them to enter them. Testing Code: 4.01

Otherwise, the values are uploaded to the data class, converted to 1 gram and then saved to firebase under a child node food items. Testing Code: 4.02

Next, outside of the binding button code and inside the main fragment code I created a function that would load the items the user has entered back into the data class and converted back into 100 grams so that they are better displayed for the user.

```
userId?.let {
    databaseReference.child(it).child("Food items").addValueEventListener(object : ValueEventListener {
        override fun onDataChange(snapshot: DataSnapshot) {
            for (foodSnapshot in snapshot.children) {
                val foodItem = foodSnapshot.getValue(FoodItem::class.java)
                if (foodItem != null) {
                    foodItem.convertto100gram() //converts back to 100g for viewing ease
                    foodList.add(foodItem) // adds each item to a list
                }
            }
            // Update adapter with fetched data
            adapter.updateData(foodList)
        }

        override fun onCancelled(error: DatabaseError) {
            Toast.makeText(requireContext(), "Failed to load data", Toast.LENGTH_SHORT).show()
        }
    })
}
```

However, for this I had to create an external adapter that would handle the process of displaying each item in a list and give each item the ability to be interacted with by the user.

```
class FoodItemAdapter(private val context: Context, private var foodList: List<FoodItem>) : RecyclerView.Adapter<FoodItemAdapter.FoodViewHolder>() {

    inner class FoodViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) {
        val nameTextView: TextView = itemView.findViewById(R.id.recTitle)
        val priorityTextView: TextView = itemView.findViewById(R.id.recPriority)
```

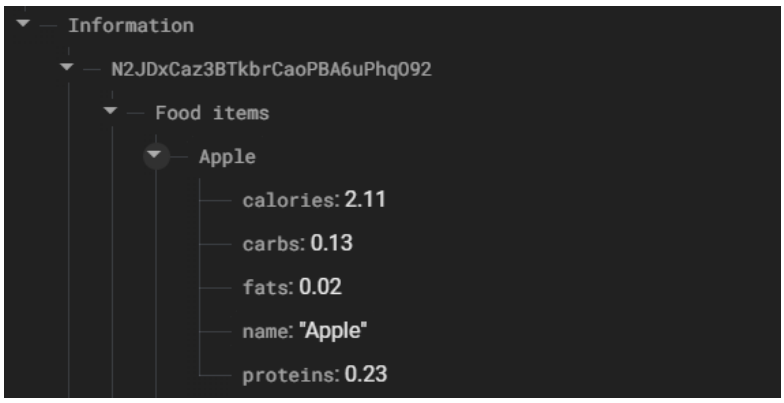
```
    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): FoodViewHolder {
        val view = LayoutInflater.from(context).inflate(R.layout.recycler_item, parent, false)
        return FoodViewHolder(view)
    }

    override fun onBindViewHolder(holder: FoodViewHolder, position: Int) {
        val foodItem = foodList[position]
        holder.nameTextView.text = foodItem.name
        holder.priorityTextView.text = "Calories: ${foodItem.calories}"
    }

    override fun getItemCount(): Int = foodList.size

    fun updateData(newList: List<FoodItem>) {
        foodList = newList
        notifyDataSetChanged()
    }
```

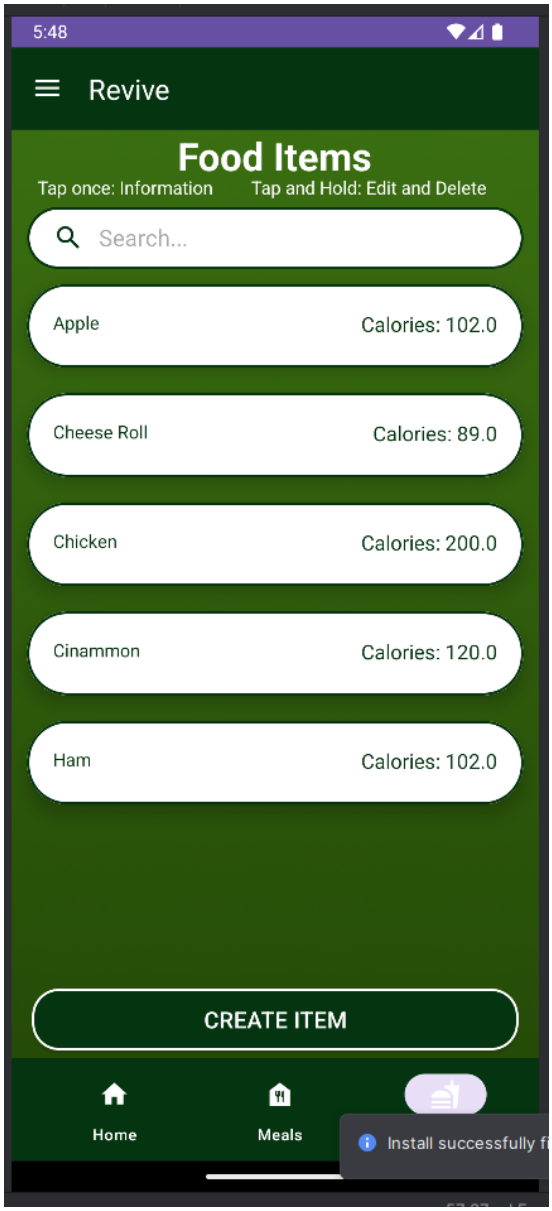
This adapter links to the xml style created for all the recycler view items recycler_item.xml from the general items at the top so that each item can be displayed as its own individual item, not as an array. The values of the title and the priority created in the xml are then found using their variables.

The onCreateViewHolder is then responsible for creating the views of the list, these are the actual items that will be displayed in the UI for the user which will use the recycler_item.xml. This essentially sets up the structure that will hold the data

The onBindViewHolder is then called by the recyler view to display the data, this will be called for each item that is created by the onCreateView so each item will have a title, which will be the name saved in firebase of the item, and the priority text, which will be the calories of the item saved in firebase. This means that each item will have values specific to the data in firebase.

This means that now in the database when a food item is created, it will be listed under its child node “food items” with a further child node for the name of the food item





Search Feature

First within my Item fragment I set up a search view quiery, so that as soon as the user enters something into the search bar the function is updated and set to true

```
searchView.setOnQueryTextListener(object : SearchView.OnQueryTextListener {
    override fun onQueryTextSubmit(query: String?): Boolean {
        return false
    }

    override fun onQueryTextChange(newText: String?): Boolean {
        searchList(newText ?: "")
        return true
    }
})
```

This then takes whatever the user has entered into the search bar and sends it over to the searchList function

```
private fun searchList(query: String) {
    if (foodList.isNotEmpty()){
        val filteredList = foodList.filter {
            it.name.toLowerCase().contains(query.toLowerCase()) // Filter food items by na
        }
        adapter.updateData(filteredList) // Update the adapter with filtered data
    }
}
```

This function validates whether the list has items, and then creates a second list based on the query so that the original list remains the same in case the user decides to delete their query, whilst the filtered one can be constantly changing. This filtered list is then sent to the adapter.

```
fun updateData(newList: List<FoodItem>) {
    foodList = newList
    notifyDataSetChanged()
}
```

Within the adapter the foodList is simply changed to the filtered list sent in, and the oncreateview and onbindview recreate the list so that only the items being searched for appear in the recycler view. Testing Code: 4.04

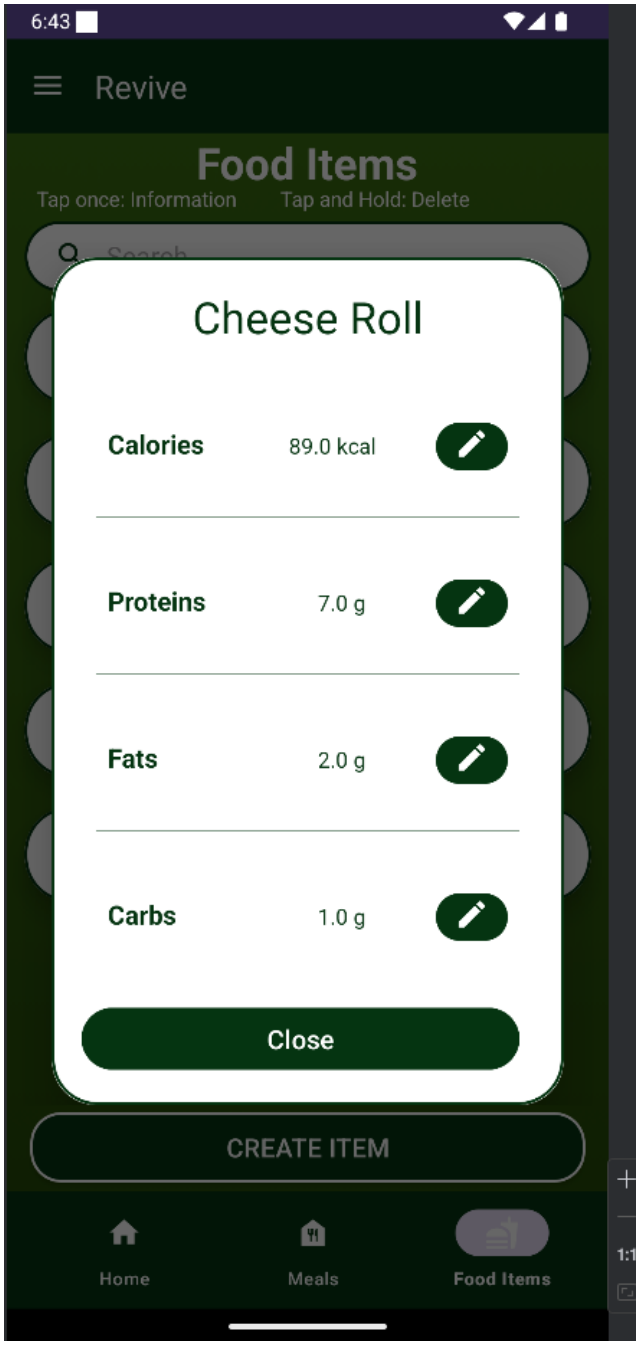
Item Interactivity

```
// Adapter for displaying food items in a RecyclerView
class FoodItemAdapter(private val context: Context, private var foodList: List<FoodItem>)
    : RecyclerView.Adapter<FoodItemAdapter.FoodViewHolder>() {

    // ViewHolder for individual food item views in the RecyclerView
    inner class FoodViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) {
        // UI elements for each food item
        val nameTextView: TextView = itemView.findViewById(R.id.recTitle)
        val priorityTextView: TextView = itemView.findViewById(R.id.recPriority)

        init {
            // Show food details dialog on click
            itemView.setOnClickListener {
                val foodItem = foodList[adapterPosition]
                showFoodItemDialog(foodItem)
            }

            // Show delete options dialog on long click
            itemView.setOnLongClickListener {
                val foodItem = foodList[adapterPosition]
                showDeleteOptionsDialog(foodItem)
                true
            }
        }
```

Within the adapter for the recycler view created earlier I added detection for when a food item is clicked or clicked and held.

```
private fun showFoodItemDialog(foodItem: FoodItem) {
        try {
            val builder = AlertDialog.Builder(context)
            val dialogView = LayoutInflater.from(context).inflate(R.layout.dialog_infoitem, null)
            val editNameBtn = dialogView.findViewById<Button>(R.id.editNameButton)
            val editCaloriesBtn = dialogView.findViewById<Button>(R.id.editCaloriesButton)
            val editProteinsBtn = dialogView.findViewById<Button>(R.id.editProteinsButton)
            val editFatsBtn = dialogView.findViewById<Button>(R.id.editFatsButton)
            val editCarbsBtn = dialogView.findViewById<Button>(R.id.editCarbsButton)

            dialogView.findViewById<TextView>(R.id.fooditemname)?.text = foodItem.name
            dialogView.findViewById<TextView>(R.id.readitemCalories)?.text = "${foodItem.calories} kcal"
            dialogView.findViewById<TextView>(R.id.readitemProteins)?.text = "${foodItem.proteins} g"
            dialogView.findViewById<TextView>(R.id.readitemFats)?.text = "${foodItem.fats} g"
            dialogView.findViewById<TextView>(R.id.readitemCarbs)?.text = "${foodItem.carbs} g"

            builder.setView(dialogView)
            val dialog = builder.create()

            dialogView.findViewById<Button>(R.id.itemClose)?.setOnClickListener {
                dialog.dismiss()
            }

            editCaloriesBtn.setOnClickListener {
                dialog.dismiss()
                showPropertyEditDialog(foodItem, "calories", "Calories", foodItem.calories.toString(), true)
            }

            editProteinsBtn.setOnClickListener {
                dialog.dismiss()
                showPropertyEditDialog(foodItem, "proteins", "Proteins", foodItem.proteins.toString(), true)
            }

            editFatsBtn.setOnClickListener {
                dialog.dismiss()
                showPropertyEditDialog(foodItem, "fats", "Fats", foodItem.fats.toString(), true)
            }

            editCarbsBtn.setOnClickListener {
                dialog.dismiss()
                showPropertyEditDialog(foodItem, "carbs", "Carbs", foodItem.carbs.toString(), true)
            }

            dialog.window?.setBackgroundDrawable(ColorDrawable(0))
            dialog.show()
        } catch (e: Exception) {
            Toast.makeText(context, "Error showing food item details", Toast.LENGTH_SHORT).show()
        }
    }
```

This is a function that is called when the user single clicks on an item to bring up the dialog_infoitem.xml which finds the users information based on the object that has been taken from firebase. Testing Code: 4.06

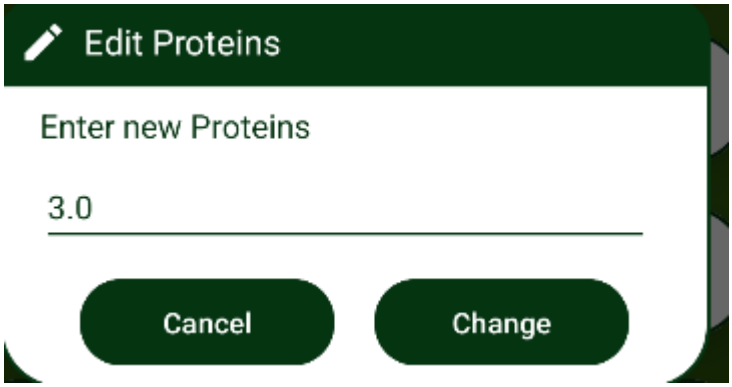


Then if one of the edit options is pressed another function is called.

Edit Items

```
// Dialog for editing individual properties of a food item (name, calories, proteins, etc.)
private fun showPropertyEditDialog(
    foodItem: FoodItem,
    property: String,
    displayName: String,
    currentValue: String,
    isNumber: Boolean
) {
    val builder = AlertDialog.Builder(context)
    // Inflate custom dialog layout
    val dialogView = LayoutInflater.from(context).inflate(R.layout.dialog_changeinfo, null)

    // Initialize UI elements from the dialog layout
    val titleTextView = dialogView.findViewById<TextView>(R.id.ChangeTitle)
    val descTextView = dialogView.findViewById<TextView>(R.id.desc)
    val editText = dialogView.findViewById<EditText>(R.id.editBoxDetails)
    val cancelButton = dialogView.findViewById<Button>(R.id.btnCancelDetails)
    val saveButton = dialogView.findViewById<Button>(R.id.btnChangeDetail)

    // Set up dialog text
    titleTextView.text = "Edit $displayName"
    descTextView.text = "Enter new $displayName"
    editText.setText(currentValue)

    // Configure input type - text for names, number for nutritional values
    if (!isNumber) {
        editText.inputType = android.text.InputType.TYPE_CLASS_TEXT
    }

    val dialog = builder.setView(dialogView).create()
```

When the edit function is called it takes in parameters to personalise the screen based on what the user is doing and updates the IDs of the XML file to reflect those values. Testing Code: 4.06



```
    // Handle save button click - updates the food item if input is valid
    saveButton.setOnClickListener {
        val newValue = editText.text.toString()
        if (newValue.isNotEmpty()) {
            // Update the appropriate property based on what's being edited
            when (property) {
                "name" -> foodItem.name = newValue
                "calories" -> foodItem.calories = newValue.toDoubleOrNull() ?: foodItem.calories
                "proteins" -> foodItem.proteins = newValue.toDoubleOrNull() ?: foodItem.proteins
                "fats" -> foodItem.fats = newValue.toDoubleOrNull() ?: foodItem.fats
                "carbs" -> foodItem.carbs = newValue.toDoubleOrNull() ?: foodItem.carbs
            }
            updateFoodItem(foodItem)
            dialog.dismiss()
        } else {
            Toast.makeText(context, "Please enter a valid value", Toast.LENGTH_SHORT).show()
        }
    }

    // Set transparent background and show dialog
    if (dialog.window != null) {
        dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
    }
    dialog.show()
}
```

Then when the user presses the save button the code finds what the value that was updated was and updates this in the object.

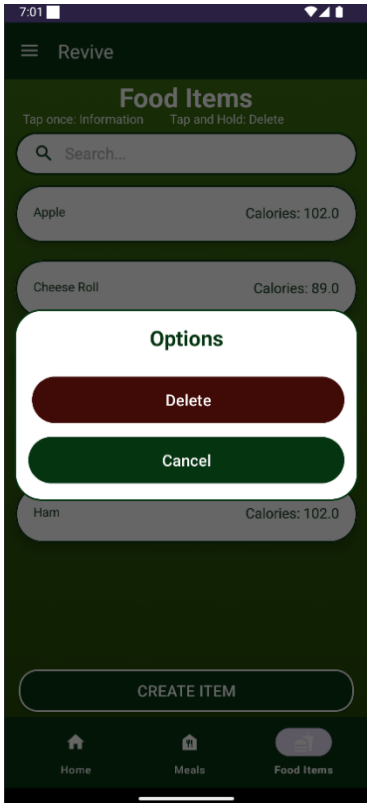
```
// Updates a food item in Firebase Realtime Database with new values
private fun updateFoodItem(foodItem: FoodItem) {
    // Get current user ID from Firebase Authentication
    val userId = FirebaseAuth.getInstance().currentUser?.uid
    if (userId != null) {
        // Create reference to specific food item in database
        val databaseRef = FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.europe-west1.firebasedatabase.app/")
            .getReference("Information")
            .child(userId)
            .child("Food items")
            .child(foodItem.name)

        foodItem.converttoigram() // Convert to 1g before saving
        databaseRef.setValue(foodItem)
            .addOnSuccessListener {
                Toast.makeText(context, "Food item updated", Toast.LENGTH_SHORT).show()
                notifyDataSetChanged()
            }
            .addOnFailureListener {
                Toast.makeText(context, "Failed to update food item", Toast.LENGTH_SHORT).show()
            }
    }
}
```

Then that food item is sent to be updated to firebase. Testing Code: 4.07

Delete Items

```
// Shows dialog with options to delete or cancel deletion of a food item
private fun showDeleteOptionsDialog(foodItem: FoodItem) {
    val builder = AlertDialog.Builder(context)
    // Inflate custom dialog layout
    val dialogView = LayoutInflater.from(context).inflate(R.layout.dialog_eidt_options, null)

    // Initialize dialog buttons
    val editButton = dialogView.findViewById<Button>(R.id.editButton)
    val deleteButton = dialogView.findViewById<Button>(R.id.deleteButton)
    val cancelButton = dialogView.findViewById<Button>(R.id.cancelButton)

    builder.setView(dialogView)
    val dialog = builder.create()
    editButton.visibility = View.GONE  // Hide edit button as it's not needed

    // Set click listeners

    deleteButton.setOnClickListener {
        dialog.dismiss()
        showDeleteConfirmationDialog(foodItem)
    }

    cancelButton.setOnClickListener {
        dialog.dismiss()
    }

    // Set transparent background
    if (dialog.window != null) {
        dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
    }

    dialog.show()
}
```

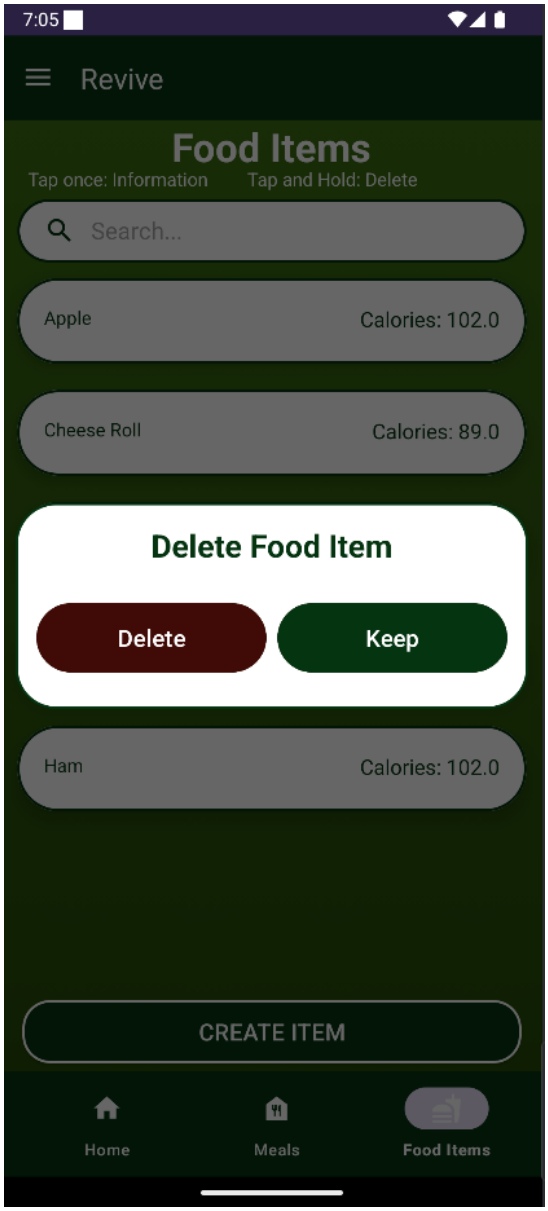
When the user clicks and holds on an item they will have the delete option displayed to them in a dialog, where the *dialog_edit_details.xml* from the general items is displayed for them and refactored to only display the delete and close button.



If the user if presses the delete button a second dialog is opened over the top asking for a final confirmation

```
private fun showDeleteConfirmationDialog(foodItem: FoodItem) {
    val builder = AlertDialog.Builder(context)
    val dialogView = LayoutInflater.from(context).inflate(R.layout.dialog_meal_options, null)

    // Get UI elements
    val titleText = dialogView.findViewById<TextView>(R.id.dialogTitle)
    val confirmButton = dialogView.findViewById<Button>(R.id.editButton)    // Reuse edit button for confirm
    val cancelButton = dialogView.findViewById<Button>(R.id.deleteButton)   // Reuse delete button for cancel
    val bottomButton = dialogView.findViewById<Button>(R.id.cancelButton)   // Hide the bottom button

    // Update UI elements
    titleText.text = "Delete Food Item"
    confirmButton.text = "Delete"
    cancelButton.text = "Keep"
    confirmButton.backgroundTintList = ColorStateList.valueOf(Color.parseColor("#400a06"))
    cancelButton.backgroundTintList = ColorStateList.valueOf(Color.parseColor("#053411"))
    bottomButton.visibility = View.GONE

    builder.setView(dialogView)
    val dialog = builder.create()

    // Set click listeners
    confirmButton.setOnClickListener {
        dialog.dismiss()
        // Inline deletion of food item instead of separate function call
        val userId = FirebaseAuth.getInstance().currentUser?.uid
        if (userId != null) {
            val databaseRef = FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.europe-west1.firebasedatabase.app/")
                .getReference("Information")
                .child(userId)
                .child("Food items")
                .child(foodItem.name)

            databaseRef.removeValue()
                .addOnSuccessListener {
                    Toast.makeText(context, "Food item deleted", Toast.LENGTH_SHORT).show()
                }
                .addOnFailureListener {
                    Toast.makeText(context, "Failed to delete food item", Toast.LENGTH_SHORT).show()
                }
        }
    }

    cancelButton.setOnClickListener {
        dialog.dismiss()
    }
```

This takes the currently selected option with the edit options dialog and changes the colour of the buttons from green to red and vice versa to swap the options so the user does not accidentally double click, and if the user chooses to confirm to delete the item it is removed from firebase.
Testing Code: 4.07



Testing

The first error that arose during testing was after an item had been created by the user, if there were any items that already existed in their list, they would be displayed twice in the UI, even though they were only saved once in firebase.

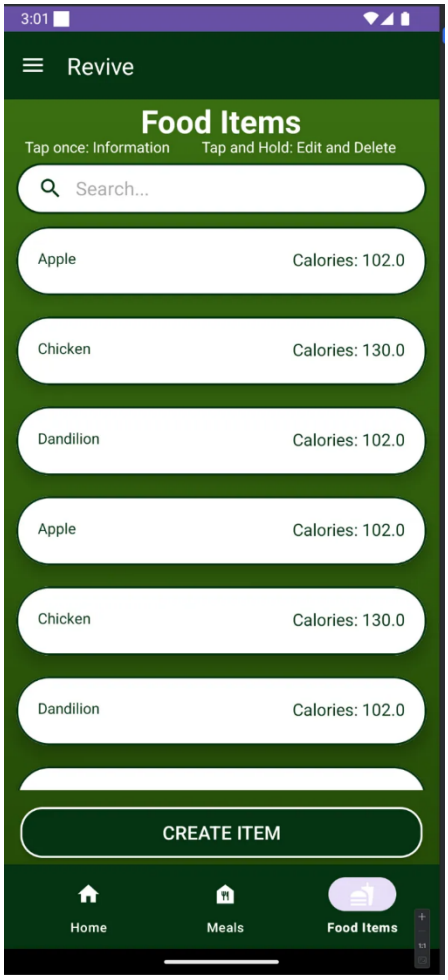
The reason for this was because the code that loaded the meal items was in the main fragment, when the dialog was dismissed, the data would be changed so the function to reload the recycler list would be called:

```
fun updateData(newList: List<FoodItem>) {
    foodList = newList
    notifyDataSetChanged()
}
```

This means the entire recycler view would be called again so items would be added to the list again, however since this code,

```
userId?.let {
    databaseReference.child(it).child("Food items").addValueEventListener(object : ValueEventListener {
        override fun onDataChange(snapshot: DataSnapshot) {
            for (foodSnapshot in snapshot.children) {
                val foodItem = foodSnapshot.getValue(FoodItem::class.java)
                if (foodItem != null) {
                    foodItem.convertto100gram() //converts back to 100g for viewing ease
                    foodList.add(foodItem) // adds each item to a list
                }
            }
            // Update adapter with fetched data
            adapter.updateData(foodList)
        }

        override fun onCancelled(error: DatabaseError) {
            Toast.makeText(requireContext(), "Failed to load data", Toast.LENGTH_SHORT).show()
        }
    })
}
```

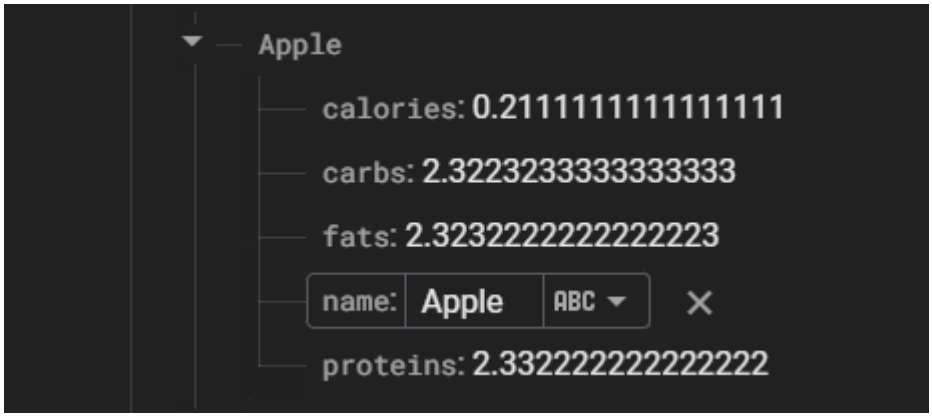
had already separated loaded the items into the list, the items that were already in the list were then loaded again, without being refreshed causing them to appear twice. Testing Code: 4.02



This was solved by adding a clear function to the ondataset changed so that the list is refreshed before the items are added: Testing Code: 4.03

```
override fun onDataChange(snapshot: DataSnapshot) {
    foodList.clear() // Add this line
    for (foodSnapshot in snapshot.children) {
        val foodItem = foodSnapshot.getValue(FoodItem::class.java)
        if (foodItem != null) {
            foodItem.convertto100gram()
            foodList.add(foodItem)
        }
    }
    adapter.updateData(foodList)
}
```

Another error that occurred during testing was when an item saves to firebase, simply dividing the inputted float by 100 would cause the database to save with an item that has a lot of recurring decimals, even though the function was to only divide by 100 of what the user entered. Testing Code: 4.04



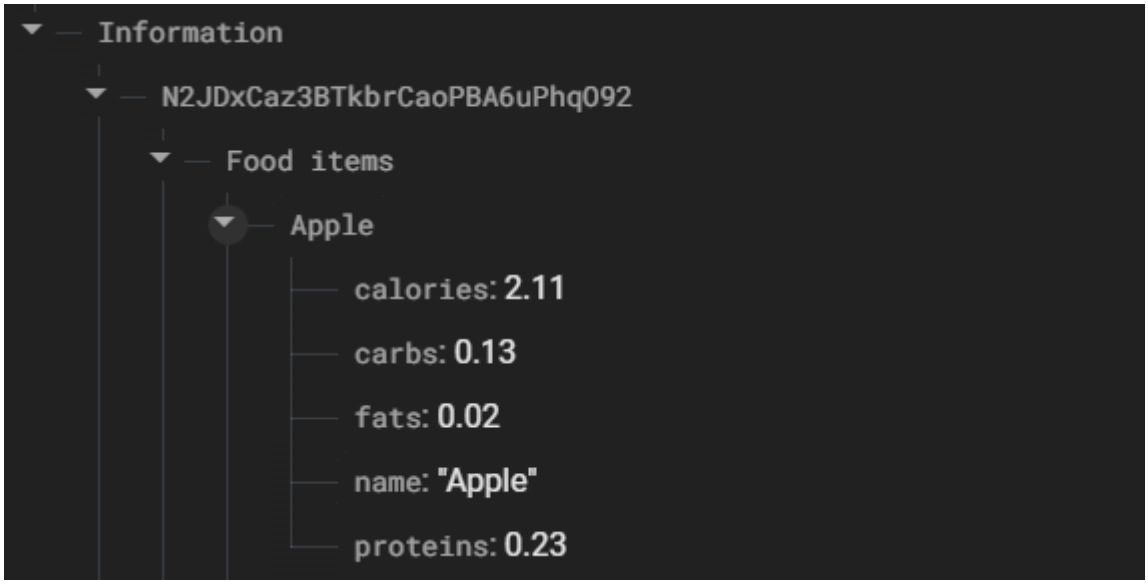
This was caused by my original conversion functions in the data class:

```
fun converttoigram() {
    calories = ((calories / 100.0))
    proteins = ((proteins / 100.0))
    fats = ((fats / 100.0))
    carbs = ((carbs / 100.0))
}

fun converttoi00gram() {
    calories = ((calories * 100.0))
    proteins = ((proteins * 100.0))
    fats = ((fats * 100.0))
    carbs = ((carbs * 100.0))
}
```

Therefore, I created a rounding function that would set the values to a low amount of decimals using a high precision. This means that if the user enters a number with a high number of decimals, the values will still be rounded to the same integer as with a low amount of decimals

```
    // Calculates 1g amounts from 100g

    fun converttoigram() {
        calories = ((calories / 100.0) * 100).roundToInt() / 100.0
        proteins = ((proteins / 100.0) * 100).roundToInt() / 100.0
        fats = ((fats / 100.0) * 100).roundToInt() / 100.0
        carbs = ((carbs / 100.0) * 100).roundToInt() / 100.0
    }

    // Calculates 1g amount to 100g
    fun converttoi00gram() {
        calories = ((calories * 100.0) * 100).roundToInt() / 100.0
        proteins = ((proteins * 100.0) * 100).roundToInt() / 100.0
        fats = ((fats * 100.0) * 100).roundToInt() / 100.0
        carbs = ((carbs * 100.0) * 100).roundToInt() / 100.0
    }
}
```

The reason this function works even though values are divided by 100 at the end is because the extra multiplicative 100s that increase the base amount of numbers before any numbers are rounded to the closest integer, then an integer divided by 100.0 should not result in any long recurring decimals, but also turns makes sure the value in the variable is still a double.



This ensured the correct rounding of values entered into the database. Testing Code: 4.05

Food Items testing video

Test ID 4	Timestamp	PURPOSE	DATA	EXPECTED RESULTS	ACTUAL RESULTS
4.01	n/a	To test whether the app will allow the user to create an item without filling in all the fields.	No data	The app displays an error message to the user to fill in all the fields	As expected
4.02	0:00	To test whether the saving of a food item created by the user, saving all variables to firebase under an object and displaying accordingly in UI.	Name: “Apple” Calories: 102 Protein: 3.2 Fats: 7.6 Carbs: 12	The app saves the data correctly to firebase and adds each item to the UI.	Pass and fail, Pass: Items are saved corectly in firebase Fail: The UI is updated so each item is stored multiple times.
4.03	0:33	To test whether the saving of a food item created by the user, saving all variables to firebase under an object and displaying accordingly in UI.	Name: “Apple” Calories: 102 Protein: 3.2 Fats: 7.6	The app saves the data correctly to firebase and adds each item to the UI.	As expected.

			Carbs: 12		
4.04	n/a	To test whether the data is saved to firebase with the correct data and to the correct decimal places.	Data from object creation	The app correctly saves the data under the correct object and user information.	Fail, values are incredibly long and not rounded to convenience
4.05	n/a	To test whether the data is saved to firebase with the correct data and to the correct decimal places.	Data from object creation	The app correctly saves the data under the correct object and user information.	As expected.
4.06	0:59	To test whether the user is able to enter a query into the search bar and the recycler list is updated accordingly.	Search Querys: “Straw” “Apple” “Da”	The app updates the recycler list visible to include only the items related to the query, and if there are none there will be no items displayed.	As expected
4.05	1:20	To test whether the data created by the user is read from firebase and displayed in the UI accordingly	N/a, data from firebase	The app updates the text views in the dialog and displays the correct, rounded values.	As expected.
4.06	1:27	Whether the user is able to press the edit button and have a dialog show up, then change a specific detail by inputting it into the dialog, then test whether the app allows them to submit a blank box.	Empty box	The app displays an error message asking the user to enter details	As expected.
4.07	1:40	Whether the user is able to press the edit button and change a specific detail by inputting it into the dialog, and then whether it updates in firebase.	New value for ham: 32.45	The app updates the UI to show the new values and updates the data in firebase.	As expected.
4.08	1:58	To test whether the app allows the user to select the delete option, confirm it and remove the food object from firebase	n/a	The app updates the UI to remove the item, and the database entry is also removed.	As expected.

Module Summary

The Food Item Creation module enables users to build a personal database of food items by inputting nutritional values based on product labels. Users can create, view, edit, and delete food items, with each item storing values per 100g for consistent calculation. The module includes search functionality to easily find existing items and validates all numerical inputs to ensure data integrity.

The prototype thus far includes account creation, personal information management, and now a foundational food database system. Users can create customized food entries that will serve as building blocks for meal creation and calorie tracking. The approach of storing values per 100g and then converting to 1g internally is justified as it matches how nutritional labels display information, making data entry intuitive for users while maintaining precise calculations. The module's search functionality and edit capabilities recognize that users need to manage an expanding food database efficiently. While users can now create food items, the prototype still needs meal creation and daily tracking features to become functionally complete. The current implementation focuses on data accuracy and usability, ensuring a reliable foundation for the more complex features to come.

Meal Creation Module

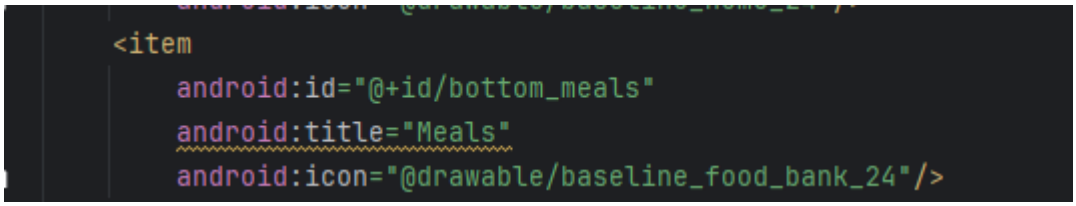
I developed a meal management system within the fitness application that allows users to create, view, edit, and delete custom meals composed of their saved food items. I structured this system using two main components: a primary MealFragment for displaying saved meals and a CreateMealFragment for meal creation and editing.

XMLS

These are all the layout files that will be used throughout this Module, with their respective names under the title as they will be referred to throughout the code explanation.

Navigation Change

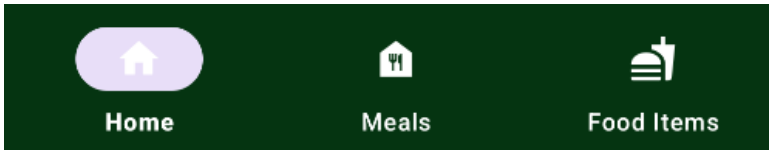
I decided that the meal module should be its own section that the user can navigate to, instead of being inside the singular meal item section, so I have added a meal items section to the bottom_nav.xml mentioned in the navigation modules with an id, name and an icon.



and added this to the function calls when fragments are opened:

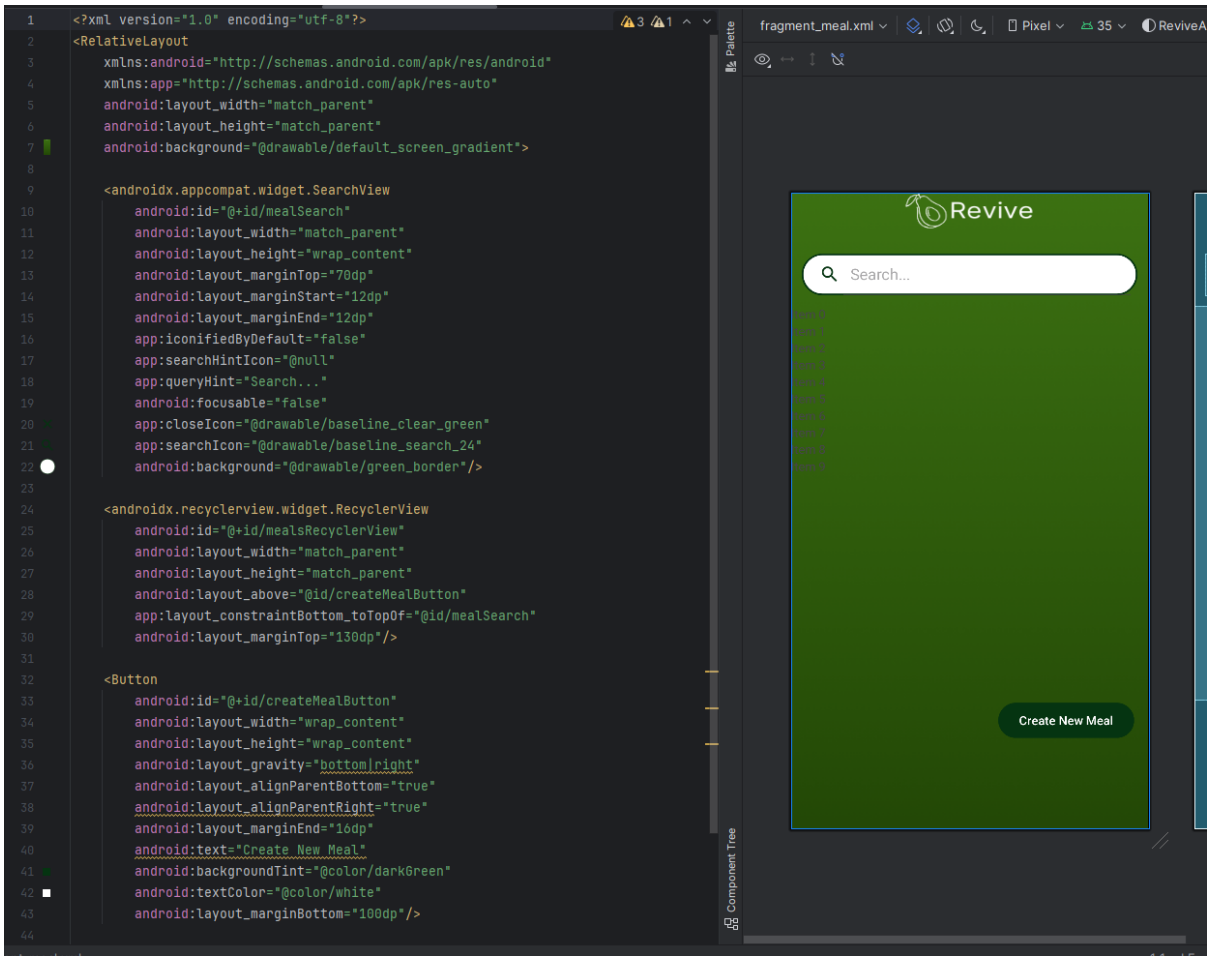
```
binding.bottomNavigation.setOnItemSelectedListener{ item -> // this listens to the BOTTOM navigation items
    when(item.itemId){
        //replace fragment based on selected items
        R.id.bottom_home -> openFragment(HomeFragment())
        R.id.bottom_foods -> openFragment(ItemFragment())
        R.id.bottom_meals -> openFragment(MealFragment())
    }
    true
}
```

So the new bottom navigation bar looks like this:



Meal Fragment

fragment_meal.xml



This XML section determines a bottom navigation fragment that will be used to create Meals out of already created food items. This XML layout defines a visually appealing Android interface using a RelativeLayout as the root container, allowing flexible positioning of its child elements. The background is set to the gradient background mentioned up top.

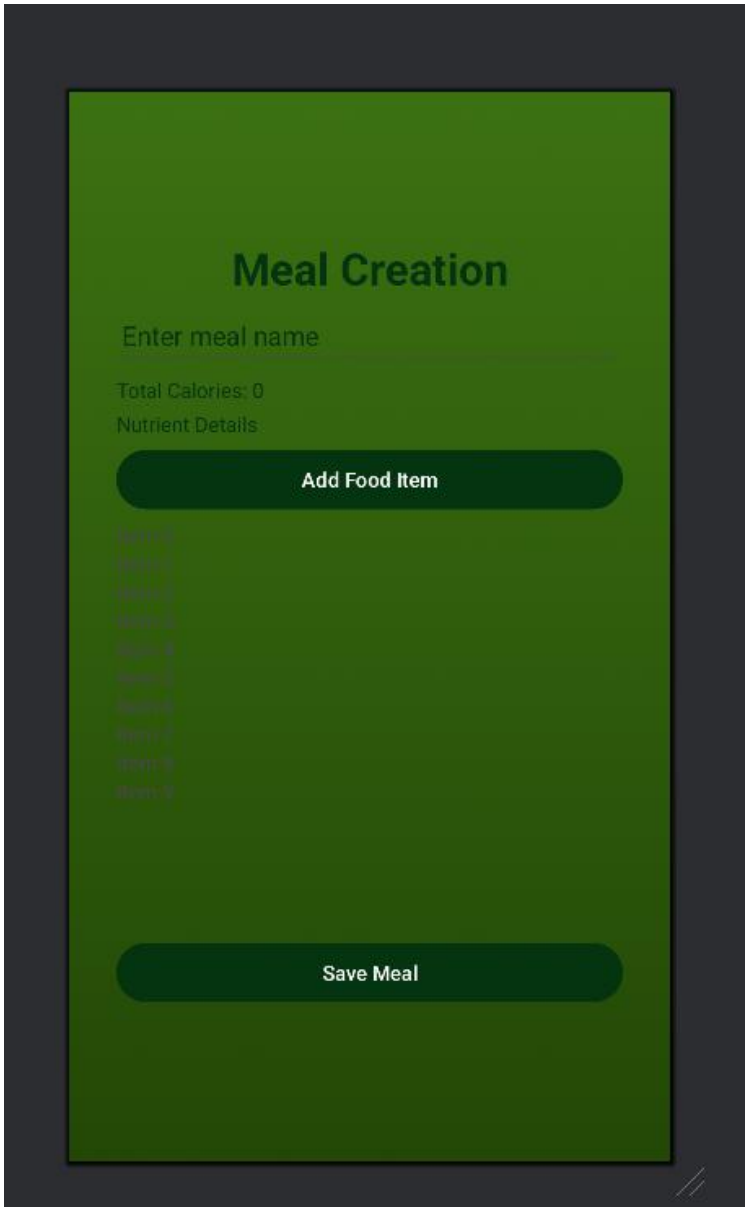
1. **Title TextView**: Displays the title "Meal Items" at the top, centered and styled with a large, bold font and white color.
2. **Horizontal LinearLayout**: Contains two TextView elements that explain user actions ("Tap once: Information" and "Tap and Hold: Edit and Delete"), positioned below the title with horizontal orientation and spacing.
3. **SearchView**: Provides a search bar for filtering meal items, styled with a custom border, placeholder text, and icons. It is positioned further down with margins.
4. **RecyclerView**: Displays a scrollable list of meal items, filling most of the available space and positioned above a floating button using layout_above. This will reuse the recycler_view.xml style created for the object creation module
5. **Create Button**: Positioned at the bottom, this button lets users create a new meal. It sits at the bottom right of the screen and is styled with padding, a dark green background, and white text.

Create Meal Screen

fragment_create_meal.xml

ProfileFragment.kt fragment_create_meal.xml fragment_profile.xml

```xml
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/ar
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@drawable/default_screen_gradient">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:layout_marginStart="12dp"
        android:layout_marginEnd="12dp"
        android:layout_marginTop="80dp"
        android:layout_marginBottom="85dp"
        android:orientation="vertical"
        android:padding="20dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Meal Creation"
            android:textAlignment="center"
            android:layout_gravity="center"
            android:textColor="@color/darkGreen"
            android:textStyle="bold"
            android:textSize="30sp"/>

        <EditText
            android:id="@+id/mealNameInput"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:hint="Enter meal name"
            android:layout_marginTop="4dp"
            android:textColor="@color/darkGreen"
            android:textColorHint="@color/darkGreen"/>

        <TextView
            android:id="@+id/mealTotalCalories"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="4dp"
            android:textColor="@color/darkGreen"
            android:text="Total Calories: 0"/>

        <TextView
            android:id="@+id/mealTotalNutrients"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="4dp"
            android:textColor="@color/darkGreen"
            android:text="Nutrient Details"/>

        <Button
            android:id="@+id/addFoodButton"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Add Food Item"
            android:layout_marginTop="4dp"
            android:backgroundTint="@color/darkGreen"
            android:textColor="@color/white"/>

        <androidx.recyclerview.widget.RecyclerView
            android:id="@+id/selectedFoodsRecyclerView"
            android:layout_width="match_parent"
            android:layout_height="0dp"
            android:layout_weight="1"
            android:layout_marginTop="4dp"
            android:layout_marginBottom="4dp"/>

        <Button
            android:id="@+id/saveMealButton"
            android:layout_width="150dp"
            android:layout_gravity="end"
            android:layout_height="wrap_content"
            android:text="Save Meal"
            android:backgroundTint="@color/darkGreen"
            android:textColor="@color/white"/>

    </LinearLayout>
</FrameLayout>
```

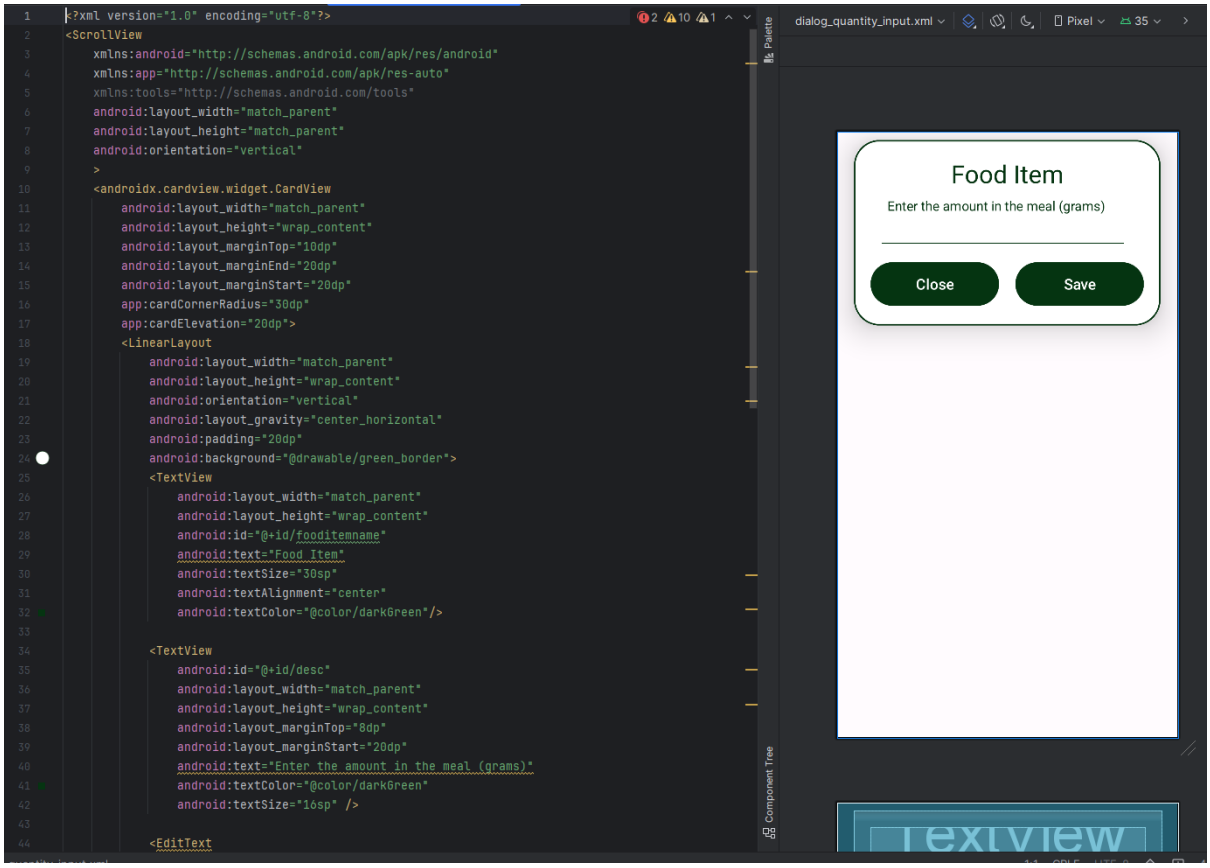


Here is the meal creation fragment that the button mentioned earlier will lead into, with the same principle as the recycler_item.xml with a recycler view. Each meal will need to be identified with a name, therefore at the top they are given that option, then there will be two textviews that initially display the total calories in the meal, which is 0 and that where the nutrient details will be updated when a food object is added. A button labeled "Add Food Item" allows users to include individual food items in the meal. It has a dark green background and white text, styled for consistency. Positioned at the bottom is a “save meals button", this button saves the meal. It is styled similarly to the "Add Food Item" button, with a margin at the bottom to prevent overlap with system UI elements.

The layout is clean and functional, designed for user interaction and data display, with a focus on clarity and aesthetics.

Dialog to enter amount in grams

dialog_quantity_input.xml


<?xml version="1.0" encoding="utf-8"?>
<ScrollView
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
>
<androidx.cardview.widget.CardView
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:layout_marginTop="10dp"
android:layout_marginEnd="20dp"
android:layout_marginStart="20dp"
app:cardCornerRadius="30dp"
app:cardElevation="20dp">
<LinearLayout
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:orientation="vertical"
android:layout_gravity="center_horizontal"
android:padding="20dp"
android:background="@drawable/green_border">
<TextView
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:id="@+id/fooditemname"
android:text="Food Item"
android:textSize="30sp"
android:textAlignment="center"
android:textColor="@color/darkGreen"/>
<TextView
android:id="@+id/desc"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:layout_marginTop="8dp"
android:layout_marginStart="20dp"
android:text="Enter the amount in the meal (grams)"
android:textColor="@color/darkGreen"
android:textSize="16sp" />
<EditText
dialog_quantity_input.xml
Pixel
35
Food Item
Enter the amount in the meal (grams)
Close
Save
Palette
Component Tree

```
                android:layout_width="300dp"
                android:layout_height="wrap_content"
                android:id="@+id/quantityInput"
                android:textColor="@color/darkGreen"
                android:textSize="16sp"
                android:layout_marginStart="10sp"
                android:backgroundTint="@color/darkGreen"
                android:layout_marginBottom="10dp"
                android:maxLines="1"
                android:inputType="numberDecimal"/>


            <!-- Upload Fields -->
            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="horizontal"
                android:layout_marginTop="1dp">

                <Button
                    android:layout_width="0dp"
                    android:layout_height="60dp"
                    android:layout_weight="1"
                    android:text="Close"
                    android:id="@+id/MealCloseBtn"
                    android:textSize="18sp"
                    android:backgroundTint="@color/darkGreen"
                    android:layout_marginEnd="10dp"
                    android:textColor="@color/white"/>

                <Button
                    android:layout_width="0dp"
                    android:layout_height="60dp"
                    android:layout_weight="1"
                    android:text="Save"
                    android:id="@+id/MealSaveBtn"
                    android:textSize="18sp"
                    android:backgroundTint="@color/darkGreen"
                    android:layout_marginStart="10dp"
```

_quantity_input.xml

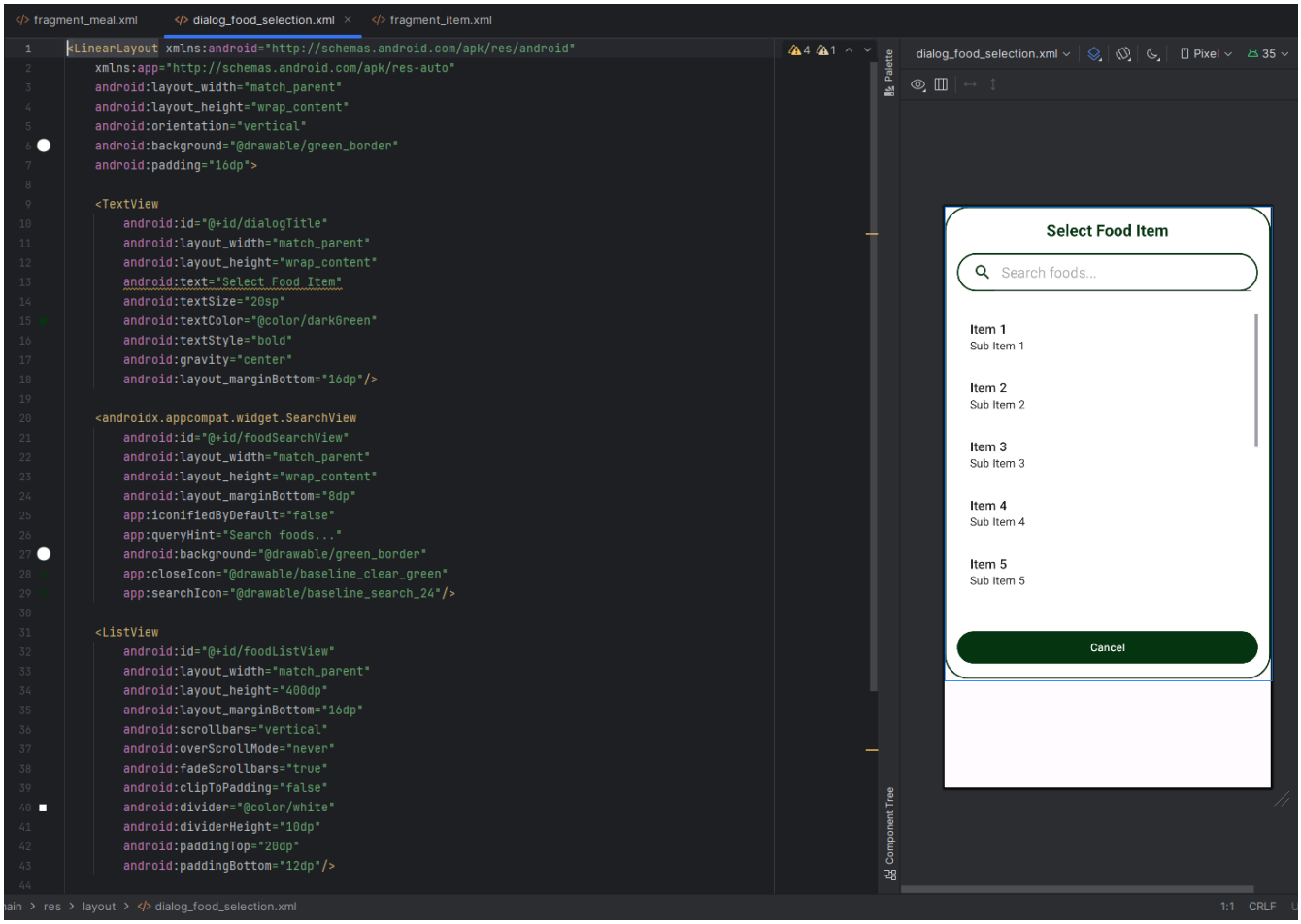
```
                    android:textColor="@color/white"/>

            </LinearLayout>
        </LinearLayout>
    </androidx.cardview.widget.CardView>
</ScrollView>
```

This XML layout file defines a user interface for an Android app using a ScrollView container to ensure the content is scrollable vertically. Inside the ScrollView, there is a CardView that provides a styled card-like design with rounded corners (app:cardCornerRadius) and elevation (app:cardElevation). The CardView contains a LinearLayout, which organizes its child views vertically and applies padding and a custom background mentioned at the which is the green_border. Inside

the layout, a TextView displays the name of a food item with large text size and centered alignment, while another TextView serves as a description prompt. Below these, an EditText allows users to input a numeric value for quantity with specific styles like color and background tint. Lastly, a LinearLayout organizes two Button elements horizontally for "Close" and "Save" actions. These buttons are styled with weights to share space equally, custom colors, and rounded backgrounds for consistent aesthetics. The entire layout uses attributes and custom resources for a visually appealing and functional design instead of system designs.

Dialog to select food

dialog_food_selection.xml

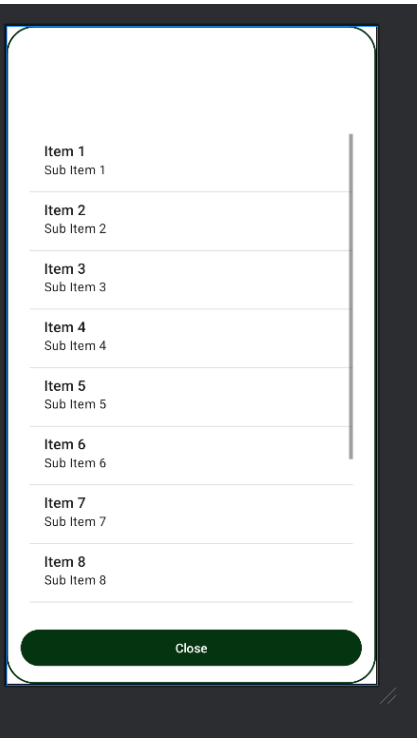


This layout defines a vertical LinearLayout for a dialog with a green border and padding. It contains a TextView for the title, "Select Food Item", styled with a bold, centered, dark green font. Below it is a SearchView allowing users to search for food items, with custom icons for searching and clearing. A ListView follows, displaying the list of food items with vertical scrolling and custom dividers. Finally, a Button labeled "Cancel" is provided at the bottom, styled with a dark green background and white text. The dialog layout has defined margins and paddings to ensure proper spacing between elements.

Dialog to display meal details

dialog_meal_details.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="16dp"
    android:background="@drawable/green_border">

    <TextView
        android:id="@+id/mealNameText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="20sp"
        android:textStyle="bold"
        android:textColor="@color/darkGreen"
        android:gravity="center"
        android:layout_marginBottom="8dp"/>

    <TextView
        android:id="@+id/totalCaloriesText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textColor="@color/darkGreen"
        android:layout_marginBottom="4dp"/>

    <TextView
        android:id="@+id/nutrientsText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textColor="@color/darkGreen"
        android:layout_marginBottom="16dp"/>

    <ListView
        android:id="@+id/foodItemsList"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:padding="10dp"
        android:layout_marginBottom="16dp"
        android:cacheColorHint="@color/black"/>

    <Button
        android:id="@+id/closeButton"
        android:layout_width="match_parent"
```



This XML layout defines a dialog for displaying meal details. It uses a vertical LinearLayout with padding and a green border as the background. The layout contains three TextView elements: one for the meal name, one for the total calories, and another for the nutritional information. These text views are styled with green color and some margin for spacing. A ListView is included to display a list of food items in the meal, with padding and space allocated using layout_weight. Finally, a Button is placed at the bottom to allow the user to close the dialog, with a dark green background and white text. The layout is structured to provide a clear and organized presentation of the meal details.

Logic

EXPLANATION: This section of the object creation module will allow the users to create and manage custom meal combination from their previously created food items. The user will be sent to a separate screen that allows them to select individual food items and save them in quantities for future use. The “meals” will be saved in firebase so they can be accessed from any device and be created as objects using OOP making it convenient and efficient.

Adapter and Meal Fragment Setup

```
data class Meal(
    var name: String = "",
    var foodItems: MutableMap<String, FoodItemWithQuantity> = mutableMapOf(),
    var totalCalories: Double = 0.0,
    var totalProteins: Double = 0.0,
    var totalFats: Double = 0.0,
    var totalCarbs: Double = 0.0
) {
    fun updateTotals() {
        totalCalories = foodItems.values.sumOf { it.calories * it.grams }
        totalProteins = foodItems.values.sumOf { it.proteins * it.grams }
        totalFats = foodItems.values.sumOf { it.fats * it.grams }
        totalCarbs = foodItems.values.sumOf { it.carbs * it.grams }
    }
}

data class FoodItemWithQuantity(
    val foodItem: FoodItem = FoodItem(), // Add default value
    var grams: Double = 0.0
) {
    // Add no-argument constructor for Firebase
    constructor() : this(FoodItem(), 0.0)

    val calories: Double get() = foodItem.calories
    val proteins: Double get() = foodItem.proteins
    val fats: Double get() = foodItem.fats
    val carbs: Double get() = foodItem.carbs
}
```

I first created the Meal class which contains a name, a map of food items with their quantities, and nutritional values like calories, proteins, fats, and carbohydrates. It includes an updateTotals method to calculate these totals by summing the nutritional values of all food items in the meal based on their quantity which is the list of foods that will be placed into the data class.

The FoodItemWithQuantity class pairs a FoodItem object with its quantity in grams and provides calculated nutritional values (calories, proteins, fats, and carbs) based on the food item's properties. This structure efficiently tracks and updates meal data dynamically.

```
class MealFragment : Fragment(), MealAdapter.MealClickListener {
    private var _binding: FragmentMealBinding? = null
    private val binding get() = _binding!!
    private lateinit var databaseReference: DatabaseReference
    private lateinit var mealAdapter: MealAdapter

    override fun onCreateView(inflater: LayoutInflater, container: ViewGroup?, savedInstanceState: Bundle?
    ): View {
        _binding = FragmentMealBinding.inflate(inflater, container, false)

        // Initialize Firebase first
        val userId = FirebaseAuth.getInstance().currentUser?.uid
        databaseReference = FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.europe-west1.firebasedatabase.app")
            .getReference("Information")
            .child(userId ?: "")
            .child("Meals")
        setupRecyclerView()
```

I then created the initial fragment, setup binding and setup the database reference and userid for later use as well as variables for the adapters that will be created.

```
    binding.createMealButton.setOnClickListener {
        val transaction = parentFragmentManager.beginTransaction()
        transaction.replace(R.id.fragment_container, CreateMealFragment())
        transaction.addToBackStack(null)
        transaction.commit()
    }

    return binding.root
```

Then I create a button that would navigate to another fragment on top of the fragment displayed, without destroying the old one. I did this using a stack so that the last one in, is the first one out so if the user wishes to, they can return to the previous fragment by popping the stack.

```
class MealItemAdapter : RecyclerView.Adapter<MealItemAdapter.MealItemViewHolder>() {
    private var mealItems = mutableListOf<FoodItemWithQuantity>()

    inner class MealItemViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) {
        val nameTextView: TextView = itemView.findViewById(R.id.recTitle)
        val detailsTextView: TextView = itemView.findViewById(R.id.recPriority)
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): MealItemViewHolder {
        val view = LayoutInflater.from(parent.context)
            .inflate(R.layout.recycler_item, parent, false) // Using your existing layout
        return MealItemViewHolder(view)
    }

    override fun onBindViewHolder(holder: MealItemViewHolder, position: Int) {
        val item = mealItems[position]
        holder.nameTextView.text = item.foodItem.name
        // Display quantity and calories in the priority TextView
        holder.detailsTextView.text = "${item.grams}g - ${(item.calories * item.grams).roun
    }

    override fun getItemCount() = mealItems.size

    fun updateMeals(items: List<FoodItemWithQuantity>) {
        mealItems.clear()
        mealItems.addAll(items)
        notifyDataSetChanged()
    }
}
```

This adapter works the same way as the previous adapter in the food creation module, using the same recycler_item.xml for appearance and loads the items the same way using onCreateViewHolder, which holds references to views for each meal item and onBindViewHolder, which adds the data to each meal item by using the IDs in the XML.

This time I made sure to use the clear() function when updating the meals for the array when data is changed so I made it its own function.

This allowed for the creation of a blank meal screen since there are no meals created just yet, where clicking the “create new meal” button sends the user to a new fragment, the meal creation fragment.

Meal Creation Fragment

EXPLANATION: This will be a separate fragment on top of the meal fragment, which separates the two, meaning one will display all the already created meal items, and the other will allow the user to create those items.

It is important that the two are split and not created using dialogs like the individual food item section as there are multiple different recycler lists that will be needed and options for the user to choose from, which means it would be harder and take more code to send information back and forth between dialogs opening and closing them constantly, which would not only dimish the efficiency of the code, but also give the user an unpleasant experience. Instead a child fragment is

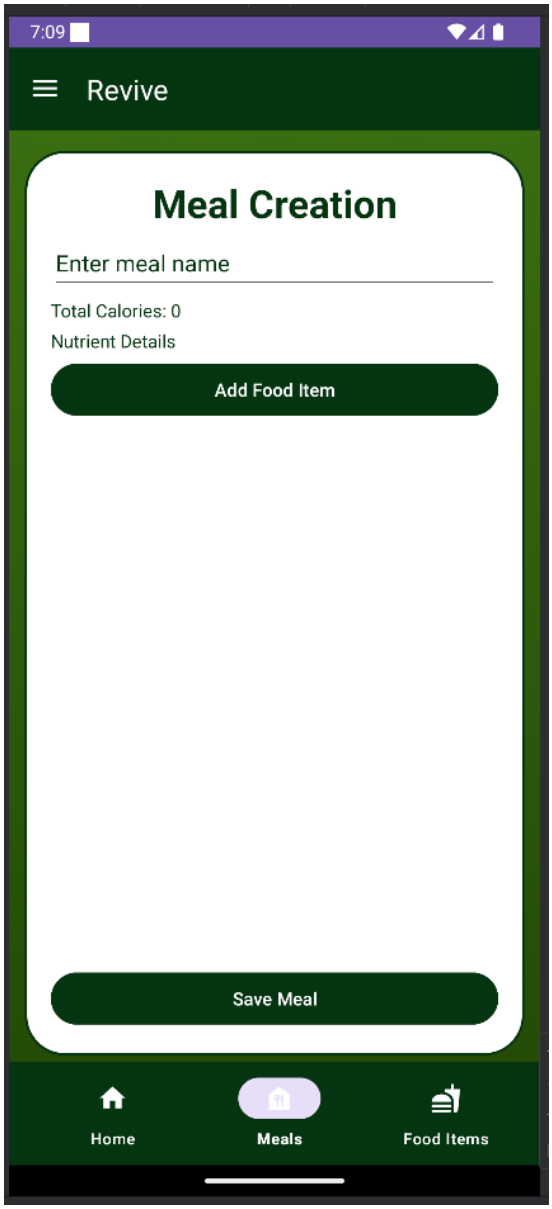
called that will be able to have all the necessary information and then be popped from the stack with an update function back to the main meal fragment.

```
class CreateMealFragment : Fragment() {
    private var _binding: FragmentCreateMealBinding? = null
    private val binding get() = _binding!!
    private lateinit var databaseReference: DatabaseReference
    private lateinit var selectedFoodsAdapter: SelectedFoodsAdapter
}
```

```
override fun onCreateView(inflater: LayoutInflater, container: ViewGroup?, savedInstanceSt
): View {
    // Initialize the view and setup necessary components
    _binding = FragmentCreateMealBinding.inflate(inflater, container, false)

    setupRecyclerView()


    binding.addFoodButton.setOnClickListener {
        // Show food selection dialog
        showFoodSelectionDialog()
    }
    binding.saveMealButton.setOnClickListener {
        saveMealToFirebase()
    }
}
```

```
    private fun setupRecyclerView() {
        mealAdapter = MealAdapter()
        binding.mealsRecyclerView.apply {
            layoutManager = LinearLayoutManager(context)
            adapter = mealAdapter
        }
    }
```

I then setup the creation fragment with two buttons to call to functions and setup binding.



This is the screen that will initially show when the user is creating a meal, and I will use a recycler view to once again list items, but this time it will be items selected through the add items button.
Testing Code: 5.01

```
class SelectedFoodsAdapter : RecyclerView.Adapter<SelectedFoodsAdapter.SelectedFoodViewHolder
    private var selectedFoods = mutableListOf<FoodItemWithQuantity>()

    inner class SelectedFoodViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) {
        val nameTextView: TextView = itemView.findViewById(R.id.recTitle)
        val detailsTextView: TextView = itemView.findViewById(R.id.recPriority)
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): SelectedFoodViewHolder
        val view = LayoutInflater.from(parent.context)
            .inflate(R.layout.recycler_item, parent, false)
        return SelectedFoodViewHolder(view)
    }

    override fun onBindViewHolder(holder: SelectedFoodViewHolder, position: Int) {
        val item = selectedFoods[position]
        holder.nameTextView.text = "${item.foodItem.name} (${item.grams}g)"
        holder.detailsTextView.text = "Cal: ${(item.calories * item.grams).roundToDecimal(1)}
    }

    override fun getItemCount() = selectedFoods.size

    fun updateItems(items: List<FoodItemWithQuantity>) {
        selectedFoods.clear()
        selectedFoods.addAll(items)
        notifyDataSetChanged()
    }
```

Therefore I created another adapter class. The setup code to this adapter is the same as the regular meal adapter, however it is justified to create another adapter as this one will be populated with individual food items that the user chooses, not all the food items as before. If I were to reuse the same adapter, since the fragments are on a stack, the previous meal list would display items chosen by the user as created meal items, which would cause logic errors, furthermore this allows me to give different functionality to this specific list.

Add food items during creation

EXPLANATION: This button will bring up a dialog that allows the user to select from a list of their already created individual food items, and then select a quantity to add to the meal, from this the create meal fragment should update to display the currently added items to the meal, which the user can then see the totals of all the food items added.

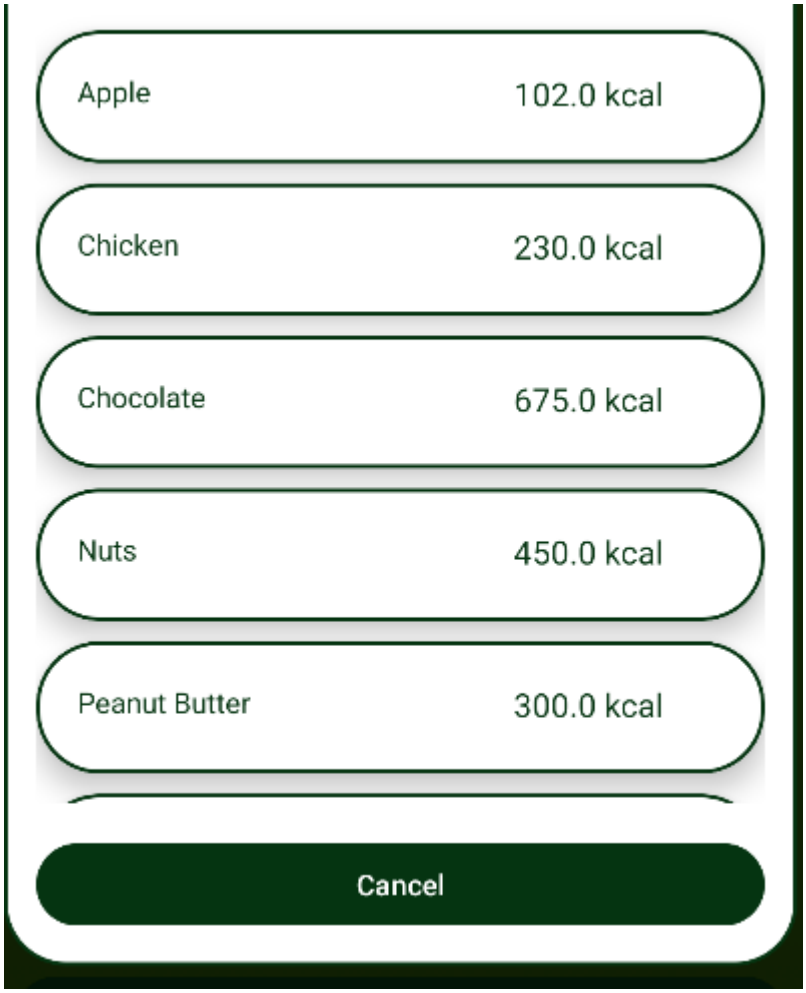
```
private fun showFoodSelectionDialog() {
    val userId = FirebaseAuth.getInstance().currentUser?.uid
    val foodsRef = FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.europe-west1.firebasedatabase.app/")
        .getReference("Information")
        .child(userId ?: "")
        .child("Food items")

    foodsRef.get().addOnSuccessListener { snapshot ->
        val foods = mutableListOf<FoodItem>()
        snapshot.children.forEach { foodSnapshot ->
            val food = foodSnapshot.getValue(FoodItem::class.java)
            food?.let {
                it.convertto100gram() // Convert to 100g for display
                foods.add(it)
            }
        }
```

This is the function that is called when the user presses the add button. It pulls all the items saved under the individual food items in firebase ”foods” and adds the objects to a list called foods.

```
if (foods.isNotEmpty()) {
    val dialogView = LayoutInflater.from(requireContext()).inflate(R.layout.dialog_food_se
    val builder = AlertDialog.Builder(requireContext())
    builder.setView(dialogView)
    val dialog = builder.create()
```

```
    // Find views in the custom layout
    val titleTextView = dialogView.findViewById<TextView>(R.id.dialogTitle)
    val foodListView = dialogView.findViewById<ListView>(R.id.foodListView)
    val cancelButton = dialogView.findViewById<Button>(R.id.cancelButton)

    // Set up the list adapter using      existing recycler_item layout
    val adapter = ArrayAdapter(
        requireContext(),
        R.layout.recycler_item,
        R.id.recTitle,  // Use      existing TextView ID
        foods.map { it.name }
    )
    foodListView.adapter = adapter
```

Then the dialog_food_selection.xml is opened and uses the adapter created to add the individual food items to the list.


Apple
102.0 kcal
Chicken
230.0 kcal
Chocolate
675.0 kcal
Nuts
450.0 kcal
Peanut Butter
300.0 kcal
Cancel

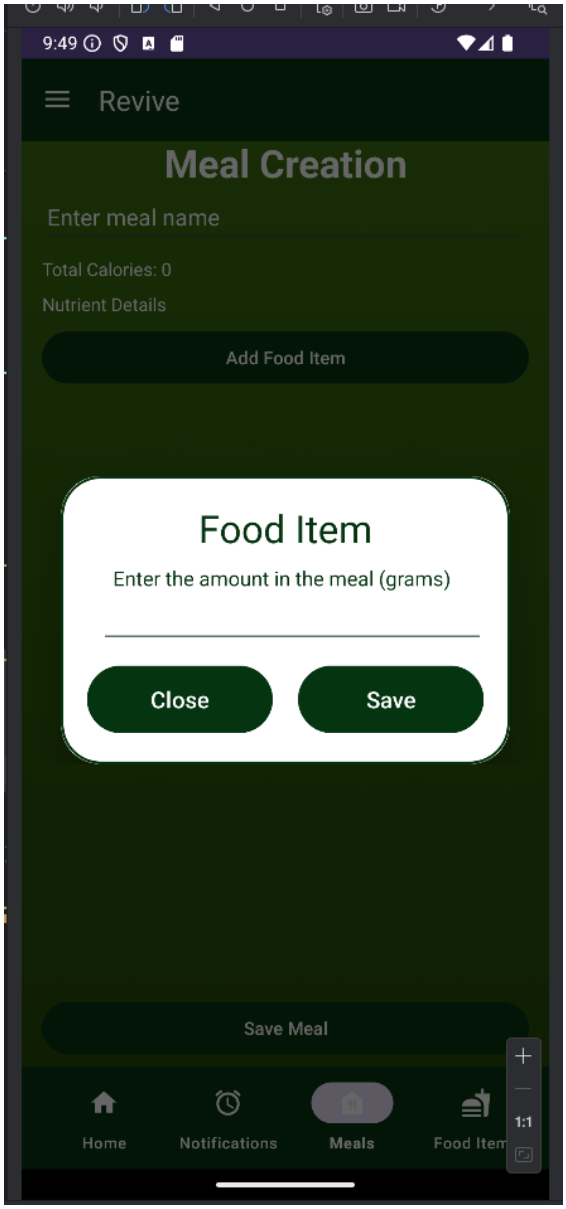
```
    foodListView.setOnItemClickListener { _, _, position, _ ->
        val selectedFood = foods[position].copy()
        selectedFood.convertto1gram() // Convert back to 1g before showing quantity dialog
        showFoodQuantityDialog(selectedFood)
        dialog.dismiss()
    }

    cancelButton.setOnClickListener {
        dialog.dismiss()
    }

    if (dialog.window != null) {
        dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
    }

    dialog.show()
} else {
    Toast.makeText(context, "No food items available. Please add some first.", Toast.LENGT
}.addOnFailureListener {
    Toast.makeText(context, "Failed to load food items", Toast.LENGTH_SHORT).show()
}
```

Then added the listeners for the close button, and for when a food item is pressed, as well as validation to make sure that the user has created food items first before they are able to create a food item, which displays an informative message. Testing Code: 5.01

```
private fun showFoodQuantityDialog(foodItem: FoodItem) {
    val builder = AlertDialog.Builder(requireContext())
    val view = layoutInflater.inflate(R.layout.dialog_quantity_input, null)

    val quantityInput = view.findViewById<EditText>(R.id.quantityInput)
    val confirmButton = view.findViewById<Button>(R.id.confirmButton)
    val cancelButton = view.findViewById<Button>(R.id.cancelButton)

    builder.setView(view)
    val dialog = builder.create()

    confirmButton.setOnClickListener {
        val quantity = quantityInput.text.toString().toDoubleOrNull()
        if (quantity != null && quantity > 0) {
            addFoodItemToMeal(foodItem, quantity)
            dialog.dismiss()
            updateMealDisplay()
        } else {
            Toast.makeText(context, "Please enter a valid quantity", Toast.LENGTH_SHOR
        }
    }

    cancelButton.setOnClickListener { dialog.dismiss() }

    if (dialog.window != null){
        dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
    }

    dialog.show()
}
```

This code triggers the dialog_quantity_input.xml style created earlier.

When the "Confirm" button is clicked, if the input is invalid meaning it is empty, a toast message prompts the user to enter a valid quantity. Testing Code: 5.02



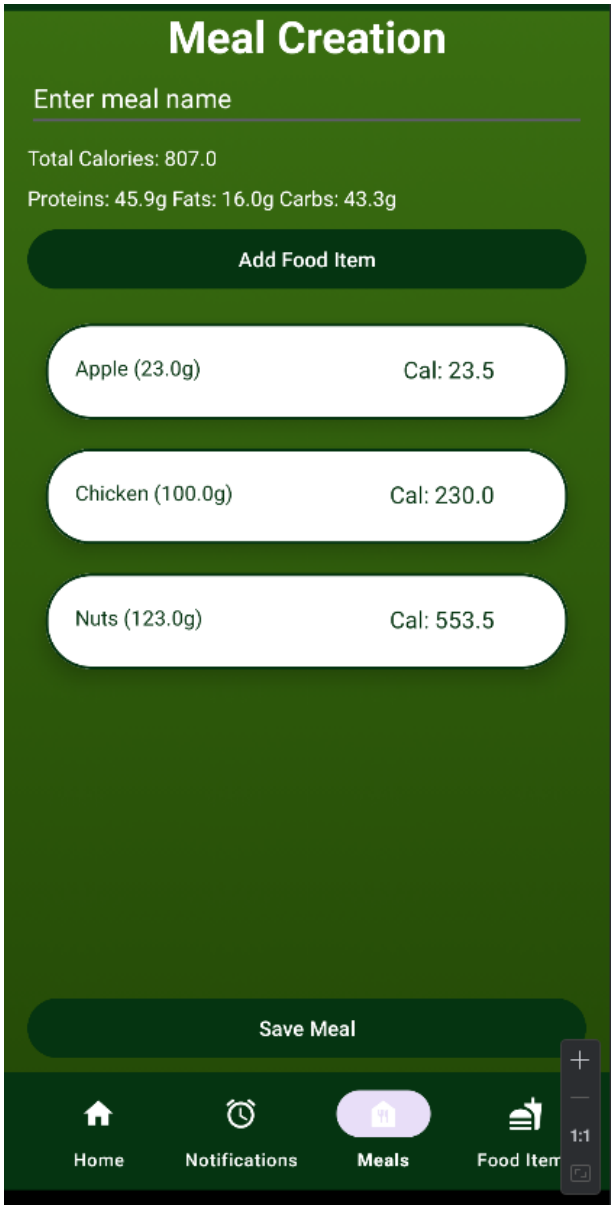
If the input is correct, it parses the entered quantity as a Double, and validates if the value is valid and greater than zero, it adds the food item with the specified quantity to a meal by calling the addFoodItemToMeal() function and calls the updatemealdisplay().

```
private fun addFoodItemToMeal(foodItem: FoodItem, grams: Double) {
    val foodItemWithQuantity = FoodItemWithQuantity(foodItem, grams)
    currentMeal.foodItems[foodItem.name] = foodItemWithQuantity
    currentMeal.updateTotals()
    selectedFoodsAdapter.updateItems(currentMeal.foodItems.values.toList())
}
```

The food items list is updated by passing the current list into the adapter.

```
private fun updateMealDisplay() {
    binding.mealTotalCalories.text = "Total Calories: ${currentMeal.totalCalories.roun
    binding.mealTotalNutrients.text =
        "P: ${currentMeal.totalProteins.roundToDecimal(1)}g " +
        "F: ${currentMeal.totalFats.roundToDecimal(1)}g " +
        "C: ${currentMeal.totalCarbs.roundToDecimal(1)}g"
}
```

Then the updateMealDisplay() updates the UI elements to include the totals for the calories, proteins, fats and carbs. Testing Code: 5.03



Remove food items during creation

EXPLANATION: During a meal creation, in case a user decides they have added the wrong amount of a food item, or that they no longer want that food item in the meal I have created another item on click listener specifically in the item creation that listens to a singular tap, to remove the food item.

```
class SelectedFoodsAdapter : RecyclerView.Adapter<SelectedFoodsAdapter.SelectedFoodViewHol
    private var selectedFoods = mutableListOf<FoodItemWithQuantity>()
    private var onItemClickListener: ((FoodItemWithQuantity) -> Unit)? = null

    fun setOnItemClickListener(listener: (FoodItemWithQuantity) -> Unit) {
        onItemClickListener = listener
    }

    inner class SelectedFoodViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView)
        val nameTextView: TextView = itemView.findViewById(R.id.recTitle)
        val detailsTextView: TextView = itemView.findViewById(R.id.recPriority)

        init {
            itemView.setOnClickListener {
                val position = adapterPosition
                if (position != RecyclerView.NO_POSITION) {
                    onItemClickListener?.invoke(selectedFoods[position])
                }
            }
        }
    }
```

For this I started by adding a listener for an item click in the SelectedFoodsAdapter:

```
private fun setupRecyclerView() {
    selectedFoodsAdapter = SelectedFoodsAdapter()
    selectedFoodsAdapter.setOnItemClickListener { foodItem ->
        // Remove the item from currentMeal
        currentMeal.foodItems.remove(foodItem.foodItem.name)
        currentMeal.updateTotals()
        // Update the display
        selectedFoodsAdapter.updateItems(currentMeal.foodItems.values.toList())
        updateMealDisplay()
        // Show confirmation toast
        Toast.makeText(context, "${foodItem.foodItem.name} removed", Toast.LENGTH_SHORT).sl
    }
    binding.selectedFoodsRecyclerView.apply {
        layoutManager = LinearLayoutManager(context)
        adapter = selectedFoodsAdapter
    }
}
```

Then all that was needed was to take the meal that is being created, find the item that was clicked

and remove the food item from selected item. This could all be done from the setup to the recycler view so that it handles everything straight to the selectedFoodsAdapter. Testing Code: 5.05

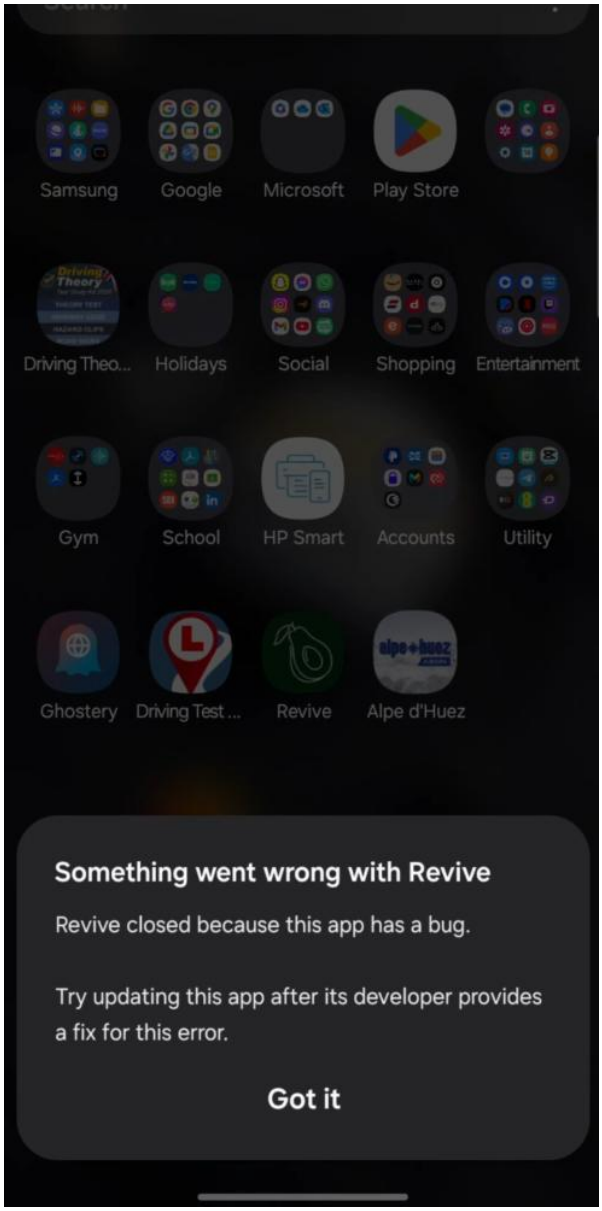
Save food items

```
private fun saveMealToFirebase() {
    val mealName = binding.mealNameInput.text.toString()
    if (mealName.isBlank()) {
        Toast.makeText(context, "Please enter a meal name", Toast.LENGTH_SHORT).show()
        return
    }

    if (currentMeal.foodItems.isEmpty()) {
        Toast.makeText(context, "Please add at least one food item", Toast.LENGTH_SHORT).s
        return
    }
```

```
    currentMeal.name = mealName
    currentMeal.updateTotals() // Make sure totals are up to date

    databaseReference.child(mealName).setValue(currentMeal)
        .addOnSuccessListener {
            Toast.makeText(context, "Meal saved successfully", Toast.LENGTH_SHORT).show()
            // Navigate back to MealFragment
            parentFragmentManager.popBackStack()
        }
        .addOnFailureListener {
            Toast.makeText(context, "Failed to save meal", Toast.LENGTH_SHORT).show()
        }
}
```

When the save button is pressed this function is called, which takes the currentMeal object and all its data and saves it to firebase. Just before saving, the code performs validation to make sure the user has entered a meal name, and that at least one food item has been added. Testing Code: 5.05

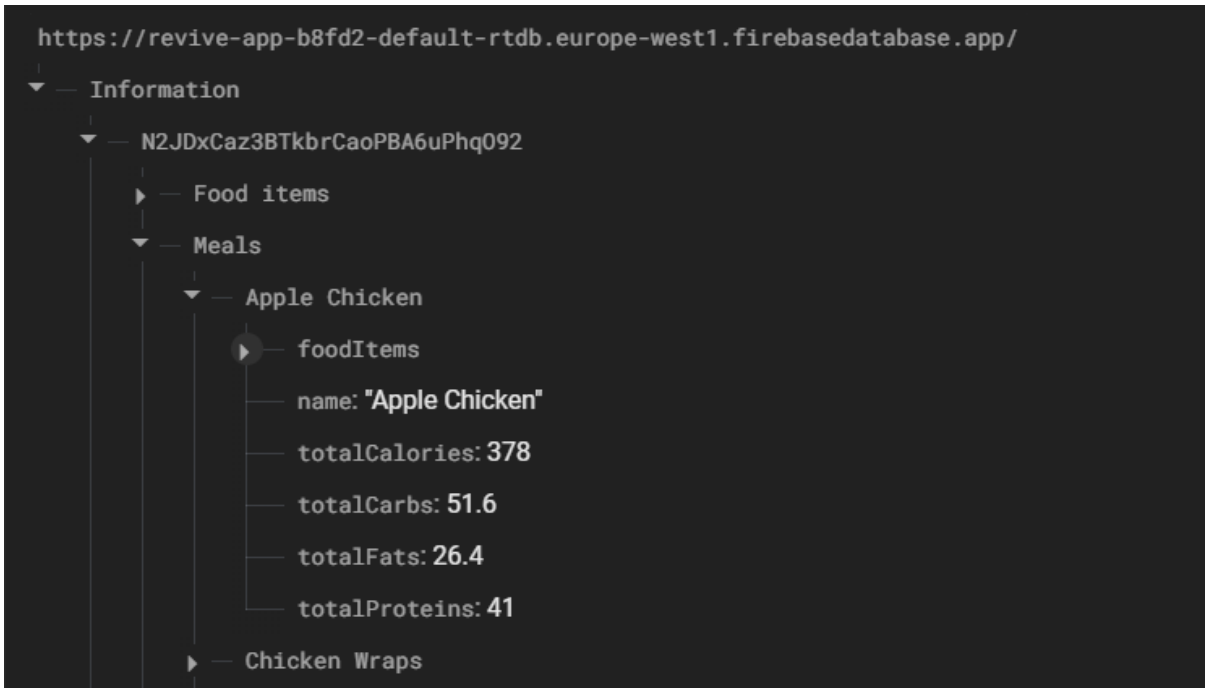
However this code would cause the app to crash when the button was pressed and nothing would save to firebase. Testing Code: 5.06



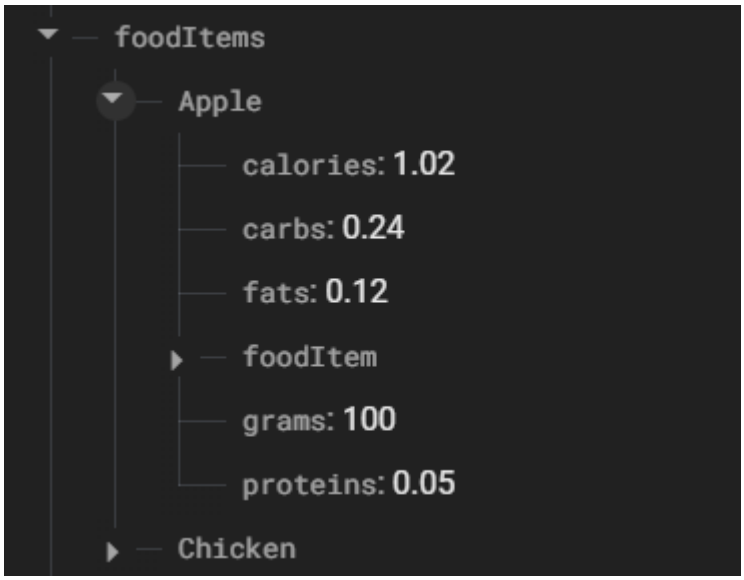

```
    currentMeal.name = mealName
    currentMeal.updateTotals() // Make sure totals are up to date

    databaseReference.child(mealName).setValue(currentMeal)
        .addOnSuccessListener {
            Toast.makeText(context, "Meal saved successfully", Toast.LENGTH_SHORT).show()
            // Navigate back to MealFragment
            parentFragmentManager.popBackStack()
        }
        .addOnFailureListener {
            Toast.makeText(context, "Failed to save meal", Toast.LENGTH_SHORT).show()
        }
}
```

This was because I had not set up the database reference yet within the fragment, and assumed it worked using the setup from the previous main fragment. This meant that when the user was saving

the meal items to thin air instead of the database, so the app would crash as a result, setting up the database call again solved this issue. Testing Code: 5.07



Each meal is saved in database like this under the users id under a child node specifically for meals, where the name of the meal is the child node and the total of each item is saved under the main child. Each individual foodItem is then also saved into the database, so that the meal can almost ”remember” what items where in the meal, this will be useful later for an edit meal option.



Where each foodItem has its original item, and then the quantity converted values by the user.

Searching meal items

EXPLANATION: The user may not want to spend time searching through all of their individual food items if they already have a specific one in mind and have already created lots of food items, this means a search feature would be necessary to find those items quickly.

Within the create meal fragment in the showFoodSelectionDialog which opens up the food selection dialog xml I have added a feature to incorporate the search view widget in the xml file.

```
private fun showFoodSelectionDialog() {
    val userId = FirebaseAuth.getInstance().currentUser?.uid
    val foodsRef = FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.eu
        .getReference("Information")
        .child(userId ?: "")
        .child("Food items")

    foodsRef.get().addOnSuccessListener { snapshot ->
        val foods = mutableListOf<FoodItem>()
        snapshot.children.forEach { foodSnapshot ->
            val food = foodSnapshot.getValue(FoodItem::class.java)
            food?.let { foods.add(it) }
        }

        if (foods.isNotEmpty()) {
            // Create a custom dialog view with search
            val dialogView = LayoutInflater.from(requireContext()).inflate(R.layout.dialo
            val builder = AlertDialog.Builder(requireContext())
            builder.setView(dialogView)

            val dialog = builder.create()

            val searchView = dialogView.findViewById<SearchView>(R.id.foodSearchView)
            val foodListView = dialogView.findViewById<ListView>(R.id.foodListView)
            val cancelButton = dialogView.findViewById<Button>(R.id.cancelButton)
```

Then further within the dialog I have added a variable to store the filtered foods.

```
// Create adapter with filtered list
val filteredFoods = mutableListOf<FoodItem>()
filteredFoods.addAll(foods)
```

```
// Setup search functionality
searchView.setOnQueryTextListener(object : SearchView.OnQueryTextListener {
    override fun onQueryTextSubmit(query: String?): Boolean = false

    override fun onQueryTextChange(newText: String?): Boolean {
        filteredFoods.clear()
        if (newText.isNullOrEmpty()) {
            filteredFoods.addAll(foods)
        } else {
            val filtered = foods.filter {
                it.name.toLowerCase().contains(newText.toLowerCase())
            }
            filteredFoods.addAll(filtered)
        }
        adapter.notifyDataSetChanged()
        return true
    }
})

foodListView.setOnItemClickListener { _, _, position, _ ->
    showFoodQuantityDialog(filteredFoods[position])
    dialog.dismiss()
}
```

This code listens for the user to enter a query into the available search bar which is then filtered the foods list and updates the adapter to only display the items that are based on the users query.

If the input of the user is empty, meaning they have removed their query then the full list is restored, and if the query does not have an item containing that filter no items will be displayed. Testing Code: 5.07

Testing: Meal Creation

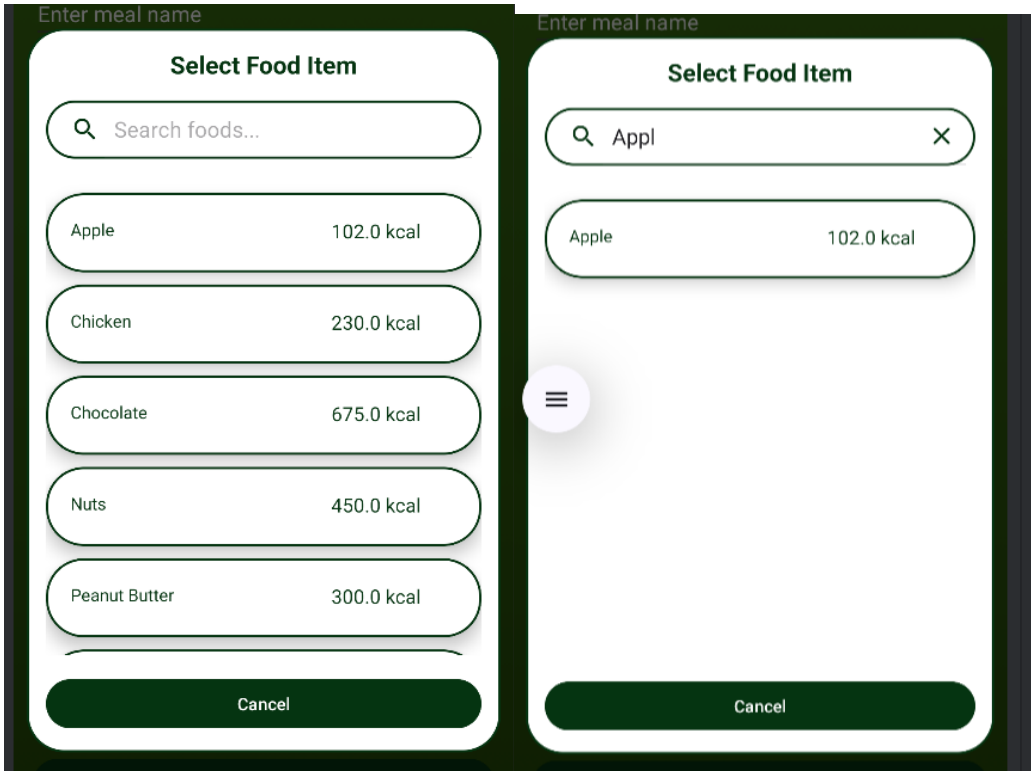
After the initial testing fail where the items were not saving to firebase, another issue emerged where when a query was input into the search bar, for individual meal items, nothing would be changed. Testing Code: 5.07

```
        // Find views in the custom layout
        val titleTextView = dialogView.findViewById<TextView>(R.id.dialogTitle)
        val foodListView = dialogView.findViewById<ListView>(R.id.foodListView)
        val cancelButton = dialogView.findViewById<Button>(R.id.cancelButton)

        // Set up the list adapter using      existing recycler_item layout
        val adapter = ArrayAdapter(
            requireContext(),
            R.layout.recycler_item,
            R.id.recTitle,  // Use      existing TextView ID
            foods.map { it.name }
        )
        foodListView.adapter = adapter
```

This was because when the list was initially set up, it was set to always display the full list of items.

```
val adapter = object : BaseAdapter() {
    override fun getCount(): Int = filteredFoods.size
    override fun getItem(position: Int): FoodItem = filteredFoods[position]
    override fun getItemId(position: Int): Long = position.toLong()

    override fun getView(position: Int, convertView: View?, parent: ViewGroup): View {
        val itemView = convertView ?: LayoutInflater.from(context).inflate(R.layout.recycler

        val nameTextView = itemView.findViewById<TextView>(R.id.recTitle)
        val caloriesTextView = itemView.findViewById<TextView>(R.id.recPriority)

        val foodItem = getItem(position)
        nameTextView.text = foodItem.name
        caloriesTextView.text = "${foodItem.calories} kcal"

        return itemView
    }
}
```

Therefore I changed the adapter to use a base adapter, which allows the manipulation of the list after it has been created, adding a count, id and item filter I am able to search for the correct items according to the user's query and update the list to the current filtered list, of course if there is no query the whole list will be displayed as usual. Testing Code: 5.08



====== Meal Creation ======

Meal Creation Testing Video

Test ID 5	Timestamp	PURPOSE	DATA	EXPECTED RESULTS	ACTUAL RESULTS
5.01	0:00	To test whether the app displays the meal creation fragment on top and tests whether the user is able to add meal items when they have no food items created	n/a	No dialog is shows and a prompt comes up telling the user to create some food items.	As expected.
5.02	0:15	To test whether the app allows the user to add a meal items without inputting a quantity first	No data inputted	The item is not added and the user has a message displayed informing them what to do.	As expected.
5.03	0:35	To test whether the user can enter a food item into the meal, the UI is updated for totals and the food item is added to the list.	Food items: Apple, 23g Chicken, 100g Nuts, 123g	The items are all added to the list and the totals in the UI are updated	As expected.
5.04	1:32	To test whether the user can click on an item while creating a meal and have it removed from that meal.	Remove items: Apple	The totals are updated to remove the selected item	As expected

				and recycler list is updated	
5.05	1:10	To test whether the user is able to create a meal without a name	No name entered	No meal is created and the user is given a message prompting them to enter a meal name.	As expected.
5.06	n/a	To test whether the user is able to create a meal by saving the object to firebase and the data is saved under the correct children.	Food items in meal: Apple, 23g Chicken, 100g Nuts, 123g	The meal object is saved to firebase, the user is returned to the initial meal fragment, and the new meal is displayed in the list.	Fail, the app crashed when items were being saved. (evidence during developmen t)
5.07	1:13	To test whether the user is able to create a meal by saving the object to firebase and the data is saved under the correct children.	Food items in meal: Apple, 23g Chicken, 100g Nuts, 123g	The meal object is saved to firebase, the user is returned to the initial meal fragment, and the new meal is displayed in the list.	As expected.
5.07	n/a	To test whether the user is able to enter a query and search for a specific item.	Queries: “Strawberr y” ”Apple” “Nuts”	The recycler items are updated to store items depending on the search, and if no items all items are displayed.	Fail, the search did not work and the list was not updated
5.08	1:52	To test whether the user is able to enter a query and search for a specific item.	Queries: “Strawberr y” ”Apple” “Nuts”	The recycler items are updated to store items depending on the search, and if no items all items are displayed.	As expected.

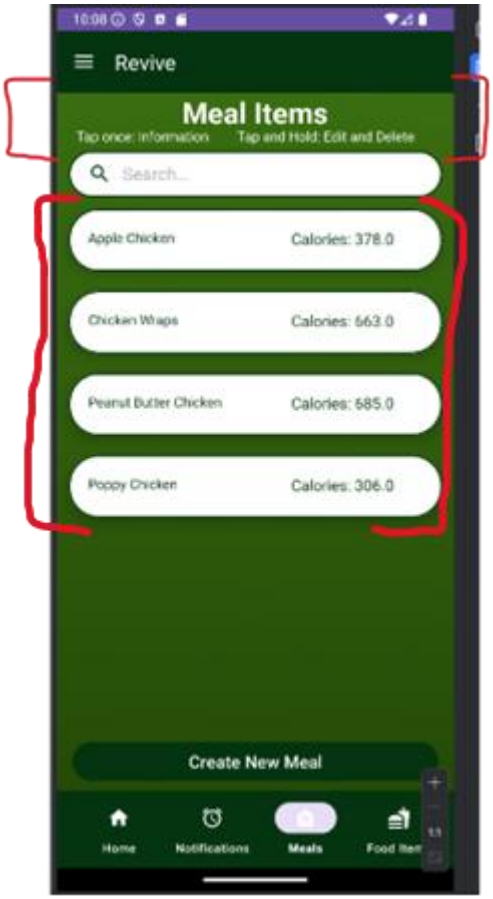
Meal Fragment

EXPLANATION: After a meal is created the stack is popped open and the user is returned to the meal fragment. This fragment will display all the already created meals, and allow the user to interact with the meals to either view, edit, or delete them.

Loading Meals

```
private fun loadMealsFromFirebase() {
    try {
        val userId = FirebaseAuth.getInstance().currentUser?.uid

        if (userId != null) {
            databaseReference = FirebaseDatabase.getInstance("https://revive-app-b8fd2-defa
                .getReference("Information")
                .child(userId)
                .child("Meals")

            databaseReference.addValueEventListener(object : ValueEventListener {
                override fun onDataChange(snapshot: DataSnapshot) {
                    val meals = mutableListOf<Meal>()
                    for (mealSnapshot in snapshot.children) {
                        val meal = mealSnapshot.getValue(Meal::class.java)
                        meal?.let { meals.add(it) }
                    }
                    mealAdapter.updateMeals(meals)
                }

                override fun onCancelled(error: DatabaseError) {
                    Toast.makeText(context, "Failed to load meals", Toast.LENGTH_SHORT).sho
                }
            })
        } else {
            Toast.makeText(context, "User not logged in", Toast.LENGTH_SHORT).show()
        }
    } catch (e: Exception) {
        Toast.makeText(context, "Error loading meals", Toast.LENGTH_SHORT).show()
    }
}
```

The first function I created retrieves meal data from Firebase for the currently logged-in user. A ValueEventListener is then added to listen for changes in the meals data. When the data is successfully fetched (onDataChange), it iterates through the meals in the snapshot, converts them to Meal objects, and updates the meal list using mealAdapter.updateMeals(). If the operation fails, a toast displays an error message. Testing Code: 5.09



Searching created meal items

EXPLANATION: If the user creates many meals they may want to find a specific one and view its details and later be able to edit or delete that meal specifically so a search feature here aswell would be incredibly beneficial.

```
class MealAdapter : RecyclerView.Adapter<MealAdapter.MealViewHolder>() {
    private var meals = mutableListOf<Meal>()
    private var filteredMeals = mutableListOf<Meal>()

    init {
        filteredMeals = meals
    }
```

Started by adding a function to the existing meal adapter class and an extra variable to use called filteredMeals,

```
fun searchMeals(query: String) {
    filteredMeals = if (query.isEmpty()) {
        meals.toMutableList()
    } else {
        meals.filter {
            it.name.toLowerCase().contains(query.toLowerCase())
        }.toMutableList()
    }
    notifyDataSetChanged()
}
```

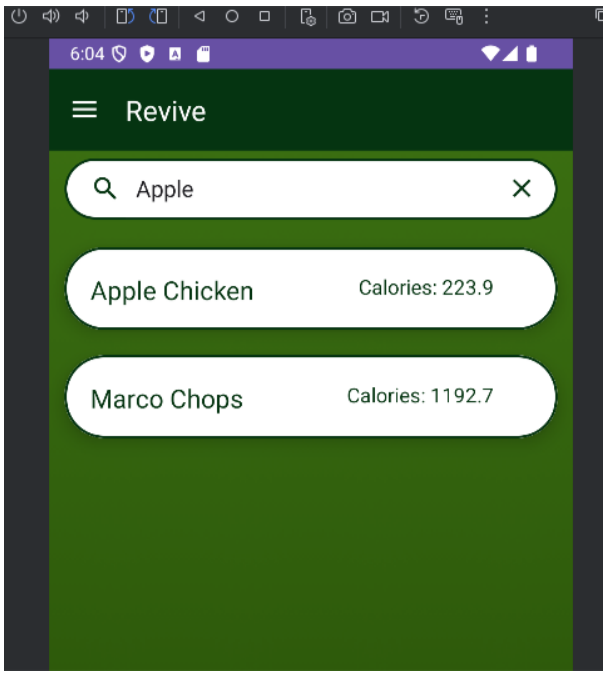
Then the same function for search is used as before, but without the need to find the search bar as binding which is used in the fragments allows the query to be found immediately. It makes sure that values are converted to lowercase so that meals are searched for in the same way regardless of what the user has placed.

```
class MealFragment : Fragment() {
    private var _binding: FragmentMealBinding? = null
    private val binding get() = _binding!!

    private lateinit var mealAdapter: MealAdapter

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        _binding = FragmentMealBinding.inflate(inflater, container, false)

        setupRecyclerView()
        setupSearch()

        loadMealsFromFirebase()
```

```
    private fun setupSearch() {
        binding.mealSearch.setOnQueryTextListener(object : SearchView.OnQueryTextListener {
            override fun onQueryTextSubmit(query: String?): Boolean {
                return false
            }

            override fun onQueryTextChange(newText: String?): Boolean {
                mealAdapter.searchMeals(newText ?: "")
                return true
            }
        })
    }
```

hen within the MealFragment I added a setupSearch function which configures a search feature for the meal list. It sets a listener on the SearchView to detect text input changes. When the user types in the search bar, the onQueryTextChange method updates the displayed list of meals by calling mealAdapter.searchMeals with the entered text. This enables real-time filtering of the meal list as the user types.

However when testing this section of the module when I entered an option into the search bar, once again, nothing was searched or updated: Testing Code: 5.10



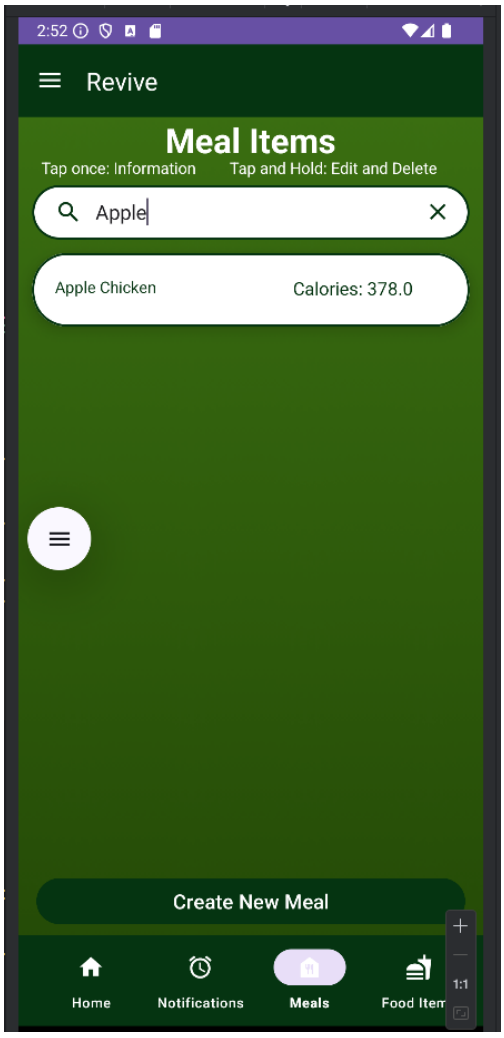
Therefore I tried updating the onBindViewHolder and getItemCount() variables in the adapter class to use the filteredMeals variable created through the search instead of the regular mealItems array and it worked:

```
override fun onBindViewHolder(holder: MealViewHolder, position: Int) {
    val meal = meals[position]
    holder.nameTextView.text = meal.name
    holder.detailsTextView.text = "Calories: ${meal.totalCalories.roundToDecimal(1)}"
}
```

To

```
override fun onBindViewHolder(holder: MealViewHolder, position: Int) {
    // Changed from meals to filteredMeals
    val meal = filteredMeals[position]
    holder.nameTextView.text = meal.name
    holder.detailsTextView.text = "Calories: ${meal.totalCalories.roundToDecimal(1)}"
}

// Changed to use filteredMeals size
override fun getItemCount() = filteredMeals.size
```

Testing Code: 5.11



Item Interactivity

EXPLANATION: This will allow the user to interact with the items in the main meal fragment, they will be able to select the items to view the individual meals inside the object, as well as see the total

amounts of macros for the meal, but also be able to select the meal to edit and delete it should they need to.

So that the user can click a meal item i added this code to the adapter class:

```
interface MealClickListener {
    fun onMealClick(meal: Meal)
    fun onMealLongClick(meal: Meal)
}
```

Where a regular click will open a dialog (that just displays the information of the meal), and long click (hold) on the item will display a different dialog (that asks the user whether they want to edit, or whether they want to delete)

```
    private var clickListener: MealClickListener? = null

    fun setMealClickListener(listener: MealClickListener) {
        clickListener = listener
    }

    inner class MealViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) {
        val nameTextView: TextView = itemView.findViewById(R.id.recTitle)
        val detailsTextView: TextView = itemView.findViewById(R.id.recPriority)

        init {
            itemView.setOnClickListener {
                val position = adapterPosition
                if (position != RecyclerView.NO_POSITION) {
                    clickListener?.onMealClick(filteredMeals[position])
                }
            }

            itemView.setOnLongClickListener {
                val position = adapterPosition
                if (position != RecyclerView.NO_POSITION) {
                    clickListener?.onMealLongClick(filteredMeals[position])
                }
                true
            }
        }
    }
```

The setMealClickListener function sets a listener to handle these events. In the MealViewHolder class, click and long-click listeners are set on the item view. When a click occurs, the listener calls the appropriate method with the selected meal, enabling interaction with meal items in the list.

```
class MealFragment : Fragment(), MealAdapter.MealClickListener {
```

Adding the mealAdapter.MealClickListener allows the fragment to listen to the type of click that is used on the recycler view item instead of simply binding a button to a click

```
    override fun onMealClick(meal: Meal) {
        showMealDetailsDialog(meal)
    }

    override fun onMealLongClick(meal: Meal) {
        showEditDeleteDialog(meal)
    }
```

Calls a specific function based on what type of click is experienced

This line added allows the mealclickListener to actually work with the recycler view without crashing on an item being clicked

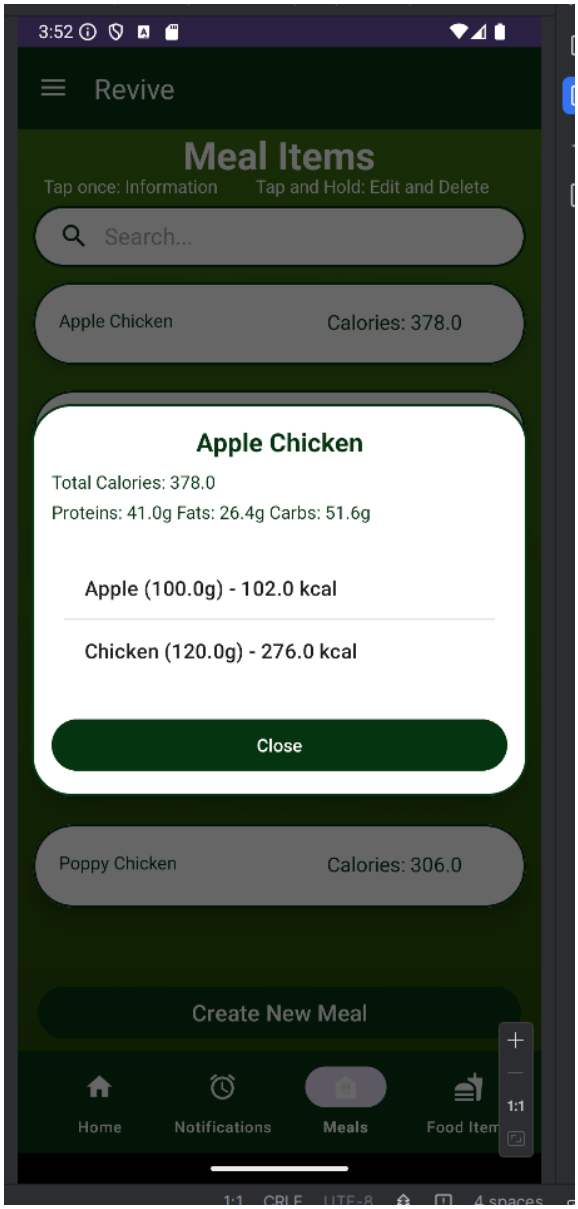
```
private fun setupRecyclerView() {
    mealAdapter = MealAdapter()
    mealAdapter.setMealClickListener(this)
    binding.mealsRecyclerView.apply {
        layoutManager = LinearLayoutManager(context)
        adapter = mealAdapter
    }
}
```

Meal details dialog

EXPLANATION: This will be a dialog that shows when the user taps on the meal item and will display all the information about that meal item, including all the items within the meal and its totals.

```
private fun showMealDetailsDialog(meal: Meal) {
    val builder = AlertDialog.Builder(requireContext())
    val dialogView = layoutInflater.inflate(R.layout.dialog_meal_details, null)

    val nameTextView = dialogView.findViewById<TextView>(R.id.mealNameText)
    val caloriesTextView = dialogView.findViewById<TextView>(R.id.totalCaloriesText)
    val nutrientsTextView = dialogView.findViewById<TextView>(R.id.nutrientsText)
    val foodItemsList = dialogView.findViewById<ListView>(R.id.foodItemsList)
    val closeButton = dialogView.findViewById<Button>(R.id.closeButton)

    nameTextView.text = meal.name
    caloriesTextView.text = "Total Calories: ${meal.totalCalories.roundToDecimal(1)}"
    nutrientsTextView.text = "P: ${meal.totalProteins.roundToDecimal(1)}g " +
            "F: ${meal.totalFats.roundToDecimal(1)}g " +
            "C: ${meal.totalCarbs.roundToDecimal(1)}g"

    // Create adapter for food items list
    val foodItems = meal.foodItems.values.toList()
    val adapter = ArrayAdapter(
        requireContext(),
        android.R.layout.simple_list_item_1,
        foodItems.map { "${it.foodItem.name} (${it.grams}g) - ${(it.calories * it.gram
    )
    foodItemsList.adapter = adapter

    builder.setView(dialogView)
    val dialog = builder.create()

    closeButton.setOnClickListener { dialog.dismiss() }

    if (dialog.window != null) {
        dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
    }

    dialog.show()
}
```

This code defines a function showMealDetailsDialog found inside the fragment_meal.xml that displays a dialog with detailed information about a meal. The dialog populates the list with data from the given Meal object. The list is populated using an ArrayAdapter to format the data. A close button is provided to dismiss the dialog. Testing Code: 5.12



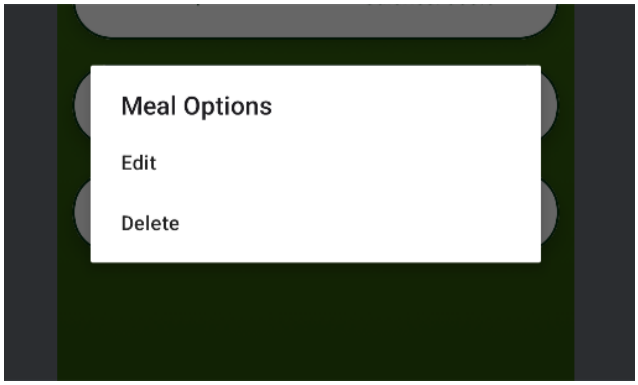
Edit or Delete and its dialog

EXPLANATION: This will be a dialog that shows up and asks the user whether they want to edit or delete the item, this is important as it allows two very different choices to be placed into one dialog, so a third way of clicking on the item is not needed, saving time and resources.

Since there is only the option to long click, and I wanted both the options to edit and delete a meal I decided to create an initial dialog that asks whether the user wants to edit or delete.

```
private fun showEditDeleteDialog(meal: Meal) {
    val options = arrayOf("Edit", "Delete")
    AlertDialog.Builder(requireContext())
        .setTitle("Meal Options")
        .setItems(options) { dialog, which ->
            when (which) {
                0 -> editMeal(meal)
                1 -> confirmDelete(meal)
            }
            dialog.dismiss()
        }
        .show()
}
```

This was the initial code that I had created after learning about dialog to dialog opening which just told me that a dialog can be dismissed and another opened with a very short gap so to the user it looks like it was opened on top, however this code created a dull system dialog.

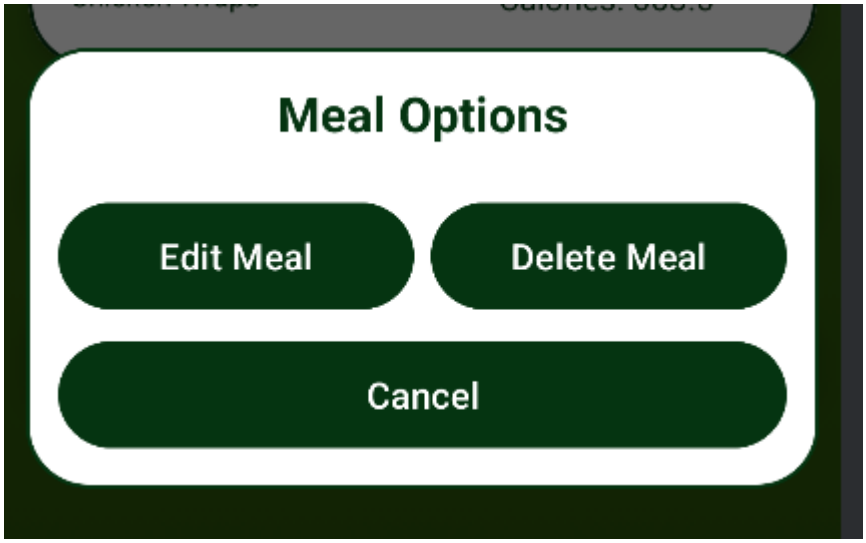


This was not in the style of my application at all and did not fit. Working test code up to this point: 5.13

Therefore, I created another dialog layout, dialog_meal_options.xml, and it also meant that upon pressing the buttons the user would be taken to a different location in the application, due to using a number theory as compared to a proper validation.

```
private fun showEditDeleteDialog(meal: Meal) {
    val builder = AlertDialog.Builder(requireContext())
    val view = layoutInflater.inflate(R.layout.dialog_meal_options, null)

    val editButton = view.findViewById<Button>(R.id.editButton)
    val deleteButton = view.findViewById<Button>(R.id.deleteButton)
    val cancelButton = view.findViewById<Button>(R.id.cancelButton)

    builder.setView(view)
    val dialog = builder.create()

    // Set click listeners
    editButton.setOnClickListener {
        dialog.dismiss()
        editMeal(meal)
    }

    deleteButton.setOnClickListener {
        dialog.dismiss()
        showDeleteConfirmationDialog(meal)
    }

    cancelButton.setOnClickListener {
        dialog.dismiss()
    }

    // Set transparent background
    if (dialog.window != null) {
        dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
    }

    dialog.show()
}
```

Which fulfills the same purpose as the first code, but instead of having one function that has a value of 0 or 1, there are buttons that are found using a view just like every dialog I have used as the dialog would simply just replace the current one as long as it is dismissed before the next one is called.
Testing Code: 5.14



Edit Meals

EXPLANATION: This will allow the user to edit the items in their already created meal, however since just showing a dialog to edit individual aspects of an item like before will not work, instead the code will treat the edit like an extension to the creation of the meal, putting the user back into the creation fragment with the already created meal and its values and allow the user to make changes as if they were first creating the meal. This is justified as it reuses already working code saving time and adding functionality.

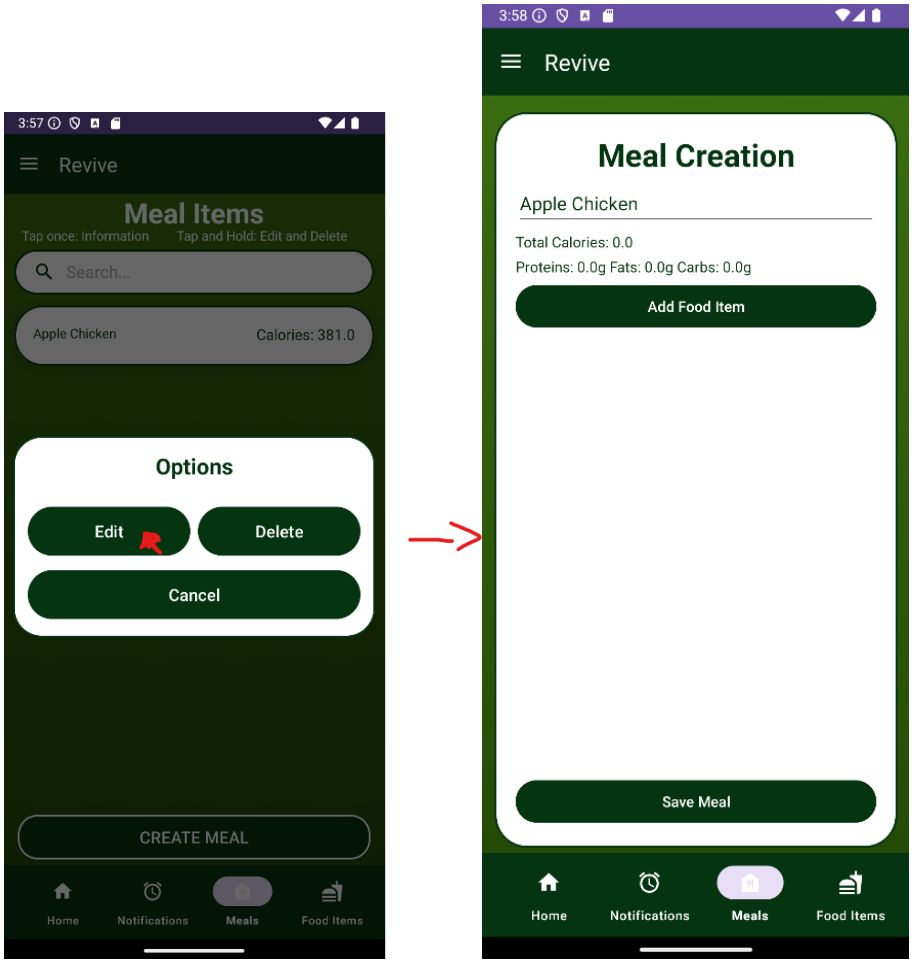
```
private fun editMeal(meal: Meal) {
    val transaction = parentFragmentManager.beginTransaction()
    val createMealFragment = CreateMealFragment().apply {
        arguments = Bundle().apply {
            putString("mealName", meal.name)
            putBoolean("isEditing", true)
        }
    }
    transaction.replace(R.id.fragment_container, createMealFragment)
    transaction.addToBackStack(null)
    transaction.commit()
}
```

To edit a meal when the edit meal option is pressed in the dialog the dialog will swap the current fragment back to the fragment_create_meal.xml and will place the name into the mealName section so that when the save button is pressed, it will overwrite the meal of the same name with the new foods.

```
class CreateMealFragment : Fragment() {
    private var _binding: FragmentCreateMealBinding? = null
    private val binding get() = _binding!!
    private lateinit var databaseReference: DatabaseReference
    private lateinit var selectedFoodsAdapter: SelectedFoodsAdapter
    private var currentMeal = Meal()
    private var isEditing = false
    private var editingMealName = ""

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        // Get arguments passed from MealFragment
        arguments?.let {
            isEditing = it.getBoolean("isEditing", false)
            editingMealName = it.getString("mealName", "")
        }
    }
```

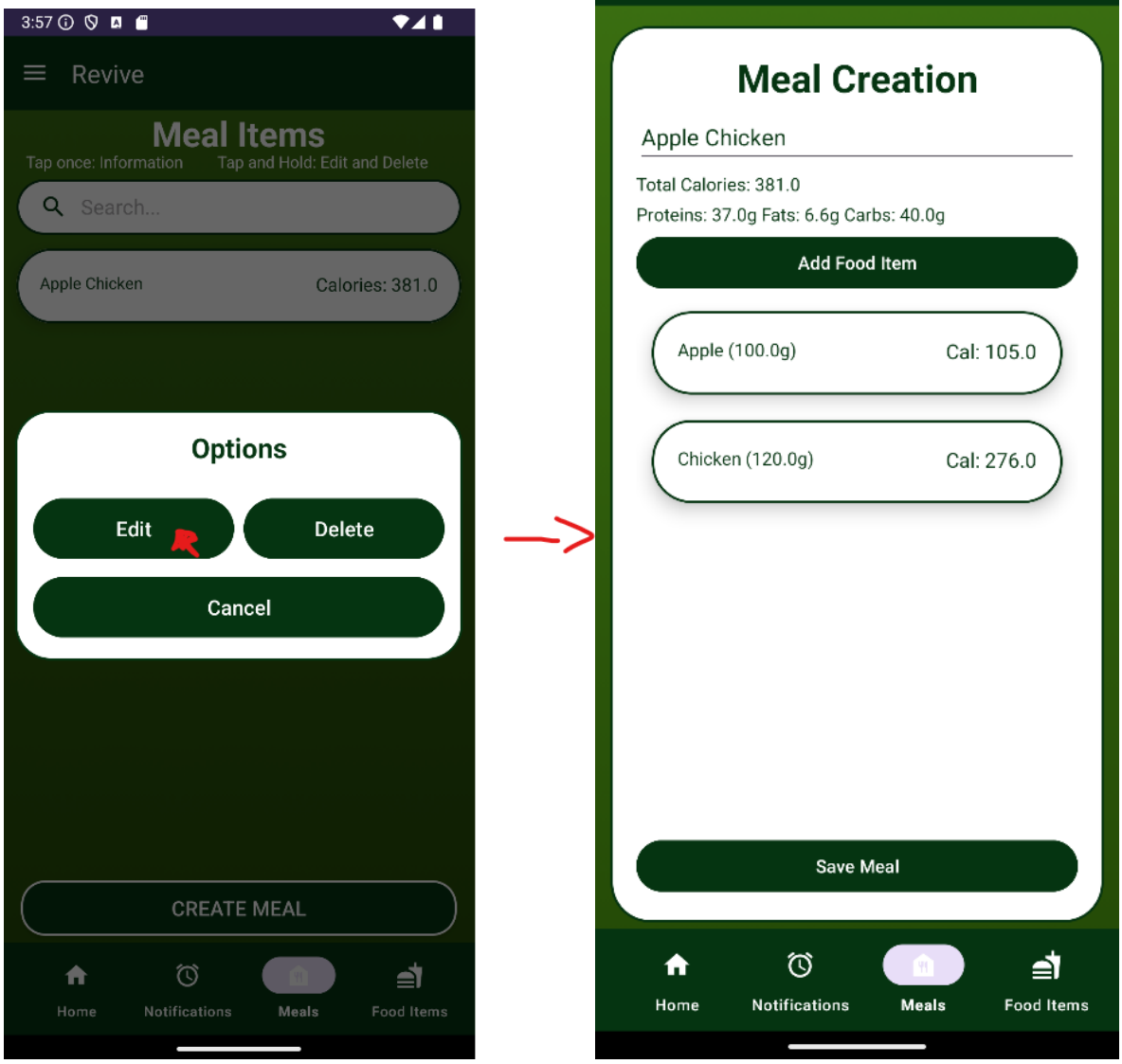
This is the function that is within the create_meal_fragment that retrieves the variables sent from the editmeal function, however if the values have not been sent as the user is creating a new meal and not editing one, the values will be set to their default, which is false for isEditing and an empty string for the meal name.

This code worked to send the meal to the creation meal fragment as an edit, however this worked on the user not changing the name of the item and updating it through that name, this meant that only the name was passed into the meal edit, so when the meal was edited, the food objects already in the meal did not appear in the create meal section: Testing Code: 5.15


3:57
Revive
Meal Items
Tap once: Information
Tap and Hold: Edit and Delete
Search...
Apple Chicken
Calories: 381.0
Options
Edit
Delete
Cancel
CREATE MEAL
Home
Notifications
Meals
Food Items
3:58
Revive
Meal Creation
Apple Chicken
Total Calories: 0.0
Proteins: 0.0g Fats: 0.0g Carbs: 0.0g
Add Food Item
Save Meal
Home
Notifications
Meals
Food Items

```
    // Load existing meal data if editing
    if (isEditing) {
        loadExistingMeal()
    }

    return binding.root
}

private fun loadExistingMeal() {
    val userId = FirebaseAuth.getInstance().currentUser?.uid
    if (userId != null && editingMealName.isNotEmpty()) {
        // Set the meal name in the input field
        binding.mealNameInput.setText(editingMealName)

        val mealRef = FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.
            .getReference("Information")
            .child(userId)
            .child("Meals")
            .child(editingMealName)

        mealRef.get().addOnSuccessListener { snapshot ->
            snapshot.getValue(Meal::class.java)?.let { meal ->
                currentMeal = meal
                selectedFoodsAdapter.updateItems(meal.foodItems.values.toList())
                updateMealDisplay()
            }
        }.addOnFailureListener {
            Toast.makeText(context, "Failed to load meal data", Toast.LENGTH_SHORT).show()
        }
    }
}
```

To solve this, under the methods of the class, I have an if statement to check whether the editing value has been sent, and if so there a meal is taken from a snapshot from Firebase and loaded into the selectedFoodsAdapter so that not only the name appears in the edit section, but also the meal items already in it and if the snapshot fails there is a failiure message displayed. Testing Code: 5.16


3:57
Revive
Meal Items
Tap once: Information
Tap and Hold: Edit and Delete
Search...
Apple Chicken
Calories: 381.0
Options
Edit
Delete
Cancel
CREATE MEAL
Home
Notifications
Meals
Food Items
3:59
Revive
Meal Creation
Apple Chicken
Total Calories: 381.0
Proteins: 37.0g Fats: 6.6g Carbs: 40.0g
Add Food Item
Apple (100.0g)
Cal: 105.0
Chicken (120.0g)
Cal: 276.0
Save Meal
Home
Notifications
Meals
Food Items

```
private fun saveMealToFirebase() {
    val mealName = binding.mealNameInput.text.toString()

    // Validation checks
    if (mealName.isBlank()) {
        Toast.makeText(context, "Please enter a meal name", Toast.LENGTH_SHORT).show()
        return
    }

    // When editing, remove old meal if name changed
    if (isEditing && editingMealName != mealName) {
        databaseReference.child(editingMealName).removeValue()
    }

    if (currentMeal.foodItems.isEmpty()) {
        Toast.makeText(context, "Please add at least one food item", Toast.LENGTH_SHORT).sh
        return
    }

    // Save meal to Firebase
    currentMeal.name = mealName
    currentMeal.updateTotals()

    databaseReference.child(mealName).setValue(currentMeal)
        .addOnSuccessListener {
            Toast.makeText(context, "Meal saved successfully", Toast.LENGTH_SHORT).show()
            parentFragmentManager.popBackStack()
        }
        .addOnFailureListener {
            Toast.makeText(context, "Failed to save meal", Toast.LENGTH_SHORT).show()
        }
}
```

In case the user decides to change the name of the meal they have created, so that the current meal is still technically “overwritten” a new meal is created so the same saveMealToFirebase() function can be used as before on edit, and the old one is deleted.

Delete Meals

EXPLANATION: The user may simply just want to remove a meal in its entirety from their meal list, so pressing the option to delete will remove the whole meal object from firebase, however in case the user accidentally pressed the delete button, a confirmation message will show up first asking them if they want to proceed, then delete the object. This is justified as it stops the user from accidentally deleting lots of data they had spent time measuring and inputting.

```
// Shows a confirmation dialog before deleting a meal
private fun showDeleteConfirmationDialog(meal: Meal) {
    val builder = AlertDialog.Builder(requireContext())
    val view = layoutInflater.inflate(R.layout.dialog_meal_options, null)

    // Repurpose existing dialog layout components
    val titleText = view.findViewById<TextView>(R.id.dialogTitle)
    val confirmButton = view.findViewById<Button>(R.id.editButton)    // Reuse edit button for confirm
    val cancelButton = view.findViewById<Button>(R.id.deleteButton)   // Reuse delete button for cancel
    val bottomButton = view.findViewById<Button>(R.id.cancelButton)   // Hide the bottom button

    // Configure UI elements for delete confirmation
    titleText.text = "Delete Meal"
    confirmButton.text = "Delete"
    cancelButton.text = "Keep"
    confirmButton.backgroundTintList = ColorStateList.valueOf(Color.parseColor("#400a06"))  // Dark red
    cancelButton.backgroundTintList = ColorStateList.valueOf(Color.parseColor("#053411"))   // Dark green
    bottomButton.visibility = View.GONE

    builder.setView(view)
    val dialog = builder.create()

    // Handle user response
    confirmButton.setOnClickListener {
        dialog.dismiss()
        deleteMeal(meal)
    }

    cancelButton.setOnClickListener {
        dialog.dismiss()
    }

    // Apply transparent background to dialog
    if (dialog.window != null) {
        dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
    }

    dialog.show()
}
```

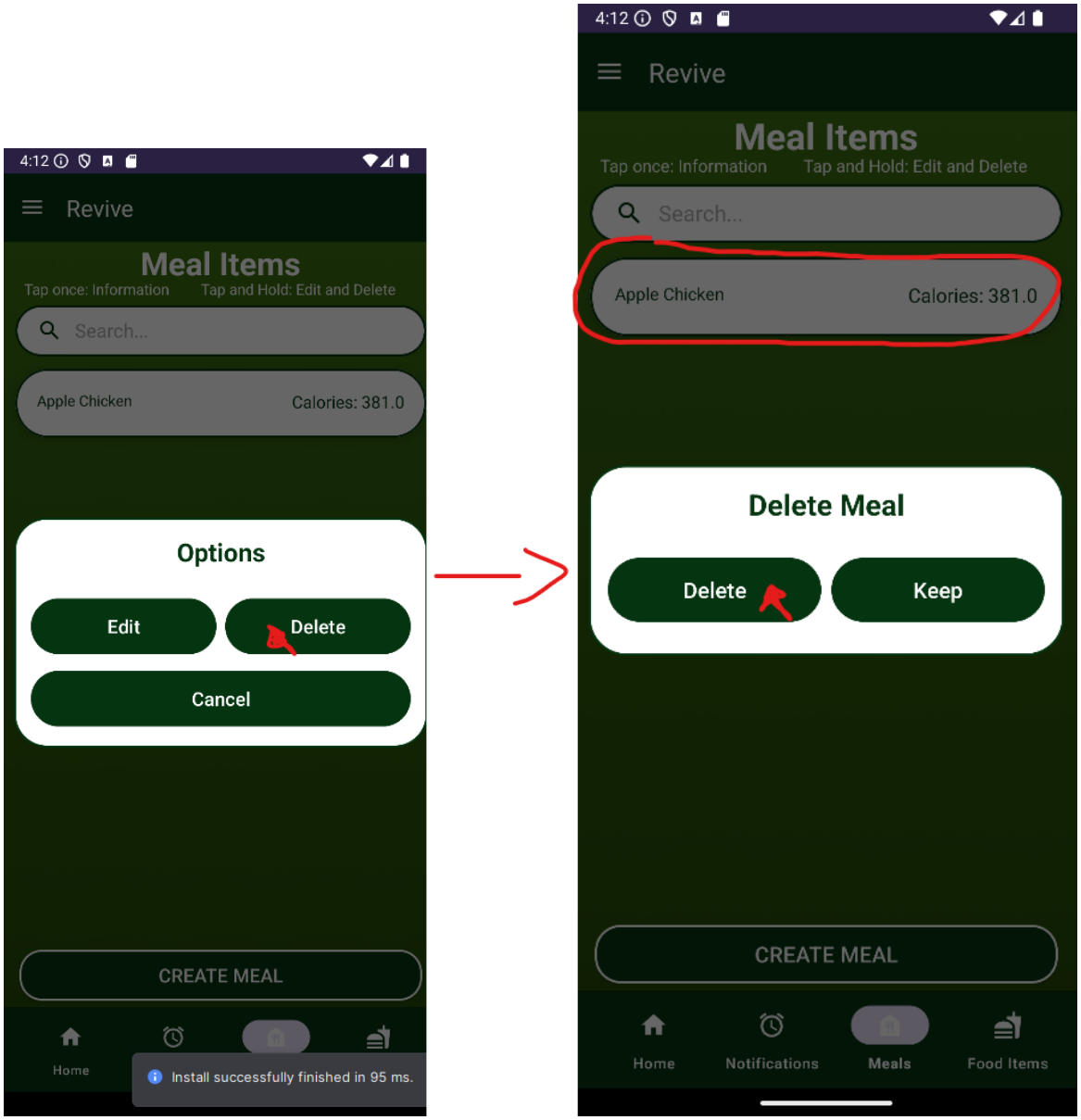
When the delete option is pressed in the edit meals dialog, the function called will be a confirmation dialog and the options one will be dismissed. Instead of creating a new dialog, I have reused the options dialog, changing the edit button to a confirm and the cancel button to stop the deletion of the item, and hid the old close button.

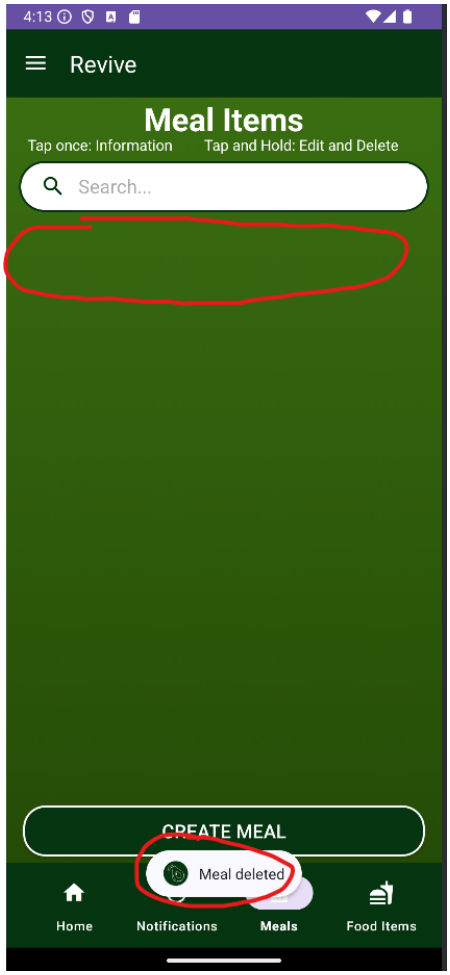
If the cancel button is pressed, the dialog is simply dismissed and the code returns to the meal creation fragment as before, however if the delete button is clicked, meaning the user has confirmed

they want to delete the item the delete Meal() function is called with an argument of the current meal.

```
// Removes specified meal from Firebase database
private fun deleteMeal(meal: Meal) {
    val userId = FirebaseAuth.getInstance().currentUser?.uid
    if (userId != null) {
        databaseReference.child(meal.name).removeValue()
            .addOnSuccessListener {
                Toast.makeText(context, "Meal deleted", Toast.LENGTH_SHORT).show()
            }
            .addOnFailureListener {
                Toast.makeText(context, "Failed to delete meal", Toast.LENGTH_SHORT).show()
            }
    }
}
```

This function simply finds the meal that the user is trying to delete by name under their own id, and deletes it in its entirety from the database, returning a confirm message if the action succeeded, or failiure if not. Testing Code: 5.17


4:12
Revive
Meal Items
Tap once: Information
Tap and Hold: Edit and Delete
Search...
Apple Chicken
Calories: 381.0
Options
Edit
Delete
Cancel
CREATE MEAL
Home
Install successfully finished in 95 ms.
4:12
Revive
Meal Items
Tap once: Information
Tap and Hold: Edit and Delete
Search...
Apple Chicken
Calories: 381.0
Delete Meal
Delete
Keep
CREATE MEAL
Home
Notifications
Meals
Food Items



Testing: Meal Fragment

Due to the long nature of this particular model testing was conducted throughout the development as usual and written during the section instead of at the end. Below will be a table listing all the tests and a video for a few.

====== Meal Fragment ======

Meal Fragment testing video

Test ID 5	Time stamp	PURPOSE	DATA	EXPECTED RESULTS	ACTUAL RESULTS
5.09	n/a	To test whether the meal items are loaded into the app when the user enters the app.	n/a	All meal items created are shown in the UI	As expected(image shown during development)
5.10	n/a	To test whether the user is able to enter a query into the search bar and the recycler list is updated accordingly.	Queries: Apple	The recycler view is updated to show the items according to the query the user has entered, and if	Fail, no change in the UI when a prompt is entered.

				there is no query all items are shown.	
5.11	2:27	To test whether the user is able to enter a query into the search bar and the recycler list is updated accordingly.	Queries: Apple	The recycler view is updated to show the items according to the query the user has entered, and if there is no query all items are shown.	As expected.
5.12	2:20	To test whether single clicking on an individual meal item shows the dialog with all information about that meal item loaded into UI	n/a data from firebase	All the totals are shown in the UI, and all the food items in the meal are displayed in a list.	As expected.
5.13	n/a	To test whether an appropriate edit and delete option is shown on the UI and whether the user is directed to the correct action.	n/a	A dialog is shown in the theme of the app and the user is able to select two options	Fail, the dialog is in a completely different style and the edit and delete buttons cause the user to be taken to a different part of the application.
5.14	n/a	To test whether an appropriate edit and delete option is shown on the UI and whether the user is directed to the correct action.		A dialog is shown in the theme of the app and the user is able to select two options.	Pass, dialog send the user to the correct options and is in the theme of the app. (evidence during development)
5.15	n/a	To test whether the user is able to select the edit button which allows them to enter the meal creation and edit the objects.	Data: Chicken object removed	When the edit button is pressed the user is sent back to the creation fragment with all the already created meal items added to the meal creation screen.	Fail, no items are displayed when the user enters the creation fragment so the user is essentially creating a new meal, which is not what is intended.
5.16	2:43	To test whether the user is able to select the edit button which allows them to enter the meal creation and edit the objects.	Data: Chicken object removed	When the edit button is pressed the user is sent back to the creation fragment with all the already	Pass, as expected. Values are saved correctly to firebase.

				created meal items added to the meal creation screen.	
5.17	3:02	To test whether the user is able to select the delete option and remove an object from the UI and database.	n/a	When the delete button is pressed the user should receive a confirmation method, then upon confirming the meal object is removed from firebase.	As expected.

Module Summary

The Meal Creation module builds upon the Food Item system by allowing users to combine individual food items into reusable meals with customizable portions. The module consists of two main components: a meal creation interface where users can select foods and specify quantities, and a meal management screen where users can view, search, edit, and delete saved meals. Each meal automatically calculates and stores its total nutritional values based on its component foods.

The prototype now encompasses a complete food and meal management system. Users can move from creating individual food items to combining them into meals, with all nutritional calculations handled automatically. This layered approach is justified as it mirrors how people actually think about food - individual ingredients that make up complete meals. Storing meals as collections of their component foods rather than just their total values is justified as it allows for granular editing and portion adjustment. The current implementation lets users build up a personalized database of both individual foods and complete meals, though the prototype still needs the daily tracking features to make practical use of these meal combinations. The consistent interface design across both food and meal management, including search functionality and edit capabilities, helps maintain a cohesive user experience despite the increasing complexity of the system.

StakeHolder review - 3

Q: How are you ensuring accurate nutritional data entry?

A: For food items, users enter values per 100g to match product labels, making data entry intuitive. All numerical inputs are validated, and the app internally converts values to per 1g for precise calculations. For meals, nutritional totals are automatically calculated based on the component foods and their quantities.

Q: The specification mentioned avoiding user frustration. How does the meal creation process address this?

A: Meal creation uses a step-by-step flow: select food, enter quantity, see it added to the meal. Users can remove items during creation if they make mistakes, and edit meals after creation. The process reuses familiar dialogs and interfaces from the food creation module to maintain consistency.

Q: What happens if users try to create meals without any food items?

A: The app validates this scenario and displays a message telling users they need to create food items first. This prevents confusion by clearly directing users to the correct first step rather than letting them attempt meal creation with an empty food database.

Updates to the Login Module

EXPLANATION: This updates the login module to include a “remember me” feature meaning that the user does not need to login to the app every single time, their session is saved and they can proceed straight into the app. This is an important adjustment and is a convenience feature for the user aligning this app closer to other professional products.

Logic

```
if (firebaseAuth.getCurrentUser() != null) {
    val userid = FirebaseAuth.getInstance().currentUser?.uid
    if (userid != null){
        databaseReference = FirebaseDatabase.getInstance().getReference("Information").child(userid)
        databaseReference.get().addOnSuccessListener { snapshot ->
            if (snapshot.exists()) {
                // If data exists, skip the form and go to the next activity
                // Navigate to the next activity
                val intent = Intent(this, MainActivity::class.java)
                startActivity(intent)

            } else {
                val intent = Intent(this, PersonalDetailsActivity::class.java)
                startActivity(intent)  // moves to the next screen if password and email are correct

            }
        }
    }
}
```

This code is added to the login activity after learning how to fetch data from firebase. The program will now begin by checking if a user is already logged in.

If the user is already logged in, the app sends a request to retrieve user data from Firebase Realtime Database. If the data exists, the app skips the additional sign in personal info forms and navigates to MainActivity. If data hasnt been recorded, meaning the user signed in then exited the app straight away, the user is directed to the PersonalDetailsActivity to complete their information. This means a

user will not have to login to the app every time they try to enter it, instead they are remembered and logged straight in. However it makes sure that if the user has not entered any of their information, they will not be taken straight into the app as this would cause the important variables to become null, which could cause crashes as calculations are made in the future.

```
binding.loginButton.setOnClickListener{   // waits for the login button to be clicked
    val email = binding.loginEmail.text.toString()
    val password = binding.loginPassword.text.toString()

    if (email.isNotEmpty() && password.isNotEmpty()) {  // makes sure the two email and password fields are not left empty
        firebaseAuth.signInWithEmailAndPassword(email, password).addOnCompleteListener{
            if (it.isSuccessful){
                Log.d("LoginActivity", "Authentication successful")
                val userid = FirebaseAuth.getInstance().currentUser?.uid
                if (userid != null){
                    databaseReference = FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.europe-west1.firebasedatab
                    databaseReference.get().addOnSuccessListener { snapshot ->
                        Log.d("LoginActivity", "Database snapshot exists: ${snapshot.exists()}")
                        if (snapshot.child("personal information").exists()) {
                            // If data exists, skip the form and go to the next activity
                            // Navigate to the next activity

                            val intent = Intent(this, MainActivity::class.java)
                            startActivity(intent)
                        } else {
                            val intent = Intent(this, PersonalDetailsActivity::class.java)
                            startActivity(intent)  // moves to the next screen if password and email are correct
                        }
                    }
                }else{
                    Toast.makeText(this, "Authentication is Failed", Toast.LENGTH_SHORT).show()
                }

            } else {
                Toast.makeText(this, "Authentication Failed", Toast.LENGTH_SHORT).show()  // returns an error if wrong
            }
        }
    } else {
        Toast.makeText(this, "Fields cannot be empty", Toast.LENGTH_SHORT).show() // returns an error if fields are empty
    }
}
```

Then in the same activity, where the login button is checked and the user is first logged in, the same check happens where if the user is logging in for the first time, there will be no data for them in firebase, so they will therefore be sent to the personal details activity which will ask them to enter their details, but if they are logging in for a second time, etc, data will exist and the app will skip that screen and send them straight to the app

Testing

Login and Sign up testing video

Test ID 1	Timestamp	PURPOSE	DATA	EXPECTED RESULTS	ACTUAL RESULTS
1.14	2:56	To test whether the app will log the user into the app if they have already	n/a	The app will take the user to the personal details	As expected

		previously logged in, but without any details entered		activity so that the user will enter their details	
1.15	3:07	To test whether the app will log the user into the app if they have already previously logged in, personal details have then already been entered.	n/a	The app will take the user straight to the home page, skipping the personal details activity	As expected

Calorie Tracking Module

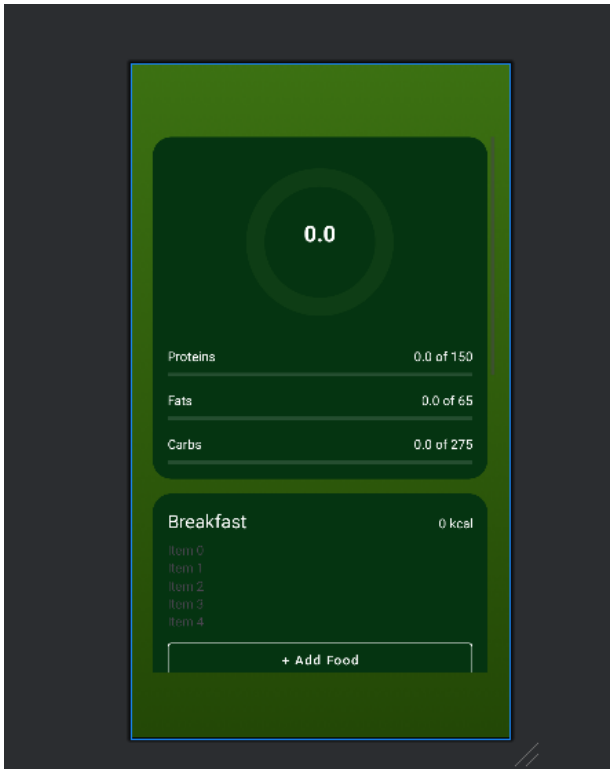
XMLS

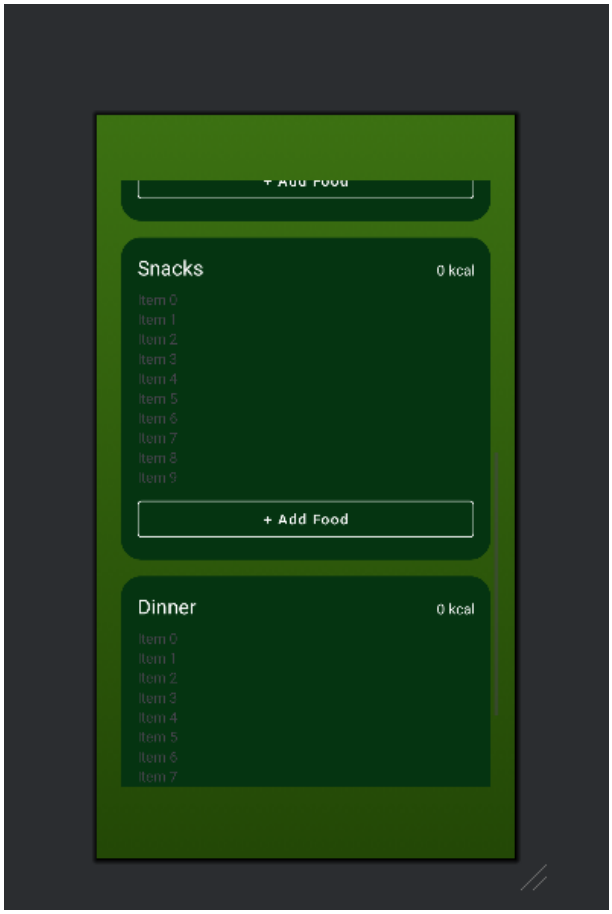
```xml
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@drawable/default_screen_gradient">

    <ScrollView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:layout_marginTop="?attr/actionBarSize"
        android:layout_marginBottom="70dp"
        android:clipToPadding="false"
        android:paddingHorizontal="16dp"
        android:paddingVertical="16dp">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:spacing="16dp"
            android:layout_marginHorizontal="8dp">

            <!-- Progress Summary Card -->
            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="vertical"
                android:padding="16dp"
                android:layout_marginBottom="16dp"
                android:background="@drawable/darkgreen_round">

                <!-- Calorie Circle -->
                <RelativeLayout
                    android:layout_width="180dp"
                    android:layout_height="180dp"
                    android:layout_gravity="center_horizontal"
                    android:layout_marginTop="8dp"
                    android:layout_marginBottom="16dp">

                    <ProgressBar
                        android:id="@+id/calorieProgress"
                        style="?android:attr/progressBarStyleHorizontal"
                        android:layout_width="match_parent"
                        android:layout_height="match_parent"
                        android:background="@drawable/circle_shape_background"
                        android:progressDrawable="@drawable/circle_shape_progress"
                        android:rotation="-90" />

                    <LinearLayout
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:layout_centerInParent="true"
                        android:gravity="center"
                        android:orientation="vertical">

                        <TextView
                            android:id="@+id/calorieCount"
                            android:layout_width="wrap_content"
                            android:layout_height="wrap_content"
                            android:textColor="@color/white"
                            android:textSize="24sp"
                            android:textStyle="bold"
                            android:text="0.0"/>

                        <TextView
                            android:id="@+id/calorieGoalText"
                            android:layout_width="wrap_content"
                            android:layout_height="wrap_content"
                            android:text=""
                            android:textColor="@color/white"/>
                    </LinearLayout>
                </RelativeLayout>

                <!-- Macro Progress Bars -->
                <LinearLayout
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:orientation="vertical"
                    android:layout_marginTop="8dp">
```

```
<!-- Proteins -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:layout_marginBottom="8dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Proteins"
        android:textColor="@color/white"/>

    <TextView
        android:id="@+id/proteinText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="0.0 of 150"
        android:textAlignment="textEnd"
        android:textColor="@color/white"/>
</LinearLayout>

<ProgressBar
    android:id="@+id/proteinProgress"
    style="@style/Widget.AppCompat.ProgressBar.Horizontal"
    android:layout_width="match_parent"
    android:layout_height="4dp"
    android:progressTint="@color/white"
    android:progressBackgroundTint="#80FFFFFF"/>

<!-- Fats -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:layout_marginTop="16dp"
    android:layout_marginBottom="8dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Fats"
        android:textColor="@color/white"/>

    <TextView
        android:id="@+id/fatsText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="0.0 of 65"
        android:textAlignment="textEnd"
        android:textColor="@color/white"/>
</LinearLayout>

<ProgressBar
    android:id="@+id/fatsProgress"
    style="@style/Widget.AppCompat.ProgressBar.Horizontal"
    android:layout_width="match_parent"
    android:layout_height="4dp"
    android:progressTint="@color/white"
    android:progressBackgroundTint="#80FFFFFF"/>

<!-- Carbs -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:layout_marginTop="16dp"
    android:layout_marginBottom="8dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Carbs"
        android:textColor="@color/white"/>
```

```
<!-- Breakfast Section -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="16dp"
    android:layout_marginBottom="16dp"
    android:background="@drawable/darkgreen_round">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Breakfast"
            android:textColor="@color/white"
            android:textSize="20sp"/>

        <TextView
            android:id="@+id/breakfastCalories"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="0 kcal"
            android:textAlignment="textEnd"
            android:textColor="@color/white"/>
    </LinearLayout>

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/breakfastRecyclerView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginVertical="8dp"/>

    <Button
        android:id="@+id/addBreakfastButton"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="+ Add Food"
        android:textColor="@color/white"
        style="@style/Widget.MaterialComponents.Button.OutlinedButton"
        android:textAllCaps="false"
        app:strokeColor="@color/white"/>
</LinearLayout>

<!-- Snacks Section -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="16dp"
    android:layout_marginBottom="16dp"
    android:background="@drawable/darkgreen_round">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Snacks"
            android:textColor="@color/white"
            android:textSize="20sp"/>

        <TextView
            android:id="@+id/snacksCalories"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="0 kcal"
            android:textAlignment="textEnd"
            android:textColor="@color/white"/>
    </LinearLayout>
```

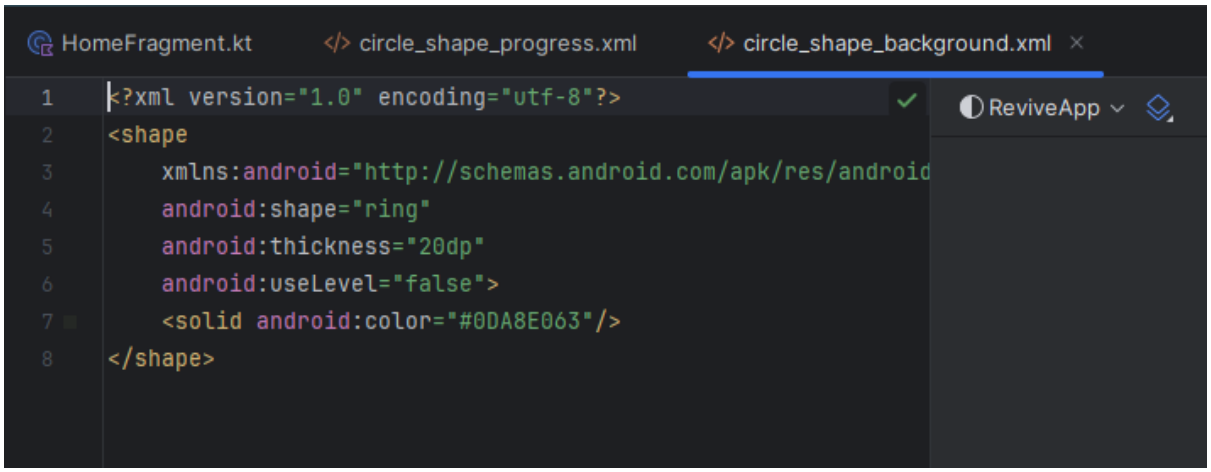
```
254
255                 <androidx.recyclerview.widget.RecyclerView
256                     android:id="@+id/snacksRecyclerView"
257                     android:layout_width="match_parent"
258                     android:layout_height="wrap_content"
259                     android:layout_marginVertical="8dp"/>
260
261                 <Button
262                     android:id="@+id/addSnacksButton"
263                     android:layout_width="match_parent"
264                     android:layout_height="wrap_content"
265                     android:text="+ Add Food"
266                     android:textColor="@color/white"
267                     style="@style/Widget.MaterialComponents.Button.OutlinedButton"
268                     android:textAllCaps="false"
269                     app:strokeColor="@color/white"/>
270             </LinearLayout>
271
272             <!-- Dinner Section -->
273             <LinearLayout
274                 android:layout_width="match_parent"
275                 android:layout_height="wrap_content"
276                 android:orientation="vertical"
277                 android:padding="16dp"
278                 android:layout_marginBottom="16dp"
279                 android:background="@drawable/darkgreen_round">
280
281                 <LinearLayout
282                     android:layout_width="match_parent"
283                     android:layout_height="wrap_content"
284                     android:orientation="horizontal">
285
286                     <TextView
287                         android:layout_width="wrap_content"
288                         android:layout_height="wrap_content"
289                         android:text="Dinner"
290                         android:textColor="@color/white"
291                         android:textSize="20sp"/>
292
293                     <TextView
294                         android:id="@+id/dinnerCalories"
295                         android:layout_width="match_parent"
296                         android:layout_height="wrap_content"
297                         android:text="0 kcal"
298                         android:textAlignment="textEnd"
299                         android:textColor="@color/white"/>
300                 </LinearLayout>
301
302                 <androidx.recyclerview.widget.RecyclerView
303                     android:id="@+id/dinnerRecyclerView"
304                     android:layout_width="match_parent"
305                     android:layout_height="wrap_content"
306                     android:layout_marginVertical="8dp"/>
307
308                 <Button
309                     android:id="@+id/addDinnerButton"
310                     android:layout_width="match_parent"
311                     android:layout_height="wrap_content"
312                     android:text="+ Add Food"
313                     android:textColor="@color/white"
314                     style="@style/Widget.MaterialComponents.Button.OutlinedButton"
315                     android:textAllCaps="false"
316                     app:strokeColor="@color/white"/>
317             </LinearLayout>
318
319         </LinearLayout>
320     </ScrollView>
321 </FrameLayout>
```


0.0
Proteins
0.0 of 150
Fats
0.0 of 65
Carbs
0.0 of 275
Breakfast
0 kcal
Item 0
Item 1
Item 2
Item 3
Item 4
+ Add Food


Snacks
0 kcal
Item 0
Item 1
Item 2
Item 3
Item 4
Item 5
Item 6
Item 7
Item 8
Item 9
+ Add Food
Dinner
0 kcal
Item 0
Item 1
Item 2
Item 3
Item 4
Item 5
Item 6
Item 7

This XML design showcases how the home fragment will look like. I have included everything inside a scrollview, meaning the user will be able to scroll all the way through. This XML will work on the fact that each food item can be added to each of the three sections, Breakfast, Snacks, Dinner and a recycler view will be responsible for each individual item in each of the three lists. At the top there will be a total of 4 progress bars, one main one tracking the users total calories for the day, and 3 sub progress bars tracking the macro progress.

This small XML is the design of the main progress bar which will be a gradient from light green to dark green.



This XML will show the background of the progress bar as a translucent green to be filled in as they progress, so that the user can see there is always a progress bar there, and not just an empty screen.

Logic

Daily Meals Class

```
package com.example.reviveapp

data class DailyMeals(
    var date: String = "",
    var breakfast: MutableMap<String, FoodItemWithQuantity> = mutableMapOf(),
    var snacks: MutableMap<String, FoodItemWithQuantity> = mutableMapOf(),
    var dinner: MutableMap<String, FoodItemWithQuantity> = mutableMapOf(),
    var totalCalories: Double = 0.0,
    var totalProteins: Double = 0.0,
    var totalFats: Double = 0.0,
    var totalCarbs: Double = 0.0
) {
    fun updateTotals() {
        totalCalories = 0.0
        totalProteins = 0.0
        totalFats = 0.0
        totalCarbs = 0.0

        // Calculate totals from all meals
        listOf(breakfast, snacks, dinner).forEach { meal ->
            meal.values.forEach { foodItem ->
                totalCalories += foodItem.calories * foodItem.grams
                totalProteins += foodItem.proteins * foodItem.grams
                totalFats += foodItem.fats * foodItem.grams
                totalCarbs += foodItem.carbs * foodItem.grams
            }
        }
    }

    fun getMealCalories(meal: MutableMap<String, FoodItemWithQuantity>): Double {
        return meal.values.sumOf { it.calories * it.grams }
    }
}
```

]

I started by creating a data class DailyMeals to manage all meals for a single day, where there will be three lists which will contains separate maps for breakfast, snacks, and dinner foods and will be separate from one another. Each meal is stored as a map where the key is the food name and the value is the FoodItemWithQuantity. Since foods are all stored under the food item class, I can use the function to retrieve a variable which will be their macro and multiply it by the grams that the user will later place into the meal, so that a total can be created for the meal. Then finally i created a get method to return the total calories of the meal.

```
class HomeMealAdapter(private val onItemClick: (FoodItemWithQuantity) -> Unit) :
    RecyclerView.Adapter<HomeMealAdapter.FoodViewHolder>() {

    private var foods = mutableListOf<FoodItemWithQuantity>()

    inner class FoodViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) {
        val nameTextView: TextView = itemView.findViewById(R.id.recTitle)
        val detailsTextView: TextView = itemView.findViewById(R.id.recPriority)

        init {
            itemView.setOnClickListener {
                val position = adapterPosition
                if (position != RecyclerView.NO_POSITION) {
                    onItemClick(foods[position])
                }
            }
        }
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): FoodViewHolder {
        val view = LayoutInflater.from(parent.context)
            .inflate(R.layout.recycler_item, parent, false)
        return FoodViewHolder(view)
    }
```

Since I knew I would be using a recycler view I created another Adapter class to be able to display each item and add interaction to it, unfortunately I am not able to reuse other adapters already created as each adapter has its own set of functions, so if I reused the adapter from the meal class my home objects would have the exact same interaction as those from the meal class, which is not what is wanted. However, this does reuse the same *recyler_item.xml* as used before. Fortunately, since all three lists in the main screen will need the same interactivity, they will all be able to reuse this one adapter class, making this a suitable way of displaying the lists.

```
    override fun onBindViewHolder(holder: FoodViewHolder, position: Int) {
        val food = foods[position]
        holder.nameTextView.text = "${food.foodItem.name} (${food.grams.roundToDecimal(1)}g)"
        holder.detailsTextView.text = "${(food.calories * food.grams).roundToDecimal(1)} kcal"
```

This function will fill each view with the actual food data, setting the text of the name and calories as the title and priority of the recycler view item. These values are rounded as they are set to make sure that no long decimal numbers are displayed.

```
    override fun getItemCount() = foods.size

    fun updateFoods(newFoods: List<FoodItemWithQuantity>) {
        foods.clear()
        foods.addAll(newFoods)
        notifyDataSetChanged()
    }

    private fun Double.roundToDecimal(decimals: Int): Double {
        return BigDecimal(this)
            .setScale(decimals, RoundingMode.HALF_UP)
            .toDouble()
    }
}
```

I then created a function that will tell me how many foods are in the list for later use and a function to update the list with newer items, and the function to make the above code work for rounding the numbers to a certain number of decimal places (1).

Home Fragment

```
class HomeFragment : Fragment() {
    private var _binding: FragmentHomeBinding? = null
    private val binding get() = _binding!!

    private lateinit var databaseReference: DatabaseReference
    private lateinit var auth: FirebaseAuth

    // Adapters for each meal section
    private lateinit var breakfastAdapter: HomeMealAdapter
    private lateinit var snacksAdapter: HomeMealAdapter
    private lateinit var dinnerAdapter: HomeMealAdapter

    // Current daily meals data
    private var currentDailyMeals = DailyMeals()

    // Nutrient goals
    private val calorieGoal = 2000.0
    private val proteinGoal = 150.0
    private val fatGoal = 65.0
    private val carbGoal = 275.0

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        _binding = FragmentHomeBinding.inflate(inflater, container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
```

Then I setup the initial HomeFragment that will be the fragment the user travels to, and once again created binding to link the xml created earlier to this code. Each of the three lists will link to the same adapter but use three different variables so they are able to be setup this way. Then a couple default values for Goals in case firebase loading does not work.

```
private fun setupFirebase() {
    auth = FirebaseAuth.getInstance()
    val userId = auth.currentUser?.uid
    databaseReference = FirebaseDatabase.getInstance()
        .getReference("Information")
        .child(userId ?: "")
        .child("DailyMeals")
}

private fun getCurrentDate(): String {
    val dateFormat = SimpleDateFormat("yyyy-MM-dd", Locale.getDefault())
    return dateFormat.format(Date())
}

private fun Double.roundToDecimal(decimals: Int): Double {
    return BigDecimal(this)
        .setScale(decimals, RoundingMode.HALF_UP)
        .toDouble()
}
```

Then I setup firebase using a function instead of having to recode it multiple times each time it is needed, as well as a utility function to once again round values taken from firebase in case of any logic errors. Then I created a function that would return the current date based on the users local phone data, so essentially whatever the user has their phone date set to will be the data that is returned.

```
private fun loadUserGoals() {
    val userId = FirebaseAuth.getInstance().currentUser?.uid
    if (userId != null) {
        FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.europe-west1.firebasedatabase.app/")
            .getReference("Information")
            .child(userId)
            .child("personal information")
            .get()
            .addOnSuccessListener { snapshot ->
                calorieGoal = snapshot.child("deficitcalories").getValue(Double::class.java) ?: 0.0
                proteinGoal = snapshot.child("protein").getValue(Double::class.java) ?: 0.0
                fatGoal = snapshot.child("fats").getValue(Double::class.java) ?: 0.0
                carbGoal = snapshot.child("carbs").getValue(Double::class.java) ?: 0.0



            }
            .addOnFailureListener {
                Toast.makeText(context, "Failed to load calorie goals", Toast.LENGTH_SHORT).show()
            }
    }
}
```

I then created a function that would load the user goals from firebase, these goals would essentially be the deficit calories created from the calories calculated that the user has set, and then protein, fats and carb values saved in firebase. Testing Code: 8.01

```
    private fun setupRecyclerViews() {
```

```
        // Setup RecyclerViews with adapters and layouts
        binding.breakfastRecyclerView.apply {
            layoutManager = LinearLayoutManager(context)
            adapter = breakfastAdapter
        }

        binding.snacksRecyclerView.apply {
            layoutManager = LinearLayoutManager(context)
            adapter = snacksAdapter
        }

        binding.dinnerRecyclerView.apply {
            layoutManager = LinearLayoutManager(context)
            adapter = dinnerAdapter
        }
    }
```

This function sets up the click listeners for the "Add Food" buttons in each meal section. When clicked, each button will show a dialog for selecting foods to add to that specific meal. Then it initializes the RecyclerViews for each meal section. It creates adapters that handle displaying the

food items and sets up click listeners for when a food item is tapped. Each RecyclerView is configured with a LinearLayoutManager for vertical scrolling and its corresponding adapter.

To handle the food selection i then had to code the showFoodItemDialog function that was called earlier **when the add button is clicked.**

Show food items function

```
private fun showFoodSelectionDialog(mealType: String) {
    val userId = FirebaseAuth.getInstance().currentUser?.uid
    if (userId != null) {
        val foodsRef = FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.
            .getReference("Information")
            .child(userId ?: "")
```

```
// Create dialog
val dialogView = LayoutInflater.from(requireContext())
    .inflate(R.layout.dialog_food_selection, null)
val builder = AlertDialog.Builder(requireContext())
builder.setView(dialogView)
val dialog = builder.create()

// Setup dialog views
val searchView = dialogView.findViewById<SearchView>(R.id.foodSearchView)
val foodListView = dialogView.findViewById<ListView>(R.id.foodListView)
val cancelButton = dialogView.findViewById<Button>(R.id.cancelButton)
val titleText = dialogView.findViewById<TextView>(R.id.dialogTitle)

titleText.text = "Select Food or Meal"

// Lists to hold our items
val allItems = mutableListOf<SelectionItem>()
val filteredItems = mutableListOf<SelectionItem>()

// Custom adapter for our combined list
val adapter = object : BaseAdapter() {
    override fun getCount(): Int = filteredItems.size
    override fun getItem(position: Int): SelectionItem = filteredItems[position]
    override fun getItemId(position: Int): Long = position.toLong()

    override fun getView(position: Int, convertView: View?, parent: ViewGroup): View {
        val itemView = convertView ?: LayoutInflater.from(context)
            .inflate(R.layout.recycler_item, parent, false)

        val nameTextView = itemView.findViewById<TextView>(R.id.recTitle)
        val detailsTextView = itemView.findViewById<TextView>(R.id.recPriority)

        val item = filteredItems[position]
        nameTextView.text = item.name
        when (item) {
            is SelectionItem.FoodItem -> {
                detailsTextView.text = "${item.food.calories} kcal/100g"
            }
            is SelectionItem.MealItem -> {
                detailsTextView.text = "${item.meal.totalCalories.roundToDecimal(1)} k
            }
        }
```

#

The dialog is then setup to find all the different components of the *dialog_food_selection.xml* that is reused and is shown and it creates a custom adapter (using BaseAdapter) that manages how each food item appears in the list. This adapter maintains two lists - the original list of all foods and a filtered list that changes based on search input that are variables that will store the foodlists from firebase. Each food item in the list shows both the food's name and its calorie content. When

displaying each item, the adapter either reuses an existing view (convertView) or creates a new one if needed, which is an efficient way to handle scrolling through long lists.

The filtered list starts off containing all foods but can be updated based on user searches.

```
// Load foods first
foodsRef.child("Food items").get().addOnSuccessListener { foodSnapshot ->
    foodSnapshot.children.forEach { foodSnap ->
        val food = foodSnap.getValue(FoodItem::class.java)
        food?.let {
            it.convertto100gram() // Convert to 100g for display
            allItems.add(SelectionItem.FoodItem(it))
        }
    }

    // Then Load meals
    foodsRef.child("Meals").get().addOnSuccessListener { mealSnapshot ->
        mealSnapshot.children.forEach { mealSnap ->
            val meal = mealSnap.getValue(Meal::class.java)
            meal?.let {
                allItems.add(SelectionItem.MealItem(it))
            }
        }

        if (allItems.isEmpty()) {
            Toast.makeText(context, "No items available", Toast.LENGTH_SHORT).show
            dialog.dismiss()
        } else {
            // Initial population of filtered list
            filteredItems.addAll(allItems)
            adapter.notifyDataSetChanged()
        }
```

This function creates a dialog for selecting foods to add to a meal. It starts by fetching all food items and all meal items from Firebase, converts any food items to 100g portions for consistent display, and shows them together in the same list. If there are no items in the list then the dialog is immediately dismissed and the user is given a message that there are not items available to perform validation, meaning they havent created any items to include in the Daily Foods. Testing Code up to this point: 8.02, 8.03

```
// Setup search functionality
searchView.setOnQueryTextListener(object : SearchView.OnQueryTextListener {
    override fun onQueryTextSubmit(query: String?): Boolean = false

    override fun onQueryTextChange(newText: String?): Boolean {
        filteredItems.clear()
        if (newText.isNullOrEmpty()) {
            filteredItems.addAll(allItems)
        } else {
            val searchText = newText.toLowerCase()
            filteredItems.addAll(allItems.filter { item ->
                when (item) {
                    is SelectionItem.FoodItem ->
                        item.food.name.toLowerCase().contains(searchText)
                    is SelectionItem.MealItem ->
                        item.meal.name.toLowerCase().contains(searchText)
                }
            })
        }
        adapter.notifyDataSetChanged()
        return true
    }
})
```

I then reused the code from the search functionality only this time the selection item does not only look for food items but it also looks for meal items too, This time I made sure to use the correct variables to not cause the same logic error as in the meal fragment. Testing code for search: 8.08

```
        // Handle item selection
        foodListView.setOnItemClickListener { _, _, position, _ ->
            val selectedItem = filteredItems[position]
            when (selectedItem) {
                is SelectionItem.FoodItem -> {
                    val selectedFood = selectedItem.food.copy()
                    selectedFood.converttoigram() // Convert back to 1g before quantity in
                    showFoodQuantityDialog(selectedFood, mealType)
                }
                is SelectionItem.MealItem -> {
                    addMealToMeal(selectedItem.meal, mealType)
                }
            }
            dialog.dismiss()
        }

        cancelButton.setOnClickListener {
            dialog.dismiss()
        }

        if (dialog.window != null) {
            dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
        }

        dialog.show()
    } else {
        Toast.makeText(context, "User not authenticated", Toast.LENGTH_SHORT).show()
    }
}
```

```
// Sealed class to represent our selection items
sealed class SelectionItem {
    abstract val name: String

    data class FoodItem(val food: com.example.reviveapp.FoodItem) : SelectionItem() {
        override val name: String get() = food.name
    }

    data class MealItem(val meal: Meal) : SelectionItem() {
        override val name: String get() = meal.name
    }
}
```

I then created handler that would validate whether the item clicked on by the user was a individual food item, which would call a different function that would show a dialog for the user to pick what quantity they wanted to add, or whether the item was a meal item, which would call a function that would add the meal to the menu. To check whether the item is a food item or a meal item I just called the item classes to check which class the item belongs to. I set breakpoints on both functions (showFoodQuantityDialog and addMealtoMeal) for now so that errors are not called and coded each one separately.

Before creating those functions I created another utility function in the HomeFragment that would refresh any of the values on the UI in case of a detail change.

```
private fun updateUI() {
    // Update progress displays
    binding.calorieCount.text = currentDailyMeals.totalCalories.roundToDecimal(1).toString()
    binding.calorieProgress.progress =
        ((currentDailyMeals.totalCalories / calorieGoal) * 100).toInt()

    // Update macro progress bars and texts
    binding.proteinProgress.progress =
        ((currentDailyMeals.totalProteins / proteinGoal) * 100).toInt()
    binding.proteinText.text =
        "${currentDailyMeals.totalProteins.roundToDecimal(1)} of $proteinGoal"

    binding.fatsProgress.progress =
        ((currentDailyMeals.totalFats / fatGoal) * 100).toInt()
    binding.fatsText.text =
        "${currentDailyMeals.totalFats.roundToDecimal(1)} of $fatGoal"

    binding.carbsProgress.progress =
        ((currentDailyMeals.totalCarbs / carbGoal) * 100).toInt()
    binding.carbsText.text =
        "${currentDailyMeals.totalCarbs.roundToDecimal(1)} of $carbGoal"

    // Update meal calorie displays
    binding.breakfastCalories.text =
        "${currentDailyMeals.getMealCalories(currentDailyMeals.breakfast).roundToDecimal(1)}
    binding.snacksCalories.text =
        "${currentDailyMeals.getMealCalories(currentDailyMeals.snacks).roundToDecimal(1)} kc
    binding.dinnerCalories.text =
        "${currentDailyMeals.getMealCalories(currentDailyMeals.dinner).roundToDecimal(1)} kc

    // Update RecyclerViews
    breakfastAdapter.updateFoods(currentDailyMeals.breakfast.values.toList())
    snacksAdapter.updateFoods(currentDailyMeals.snacks.values.toList())
    dinnerAdapter.updateFoods(currentDailyMeals.dinner.values.toList())
}
```

The updateUI function manages the visual display of all nutritional tracking elements in the user interface. It systematically updates three main aspects of the display:

1. First, it updates the central calorie tracking display by modifying both the numerical calorie count and the visual progress indicator, converting the current calories into a percentage of the daily goal.
2. Second, it refreshes all macro-nutrient displays (proteins, fats, and carbohydrates), updating both the progress bars and their corresponding text displays to show current values against daily targets. The function ensures numerical precision through decimal rounding.
3. Updated a small variable at the right of the display that will show the total calories for that section.

4. Lastly, it updates the meal-specific sections by refreshing the calorie displays for each meal type (breakfast, snacks, and dinner) and updating their respective food item lists through RecyclerView adapters.

The function maintains synchronization between the underlying data and its visual representation, ensuring the interface accurately reflects the current nutritional status. Testing Code: 8.02

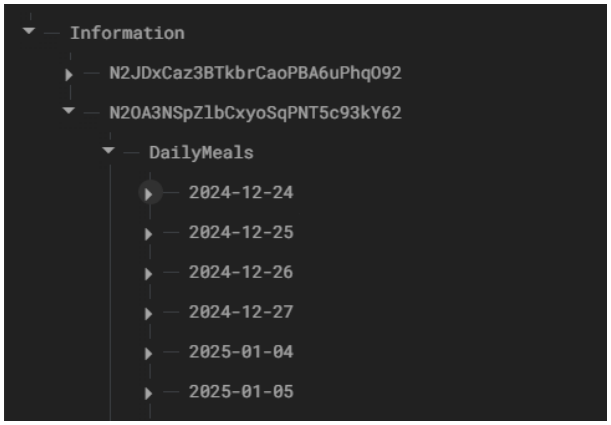
Then I started working on what happens when food items are added.

```
private fun showFoodQuantityDialog(foodItem: FoodItem, mealType: String) {
    Log.d("HomeFragment", "Opening quantity dialog for ${foodItem.name}")
    val builder = AlertDialog.Builder(requireContext())
    val view = layoutInflater.inflate(R.layout.dialog_quantity_input, null)

    val quantityInput = view.findViewById<EditText>(R.id.quantityInput)
    val confirmButton = view.findViewById<Button>(R.id.MealSaveBtn)
    val cancelButton = view.findViewById<Button>(R.id.MealCloseBtn)

    builder.setView(view)
    val dialog = builder.create()

    confirmButton.setOnClickListener {
        val quantity = quantityInput.text.toString().toDoubleOrNull()
        Log.d("HomeFragment", "Quantity entered: $quantity")
        if (quantity != null && quantity > 0) {
            addFoodToMeal(foodItem, quantity, mealType)
            dialog.dismiss()
        } else {
            Toast.makeText(context, "Please enter a valid quantity", Toast.LENGTH_SHORT).show()
        }
    }

    cancelButton.setOnClickListener { dialog.dismiss() }

    if (dialog.window != null) {
        dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
    }

    dialog.show()
}
```

When an individual food item is selected I want the same dialog to show that was used for the meal creation class, which asks the user how much of the item they would like to add, so all I needed to do was reuse the same code from the meal creation class, but change the function it calls to include the mealType variable added earlier. However the code also validates whether the user has entered a value into the amount dialog, if they haven't a message is displayed. Testing Code: 8.04

```
private fun addFoodToMeal(foodItem: FoodItem, grams: Double, mealType: String) {
    val foodItemWithQuantity = FoodItemWithQuantity(foodItem, grams)
    val currentDate = getCurrentDate()

    // Add to local data structure
    when (mealType) {
        "Breakfast" -> currentDailyMeals.breakfast[foodItem.name] = foodItemWithQuantity
        "Snacks" -> currentDailyMeals.snacks[foodItem.name] = foodItemWithQuantity
        "Dinner" -> currentDailyMeals.dinner[foodItem.name] = foodItemWithQuantity
    }
    currentDailyMeals.updateTotals()

    // Update Firebase
    databaseReference.child(currentDate)
        .setValue(currentDailyMeals)
        .addOnSuccessListener {
            currentDailyMeals.updateTotals()
            updateUI()
        }
        .addOnFailureListener {
            Toast.makeText(context, "Failed to save food item", Toast.LENGTH_SHORT).show()
        }
}
```

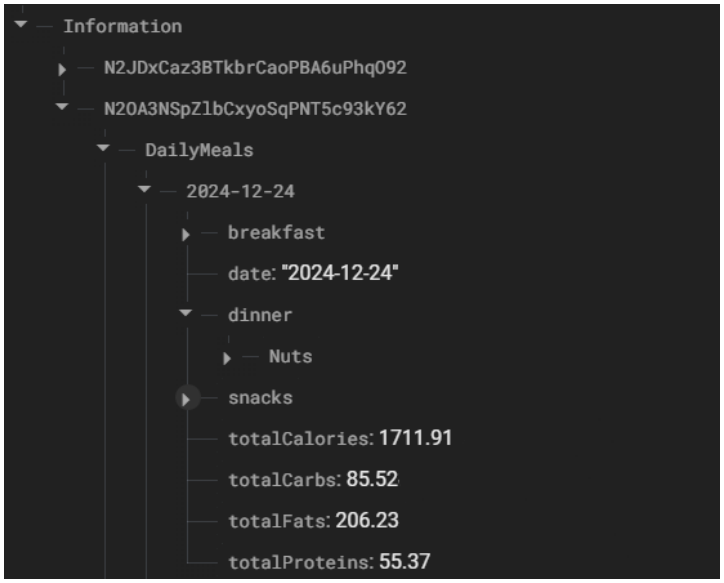
Then the food item is passed into the addFoodToMeal function from the dialog, which just handles saving the food item to the daily meals section in firebase, nesting under whether it is a breakfast, snack or dinner item. The updateUI function from earlier is also called so that any progress bars are updated and foods are added to the lists.

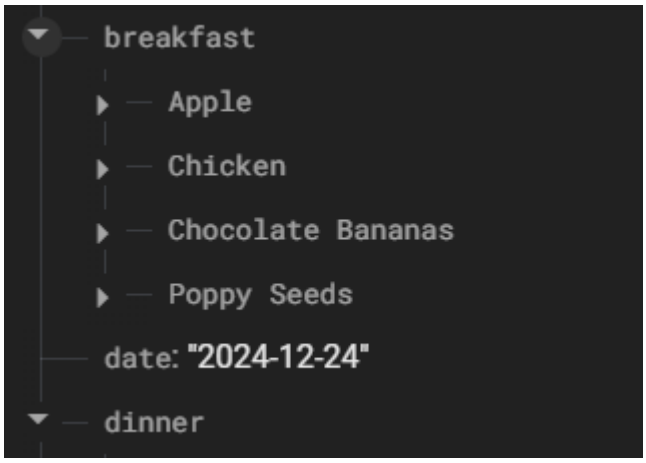


Then I swapped to adding what happens when a meal Item is clicked on, since the user has already entered the values of their items when creating the meal, they do not need to re-enter all the quantity amounts and therefore the meals can just be added straight to the daily meals in database, and ui updated. Testing Code: 8.05

```
// Function to add a meal's items to current meal
private fun addMealToMeal(mealToAdd: Meal, mealType: String) {
    val currentDate = getCurrentDate()

    // Add all food items from the selected meal
    mealToAdd.foodItems.values.forEach { foodItem ->
        when (mealType) {
            "Breakfast" -> currentDailyMeals.breakfast[foodItem.foodItem.name] = foodItem
            "Snacks" -> currentDailyMeals.snacks[foodItem.foodItem.name] = foodItem
            "Dinner" -> currentDailyMeals.dinner[foodItem.foodItem.name] = foodItem
        }
    }

    // Update Firebase
    databaseReference.child(currentDate)
        .setValue(currentDailyMeals)
        .addOnSuccessListener {
            currentDailyMeals.updateTotals()
            updateUI()
            Toast.makeText(context, "Meal added", Toast.LENGTH_SHORT).show()
        }
        .addOnFailureListener {
            Toast.makeText(context, "Failed to add meal", Toast.LENGTH_SHORT).show()
        }
}
```

While these functions may be similar, they cannot be abstracted down to one as the individual foods function adds a singular item, whereas this one adds each item in the meal one by one going through the list. This means that the meal is not stored as a meal in the DailyMeals, instead all its sub meal items get placed into the lists. For example if there were to be a meal with the name “eggs on toast” consisting of eggs and toast, the code would have eggs and toast placed into the breakfast list as separate items, instead of “eggs on toast” as a singular item, this is also a suitable way of doing this as the user is then able to individually change the values of the items in case they were to differ for that day, instead of having to go back and edit the entire meal. Testing Code: 8.07





Finally, I added a function that would load all the current meals from the database that have been added during a day, in case the user has left the app and is returning at a later time.

```
private fun loadTodaysMeals() {
    val currentDate = getCurrentDate()

    databaseReference.child(currentDate)
        .addValueEventListener(object : ValueEventListener {
            override fun onDataChange(snapshot: DataSnapshot) {
                // Check if this is a new day
                val savedDate = snapshot.child("date").getValue(String::class.java)
                if (savedDate != currentDate) {
                    // It's a new day, reset the meals
                    currentDailyMeals = DailyMeals(date = currentDate)
                    // Save the empty day to Firebase
                    databaseReference.child(currentDate).setValue(currentDailyMeals)
                } else {
                    // Load existing meals for today
                    currentDailyMeals = DailyMeals()
                    currentDailyMeals.date = currentDate

                    // Load meals from snapshot
                    listOf("breakfast", "snacks", "dinner").forEach { mealType ->
                        snapshot.child(mealType).children.forEach { foodSnapshot ->
                            val food = foodSnapshot.getValue(FoodItemWithQuantity::class.java)
                            food?.let {
                                when (mealType) {
                                    "breakfast" -> currentDailyMeals.breakfast[it.foodItem.name] = it
                                    "snacks" -> currentDailyMeals.snacks[it.foodItem.name] = it
                                    "dinner" -> currentDailyMeals.dinner[it.foodItem.name] = it
                                }
                            }
                        }
                    }

                    currentDailyMeals.updateTotals()
                    updateUI()
                }
            }

            override fun onCancelled(error: DatabaseError) {
                Toast.makeText(context, "Failed to load meals", Toast.LENGTH_SHORT).show()
            }
        })
}
```

This takes in a list of all the items that have been added based on their mealType from the current date (breakfast, snacks, dinner) and updates the UI so that they can be shown in the recycler views.

Item interactivity

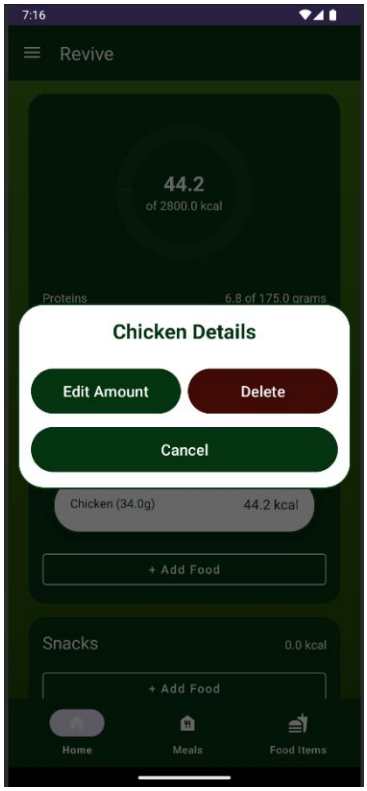
I want the user to be able to interact with the items they have added to their daily amounts, given the option to either change an amount they have entered, or remove it entirely by tapping on it, I think clicking on each individual item to interact with it will be better than having to create an entirely new fragment just to add an ability to edit the items, this also means that a dialog can be reused from the meal fragment, which is the one to edit or delete meals, *dialog_edit_meal_options.xml.*

```
private fun setupRecyclerViews() {
    // Initialize adapters with click handlers
    breakfastAdapter = HomeMealAdapter { foodItem ->
        showFoodItemDialog(foodItem, "Breakfast")
    }
    snacksAdapter = HomeMealAdapter { foodItem ->
        showFoodItemDialog(foodItem, "Snacks")
    }
    dinnerAdapter = HomeMealAdapter { foodItem ->
        showFoodItemDialog(foodItem, "Dinner")
    }

    // Setup RecyclerViews with adapters and layouts
    binding.breakfastRecyclerView.apply {
        layoutManager = LinearLayoutManager(context)
        adapter = breakfastAdapter
    }

    binding.snacksRecyclerView.apply {
        layoutManager = LinearLayoutManager(context)
        adapter = snacksAdapter
    }

    binding.dinnerRecyclerView.apply {
        layoutManager = LinearLayoutManager(context)
        adapter = dinnerAdapter
    }
}
```

The first thing I did was add code to the initial setup of the recycler views to also add a click handler, which I already implemented in the HomeMealAdapter, this would call a function that would show the dialog mentioned above, so that the user can select what option they would like to proceed with.

```
private fun showFoodItemDialog(foodItem: FoodItemWithQuantity, mealType: String) {
    val builder = AlertDialog.Builder(requireContext())
    val view = layoutInflater.inflate(R.layout.dialog_meal_options, null)

    // Set up dialog views
    val titleText = view.findViewById<TextView>(R.id.dialogTitle)
    val editButton = view.findViewById<Button>(R.id.editButton)
    val deleteButton = view.findViewById<Button>(R.id.deleteButton)
    val cancelButton = view.findViewById<Button>(R.id.cancelButton)

    // Customize dialog for meal items
    titleText.text = "${foodItem.foodItem.name} Details"
    editButton.text = "Edit Amount"

    builder.setView(view)
    val dialog = builder.create()

    // Handle button clicks
    editButton.setOnClickListener {
        dialog.dismiss()
        showQuantityEditDialog(foodItem, mealType)
    }

    deleteButton.setOnClickListener {
        dialog.dismiss()
        removeFoodFromMeal(foodItem, mealType)
    }

    cancelButton.setOnClickListener {
        dialog.dismiss()
    }

    if (dialog.window != null) {
        dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
    }

    dialog.show()
}
```

This function is reused once again from the meal fragment to display edit options, this time a few variables are changed around to display things according to what is needed such as the top title displaying the name of the item being edited so the user knows what item they have clicked on, and the function names that will be used have been changed so that they are better to the context of the home fragment to what they are doing.



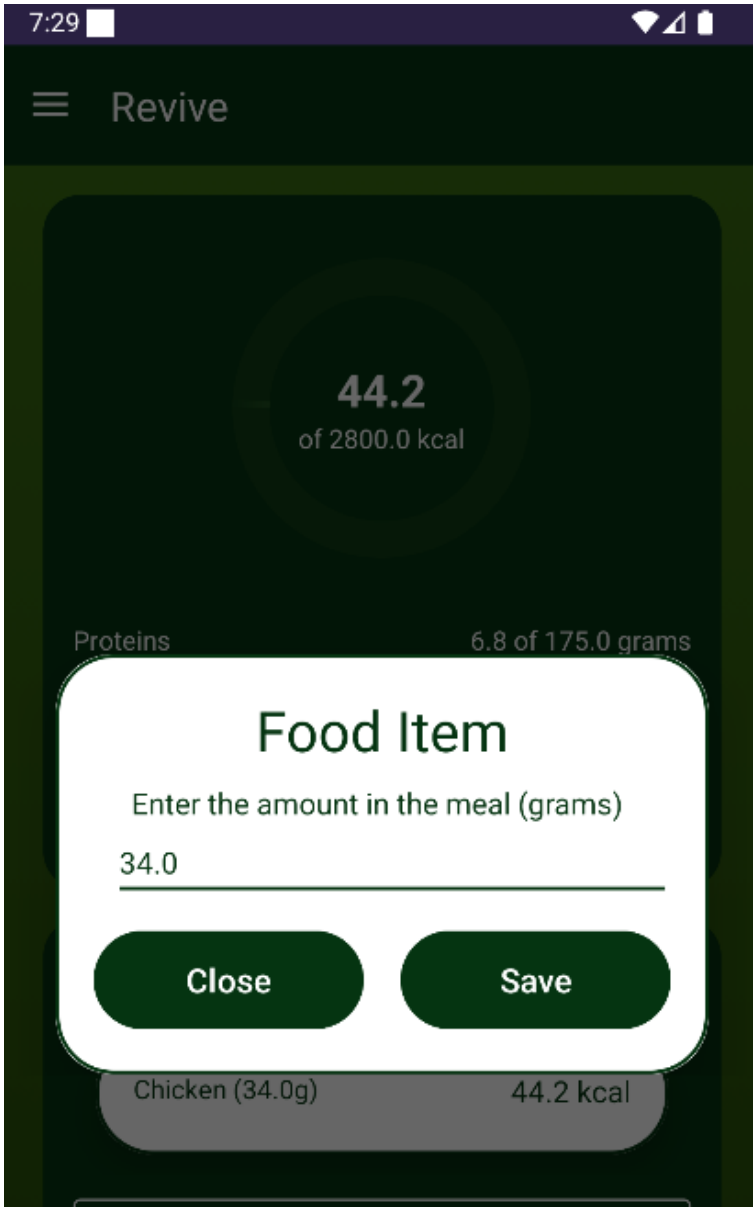
If the user pressed the close button the dialog is simply dismissed and nothing is changed, however the user can also press the edit or delete options. Testing Code: 8.09

Edit

```
private fun showQuantityEditDialog(foodItem: FoodItemWithQuantity, mealType: String) {
    val builder = AlertDialog.Builder(requireContext())
    val view = layoutInflater.inflate(R.layout.dialog_quantity_input, null)

    val quantityInput = view.findViewById<EditText>(R.id.quantityInput)
    val confirmButton = view.findViewById<Button>(R.id.MealSaveBtn)
    val cancelButton = view.findViewById<Button>(R.id.MealCloseBtn)

    // Show current quantity
    quantityInput.setText(foodItem.grams.toString())

    builder.setView(view)
    val dialog = builder.create()

    confirmButton.setOnClickListener {
        val newQuantity = quantityInput.text.toString().toDoubleOrNull()
        if (newQuantity != null && newQuantity > 0) {
            addFoodToMeal(foodItem, newQuantity, mealType)
            dialog.dismiss()
        } else {
            Toast.makeText(context, "Please enter a valid quantity", Toast.LENGTH_SHORT).show()
        }
    }

    cancelButton.setOnClickListener { dialog.dismiss() }

    if (dialog.window != null) {
        dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
    }

    dialog.show()
}
```

To save memory and efficiency I will reuse the dialog used for the creation of a meal item, where the item amount is being entered, *dialog_quantity_input.xml* and this dialog simply gets the current quantity that is already entered from the meal, and gives the user a box to enter a new value, if there is a correct value entered then the dialog will call the same addFoodToMeal() function from earlier since the values of the same name will simply be overwritten in database with the new values, instead of creating new ones.

This was the main reason that I had the items from the meals get placed into the lists as individual items, so that each item could be treated as an individual item once in the list, and updated in the same way, instead of having to code extra ways because the user may want to edit one specific thing inside a meal item, saving both time and efficiency in my development. Testing Code: 8.09


7:29
Revive
44.2
of 2800.0 kcal
Proteins
6.8 of 175.0 grams
Food Item
Enter the amount in the meal (grams)
34.0
Close
Save
Chicken (34.0g)
44.2 kcal

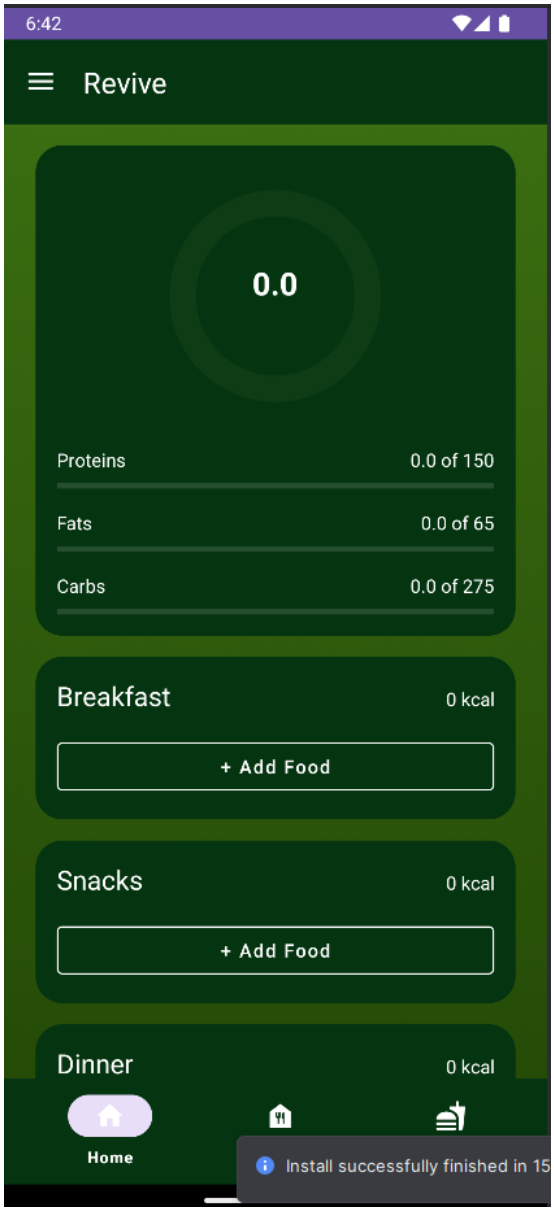
Delete

```
private fun removeFoodFromMeal(foodItem: FoodItemWithQuantity, mealType: String) {
    val currentDate = getCurrentDate()

    // Remove from local data
    when (mealType) {
        "Breakfast" -> currentDailyMeals.breakfast.remove(foodItem.foodItem.name)
        "Snacks" -> currentDailyMeals.snacks.remove(foodItem.foodItem.name)
        "Dinner" -> currentDailyMeals.dinner.remove(foodItem.foodItem.name)
    }

    // Update Firebase
    databaseReference.child(currentDate)
        .setValue(currentDailyMeals)
        .addOnSuccessListener {
            currentDailyMeals.updateTotals()
            updateUI()
            Toast.makeText(context, "Food item removed", Toast.LENGTH_SHORT).show()
        }
        .addOnFailureListener {
            Toast.makeText(context, "Failed to remove food item", Toast.LENGTH_SHORT).show()
        }
}
```

The code to remove an item from the dialog was very simple, as in the UI it only had to be removed from the lists of food and that list reset in database, with the updateUI function called to refresh the lists. The user would then recieve a confirmation message telling them the item has been removed from the daily lists. Testing Code: 8.10

Testing

Upon first testing the program I discovered that none of the goals I had created previously were loading from firebase, the home screen looked like this:

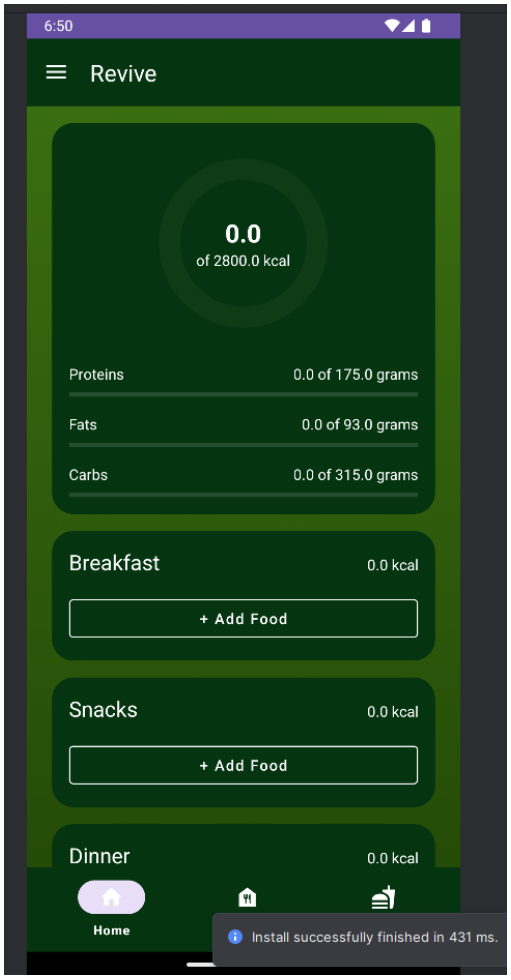


Where the protein fats and carbs goals are the default goals set by the home fragment for when none are detected in firebase. Testing Code: 8.01

```
private fun loadUserGoals() {
    val userId = FirebaseAuth.getInstance().currentUser?.uid
    if (userId != null) {
        FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.europe-west1.firebasedatabase.app/")
            .getReference("Information")
            .child(userId)
            .child("personal information")
            .get()
            .addOnSuccessListener { snapshot ->
                calorieGoal = snapshot.child("deficitcalories").getValue(Double::class.java) ?: 0.0
                proteinGoal = snapshot.child("protein").getValue(Double::class.java) ?: 0.0
                fatGoal = snapshot.child("fats").getValue(Double::class.java) ?: 0.0
                carbGoal = snapshot.child("carbs").getValue(Double::class.java) ?: 0.0



            }
            .addOnFailureListener {
                Toast.makeText(context, "Failed to load calorie goals", Toast.LENGTH_SHORT).show()
            }
    }
}
```

This was caused by the loadUserGoals function, where even after the goals were loaded, the UI function was not updated as it had not existed while being coded, therefore I had to go back into the code and add the updateUI function. Testing Code: 8.02

```
private fun loadUserGoals() {
    val userId = FirebaseAuth.getInstance().currentUser?.uid
    if (userId != null) {
        FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.europe-west1.firebasedatabase.app/")
            .getReference("Information")
            .child(userId)
            .child("personal information")
            .get()
            .addOnSuccessListener { snapshot ->
                calorieGoal = snapshot.child("deficitcalories").getValue(Double::class.java) ?: 0.0
                proteinGoal = snapshot.child("protein").getValue(Double::class.java) ?: 0.0
                fatGoal = snapshot.child("fats").getValue(Double::class.java) ?: 0.0
                carbGoal = snapshot.child("carbs").getValue(Double::class.java) ?: 0.0
                updateUI()



            }
            .addOnFailureListener {
                Toast.makeText(context, "Failed to load calorie goals", Toast.LENGTH_SHORT).show()
            }
    }
}
```



Later I realised that foods that the foods that have already been added to firebase, when the app was reloaded would not show up as the loadTodaysMeals function was created, but not called.
Testing Code: 8.05

I therefore decided that as everything is loaded from firebase, these two functions should be called to display the correct details:

```
private fun loadUserGoals() {
    val userId = FirebaseAuth.getInstance().currentUser?.uid
    if (userId != null) {
        FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.europe-west1.firebasedatabase.app/")
            .getReference("Information")
            .child(userId)
            .child("personal information")
            .get()
            .addOnSuccessListener { snapshot ->
                calorieGoal = snapshot.child("deficitcalories").getValue(Double::class.java) ?: 0.0
                proteinGoal = snapshot.child("protein").getValue(Double::class.java) ?: 0.0
                fatGoal = snapshot.child("fats").getValue(Double::class.java) ?: 0.0
                carbGoal = snapshot.child("carbs").getValue(Double::class.java) ?: 0.0
                updateUI()
                // Once we have the goals, load the meals
                loadTodaysMeals()
            }
            .addOnFailureListener {
                Toast.makeText(context, "Failed to load calorie goals", Toast.LENGTH_SHORT).show()
            }
    }
}
```

This made sure that all the goals are correctly fetched and displayed and placed where they should be, as well as items appeared after the phone was refreshed. Testing Code: 8.06

Calorie Tracking Module testing video

Test ID 8	Timestamp	PURPOSE	DATA	EXPECTED RESULTS	ACTUAL RESULTS
8.01	0:00	To test whether the goals calculated for the user based on their percentage splits from the daily goals will load into the calorie tracking module	Data entered in firebase by user.	All data is loaded into the variables and UI is updated to show this data	Fail: data was loaded into variables but nothing showed in the UI.
8.02	0:08	To test whether the goals calculated for the user based on their percentage splits from the daily goals will load into the calorie tracking module as well as testing whether the user can load up the entry dialog without already created food items.	Data entered in firebase by user.	All data is loaded into the variables and UI is updated to show this data, dialog is closed without data and a message is displayed.	As expected.
8.03	0:26	To test whether the dialog to input food is shown when food items are available	Food Items: Apple Chicken Cinamon Meal Items: Apple Cinamon	Food items are displayed using the same recycler view list and the food items are displayed first	As expected.

				and afterwards the meal items.	
8.04	0:44	To test whether the user is able to add an item without entering a quantity for it.	No data added	The dialog returns a message without the dialog dismissing.	As expected.
8.05	0:52	To test whether the user can enter an individual food item, but also whether those items are correctly saved to firebase and loaded when the app is reloaded	Food Item added: Chicken: 50grams	The food item is saved under the daily meals child in firebase and UI is updated to show the item in firebase, then when the app is reloaded the food items are loaded back into the app.	Pass: Food items are saved correctly and progress bars updated Fail: Food items are not loaded back after app reload
8.06	1:29	To test whether the user can enter an individual food item, but also whether those items are correctly saved to firebase and loaded when the app is reloaded	Food Item added: Chicken: 50grams	The food item is saved under the daily meals child in firebase and UI is updated to show the item in firebase, then when the app is reloaded the food items are loaded back into the app.	As expected.
8.07	2:13	To test when the user enters a meal item, the code does not ask for quantity input and instead fetches the quantity from what is saved in firebase, and converts the meal into individual food items.	Meal with items: Apple: 100g Cinamon: 30g	The meal items are added as individual items and progress bars are updated as if an individual item was added.	As expected.
8.08	2:36	To test whether the recycler view list updates dynamically with the query the user has entered.	Query entered	The list is updated with the items based on the user search	As expected.

8.09	2:59	To test whether the dialog that asks the user whether they want to edit appears, and for when the user clicks the edit option the quantity edit dialog from earlier appears with their current amount already in the edit box.	Value entered by user already in firebase. New value: 100g	The progress bars are updated and amounts of objects are updated.	As expected.
8.10	3:33	To test whether the user is able to delete an item from the daily amount by pressing the button.	n/a	Item is deleted from the UI and the progress bars.	As expected.
8.11	3:45	Progress bars are updated according to user action	n/a	All 3 progress bars updated upon user action	As expected.

Development Summary

The Calorie Tracking module serves as the central hub where all previous modules converge into practical functionality. It provides users with a daily tracking interface divided into three meal sections (breakfast, snacks, dinner), each with the ability to add both individual foods and complete meals. Progress bars visualize daily calories and macro targets, updating in real-time as users add or modify entries. Users can edit quantities or remove items through direct interaction with list entries, maintaining the consistent interface patterns established in earlier modules.

The app now represents a complete basic fitness tracking system. From initial user signup through personal information management, food and meal creation, to daily tracking. All components work together cohesively. Users can progress logically from setting up their profile and goals, creating their food database, building meals, and finally tracking their daily nutrition. This approach is justified as it provides all essential features needed for basic fitness tracking while maintaining consistent interaction patterns throughout. The decision to separate meals into three distinct sections, reuse existing dialogs for food selection, and provide visual feedback through progress bars creates an intuitive experience that caters to both beginners and experienced users. While additional features could enhance the app, the current implementation delivers a functional fitness tracking solution that fulfils the core requirements of accurate tracking, easy data entry, and clear progress visualization.

Evaluation

Testing to inform evaluation

The timestamps in this testing table may repeat, this is because the timestamps are the main modules rather than the individual tests to test the program, the test mentioned will occur within that module.

Test ID	Timestamp	Purpose	Expected Result	Actual Result	Success Criteria
9.1	00:00	Check user registration and login for a new user	Successful registration and login leading to collecting correct user information, if wrong information is input error messages should be returned.	Result as expected.	#1 – Registration
9.2	00:00	Check user registration and login for an existing user.	Successful login to app without the need to collect information as it is already stored	Result as expected.	#1 – Registration
10.1	03:02	Check for successful collection and storage of data in firebase from the user after input and submission.	Firebase stores correct information, correctly listed under the correspondin g child nodes.	Partially successful. The data is nested under the correct nodes; however, the user is only able to input full integer values as rounding proves an issue if not.	#2 – User registers details
11.1	04:07	Check to make sure the user can navigate through different main screens of the app through a bar at the bottom with different buttons.	The user can press a button at through the bottom navigation bar and be taken to the correspondin g fragment.	After some issues, result as expected.	#3 – App Navigation
11.2	04:07	Check to make sure the user is able to	The user can press a button at the	As expected.	#3 – App Navigation.

		navigation through different, less important screens by showing a side menu with all the different options necessary.	top right of their screen, loading a side navigation view. From there any of the options can be pressed and the user will be taken to the correspondin g fragment.		
12	04:28	Check whether the user is able to edit the personal information they have entered if it were to change.	The user is able to choose to edit any personal information they have entered. Information is then updated in firebase and any new calculations are performed.	As expected.	#11 – Ability to edit information
12	05:37	To check whether the BMR and AMR are correctly calculated after the details are collected.	The correct details according to the user input will be calculated, then stored in firebase according to the correct child node	As expected.	#4 - Calculations
13	05:37	To check whether the code calculates the users nutrients using the calories and saves them correctly. Then ask the user whether they	The correct details calculated according to the values calculated from the user input, then stored into firebase,	Partially successful, the user is not given a choice during the creation, but values are calculated and saved as expected.	#4 - Calculations

		want to use these values.	nested correctly.		
14	06:50	To check whether the user can create individual food objects on the food items screen.	A food object is saved to firebase, where all its values are listed under the name of the object nested in the correct data.	Partially successful, values entered are taken and saved correctly, upon rounding any numbers that are not whole numbers have their decimal removed instead of being saved upon rounding.	#4 – 6 Object creation system
15	06:50	To check whether the user can edit and delete the individual food items	When the user wants to edit they are given an option to edit the specific item they want, if they want to delete their is a confirmation, and after the confirmation the item is removed. Any changes are updated in firebase.	As expected.	#11 – Ability to edit information
16	08:37	To check whether the user is able to take individual food items and form a meal out of an item and its quantity.	The user is able to select an individual food item, enter a quantity and pair it with other items to save a meal item. The meal should then save	As expected.	#7 – Meal Creation System

			correctly in firebase with all data.		
17	08:37	To check whether the user can edit and delete the contents of a meal, and an delete a meal entirely.	The user is able to "re-create" a meal by entering an edit mode and change any details they wish, the new details will be updated to firebase.	Partially successful. Most of the time everything as expected but on occasions where the name is changed, the old meal is not deleted and the app acts as if a separate meal has been created	#11 – Ability to edit information
18	10:40	To check whether the calorie tracking module involves information from all other modules and displays them accordingly to UI.	Any progress bars are filled with their correspondin g goals calculated earlier. Food items appear where needed even when user leaves the app.	As expected.	#8 – Calorie tracking system
18	10:40	To check whether the user is able to add any meals and food items as well as a quantity to a daily amount where progress bars are updated and saved.	Any items added should have their values returned from firebase and used to update progress bars. Anything added to a current date should be saved to firebase under the correct node.	Partially successful, all data is saved correctly and dates are added automatically. However dates are also saved for good so as the days in the year go on more and more data will be wasted in the database without a way	#8 – Calorie Tracking system

				of deleting data	
19	n/a edits made throughout video	The user is able to edit or delete any items and their quantity to the daily progress.	Any updates are saved to firebase correctly and the UI updates any progress bars accordingly.	As expected.	#11 – Ability to edit information
20	n/a searches made throughout video	To check any search bars throughout the app, fulfil their purpose.	The user is able to enter queries into any search bars that are in the app and the correspondin g information is shown if it exists.	As expected.	#12 – Ability to search information
21	0:00	To test if any data created on one device also shows up and can be retrieved on the same account on a different device.	The user can access their data on a different device seamlessly and is able to update and change data on that new device, which will also impact the database.	As expected.	#11 – Ability to edit information
22	0:41	To test if objects saved in daily calorie tracking module reset when the world date changes so that the user can enter values for a new day.	There will be no interaction from the user and the values will be reset by themselves each day.	As expected.	#8 – Calorie tracking system.

Full Testing Video for Evaluation table tests 9-20 and Success Criteria Comparison:
https://www.youtube.com/watch?v=xdYnTLQ73Pg

Testing video for tests 21-22:

https://www.youtube.com/watch?v=Uo1hK3yYh5Q

Success Criteria Comparison

Success Criteria	Status	Justification
#1 – Registration	Fully Achieved	This object was fully met. The user can register an account based on their email and password, if the email entered is not real an account will not be created. The user then has the ability to change their password should they forget it. All information is stored correctly and securely in firebase authentication. (Testing codes 1.00-1.13) Furthermore, there is a convenience feature for the user where they are automatically logged in if they have previously logged in. (Testing codes 1.14-1.15) User feedback from this stage 1 of development (stakeholder review) suggested that they would like to be able to sign up with different methods through integrating different companies such as google or Facebook. This improvement can be made through integrating the API of the companies through firebase through built in methods, and setting their login to a specific window opened by a button on the main page.
#2 – User registers details	Partially Achieved	The object was met to fulfil its purpose. It takes the correct values and where needed specifies values for the user to input in order to prevent any kind of errors occurring. (Testing codes 2.01 - 2.07)However the user is only able to enter full number integers, so for those who want to keep track of accurate data this may be an issue. The user feedback also mentioned this issue, how the difference in calculations could be small, but for a user who wants to proactively measure their numbers, this may be crucial. This improvement could be made through removing the rounding function created to round to an integer, and instead of rounding to firebase, the code could round **from** firebase instead to display accordingly without changing values.
#3 – App Navigation	Fully Achieved.	The objective was fully met. The app allows flawless transition between screens throughout the app, keeps things memory efficient and brings about a hierarchy for screens. (Testing codes 3.1-3.3) This criteria also allows for high amounts of modularity as new modules can simply be added to the navigation to be accessed.

		The user feedback for this section was predominantly positive, mentioning how there is just the right amount of navigation tools, any more would make the app feel cluttered. Some mentioned how there may have been a search feature for specific items in the side navigation to help with navigating when the number of items increase. This improvement could be made the same way the searches were made throughout the app, but instead of filtering the list of food items taken from firebase, the search could simply filter through the list of menu items stored in the strings.
#4 – Calculations	Fully Achieved.	The app takes any values entered by the user and calculates and saves values according to what is needed. 1. AMR, BMR and therefore Calories are calculated based on user input from the personal details (Testing code 2.07) 2. Calculated all the daily targets for a person based on previous calculations (Testing codes 2.17-2.23 3. Any individual foods have their entered values converted and rounded in firebase. (Testing Codes 4.04 - 4.05) 4. Any meal items take the individual food items and calculate the totals for a meal based on quantities entered. (Testing codes 5.03 - 5.07) 5. Any foods added to the day adds to any progress bars based on the user targets created earlier by finding the food in firebase and adding or subtracting the value based on user action. (Testing codes 8.01-8.11) The user feedback for this section was also positive, stating that the time to complete the calculations and have them show up was very fast. Yet, some users said there should be more restrictions within the app, to do with setting limits on inputs such as not being able to enter an unrealistic height above 250cm, or restrict the app from calculating values for those under 12. These improvements can be made by creating additional conditional statements that check the inputs before proceeding with the calculations for calories and the rest.
#5 – Tutorials	Not achieved.	Implementing user tutorials was challenging due to the app's complex navigation structure with multiple fragments, activities, and overlapping UI elements.

		The app already uses a combination of bottom navigation, drawer navigation, and fragment transitions, making it difficult to overlay or integrate tutorial elements without disrupting the existing UI flow. Instead, the app prioritizes core functionality like meal tracking, nutritional calculations, etc establishing a solid foundation, unfortunately this creates limitations for introducing additional UI layers needed for tutorials. The existing dialog-heavy interaction pattern would make it difficult to introduce guided tours or contextual hints without significant refactoring of the underlying navigation and display logic. User feedback talked about the missing tutorials, how they would have been very helpful for newer users to have an interactive experience, however the stakeholders also mentioned that the app's simplicity means the elimination of dedicated tutorials is an acceptable solution, as the app is easy enough to understand without help since the user is not flooded with many options right from the start, they can explore the app themselves at their own pace. One way tutorials could have been added rather than messing with the app's navigation could have been to use dialog screens that show an image of what is to be done, this would mean that the user could dismiss the dialog when they are done reading and have understood what to do.
#6 Object Creation System	Fully achieved	This objective was fully met. The system allows the user to create individual food items quickly and efficiently with just a few input values and all values are saved correctly to firebase. The food items are saved and retrieved as objects, meaning all variables are correctly grouped together and methods can be used to extract data later. Testing codes (4.01-4.08) The criteria for this section were fully met providing users with a fully functioning object creation system.
#7 Meal Creation System	Fully Achieved.	The objective was fully met. The system allows the user to select individual food items along with a quantity and group them together with a name. Once again, all meal items are saved and

		retrieved as objects for the same reason as the object creation system. Testing codes (5.1-5.16) The criteria for this section were also fully met with a functioning creation system that allows the users to group together individual food items to add to their meals later. Feedback talked about some quality of life features such as being able to group together meals within the meal list for better organisation, as well as the ability to favourite certain meals to appear at the top. These improvements could be made by adding a "Favourite" button to the meal creation and viewing interfaces, then when the meals are being loaded into the recycler view, the items with the favourite tag are displayed first, then not displayed later. If the users wanted to group a meal they'd like to save as a template, they would tap a button, and the meal would be stored in a dedicated "Templates" section. This section would be accessible from the main meals screen via a tab or button. This would save time for users who regularly prepare similar meals and reduce the need to repeatedly add the same food items.
#8 Calorie Tracking system	Partially achieved.	The system performs what it is required to do but the implementation differs from the criteria. Sections of the day have been split apart instead of storing everything in one large section like in the criteria. There are no individual buttons that allow the user to add, edit or remove values, those buttons have been abstracted into the individual items themselves. All goals are displayed according to calculations performed and are accurate and the values reset each day. Each section can only store one food item as saving a second item overwrites the first which could be an inconvenience. The user feedback was generally positive for this section talking about how the progress bars felt very dynamic and grouping together different meals for a specific time is very useful. However the main feedback was to do with customizability, where some people may not eat breakfast, etc and would want the ability to customize the sections instead of relying on

		predefined ones, this could be implemented through an edit feature where the user can input a new string value through the already created change detail dialog. Another section of feedback was to do with the overwriting of items, an improvement that could be made regarding this is to check if the item is already added, if so then the total quantity is increased with the new item, instead of letting it override in firebase.
#9 Notification System	Not Achieved.	Implementing notifications was not able to happen due to the complexity of the application, the notification system would require integration with Android's background processing and scheduling components, meaning the feature would require extensive knowledge that would be difficult to gather during the time frame available. The current application architecture focuses heavily on immediate user interactions through fragments and dialogs but lacks the background service infrastructure needed to trigger notifications at specific times regardless of app state. Implementing this would require creating dedicated worker classes, handling permission requests for notifications on newer Android versions, and establishing persistent alarm scheduling that survives device reboots. Additionally, my firebase implementation currently centres on the real-time database functions for storing user data but would need expansion to include Firebase Cloud Messaging or a custom notification manager system to deliver timely alerts. This represents a significant architectural addition that would require careful integration to maintain the existing user experience flow, so this feature is simply not possible to be implemented at this time. Stakeholders spoke about how implementing a notification system would have enhanced the user experience by providing timely reminders for meal logging and nutritional goal tracking, therefore they have acknowledged this as an acceptable omission in the current version. The feature represents a specialized functionality that, while beneficial, isn't critical to the core value proposition of nutritional tracking and meal planning.

		The stakeholders have indicated that the notification functionality could be considered for a future update once the core features have been thoroughly validated with users. This phased approach allows for gathering user feedback on the existing functionality before investing resources in more technically complex enhancements.
#10 – Design	Fully achieved.	The colour scheme of nature is consistent throughout the app by contributing IDs to every UI element in both xml and the logic where colours were changed, this means that the value of the ID can be changed in one place to change the colour of every element under the value making sure that the colours will always stay consistent. Company logos are visible both upon opening the app and inside the app where convenient and the name is constantly reminded to the user.
#11 – Ability to edit information	Partially Achieved.	Most elements that require the ability to be changed can be changed through either a dialog screen or through a fragment. 1. Personal information and daily goals are able to be edited through a dedicated fragment on the side menu (Testing code 2.13, 2.21) 2. User can edit the individual values of their food items using a dialog screen in an already created food item (Testing codes 4.06-4.08) 3. User can edit which item, and the quantity of each item in an already created meal (Testing codes 5.15-5.17) 4. User is able to edit or remove the items they have added to the calorie tracking module for the day. (Testing codes 8.10) 5. User is able to edit the quantity of any food items added to the daily calories (Testing codes 8.09) Unfortunately to save development time as well as memory there are no additional functions to change certain factors such as the name the user enters at the start, as well as their email which could become an inconvenience if the user changes their email and does not want to lose any previously created data. Stakeholders spoke about how the ability to edit information is an extremely useful feature

		across the entire program. However, there are a few sections that could improve the quality of the program. For example, the ability to edit a user email was also brought up by the stakeholders, as being able to store account details across emails could be a very useful feature should an email be compromised, etc. This could be improved by adding an edit button to the email within the user's profile section that reuses the forgot password dialog and refactors it, to link to the firebases email change function instead. However other sections, such adding the ability for the user to change their name the stakeholders found to be a great omission, since the one feature would require its own separate functions that, should not be needed as a legal name have a very small chance of changing wasting time and resources.
#12 – Ability to search information	Fully achieved.	Any sections of the app that require the user to enter a query into a search bar comes back with the corresponding information and does not cause any errors when the user enters a query for non-existing information. The search functionality integrates with Firebase to ensure efficient performance even with larger datasets. Custom filter methods work effectively with complex data structures including meals, food items, an O(n) linear time complexity means that the time to search will increase with the number of items the user has created, however it is unlikely for the user to create even close to 1000 items, therefore an O(n) complexity is acceptable. The adapter-based approach refreshes results in real-time, maintaining a responsive interface throughout the search process. Stakeholders spoke about how the search functionality implementation successfully achieves quick and responsive filtering across various sections of the application. User feedback suggests potential improvements could include adding advanced filtering options beyond simple text searches. For example, users have expressed interest in filtering food items by nutritional ranges (such as "foods with less than 200 calories" or "high protein meals"). Additionally, some users have requested search history functionality to quickly access previously searched terms, which would further streamline

		the experience for those who frequently search for the same items.

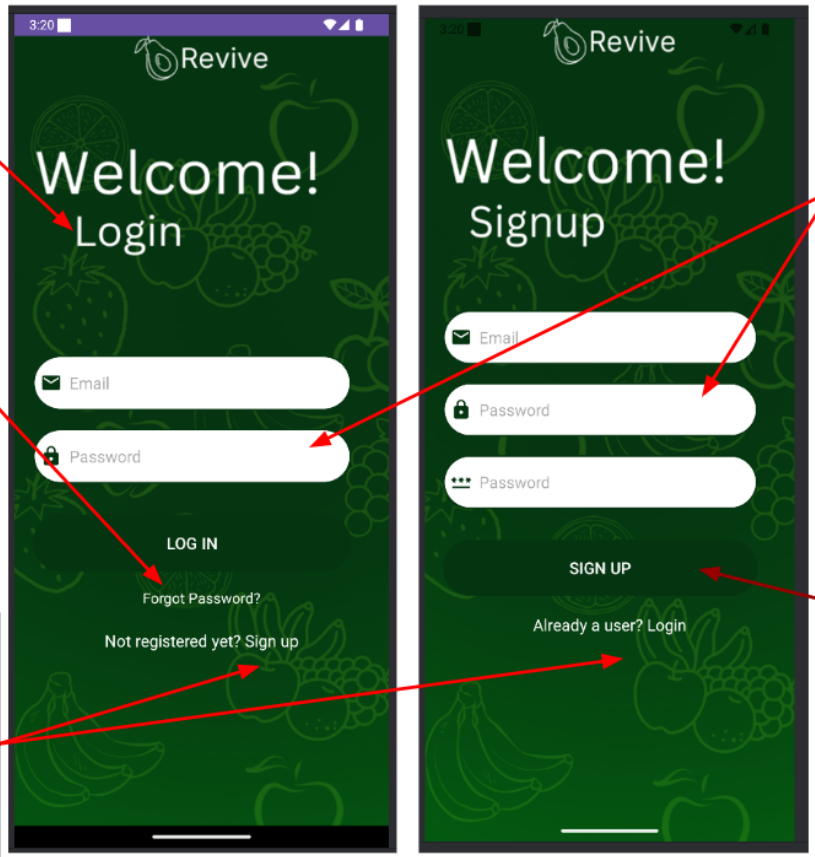
Usability Features

Registration Feature

Stakeholders also enjoyed the appearance of the screen, labelling it as “professional” and “elegant”. This means the apps appearance is on par with its existing competition.

The forgot password functionality is another successful usability feature, it introduced the use of dialogs throughout the app and links directly to firebase API so that the user can securely change their password if they were to forget it. Most users should not forget this as it is an important detail, however for user convenience this feature contributes to the overall impression of the app.

Successful redirection functionality meaning the user can swap between the login and sign up screen seamlessly. This means the two screens can be split apart from one another, and there is no need for dialogs, etc to be placed over the top to create an account or sign in, simplifying the process for the user

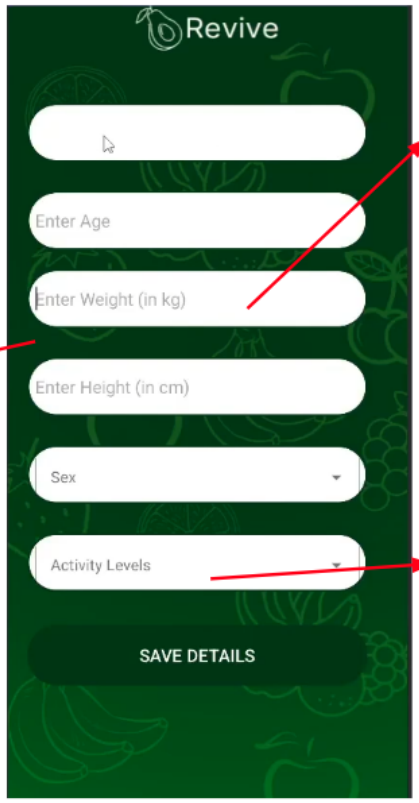


Successfully implemented a password input, the reason this is successful is because the password is hidden as it is entered, meaning if there is another person watching over the user they cannot see their password as it is being entered, protecting their account. Stakeholders say that they would like a button to be able to view the text that they have entered when needed.

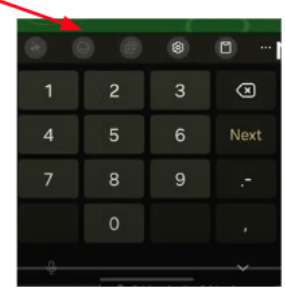
Sign up functionality is a successful usability feature as it provides users of an incredibly simple way of creating an account, without needing to enter many details, only taking the key specific ones. The simplicity caters to a range of users as almost everyone should have an email and a password for accounts when they are old enough, but not everyone may use other details, such as having their own username.

Input of Personal Information

The background is partially successful as it is consistent with the current login feature overall leading to a professional look, however it is greatly reduced as when the user presses an input box to enter a value, the background tries to fit the new screen with the keyboard instead of staying the same and the background appears squashed and unpleasant to look at. This could be changed by setting the background to a specific value or a specific size to match the parent of the main screen instead of dynamic, however this may cause the background to appear too small or large on other devices.



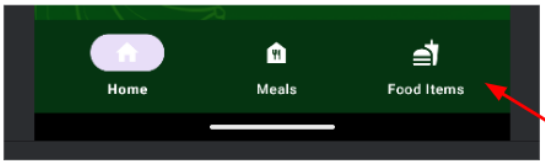
The input of personal details after a first time login of the user is a partially successful usability feature. It introduced the use of specific keyboards where the user is only allowed to enter a certain type of value, ensuring robustness throughout the app. However this feature could be improved by allowing the user to enter decimal value so that the accuracy of the calculations performed due to this screen are of a higher accuracy. This could be implemented by changing the input types of the objects, and changing the rounding features to accommodate float or double values.

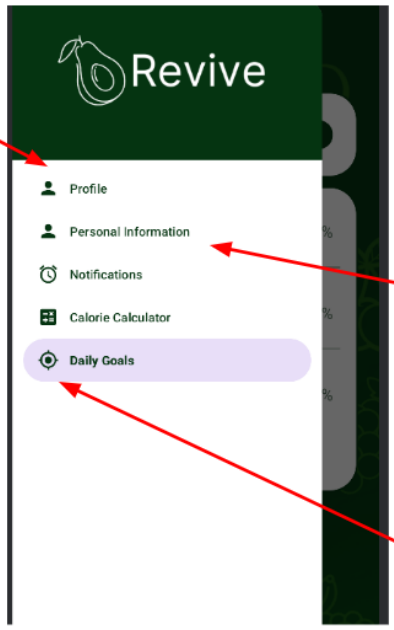


The dropdown input is another successful feature as it further adds to the robustness of the app, meaning there can be specific conditional statements that react and perform specific actions due to these choices, having a string input here instead would leave room for too much variation which could cause issues throughout the app later, and having to guess the correct inputs may reduce the user experience.

Navigation Module

Stakeholders talked about including an additional usability feature above each entry so that when the number of feature start to increase if the app were to be further developed in the future, the user would simply be able to search for the feature they need in the correct location. Future development could implement this by adding a dedicated search bar above each functional section of the app, allowing users to filter and find specific features within their contextual areas. This localized search approach would help maintain the app's intuitive organization while ensuring users can quickly access the exact tools they need as the feature set expands, without having to navigate through multiple menus or screens.





The bottom navigation bar provides persistent access to core functions including home, meals, and food items with clear iconography and labels. This implementation succeeds as a usability feature by following mobile design conventions that place primary navigation within thumb reach, allowing users to switch between major sections with minimal effort. For users, this means efficient movement between key app functions without having to return to a central menu or use back buttons repeatedly, reducing navigation fatigue during daily app usage.

The side drawer navigation presents additional personalized features like profile, personal information, and calorie calculator in an organized menu that can be accessed with a simple swipe or tap. This pattern enhances usability by implementing the principle of progressive disclosure, keeping secondary functions accessible but not cluttering the main interface. Users benefit by having a complete overview of all available app sections while maintaining a clean, focused main screen that prioritizes daily tracking activities.

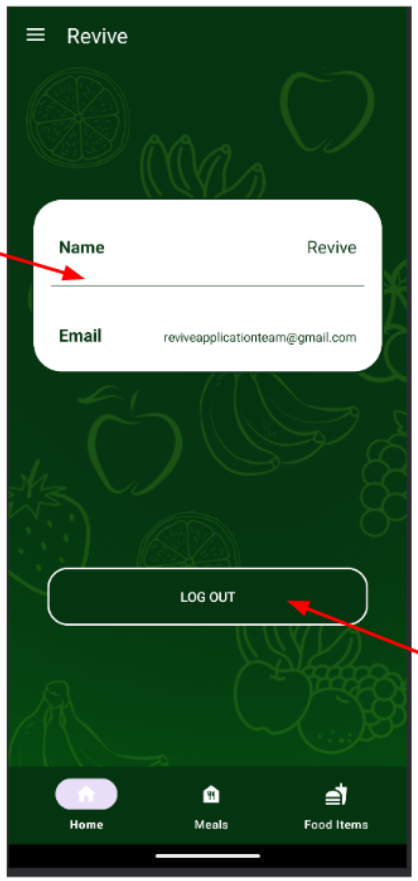
The addition of images for both the bottom and side navigation provides a visualisation of each feature in the app. Making it easier for users to understand their significance and purpose. The visual aid supplements the text description and allows users to quickly grasp the concept without relying solely on the writing.

The smooth transitions between fragments maintain context while updating content, providing visual continuity during navigation. This feature enhances usability by reducing cognitive load during screen changes and helping users understand their location within the app's structure. Users experience a more intuitive, flowing interaction that makes the application feel responsive and well-designed, increasing overall satisfaction and reducing the frustration associated with abrupt or disorienting screen changes.

Personal Details Module

Profile Screen

The profile section prominently displays the user's name and email in a clean, readable format with clear visual separation between fields. This immediate identification feature successfully addresses usability principles by providing instant account verification without requiring additional actions from the user. The high-contrast text ensures good readability, allowing users to confirm they're logged into the correct account even in varied lighting conditions.

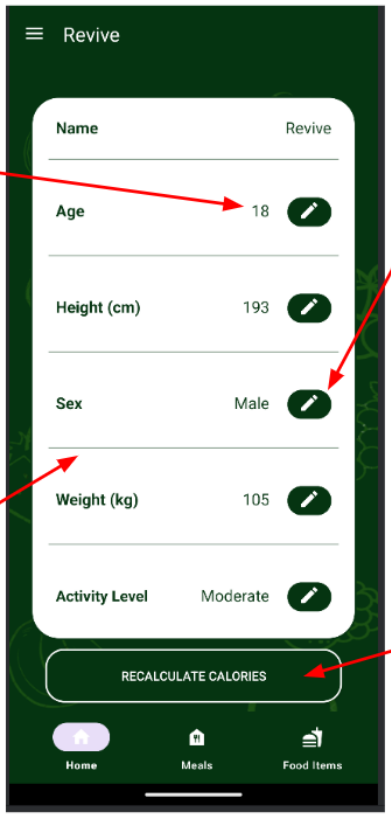


The logout button is strategically positioned at the bottom of the profile screen with high visibility and ample touch area. This placement succeeds as a usability feature by following the principle of expected locations—users typically look for logout options at the bottom of profile pages. The distinct styling and prominent size reduce cognitive load by making the function immediately recognizable, preventing accidental logouts while ensuring the feature remains accessible when needed. Furthermore, providing users with autonomy to log out of their account according to their will and swap to a different one when needed.

Personal Information Screen

The personal information fragment effectively displays user data in a structured, easy-to-scan format with distinct sections for each category of information. This implementation succeeds as a usability feature by organizing related data points with consistent spacing and clear labels, allowing users to quickly locate specific information without excessive scrolling or searching. For users, this means effortlessly checking their recorded health metrics and personal details whenever they need to verify or reference this information.

The subtle dividing lines between information categories create visual separation without adding clutter to the interface. This design element enhances usability by implementing the principle of progressive disclosure, helping users mentally group related information while maintaining a clean, uncluttered presentation. For users, this improves readability and focus, making it easier to distinguish between different types of personal data and reducing the mental effort required to process the information.



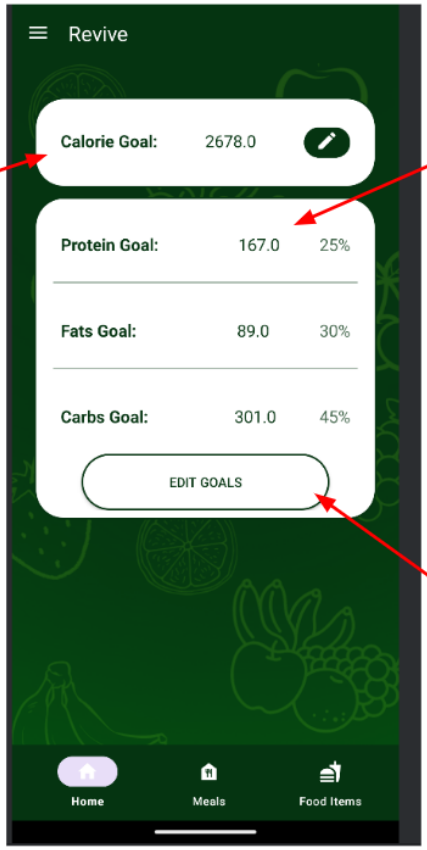
The small edit buttons positioned beside each information field provide immediate access to modification options right where users need them. Users benefit by being able to make quick, targeted changes to individual fields without navigating through multiple screens or forms, saving time and reducing frustration. However while well-positioned, stakeholders said the buttons have a limited touch target size, where those with larger fingers may experience frustration when attempting to make quick edits, potentially needing multiple attempts to select this feature. This could be improved by increasing the touch target size while maintaining the current visual design, possibly by extending the invisible touch area beyond the visible button boundaries.

The subtle dividing lines between information categories create visual separation without adding clutter to the interface. This design element enhances usability by implementing the principle of progressive disclosure, helping users mentally group related information while maintaining a clean, uncluttered presentation. For users, this improves readability and focus, making it easier to distinguish between different types of personal data and reducing the mental effort required to process the information.

Daily Goals Feature

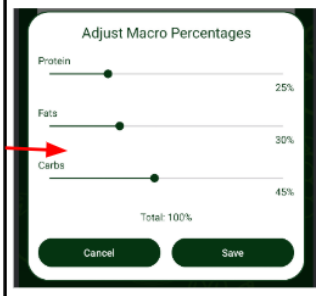
The daily goals fragment presents the calorie target in a dedicated section with clear, high-contrast text that immediately captures user attention. This implementation succeeds as a usability feature by making the most critical nutritional goal highly visible, allowing users to quickly reference their daily calorie target without searching through menus or screens. For users, this means having constant, easy access to their primary nutritional benchmark, helping them make informed food choices throughout the day.

The screen maintains the same card-based layout, color scheme, and typography as other sections of the application. This visual consistency enhances usability by reducing learning curves and creating a seamless experience across the app. Users benefit from familiar interaction patterns that build confidence and efficiency, allowing them to focus on their nutritional goals rather than figuring out how to use the interface.



The protein, fats, and carbs goals are displayed with both gram values and percentage allocations, providing a comprehensive view of nutritional targets. This dual representation enhances usability by catering to different user preferences and understanding levels, allowing both precise tracking and quick ratio assessment. Users benefit by gaining deeper insight into their nutritional balance, enabling them to make more targeted adjustments to their diet beyond simple calorie counting.

The prominently positioned "Edit Goals" button provides direct access to adjustment controls for macronutrient percentages. This feature succeeds by following the principle of discoverable controls, making it obvious how users can personalize their nutritional targets. For users, this means being able to easily adapt their nutritional goals to different dietary approaches (such as high-protein or low-carb) without navigating complex menu hierarchies, supporting various fitness and health objectives.

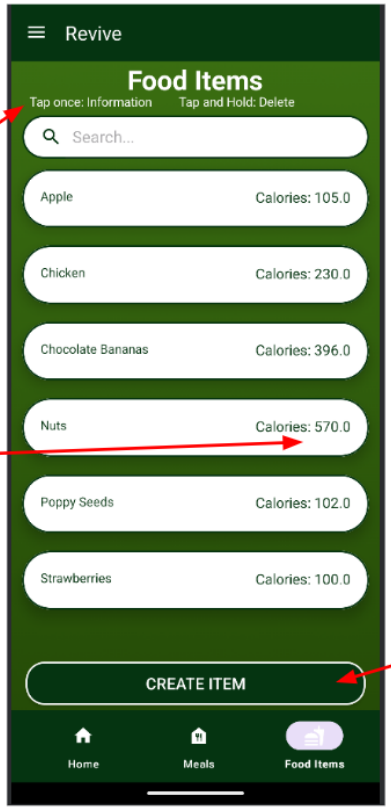


Object Creation Module

Individual Food items screen

The food item creation interface features a prominently positioned name field with clear labeling and ample input space at the top of the form. This design succeeds as a usability feature by establishing a logical information hierarchy that starts with the most basic identifying information. For users, this means a natural, intuitive flow that matches their mental model of creating a food item, starting with naming it before adding nutritional details.

The interface clearly indicates that nutritional values should be entered per 100g of the food item, establishing a consistent standard for all entries. This approach succeeds as a usability feature by eliminating confusion about serving sizes and creating a normalized database of food items. For users, this means more accurate meal calculations later when adding these items to their daily intake, as portions can be easily scaled from this standard measure.



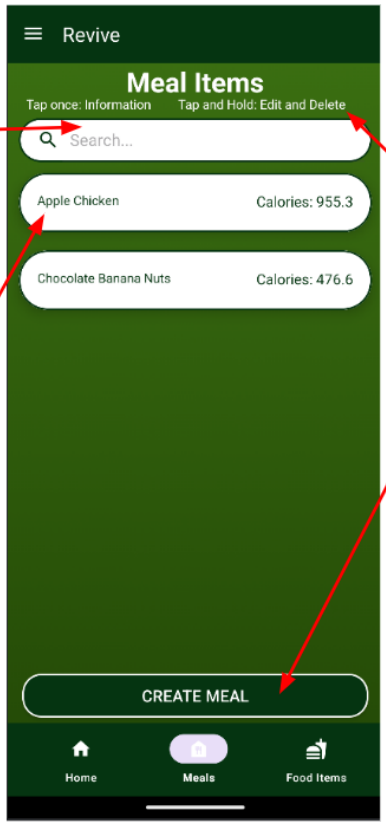
The nutritional input fields for calories, protein, fats, and carbs are presented in a consistent, vertically aligned format with appropriate input constraints for numerical values. This implementation enhances usability by creating a predictable pattern that allows users to quickly enter multiple values in sequence. Users benefit from clear visual cues about which nutritional components need to be entered, reducing errors and ensuring comprehensive nutritional profiles for each food item.

Meal Items screen

The search bar positioned at the top of the meal items screen offers immediate filtering of the meal library as users type. This feature enhances usability by implementing real-time response to search queries, eliminating the need for separate search submission actions. Users benefit by rapidly locating specific meals in expanding libraries, reducing frustration and supporting efficient meal planning even as their collection of saved meals grows.

The implementation of different actions for tap versus long-press interactions provides access to both viewing and editing functions without cluttering the interface. This approach succeeds by offering contextual operations appropriate to the user's intent, following touch interaction best practices. For users, this creates an efficient workflow where they can quickly view meal details with a tap or access edit/delete options with a long-press, adapting to their current needs.

Stakeholders talked about additional usability features in the future, regarding a possibility to set “favourites” to meals so that they appear towards the top, or to introduce meal categorization by type. This would most likely be implemented by tags to the meals and allow users to quickly locate corresponding meals when they need them. Implementing categorization, recently used items, and alphabetical jump lists would make navigation more efficient as users' food libraries grow over time.



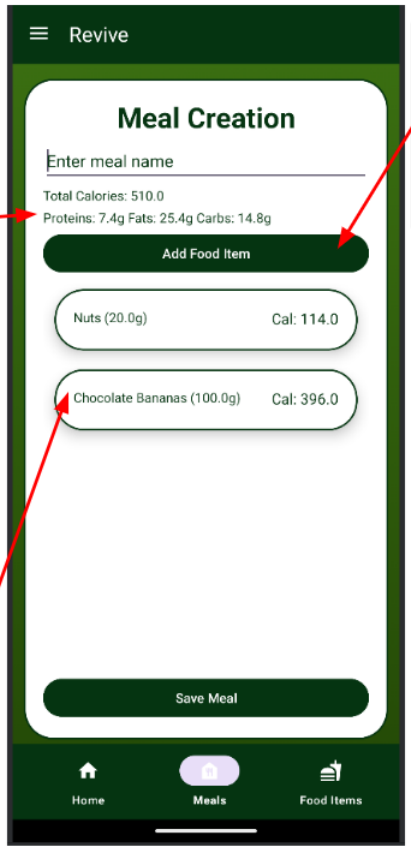
The text explaining "Tap once: Information" and "Tap and Hold: Edit and Delete" at the top of the screen provides necessary guidance but consumes valuable screen space permanently. This implementation is partially successful because while it helps new users understand interaction patterns, it remains visible for experienced users who no longer need the instruction, reducing the space available for actual meal items. For users, this means slightly more scrolling required to view their meal library, potentially creating friction during frequent use of the application.

The implementation of the create meal button, being identical to the create item button, succeeds as a usability feature by leveraging spatial consistency and visual recognition principles, reducing the learning curve by having similar functions appear in similar locations with matching design language. For users, this means being able to intuitively find creation functions across different sections of the app without having to search or learn new interaction patterns, allowing them to efficiently add content regardless of whether they're working with meals or individual food items.

Meal creation screen

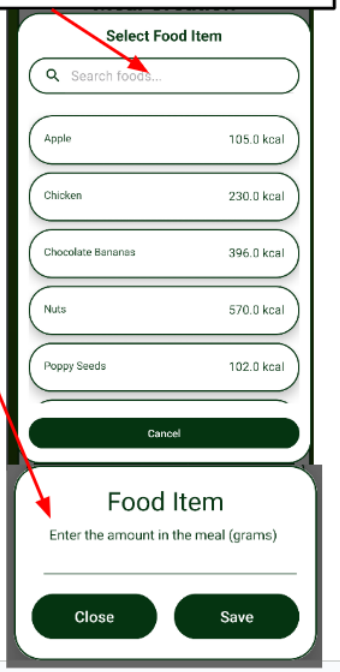
The interface displays automatically updating nutritional totals as users add food items to the meal, showing calorie counts and macronutrient breakdowns. This feature enhances usability by providing immediate feedback on the nutritional profile of the meal as it's being built. Users benefit from seeing how each added ingredient affects the overall nutritional composition of their meal, enabling informed decisions about portion sizes and ingredient combinations.

The scrollable list of selected food items presents clear information about each ingredient including name and portion size with calorie content. This design succeeds by giving users a comprehensive overview of the meal composition with the ability to review and modify components. For users, this means having confidence that their meal has been properly constructed before saving it, reducing the need for later edits and ensuring accuracy in their nutritional tracking.



The create meal fragment features a dedicated "Add Food Item" button that launches a searchable selection dialog for adding ingredients to the meal. This implementation succeeds as a usability feature by providing a streamlined process for building meals from existing food items with minimal navigation steps. For users, this means quickly assembling meals without navigating away from the creation page, streamlining the entire meal planning process.

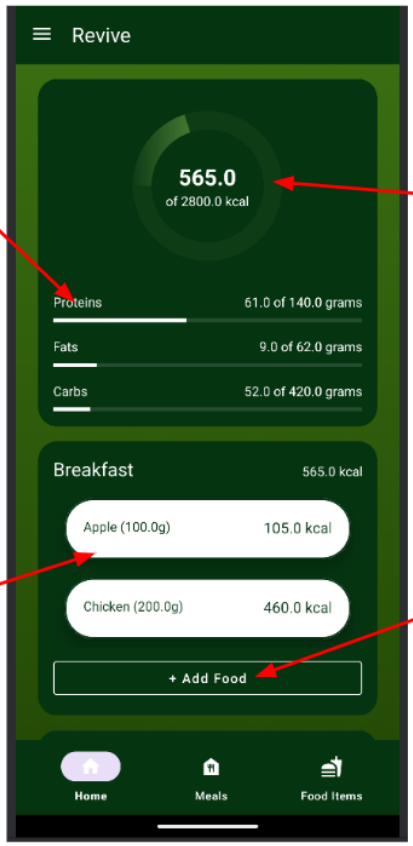
The gram quantity input dialog lets users enter portion sizes but doesn't offer suggested servings or common measurements. This is partially successful because while it allows precise measurements, users must determine appropriate portions without guidance. For users, this creates extra work when adding foods with unfamiliar serving sizes, slightly complicating an otherwise straightforward meal creation process. This limitation could be addressed in future development by adding common serving size suggestions and visual portion guides to help users quickly select appropriate quantities without requiring nutritional expertise.



Calorie Tracking Module

The horizontal progress bars for protein, fats, and carbs constitute a usability feature that provides standardized visual comparison of different nutritional metrics. Their success comes from using a familiar progress indicator pattern that's easy to interpret while keeping numerical details available for users who want precision. This helps users balance their macronutrients without performing mental math, enabling them to quickly identify which nutrients they need more or less of to meet their dietary goals.

The distinct meal sections create a usability feature that provides clear information in helping users mentally categorize their food intake across the day. This approach succeeds by decomposing the complex task of tracking daily nutrition, making the data entry process less overwhelming. Users benefit from being able to identify patterns in their eating habits (like which meal contributes most to their calorie intake) and experience an organizational structure that mirrors how they naturally think about their eating schedule. However this is also a partially successful usability feature as the stakeholders talk about added personalisation, where a person may need more than 3 separate sections, or how they may not eat their meals in the same way the app shows. Future development could address this limitation by implementing customizable meal categories, allowing users to add, rename, or remove meal sections to better reflect their individual eating patterns, cultural practices, or specific dietary regimens, thereby creating a more inclusive and personalized tracking experience.



The circular progress indicator is an effective usability feature because it transforms abstract calorie data into visual representation that requires minimal effort to understand. It's successful because it allows users to perceive their progress toward daily goals in milliseconds without mental calculation. For users, this visual feedback creates immediate awareness of their nutritional status and can boost motivation as they see the circle fill throughout the day, helping them make more informed decisions about their remaining meals.

The prominently placed "+ Add Food" buttons create a usability feature that offers clear visual affordance for the primary task of adding food items. This approach succeeds by implementing the design principle of making common actions visible and accessible, reducing the steps needed to complete the most frequent task in a calorie tracking app. For users, this streamlines the food logging process and increases the likelihood that they'll consistently track their meals—essential for both accurate data collection and habit formation. This loads a familiar screen, identical to the meal creation screen so the user will easily understand what they need to do.

Stakeholder Review

1. *What are your thoughts on the registration of an account with the application, login in and entering of personal details?*

Stakeholder 1: I like that the whole registration process adheres to conventional methods, it is exactly what I am used to when creating a new account for an application, without immediately asking for unnecessary extra details such as date of birth, etc that end up just being clutter and make the process take longer. However, I know that a lot of my friends do not like creating many different accounts, as many find it tedious to have to save every one of their details and remember them, so maybe a feature that integrates google, or any of the other common sign in methods that people use could be used, and that way you could remove some of the inputs such as the name, as it is already saved in google.

Stakeholder 2: I like the simplicity of creating an account, simply needing an email and password instead of having to think of new usernames, having yours potentially taken and needing new twists on it. I also like the confirmation of the password, as even when creating the account, I almost got saved my password wrong due to a typo and the only reason I realised is because the passwords did not match. However, I would like the ability to add a button for the password to be shown to me so that if I do get them wrong, I know which section I need to retype, instead of removing the entire password and starting again. For the personal details I feel the app should have some sort of limitation on what can be entered, because entering an age value well above 200 would cause errors in the calculations where details are calculated incorrectly, I know a serious user would not do this however this means accounts could be created with the sole purpose to waste memory space in the database.

EVALUATION: The registration process is praised for its simplicity and adherence to conventional methods, eliminating unnecessary fields that might complicate the user experience. Stakeholder 1 appreciated the streamlined approach but suggested integrating third-party authentication options like Google sign-in to further reduce friction and data entry requirements. Stakeholder 2 specifically valued the password confirmation feature that prevented errors during account creation, highlighting its effectiveness as a safeguard against typos. However, they identified the need for a password visibility toggle to improve user experience when correcting mistakes. Overall, the registration system and personal details input successfully balances simplicity with security features.

2. *How easy did you find it to navigate throughout the app?*

Stakeholder 1: I found the navigation experience to be very intuitive. The combination of bottom navigation for primary functions and the side drawer for additional features creates a clean, uncluttered interface that makes it easy to move between different sections of the app. One thing I'd suggest for future development would be implementing a search functionality within the side

navigation drawer. As you continue to add features and the list of menu items grows, this would help users quickly find specific settings or features without scrolling through a lengthy menu. I understand you've considered this and could implement it using the same filtering technique you've used for food items.

Stakeholder 2: The navigation structure is one of the strongest aspects of this app. The bottom navigation provides quick access to the core features I use daily, while the drawer menu keeps secondary functions accessible but out of the way. This approach prevents the interface from feeling overwhelming while still maintaining good access to all features. I appreciate how you've designed the navigation to be modular. This forward-thinking approach means you can easily add new features without disrupting the existing navigation patterns that users have grown accustomed to. The transitions between screens are smooth and predictable, which helps maintain my sense of location within the app.

Evaluation: Stakeholders found the navigation system to be a significant strength of the app. The dual approach of bottom navigation for core features and side drawer for secondary functions creates an innovative, uncluttered interface that facilitates seamless movement between sections. The modular design successfully supports future expansion while maintaining efficiency. The only suggested improvement was implementing a search functionality within the side navigation to help users quickly locate specific features as the app continues to grow, which could be implemented using the same filtering technique already used.

3. *Did you find any issues with changing any information throughout the app?*

Stakeholder 1: I found the process of changing personal information to be straightforward but with room for improvement. It provides good organization with clear edit buttons next to each field, making it obvious how to modify details. The input validation works well when changing numerical values like age, height, and weight. Even so, I encountered two issues: First, I noticed that once I've set my name during initial registration, I can't modify it later, this isn't communicated clearly to users as the hint can easily be ignored or go unseen. I can understand why the app has been created this way, adding the feature to change a string would have required entirely new functions and dialogs to exist, wasting more memory and therefore I am happy to omit this feature. Second, the dialog box for editing values doesn't adapt its input type based on what I'm editing, so when changing weight or height, there's no indication that only integer values are accepted until after submission.

Stakeholder 2: The personal information editing functionality is well-implemented but could benefit from additional polish. I appreciate how the app separates personal information editing from the profile screen, allowing for more detailed modification options. The ability to change sex and activity level through dropdown menus is intuitive, and the recalculate calories button at the bottom provides a clear action after making changes. One issue I found is that there's minimal feedback after making changes - a simple confirmation message appears, but there's no visual indication showing how the changes affected my calorie calculations or nutritional targets.

Evaluation: Stakeholders found the personal details modification system to be functional but with several usability issues. The clear layout with dedicated edit buttons next to each field creates a great editing experience. However, the inability to edit the name field after initial registration isn't properly communicated to users. The input validation for numerical fields works correctly, but the dialog boxes don't adapt their input types appropriately, potentially causing confusion. Additionally, there's limited feedback about how changes affect calculations, making it difficult for users to understand the impact of their modifications. Implementing contextual input types, clearer communication about unchangeable fields, and better visual feedback would significantly improve the user experience.

4. *Did you find any issues with the calculations throughout the app?*

Stakeholder 1: I found the calorie and macro calculations to be generally reliable, though I did notice some inconsistencies. The BMR formula (Harris-Benedict equation) is correctly implemented. Though, I noticed potential precision issues with how the app handles rounding in different places. For example, the rounding function is used inconsistently across different adapters and fragments, sometimes rounding to 1 decimal place and other times to 2. This creates small discrepancies when viewing the same nutritional values in different screens. Additionally, when adjusting macro percentages, the total occasionally doesn't add up to exactly 100% due to rounding errors, which can be confusing.

Stakeholder 2: The calculations throughout the app work well and provide great functionality and I appreciate how the calculations are handled quickly and efficiently for the user. I have no issues with the way calculations are performed however a good way to make sure the app is robust would be to have limits on values all round the app, such that a user cannot purposely try to crash the app by entering absurd amounts, such as 10000g of an item which is just not true.

Evaluation: Stakeholders found the core calculation functions to be implemented correctly, but identified issues that could affect precision and user experience. Inconsistent decimal rounding across different parts of the app creates discrepancies when viewing the same nutritional values in different screens.

5. *What are your thoughts about the unfortunate lack of tutorial features throughout the app?*

Stakeholder 1: While the app lacks explicit tutorial features, I found the interface obvious enough that I didn't miss them during my testing. The app follows standard mobile design patterns with clear labels and visual cues that guide users through each section. For example, the meal section has helpful text explaining "Tap once: Information" and "Tap and hold: Edit and Delete" right at the top of the screen. That said, some areas could benefit from minimal guidance—particularly the home screen's nutritional tracking system, where the relationship between goals and current consumption might not be immediately obvious to new users unfamiliar with calorie tracking apps.

Stakeholder 2: The absence of tutorials doesn't significantly impact the user experience due to the straightforward design of the app. The navigation structure with bottom tabs for primary functions is

instantly recognizable, and the edit buttons consistently positioned next to modifiable fields create a predictable pattern throughout the app. I appreciate that the app doesn't overload me with tooltips and popups that I'd just click through anyway. However, for users new to fitness tracking, a simple optional sequence showing how daily food logging affects the calorie circle and macro bars on the home screen might help demonstrate the app's value more quickly.

Evaluation: Stakeholders generally found that the lack of tutorials isn't a significant issue due to the app's intuitive design and adherence to familiar mobile patterns. The straightforward interface with consistent visual cues and some descriptive labels allows users to navigate and understand core functions without explicit guidance. The app successfully balances simplicity with functionality, avoiding overwhelming users with complex features that would need complex tutorials.

6. *What are your thoughts on using objects for items instead of a database, and the individual item and meal creation segments overall?*

Stakeholder 1: I find the object-based approach for managing food items and meals to be practical and user-friendly. The creation process is easy; I can easily add my own custom foods and combine them into meals that reflect my eating habits. The search feature helps me quickly find items I've created before, which streamlines the daily logging process. However, as my collection of foods and meals grows, I'm starting to wish for better organization options. Adding a favourites system would be tremendously helpful for accessing my most used meals with fewer taps.

Stakeholder 2: The item and meal creation systems are straightforward and meet my basic needs well. I appreciate that I can build a personal database of foods I regularly consume rather than searching through a massive pre-populated database where most items aren't relevant to me. The app keeps things simple while still being functional. What would take this experience to the next level is better meal organization. Currently, all my meals are in one list, but having the ability to group meals by type, possibly by breakfast, lunch, or dinner like in the calorie tracking section would help tremendously with my planning. As *stakeholder 1* said, having a favourites section for my go-to meals would save me time during busy mornings when I'm trying to quickly log what I've eaten.

Evaluation: Stakeholders found the object-based approach for food and meal management to be appropriately simple and effective for the app's needs. The creation interfaces are praised for being straightforward, allowing users to build a personalized food database without unnecessary complexity. The search functionality provides support for finding items as the user's food library grows. But as users accumulate more items over time, they will desire better organization capabilities. Two key points were identified as valuable additions: a favourites system to quickly access frequently used meals, and meal categorization by type (breakfast, lunch, dinner) to better align with natural eating patterns. These improvements would enhance the user experience without sacrificing simplicity, building logically on the existing framework while addressing real user needs for better organization as their food libraries expand.

7. *How well does the app combine all the features together in the main calorie tracking module and how effective did you find this module?*

Stakeholder 1: I'm impressed with how the app brings different features together in the calorie tracking module. The visual progress indicators showing calorie consumption relative to my daily goal, combined with the breakdown of macronutrients, give me a clear picture of my nutritional intake at a glance. Adding foods across different meal categories is straightforward, and I appreciate being able to add both individual food items and complete meals. However, the preset meal sections (breakfast, snacks, dinner) feel somewhat restrictive. As someone who follows intermittent fasting with just two main meals, I'd benefit from customizable meal categories that match my eating pattern. Additionally, as I've built up my library of meals, I've accidentally overwritten existing meals with the same name, some safeguards against this would be helpful.

Stakeholder 2: The main tracking interface effectively combines the app's core features to deliver a valuable daily nutrition dashboard. I can easily see how my current consumption relates to my personal goals, and the detailed macro breakdown helps me balance my nutrition beyond just counting calories. The ability to reuse previously created meals saves significant time, especially for recurring meals. That said, the tracking becomes less efficient as I add more items throughout the day. With many entries, it becomes difficult to find specific items I've added, especially when I want to edit or remove something. A search function within each meal section would be incredibly useful. I'd also appreciate more flexibility in how meal periods are categorized—perhaps the ability to create custom meal categories would better accommodate different eating schedules and dietary approaches.

Evaluation: Stakeholders found the calorie tracking module generally effective at integrating the app's features into a cohesive experience. The visual presentation of nutritional data relative to personalized goals provides valuable and quick information, while the structured meal categories help organize daily intake. The integration of both individual food items and complete meals into the tracking system was particularly appreciated for its efficiency. Three main improvement areas were identified: First, the fixed meal categories (breakfast, snacks, dinner) don't accommodate diverse eating patterns, customizable meal sections would better serve users with different dietary approaches. Second, the lack of safeguards against accidentally overwriting existing meals with the same name creates risk of data loss. Third, as users add more items throughout the day, finding specific entries becomes challenging, implementing search functionality within meal sections would improve usability. These enhancements would maintain the effective core experience while adding flexibility and scalability to support diverse user needs.

8. *What are your thoughts on the unfortunate lack of the ability to set yourself notifications within the app?*

Stakeholder 1: I initially expected to see notification functionality within the app, but after using the app for a while, I don't find its absence to be a major drawback. The core purpose of nutritional tracking is well-served without notifications, and I've found that I naturally check my progress throughout the day. Nutrition tracking is something I'm actively engaged with rather than something

I need reminders for. That said, simple meal time reminders could be useful for those establishing new eating routines. I appreciate the development has seemed to have focused on perfecting the essential tracking features first rather than adding peripheral functionality that might go unused. This suggests a thoughtful approach to development that prioritizes core user needs.

Stakeholder 2: The lack of notification capabilities is noticeable but not critical to my experience with the app. The nutritional tracking fundamentals are solid, which matters most for my daily use. I can always use my phone's built-in alarm or reminder apps if I really need time-based alerts. What I find encouraging is that this would give users an opportunity to provide meaningful feedback. If notifications become a frequently requested feature, the developers can implement it with a better understanding of how users would utilize it, whether for meal timing, reminder to log entries, or progress celebrations. This approach likely prevents feature bloat and ensures that when notifications are added, they'll be designed around genuine user needs rather than assumptions.

Evaluation: Stakeholders generally viewed the absence of notification capabilities as a reasonable omission rather than a critical flaw. They recognized that notifications represent a specialized functionality that isn't essential to the app's core purpose of nutritional tracking. The focus on perfecting fundamental features before adding peripheral ones was seen as evidence of prioritizing development resources effectively. Stakeholders also appreciated that this approach allows for more user-informed implementation in future updates, where notification features can be designed based on actual feedback rather than assumptions. This creates an opportunity for users to contribute to the app's evolution, potentially increasing engagement and satisfaction. When notifications are eventually implemented, they'll likely better serve real user needs, whether for meal reminders, logging prompts, or goal achievement celebrations.

Maintenance

My code makes use of hardly any global variables, meaning as the scale of the program increases memory will not be wasted to run the program. This happens because I have decomposed my program into simple functions, where each fragment makes use of its own variables and classes increasing the modularity and potential reusability of code in the future. This means it is less challenging to isolate and debug issues as changes to one section should not have consequences elsewhere, unless calculations that impact the database are made. Modularity is also great for future development as new modules can simply be added to the program that have been separately tested to add to the solution.

My code is generally well documented, including comments and meaningful variable names which aid understanding and make it easier for others to be able to read and make changes to my code in the future if needed.

By mainly utilising class and class functions in my code, it was simpler to define and use reducing overall time spent getting a complex functionality working. Functions allowed me to break down complex tasks into smaller more manageable sections of code. Individual functions can be easily called from multiple parts of my code. Additionally, since functions are a fundamental programming

construct supported by majority of the programming languages it makes them portable to be applied across different platforms.

However, as the application grows, maintaining the database may be difficult as every user has their entry in the same place, while there are rules in place to make sure the users only have access to their own data, thousands of users relying on the same database may diminish the efficiency. Furthermore, there is currently no way of deleting daily meals, so as the users keep adding items to their daily meals, they will have an increased history in their database that will waste storage, as past dates are currently not a usable feature.

Overall, my solution shows a solid foundation with considerable potential for architectural improvements that could enhance its maintainability, testability, and scalability. If development were to continue, there are multiple ways of addressing these considerations to create a more robust and flexible solution.

Limitations and improvements of the solution

The application heavily relies on Firebase Realtime Database for storing and retrieving user data, including personal information, food items, meals, and daily consumption records. This creates a fundamental dependency on internet connectivity for all core app functions. When users are in areas with poor network coverage or experiencing connectivity issues, they may be unable to log meals, access their nutritional data, or track progress. This limitation is particularly problematic for an app focused on daily health tracking, as consistency is crucial for effective habit formation and monitoring. The application could be enhanced by implementing an offline mode utilizing Room Database or SQLite for local caching, allowing users to continue core functionality during connectivity gaps. There could be functions in the background that save the locally saved data to the firebase database when the user is online, or a periodic backup to the database every so often to keep information up to date.

Moreover, the application is built exclusively for Android using Kotlin, which restricts access to users of Android devices only. This excludes a significant portion of potential users who own iOS devices (approximately 27% of the global smartphone market) or prefer accessing health and nutrition services via web browsers. This platform limitation not only reduces the potential user base but also creates friction for users who switch between platforms or want to access their nutritional data from multiple devices. Future development could focus on rebuilding the application using cross-platform frameworks to support both Android and iOS from a single codebase. Developing a complementary progressive web app would make the service accessible on any device with a browser, significantly expanding reach. This multi-platform approach would not only expand the potential user base but also provide a seamless experience as users move between devices throughout their day.

Furthermore, users must manually input nutritional information for each food item they consume, including calories, proteins, fats, and carbohydrates. This process is time-consuming and prone to errors, potentially leading to inaccurate tracking. The tedious nature of manual data entry can significantly reduce user engagement over time, as it creates friction in the daily logging process, a

critical touchpoint for app retention. Additionally, users may input inconsistent nutritional values for the same foods, leading to unreliable trend analysis. The application could incorporate a nutritional database with thousands of common foods and their verified nutritional information. Other features could include a camera-based barcode scanning functionality which would allow instant import of nutritional information for packaged foods, drastically reducing manual entry. These improvements would significantly reduce the entry barrier for daily food logging, leading to more consistent use and more accurate nutritional data.

While the application tracks nutritional data, the visualization capabilities are relatively basic, primarily displaying simple progress bars and daily totals. This approach limits users' ability to gain meaningful insights from their data over time. Without visualization tools, users may struggle to identify patterns in their eating habits, understand the relationship between nutrition and their goals, or visualize their progress over weeks and months, all crucial factors for sustained engagement and behaviour change. The application could be enhanced with a diverse range of visualization options including a feature for the user to track their weight over time, using a line graph to view progress, other graphs to visualise how much food or how much progress on each day towards the targets has been achieved over a certain time period, this would not only be very beneficial for the user, but would also justify storing the dates where food items have been added and are not deleted from firebase. These could help in creating a personalized analytics experience that motivates continued engagement for the user.

While the application allows users to create and track individual meals, it lacks advanced meal planning capabilities that would help users strategically organize their nutrition over longer periods. The current approach focuses primarily on retrospective tracking rather than proactive planning. Without meal planning features, users may struggle to consistently meet their nutritional goals, prepare adequately for their dietary needs, or efficiently manage grocery shopping and food preparation, these are all critical components of successful long-term nutrition management. The application could be enhanced with a calendar interface for planning meals days or weeks in advance, automatically calculating nutritional totals for the planned period. An AI-driven suggestion system could recommend meals based on nutritional goals, previous preferences, and available ingredients. Recipe scaling functionality could allow users to easily adjust portions while maintaining accurate nutritional information. These planning-focused improvements would transform the app from a reactive tracking tool to a proactive meal management system that supports users throughout their entire nutritional journey.

A limitation that could be considered quite a large issue is that the interface is designed with specific dimensions and layouts that primarily target standard smartphone form factors. Many UI components use hard-coded dimensions (fixed dp values for heights, widths, and margins) rather than responsive design principles. This rigid approach to layout design fails to accommodate the diverse range of Android devices in the market today. Tablets with their larger screens will display the app with awkward spacing and poor use of available screen size. Foldable phones, which can transition between different screen configurations, may experience layout issues during fold/unfold transitions. On extremely small devices, elements might overlap or become inaccessible as the layout fails to compress. This limitation creates an inconsistent user experience across devices and potentially excludes users with non-standard screen sizes or configurations. The application's layout

system could be redesigned using responsive design principles that dynamically adapt to different screen sizes and orientations. Creating a flexible grid system would help maintain visual harmony across various screen sizes while ensuring that touch targets remain appropriately sized for all devices. Testing routines could be expanded to include a matrix of device types, including various tablets, foldable phones, and ultra-compact devices. This expansive testing approach would identify edge cases where the layout breaks down. These improvements would transform the application into a truly responsive experience that feels native across the entire spectrum of Android devices, from the smallest phones to the largest tablets and emerging form factors like foldables, ensuring that all users receive a consistent, high-quality experience regardless of their device choice.

Further improvements are also discussed in the comparison to the success criteria.

OCR PROGRAMMING PROJECT

Dominic Stachura

Candidate Number: 9418

Code Appendix

This appendix will contain all the **Logic** code for my programming project.

Login and Signup

```
class LoginActivity : AppCompatActivity() {
    private lateinit var binding: ActivityLoginBinding
    private lateinit var firebaseAuth: FirebaseAuth
    private lateinit var databaseReference: DatabaseReference

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Thread.sleep(3000)  // waits with the splashscreen for 2 seconds
        installSplashScreen()  // sets the splashscreen
        binding = ActivityLoginBinding.inflate(layoutInflater)
        setContentView(binding.root)
        firebaseAuth = FirebaseAuth.getInstance()   // sets variable for the firebase imported procedure

         if (firebaseAuth.getCurrentUser() != null) {
             val userid = FirebaseAuth.getInstance().currentUser?.uid
             if (userid != null){
                 databaseReference = FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.euro
                 databaseReference.get().addOnSuccessListener { snapshot ->
                     if (snapshot.exists()) {
                         // If data exists, skip the form and go to the next activity
                         // Navigate to the next activity
                         val intent = Intent(this, MainActivity::class.java)
                         startActivity(intent)

                     } else {
                         val intent = Intent(this, PersonalDetailsActivity::class.java)
                         startActivity(intent)  // moves to the next screen if password and email are correct

                     }
                 }
             }
         }

        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->
            val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
            insets
        }

        binding.loginButton.setOnClickListener{   // waits for the login button to be clicked
            val email = binding.loginEmail.text.toString()
            val password = binding.loginPassword.text.toString()

            if (email.isNotEmpty() && password.isNotEmpty()) {  // makes sure the two email and password fiel
                firebaseAuth.signInWithEmailAndPassword(email, password).addOnCompleteListener{
                    if (it.isSuccessful){
                        Log.d("LoginActivity", "Authentication successful")
                        val userid = FirebaseAuth.getInstance().currentUser?.uid
                        if (userid != null){
                            databaseReference = FirebaseDatabase.getInstance("https://revive-app-b8fd2-defaul
                            databaseReference.get().addOnSuccessListener { snapshot ->
                                Log.d("LoginActivity", "Database snapshot exists: ${snapshot.exists()}")
                                if (snapshot.child("personal information").exists()) {
                                    // If data exists, skip the form and go to the next activity
                                    // Navigate to the next activity

                                    val intent = Intent(this, MainActivity::class.java)
                                    startActivity(intent)
                                } else {
                                    val intent = Intent(this, PersonalDetailsActivity::class.java)
                                    startActivity(intent)  // moves to the next screen if password and email
                                }
                            }
                        }else{
                            Toast.makeText(this, "Authentication is Failed", Toast.LENGTH_SHORT).show()
                        }

                    } else {
                        Toast.makeText(this, "Authentication Failed", Toast.LENGTH_SHORT).show()  // returns
                    }
                }
            } else {
                Toast.makeText(this, "Fields cannot be empty", Toast.LENGTH_SHORT).show() // returns an error
            }
        }
```

```
        binding.forgotPassword.setOnClickListener{  // listens to see if forgot password text is clicked
            val builder = AlertDialog.Builder(this)
            val view = layoutInflater.inflate(R.layout.dialog_forgot, null) // calls dialog_forgot xml
            val userEmail = view.findViewById<EditText>(R.id.editBox)
            builder.setView(view)
            val dialog = builder.create()
            view.findViewById<Button>(R.id.btnReset).setOnClickListener {
                compareEmail(userEmail) // makes sure email is real
                dialog.dismiss()
            }
            view.findViewById<Button>(R.id.btnCancel).setOnClickListener {
                dialog.dismiss() // cancels operation
            }
            if (dialog.window != null){
                dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
            }
            dialog.show()
        }

        binding.signupRedirectText.setOnClickListener{  // listens for someone to click the "sign up now" tex
            val signupIntent = Intent(this, SignupActivity::class.java)
            startActivity(signupIntent)
        }
    }
    //Outside onCreate
    private fun compareEmail(email: EditText){
        if (email.text.toString().isEmpty()){
            Toast.makeText(this, "Please enter your email", Toast.LENGTH_SHORT).show()
            return
        }
        if (!Patterns.EMAIL_ADDRESS.matcher(email.text.toString()).matches()){
            Toast.makeText(this, "Please enter a valid email", Toast.LENGTH_SHORT).show()
            return
        }

        firebaseAuth.fetchSignInMethodsForEmail(email.text.toString())
            .addOnCompleteListener { task ->
                if (task.isSuccessful) {
                    val signInMethods = task.result?.signInMethods
                    if (signInMethods.isNullOrEmpty()) {
                        // Email doesn't exist in Firebase
                        Toast.makeText(this, "Email does not exist", Toast.LENGTH_SHORT).show()
                    } else {
                        // Email exists, send reset email
                        firebaseAuth.sendPasswordResetEmail(email.text.toString())
                            .addOnCompleteListener { resetTask ->
                                if (resetTask.isSuccessful) {
                                    Toast.makeText(this, "Check your email", Toast.LENGTH_SHORT).show()
                                } else {
                                    Toast.makeText(this, "Failed to send reset email", Toast.LENGTH_SHORT).sh
                                }
                            }
                    }
                } else {
                    Toast.makeText(this, "Error checking email", Toast.LENGTH_SHORT).show()
                }
            }
    }
}
```

```
class SignupActivity : AppCompatActivity() {

    // Declaring binding for accessing views and FirebaseAuth for user authentication
    private lateinit var binding: ActivitySignupBinding
    private lateinit var firebaseAuth: FirebaseAuth

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge() // Enables edge-to-edge UI support
        binding = ActivitySignupBinding.inflate(layoutInflater) // Initializing binding
        setContentView(binding.root) // Setting the content view to the binding's root layout

        firebaseAuth = FirebaseAuth.getInstance() // Initializing FirebaseAuth instance

        // Adjust padding for system bars (like status and navigation bars) dynamically
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->
            val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
            insets
        }

        // Listener for the "Sign Up" button
        binding.signupButton.setOnClickListener {
            val email = binding.signupEmail.text.toString() // Get email from input
            val password = binding.signupPassword.text.toString() // Get password from input
            val confirmPassword = binding.signupConfirm.text.toString() // Get confirm password from input

            // Check if all fields are filled
            if (email.isNotEmpty() && password.isNotEmpty() && confirmPassword.isNotEmpty()) {
                // Check if password and confirm password match
                if (password == confirmPassword) {
                    // Attempt to create a new user in Firebase
                    firebaseAuth.createUserWithEmailAndPassword(email, password).addOnCompleteListener {
                        if (it.isSuccessful) {
                            // Redirect to login activity on successful sign-up
                            val intent = Intent(this, LoginActivity::class.java)
                            startActivity(intent)
                        } else {
                            // Show error message if sign-up fails
                            Toast.makeText(this, "Incorrect Input", Toast.LENGTH_SHORT).show()
                        }
                    }
                } else {
                    // Show error if passwords do not match
                    Toast.makeText(this, "Password does not match", Toast.LENGTH_SHORT).show()
                }
            } else {
                // Show error if any field is empty
                Toast.makeText(this, "Fields cannot be empty", Toast.LENGTH_SHORT).show()
            }
        }

        // Listener for "Log In" text to redirect users to the login screen
        binding.loginRedirectText.setOnClickListener {
            val loginIntent = Intent(this, LoginActivity::class.java)
            startActivity(loginIntent)
        }
    }
}
```

Personal Info Module

```
class PersonalDetailsActivity : AppCompatActivity() {

    private lateinit var binding: ActivityPersonalDetailsBinding
    private lateinit var databaseReference: DatabaseReference

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityPersonalDetailsBinding.inflate(layoutInflater) // layout binding to be able to see b
        setContentView(binding.root)


        val sexes = resources.getStringArray(R.array.Sexes)  // sets up drop down for sexes option
        val sexadapter = ArrayAdapter(this,R.layout.dropdown_item,sexes)
        binding.sexInput.setAdapter(sexadapter)

        val activitylevels = resources.getStringArray(R.array.Activity_Levels)  // sets up drop down for activ
        val activitylevelsadapter = ArrayAdapter(this,R.layout.dropdown_item,activitylevels)
        binding.fitnessInput.setAdapter(activitylevelsadapter)


        binding.personalinfoSaveButton.setOnClickListener{  // waits until "SAVE DETAILS" is pressed
            val name = binding.nameInput.text.toString() // assigns values to variables
            val age = binding.ageInput.text.toString().toIntOrNull() ?: 0
            val height = binding.heightInput.text.toString().toIntOrNull() ?: 0
            val weight = binding.weightInput.text.toString().toIntOrNull() ?: 0
            val sex = binding.sexInput.text.toString()
            val activitylevel = binding.fitnessInput.text.toString()

            if (name.isEmpty() || age == 0 || height == 0 || weight == 0 || sex.isEmpty() || activitylevel.isE
                Toast.makeText(this, "Please fill in all fields", Toast.LENGTH_SHORT).show()
                return@setOnClickListener // makes sure fields arent empty
            }
            if(age < 12){
                Toast.makeText(this, "Must be at least 12 years old", Toast.LENGTH_SHORT).show()
                return@setOnClickListener
            }
            if(height >= 250){
                Toast.makeText(this, "Height cannot be above 250cm", Toast.LENGTH_SHORT).show()
                return@setOnClickListener
            }

            //get the current user id
            val userId = FirebaseAuth.getInstance().currentUser?.uid
            // makes sure current user id is not null
            if (userId != null){
                databaseReference = FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.europe

                val personalinformation = PersonalInfo(name,age,height,weight,sex,activitylevel,0.0,0.0, 0, 0,
                personalinformation.calculatecalories() // calculates the calories
                personalinformation.calculatemacros() // calculates macros
                databaseReference.child(userId).child("personal information").setValue(personalinformation).ad
                    binding.nameInput.text.clear()
                    binding.ageInput.text.clear() // clears values from variables as they are not needed anymo
                    binding.heightInput.text.clear()
                    binding.weightInput.text.clear()
                    binding.sexInput.text.clear()
                    binding.fitnessInput.text.clear()

                    Toast.makeText(this,"Saved",Toast.LENGTH_SHORT).show() // shows a success message
                    val intent = Intent(this, MainActivity::class.java)
                    startActivity(intent)
                    finish()
                }.addOnFailureListener{
                    Toast.makeText(this,"Failed",Toast.LENGTH_SHORT).show() // in case of failiure shows a mes
                }
            }else {
                // Handle the case if user is not authenticated (userId is null)
                Toast.makeText(this, "User not authenticated", Toast.LENGTH_SHORT).show()
            }
        }

    }
}
```

```
data class PersonalInfo(
    var name: String = "",
    var age: Int = 0,
    var height: Int = 0,
    var weight: Int = 0,
    var sex: String = "",
    var activityLevel: String = "",
    var calories: Double = 0.0,
    var deficitcalories: Double = 0.0,
    var protein: Int = 0,
    var fats: Int = 0,
    var carbs: Int = 0,
    var proteinpercent: Double = 0.3,
    var fatspercent: Double = 0.3,
    var carbspercent: Double = 0.4
){
    fun calculatecalories(){
        if (sex == "Male"){
            calories = 66.5 + (13.75 * weight) + (5.003 * height) - (6.775 * age)
                if (calories < 1200){
                    calories = 1200.0
                }
        }else {
            calories = 655.1 + (9.563* weight) + (1.85 * height) - (4.676 * age)
            if (calories < 1200){
                calories = 1200.0
            }
        }

        if (activityLevel == "Little"){
            calories = 1 * calories
        } else if (activityLevel == "Light"){
            calories = 1.2 * calories
        } else if (activityLevel == "Moderate"){
            calories = 1.35 * calories
        } else if (activityLevel == "Active"){
            calories = 1.5 * calories
        } else{
            calories = 1.7 * calories
        }

        // rounds
        calories = kotlin.math.round(calories)
        deficitcalories = kotlin.math.round(calories - 500)
    }

    fun calculatemacros(){
        if (calories != 0.0){
            protein = ((deficitcalories * proteinpercent) / 4).toInt()
            carbs = ((deficitcalories * carbspercent) / 4).toInt()
            fats = ((deficitcalories * fatspercent) / 9).toInt()
        }
    }

}
```

```
class PersonalInfoFragment : Fragment() {

    private var _binding: FragmentPersonalInfoBinding? = null
    private val binding get() = _binding!! // safe access to binding
    private lateinit var databaseReference: DatabaseReference
    private lateinit var firebaseAuth: FirebaseAuth

    private var name = ""
    private var age = 0
    private var height = 0
    private var weight = 0
    private var sex = ""
    private var alevels = ""

    private var calories = 0.0
    private var proteinpercent = 0.0
    private var fatspercent = 0.0
    private var carbspercent = 0.0


    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        // Inflate the layout for this fragment
        _binding = FragmentPersonalInfoBinding.inflate(inflater, container, false)
        val userid = FirebaseAuth.getInstance().currentUser?.uid

        if (userid != null) {
            databaseReference =
                FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.europe-west1.firebasedatabas
                    .getReference("Information").child(userid)
            databaseReference.get().addOnSuccessListener { snapshot ->
                if (snapshot.exists()) {
                    name = snapshot.child("personal information").child("name").getValue(String::class.java) ?:
                    age = snapshot.child("personal information").child("age").getValue(Int::class.java) ?:0
                    height = snapshot.child("personal information").child("height").getValue(Int::class.java) ?:
                    sex = snapshot.child("personal information").child("sex").getValue(String::class.java) ?: "
                    weight = snapshot.child("personal information").child("weight").getValue(Int::class.java) ?:
                    alevels = snapshot.child("personal information").child("activityLevel").getValue(String::cla

                    binding.readName.text = name.toString()
                    binding.readAge.text = age.toString()
                    binding.readHeight.text = height.toString()
                    binding.readSex.text = sex.toString()
                    binding.readWeight.text = weight.toString()
                    binding.readActivityLevels.text = alevels.toString()
                } else {
                    Toast.makeText(requireContext(), "No data found", Toast.LENGTH_SHORT).show()
                }
            }.addOnFailureListener {
                Toast.makeText(requireContext(), "Something went wrong", Toast.LENGTH_SHORT).show()
            }
        }

        //calls for buttons pressed so that the same dialog popup can be used with different calls depending on
        // is to be changed
        binding.btnEditAge.setOnClickListener{
            call_DetailChange("age")
        }

        binding.btnEditHeight.setOnClickListener{
            call_DetailChange("height")
        }
        binding.btnEditWeight.setOnClickListener{
            call_DetailChange("weight")
        }

        binding.btnEditSex.setOnClickListener{
            call_detailChange_dropdown("sex")
        }

        binding.btnEditActivityLevel.setOnClickListener(){
            call_detailChange_dropdown("activityLevel")
        }

```

```
108
109     private fun call_DetailChange(option: String) { // listens to see if forgot password text is clicked
110         val builder = AlertDialog.Builder(requireContext())
111         val view = layoutInflater.inflate(R.layout.dialog_changeinfo, null) // calls dialog_changeinfo xml
112         val changedetail = view.findViewById<EditText>(R.id.editBoxDetails)
113         builder.setView(view)
114         val dialog = builder.create()
115
116         fun validateInput(value: Int): Boolean {
117             return when (option) {
118                 "age" -> {
119                     if (value < 12) {
120                         Toast.makeText(requireContext(), "Age must be 12 or above", Toast.LENGTH_SHORT).show()
121                         false
122                     } else true
123                 }
124                 "height" -> {
125                     if (value > 250) {
126                         Toast.makeText(requireContext(), "Height must be below 250", Toast.LENGTH_SHORT).show()
127                         false
128                     } else true
129                 }
130                 "weight" -> {
131                     if (value <= 0) {
132                         Toast.makeText(requireContext(), "Weight must be greater than 0", Toast.LENGTH_SHORT).sh
133                         false
134                     } else true
135                 }
136                 else -> true
137             }
138         }
139
140         view.findViewById<Button>(R.id.btnChangeDetail).setOnClickListener {
141             val userid = FirebaseAuth.getInstance().currentUser?.uid
142
143             if (userid != null){
144                 var detailText = changedetail.text.toString()
145                 if (detailText.isNotEmpty()) {
146                     try {
147                         // Convert to integer
148                         val detailValue = detailText.toInt()
149
150                         // Update the database with the integer value
151                         if (validateInput(detailValue)) {
152                             // Update the database with the validated integer value
153                             databaseReference.child("personal information").child(option)
154                                 .setValue(detailValue)
155                                 .addOnSuccessListener {
156                                     updateUI(option, detailText)
157                                     updatevariableints(option, detailValue)
158                                     dialog.dismiss()
159                                 }.addOnFailureListener {
160                                     Toast.makeText(
161                                         requireContext(),
162                                         "Failed to save",
163                                         Toast.LENGTH_SHORT
164                                     ).show()
165                                 }
166                         }
167                     } catch (e: NumberFormatException) {
168                         // Handle invalid number input
169                         Toast.makeText(requireContext(), "Please enter a valid number", Toast.LENGTH_SHORT).sho
170                     }
171                 } else {
172                     Toast.makeText(requireContext(), "Field cannot be empty", Toast.LENGTH_SHORT).show()
173                 }
174             } else{
175                 Toast.makeText(requireContext(),"Failed",Toast.LENGTH_SHORT).show()
176                 dialog.dismiss()
177             }
178
179         }
180         view.findViewById<Button>(R.id.btnCancelDetails).setOnClickListener {
181             dialog.dismiss() // cancels operation
182         }
```

```
188
189     private fun call_detailChange_dropdown(option: String){
190         val builder = AlertDialog.Builder(requireContext())
191         val view = layoutInflater.inflate(R.layout.dialog_changeinfo_dropdown, null) // calls dialog_forgot xml
192         val changedetail = view.findViewById<AutoCompleteTextView>(R.id.Detail_input_dropdown)
193
194         if (option == "sex"){
195             val sexes = resources.getStringArray(R.array.Sexes)  // sets up drop down for sexes option
196             val sexadapter = ArrayAdapter(requireContext(),R.layout.dropdown_item,sexes)
197             changedetail.setAdapter(sexadapter)
198         }else{
199             val activitylevels = resources.getStringArray(R.array.Activity_Levels)  // sets up drop down for act
200             val activityleveladapter = ArrayAdapter(requireContext(),R.layout.dropdown_item,activitylevels)
201             changedetail.setAdapter(activityleveladapter)
202         }
203
204         builder.setView(view)
205
206
207         val dialog = builder.create()
208         view.findViewById<Button>(R.id.btnChangeDetailDropdown).setOnClickListener {
209             val userid = FirebaseAuth.getInstance().currentUser?.uid
210
211             if (userid != null){
212
213                 var detailText = changedetail.text.toString()
214                 if (detailText.isNotEmpty()) {
215                     try {
216
217                         // Update the database with the integer value
218                         databaseReference.child("personal information").child(option).setValue(detailText)
219                             .addOnSuccessListener {
220                                 updateUI(option,detailText)
221                                 updatevariablestrings(option, detailText)
222
223                             }.addOnFailureListener {
224                                 Toast.makeText(requireContext(), "Failed to save", Toast.LENGTH_SHORT).show()
225                             }
226                     } catch (e: NumberFormatException) {
227                         // Handle invalid number input
228                         Toast.makeText(requireContext(), "Please enter a valid number", Toast.LENGTH_SHORT).sho
229                     }
230                 } else {
231                     Toast.makeText(requireContext(), "Field cannot be empty", Toast.LENGTH_SHORT).show()
232                 }
233
234             }else {
235                 Toast.makeText(requireContext(), "Failed", Toast.LENGTH_SHORT).show()
236                 dialog.dismiss()
237             }
238
239             dialog.dismiss()
240         }
241         view.findViewById<Button>(R.id.btnCancelDetailDropdown).setOnClickListener {
242             dialog.dismiss() // cancels operation
243         }
244         if (dialog.window != null){
245             dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
246         }
247         dialog.show()
248     }
```

```
250     private fun recalculatecalories() {
251
252         val userid = FirebaseAuth.getInstance().currentUser?.uid
253
254         if (userid != null) {
255             databaseReference =
256                 FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.europe-west1.firebaseda
257                     .getReference("Information").child(userid)
258             databaseReference.get().addOnSuccessListener { snapshot ->
259                 proteinpercent = snapshot.child("personal information").child("proteinpercent")
260                     .getValue(Double::class.java) ?: 0.0
261                 fatspercent = snapshot.child("personal information").child("fatspercent")
262                     .getValue(Double::class.java) ?: 0.0
263                 carbspercent = snapshot.child("personal information").child("carbspercent")
264                     .getValue(Double::class.java) ?: 0.0
265                 calories = snapshot.child("personal information").child("calories").getValue(Double::class.
266                 val personalInfo = PersonalInfo(
267                     age = age,
268                     height = height,
269                     weight = weight,
270                     sex = sex,
271                     activityLevel = alevels,
272                     proteinpercent = proteinpercent, // this ensures that if the user has changed their %,
273                     fatspercent = fatspercent,
274                     carbspercent = carbspercent
275                 )
276
277                 personalInfo.calculatecalories()
278
279                 if (calories == personalInfo.calories){
280                     Toast.makeText(requireContext(),"No need to recalculate", Toast.LENGTH_SHORT).show()
281                 } else{
282                     personalInfo.calculatemacros()
283
284                     val updates = mapOf(
285                         "personal information/calories" to personalInfo.calories,
286                         "personal information/deficitcalories" to personalInfo.deficitcalories,
287                         "personal information/protein" to personalInfo.protein,
288                         "personal information/fats" to personalInfo.fats,
289                         "personal information/carbs" to personalInfo.carbs,
290                     )
291                     databaseReference.updateChildren(updates).addOnSuccessListener {
292                         showConfirmationDialog(personalInfo.calories, personalInfo.deficitcalories)
293                     }.addOnFailureListener {
294                         Toast.makeText(requireContext(), "Failed to ReCalculate", Toast.LENGTH_SHORT).show(
295                     }.addOnFailureListener {
296                         Toast.makeText(
297                             requireContext(),
298                             "Failed to load macronutrient percentages",
299                             Toast.LENGTH_SHORT
300                         ).show()
301                     }
302                 }
303             }
304         }
305
306
307     }
308     // Moved dialog creation to separate function for clarity
```

```
307         }
308         // Moved dialog creation to separate function for clarity
309         private fun showConfirmationDialog(calories: Double, deficitcalories: Double) {
310             val builder = AlertDialog.Builder(requireContext())
311             val view = layoutInflater.inflate(R.layout.dialog_confirm, null)
312             builder.setView(view)
313 
314             val calorietext = view.findViewById<TextView>(R.id.caloriebox)
315             val deficittext = view.findViewById<TextView>(R.id.deficitcaloriebox)
316             calorietext.setText("Calories: $calories")
317             deficittext.setText("Deficit Calories: $deficitcalories")
318 
319             val dialog = builder.create()
320             view.findViewById<Button>(R.id.btnClose).setOnClickListener {
321                 dialog.dismiss()
322             }
323 
324             if (dialog.window != null) {
325                 dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
326             }
327 
328             dialog.show()
329         }
330 
331         private fun updateUI(option: String, newValue: String) {
332             when (option) {
333                 "name" -> binding.readName.text = newValue
334                 "age" -> binding.readAge.text = newValue
335                 "height" -> binding.readHeight.text = newValue
336                 "sex" -> binding.readSex.text = newValue
337                 "weight" -> binding.readWeight.text = newValue
338                 "activityLevel" -> binding.readActivityLevels.text = newValue
339             }
340         }
341 
342         private fun updatevariableints(option: String, newValue: Int) {
343             when (option) {
344                 "age" -> age = newValue
345                 "height" -> height = newValue
346                 "weight" -> weight = newValue
347             }
348         }
349 
350         private fun updatevariablestrings(option: String, newValue: String) {
351             when (option) {
352                 "sex" -> sex = newValue
353                 "activityLevel" -> alevels = newValue
354             }
355         }
356     }
357 
```

```
class ProfileFragment : Fragment() {

    private var _binding: FragmentProfileBinding? = null
    private val binding get() = _binding!! // safe access to binding
    private lateinit var databaseReference: DatabaseReference
    private lateinit var firebaseAuth: FirebaseAuth

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        // Inflate the layout for this fragment
        _binding = FragmentProfileBinding.inflate(inflater, container, false)
        readData()


        binding.logoutButton.setOnClickListener{
            Firebase.auth.signOut()
            val intent = Intent(requireContext(), LoginActivity::class.java)
            startActivity(intent)

            Toast.makeText(requireContext(),"Logout Successful", Toast.LENGTH_SHORT).show()
        }
        return binding.root
    }

    private fun readData(){
        val user = FirebaseAuth.getInstance().currentUser
        val userid = FirebaseAuth.getInstance().currentUser?.uid
        if (userid != null){
            databaseReference = FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.europe-w
            databaseReference.get().addOnSuccessListener { snapshot ->
                if (snapshot.exists()) {
                    val name = snapshot.child("personal information").child("name").value
                    val email = user?.email
                    binding.readName.text = name.toString()
                    binding.readEmail.text = email.toString()

                } else {
                    Toast.makeText(requireContext(),"No data found", Toast.LENGTH_SHORT).show()
                }
            }.addOnFailureListener{
                Toast.makeText(requireContext(),"Something went wrong", Toast.LENGTH_SHORT).show()
            }
        }
    }



    override fun onDestroyView() {
        super.onDestroyView()
        // Avoid memory leaks by clearing binding
        _binding = null
    }
}
```

Daily goals

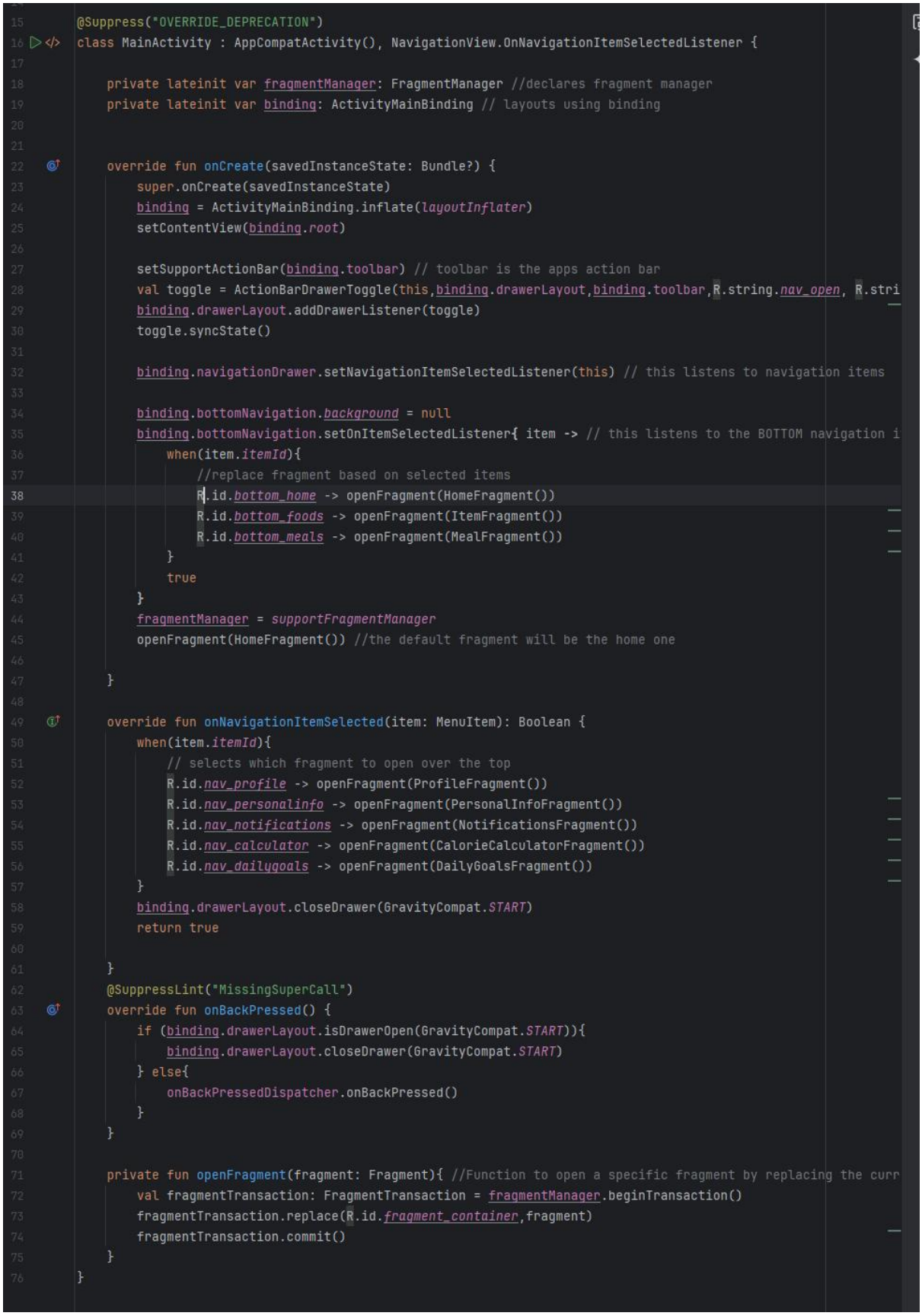
```
lass DailyGoalsFragment : Fragment() {

    private var _binding: FragmentDailygoalsBinding? = null
    private val binding get() = _binding!! // safe access to binding
    private lateinit var databaseReference: DatabaseReference

    private var calorieG = 0.0
    private var proteinG = 0.0
    private var fatG = 0.0
    private var carbG = 0.0

    private var proteinpercent = 0.0
    private var fatspercent = 0.0
    private var carbpercent = 0.0

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        _binding = FragmentDailygoalsBinding.inflate(inflater,container,false)

        setupFirebase()
        readData()

        binding.btnEditCalories.setOnClickListener{
            call_DetailChangeCalories()
        }
        binding.btnEditMacros.setOnClickListener{
            showMacrosAdjustmentDialog()
        }

        return binding.root
    }


    private fun setupFirebase() {
        val userid = FirebaseAuth.getInstance().currentUser?.uid
        databaseReference = FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.europe-west1.fire
    }

    private fun readData(){
        val userid = FirebaseAuth.getInstance().currentUser?.uid
        if (userid != null){
            databaseReference.get().addOnSuccessListener{ snapshot ->
                calorieG = snapshot.child("deficitcalories").getValue(Double::class.java) ?: 0.0
                proteinG = snapshot.child("protein").getValue(Double::class.java) ?: 0.0
                fatG = snapshot.child("fats").getValue(Double::class.java) ?: 0.0
                carbG = snapshot.child("carbs").getValue(Double::class.java) ?: 0.0

                proteinpercent = snapshot.child("proteinpercent").getValue(Double::class.java) ?: 0.0
                fatspercent = snapshot.child("fatspercent").getValue(Double::class.java) ?: 0.0
                carbpercent = snapshot.child("carbspercent").getValue(Double::class.java) ?: 0.0

                binding.readCalories.text = calorieG.toString()
                binding.readProtein.text = proteinG.toString()
                binding.readFats.text = fatG.toString()
                binding.readCarbs.text = carbG.toString()

                binding.readProteinPercent.text = "${(proteinpercent * 100).toInt()}%"
                binding.readFatsPercent.text = "${(fatspercent * 100).toInt()}%"
                binding.readCarbsPercent.text = "${(carbpercent * 100).toInt()}%"

            }.addOnFailureListener{
                Toast.makeText(requireContext(),"Something went wrong...", Toast.LENGTH_SHORT).show()
            }
        }

    }
```

```
89
90      private fun call_DetailChangeCalories() {
91          val builder = AlertDialog.Builder(requireContext())
92          val view = layoutInflater.inflate(R.layout.dialog_changeinfo, null) // calls dialog_changeinfo xml
93
94          val changedetail = view.findViewById<EditText>(R.id.editBoxDetails)
95
96          changedetail.setText(calorie6.toString())
97
98          builder.setView(view)
99          val dialog = builder.create()
100         view.findViewById<Button>(R.id.btnChangeDetail).setOnClickListener{
101             val userid = FirebaseAuth.getInstance().currentUser?.uid
102             if (userid != null){
103                 val detailText = changedetail.text.toString()
104                 if (detailText.isNotEmpty()) {
105                     val detailValue = detailText.toInt()
106
107                     val newProtein = ((detailValue * proteinpercent) / 4).toInt()
108                     val newFats = ((detailValue * fatspercent) / 9).toInt()
109                     val newCarbs = ((detailValue * carbpercent) / 4).toInt()
110
111                     val updates = hashMapOf<String, Any>(
112                         "calories" to detailValue,
113                         "deficitcalories" to detailValue,
114                         "protein" to newProtein,
115                         "fats" to newFats,
116                         "carbs" to newCarbs
117                     )
118
119                     databaseReference.updateChildren(updates)
120                         .addOnSuccessListener {
121                             calorie6 = detailValue.toDouble()
122                             protein6 = newProtein.toDouble()
123                             fat6 = newFats.toDouble()
124                             carb6 = newCarbs.toDouble()
125
126                             binding.readCalories.text = detailValue.toString()
127                             binding.readProtein.text = newProtein.toString()
128                             binding.readFats.text = newFats.toString()
129                             binding.readCarbs.text = newCarbs.toString()
130
131                             dialog.dismiss()
132                             Toast.makeText(context, "Goals updated successfully", Toast.LENGTH_SHORT).show()
133                         }
134                         .addOnFailureListener {
135                             Toast.makeText(requireContext(), "Failed to save", Toast.LENGTH_SHORT).show()
136                         }
137
138                 } else {
139                     Toast.makeText(requireContext(), "Fields cannot be empty", Toast.LENGTH_SHORT).show()
140                 }
141
142             } else {
143                 Toast.makeText(requireContext(), "Failed", Toast.LENGTH_SHORT).show()
144                 dialog.dismiss()
145             }
146         }
147         view.findViewById<Button>(R.id.btnCancelDetails).setOnClickListener {
148             dialog.dismiss() // cancels operation
149         }
150         if (dialog.window != null) {
151             dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
152         }
153         dialog.show()
154     }
155
```

```
private fun showMacrosAdjustmentDialog() {
    val builder = AlertDialog.Builder(requireContext())
    val view = layoutInflater.inflate(R.layout.dialog_macros_adjustment, null)

    // Initialize views
    val proteinSeekBar = view.findViewById<SeekBar>(R.id.proteinSeekBar)
    val fatsSeekBar = view.findViewById<SeekBar>(R.id.fatsSeekBar)
    val carbsSeekBar = view.findViewById<SeekBar>(R.id.carbsSeekBar)

    val proteinPercentageText = view.findViewById<TextView>(R.id.proteinPercentage)
    val fatsPercentageText = view.findViewById<TextView>(R.id.fatsPercentage)
    val carbsPercentageText = view.findViewById<TextView>(R.id.carbsPercentage)
    val totalPercentageText = view.findViewById<TextView>(R.id.totalPercentage)

    // Set initial values rounded to nearest 5%
    proteinSeekBar.progress = (((proteinpercent * 100) + 2.5) / 5).toInt() * 5
    fatsSeekBar.progress = (((fatspercent * 100) + 2.5) / 5).toInt() * 5
    carbsSeekBar.progress = (((carbpercent * 100) + 2.5) / 5).toInt() * 5

    // Update text views
    fun updatePercentageTexts() {
        proteinPercentageText.text = "${proteinSeekBar.progress}%"
        fatsPercentageText.text = "${fatsSeekBar.progress}%"
        carbsPercentageText.text = "${carbsSeekBar.progress}%"
        val total = proteinSeekBar.progress + fatsSeekBar.progress + carbsSeekBar.progress
        totalPercentageText.text = "Total: $total%"
        totalPercentageText.setTextColor(if (total == 100)
            resources.getColor(R.color.darkGreen)
        else resources.getColor(android.R.color.holo_red_dark))
    }

    // Set listeners for seek bars
    val seekBarChangeListener = object : SeekBar.OnSeekBarChangeListener {...}

    proteinSeekBar.setOnSeekBarChangeListener(seekBarChangeListener)
    fatsSeekBar.setOnSeekBarChangeListener(seekBarChangeListener)
    carbsSeekBar.setOnSeekBarChangeListener(seekBarChangeListener)

    updatePercentageTexts()

    builder.setView(view)
    val dialog = builder.create()

    // Handle save button click
    view.findViewById<Button>(R.id.saveButton).setOnClickListener {
        val total = proteinSeekBar.progress + fatsSeekBar.progress + carbsSeekBar.progress
        if (total != 100) {
            Toast.makeText(context, "Total percentage must equal 100%", Toast.LENGTH_SHORT).show()
            return@setOnClickListener
        }

        // Convert to decimals for storage
        val newProteinPercent = proteinSeekBar.progress / 100.0
        val newFatsPercent = fatsSeekBar.progress / 100.0
        val newCarbsPercent = carbsSeekBar.progress / 100.0

        // Save to Firebase and update local values
        val updates = hashMapOf<String, Any>(
            "proteinpercent" to newProteinPercent,
            "fatspercent" to newFatsPercent,
            "carbspercent" to newCarbsPercent
        )

        // Calculate new macro values based on current calories
        val newProtein = ((calorieG * newProteinPercent) / 4).toInt()
        val newFats = ((calorieG * newFatsPercent) / 9).toInt()
        val newCarbs = ((calorieG * newCarbsPercent) / 4).toInt()

        updates["protein"] = newProtein
        updates["fats"] = newFats
        updates["carbs"] = newCarbs
```

```
            databaseReference.updateChildren(updates)
                .addOnSuccessListener {
                    // Update local variables
                    proteinpercent = newProteinPercent
                    fatspercent = newFatsPercent
                    carbpercent = newCarbsPercent
                    protein0 = newProtein.toDouble()
                    fat0 = newFats.toDouble()
                    carb0 = newCarbs.toDouble()

                    // Update UI
                    binding.readProtein.text = newProtein.toString()
                    binding.readFats.text = newFats.toString()
                    binding.readCarbs.text = newCarbs.toString()
                    binding.readProteinPercent.text = "${(newProteinPercent * 100).toInt()}%"
                    binding.readFatsPercent.text = "${(newFatsPercent * 100).toInt()}%"
                    binding.readCarbsPercent.text = "${(newCarbsPercent * 100).toInt()}%"

                    Toast.makeText(context, "Macros updated successfully", Toast.LENGTH_SHORT).show()
                    dialog.dismiss()
                }
                .addOnFailureListener {
                    Toast.makeText(context, "Failed to update macros", Toast.LENGTH_SHORT).show()
                }
        }

        view.findViewById<Button>(R.id.cancelButton).setOnClickListener {
            dialog.dismiss()
        }

        if (dialog.window != null) {
            dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
        }

        dialog.show()
    }
```

Navigation Module



```
@Suppress("OVERRIDE_DEPRECATION")
class MainActivity : AppCompatActivity(), NavigationView.OnNavigationItemSelectedListener {

    private lateinit var fragmentManager: FragmentManager //declares fragment manager
    private lateinit var binding: ActivityMainBinding // layouts using binding


    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        setSupportActionBar(binding.toolbar) // toolbar is the apps action bar
        val toggle = ActionBarDrawerToggle(this,binding.drawerLayout,binding.toolbar,R.string.nav_open, R.stri
        binding.drawerLayout.addDrawerListener(toggle)
        toggle.syncState()

        binding.navigationDrawer.setNavigationItemSelectedListener(this) // this listens to navigation items

        binding.bottomNavigation.background = null
        binding.bottomNavigation.setOnItemSelectedListener{ item -> // this listens to the BOTTOM navigation i
            when(item.itemId){
                //replace fragment based on selected items
                R.id.bottom_home -> openFragment(HomeFragment())
                R.id.bottom_foods -> openFragment(ItemFragment())
                R.id.bottom_meals -> openFragment(MealFragment())
            }
            true
        }
        fragmentManager = supportFragmentManager
        openFragment(HomeFragment()) //the default fragment will be the home one

    }

    override fun onNavigationItemSelected(item: MenuItem): Boolean {
        when(item.itemId){
            // selects which fragment to open over the top
            R.id.nav_profile -> openFragment(ProfileFragment())
            R.id.nav_personalinfo -> openFragment(PersonalInfoFragment())
            R.id.nav_notifications -> openFragment(NotificationsFragment())
            R.id.nav_calculator -> openFragment(CalorieCalculatorFragment())
            R.id.nav_dailygoals -> openFragment(DailyGoalsFragment())
        }
        binding.drawerLayout.closeDrawer(GravityCompat.START)
        return true

    }
    @SuppressLint("MissingSuperCall")
    override fun onBackPressed() {
        if (binding.drawerLayout.isDrawerOpen(GravityCompat.START)){
            binding.drawerLayout.closeDrawer(GravityCompat.START)
        } else{
            onBackPressedDispatcher.onBackPressed()
        }
    }

    private fun openFragment(fragment: Fragment){ //Function to open a specific fragment by replacing the curr
        val fragmentTransaction: FragmentTransaction = fragmentManager.beginTransaction()
        fragmentTransaction.replace(R.id.fragment_container,fragment)
        fragmentTransaction.commit()
    }
}
```

Food Item Creation Module

```
data class FoodItem(
    var name: String = "",
    var calories: Double = 0.0,
    var proteins: Double = 0.0,
    var fats: Double = 0.0,
    var carbs: Double = 0.0
) {
    // Calculates 1g amounts from 100g

    fun convertto1gram() {
        calories = ((calories / 100.0) * 100).roundToInt() / 100.0
        proteins = ((proteins / 100.0) * 100).roundToInt() / 100.0
        fats = ((fats / 100.0) * 100).roundToInt() / 100.0
        carbs = ((carbs / 100.0) * 100).roundToInt() / 100.0
    }

    // Calculates 1g amount to 100g
    fun convertto100gram() {
        calories = ((calories * 100.0) * 100).roundToInt() / 100.0
        proteins = ((proteins * 100.0) * 100).roundToInt() / 100.0
        fats = ((fats * 100.0) * 100).roundToInt() / 100.0
        carbs = ((carbs * 100.0) * 100).roundToInt() / 100.0
    }
}

data class Meal(
    var name: String = "",
    var foodItems: MutableMap<String, FoodItemWithQuantity> = mutableMapOf(),
    var totalCalories: Double = 0.0,
    var totalProteins: Double = 0.0,
    var totalFats: Double = 0.0,
    var totalCarbs: Double = 0.0
) {
    fun updateTotals() {
        totalCalories = foodItems.values.sumOf { it.calories * it.grams }
        totalProteins = foodItems.values.sumOf { it.proteins * it.grams }
        totalFats = foodItems.values.sumOf { it.fats * it.grams }
        totalCarbs = foodItems.values.sumOf { it.carbs * it.grams }
    }
}

data class FoodItemWithQuantity(
    val foodItem: FoodItem = FoodItem(), // Add default value
    var grams: Double = 0.0
) {
    // Add no-argument constructor for Firebase
    constructor() : this(FoodItem(), 0.0)

    val calories: Double get() = foodItem.calories
    val proteins: Double get() = foodItem.proteins
    val fats: Double get() = foodItem.fats
    val carbs: Double get() = foodItem.carbs
}
```

```
class FoodItemAdapter(private val context: Context, private var foodList: List<FoodItem>)
    : RecyclerView.Adapter<FoodItemAdapter.FoodViewHolder>() {

    inner class FoodViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) {
        val nameTextView: TextView = itemView.findViewById(R.id.recTitle)
        val priorityTextView: TextView = itemView.findViewById(R.id.recPriority)

        init {
            itemView.setOnClickListener {
                val foodItem = foodList[adapterPosition]
                showFoodItemDialog(foodItem)
            }

            itemView.setOnLongClickListener {
                val foodItem = foodList[adapterPosition]
                showDeleteOptionsDialog(foodItem)
                true
            }
        }

        private fun showDeleteOptionsDialog(foodItem: FoodItem) {
            val builder = AlertDialog.Builder(context)
            val dialogView = LayoutInflater.from(context).inflate(R.layout.dialog_meal_options, null)

            val editButton = dialogView.findViewById<Button>(R.id.editButton)
            val deleteButton = dialogView.findViewById<Button>(R.id.deleteButton)
            val cancelButton = dialogView.findViewById<Button>(R.id.cancelButton)

            builder.setView(dialogView)
            val dialog = builder.create()
            editButton.visibility = View.GONE

            // Set click listeners

            deleteButton.setOnClickListener {
                dialog.dismiss()
                showDeleteConfirmationDialog(foodItem)
            }

            cancelButton.setOnClickListener {
                dialog.dismiss()
            }

            // Set transparent background
            if (dialog.window != null) {
                dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
            }

            dialog.show()
        }

        private fun showEditFieldsDialog(foodItem: FoodItem) {...}

        private fun showDeleteConfirmationDialog(foodItem: FoodItem) {
            val builder = AlertDialog.Builder(context)
            val dialogView = LayoutInflater.from(context).inflate(R.layout.dialog_meal_options, null)

            // Get UI elements
            val titleText = dialogView.findViewById<TextView>(R.id.dialogTitle)
            val confirmButton = dialogView.findViewById<Button>(R.id.editButton)    // Reuse edit button fo
            val cancelButton = dialogView.findViewById<Button>(R.id.deleteButton)   // Reuse delete button
            val bottomButton = dialogView.findViewById<Button>(R.id.cancelButton)   // Hide the bottom butt

            // Update UI elements
            titleText.text = "Delete Food Item"
            confirmButton.text = "Delete"
            cancelButton.text = "Keep"
            confirmButton.backgroundTintList = ColorStateList.valueOf(Color.parseColor("#400a06"))
            cancelButton.backgroundTintList = ColorStateList.valueOf(Color.parseColor("#053411"))
            bottomButton.visibility = View.GONE

            builder.setView(dialogView)
            val dialog = builder.create()

            // Set click listeners
            confirmButton.setOnClickListener {
                dialog.dismiss()
                deleteFoodItem(foodItem)
            }
```

```
        // Update UI elements
        titleText.text = "Delete Food Item"
        confirmButton.text = "Delete"
        cancelButton.text = "Keep"
        confirmButton.backgroundTintList = ColorStateList.valueOf(Color.parseColor("#400a06"))
        cancelButton.backgroundTintList = ColorStateList.valueOf(Color.parseColor("#053411"))
        bottomButton.visibility = View.GONE

        builder.setView(dialogView)
        val dialog = builder.create()

        // Set click listeners
        confirmButton.setOnClickListener {
            dialog.dismiss()
            deleteFoodItem(foodItem)
        }

        cancelButton.setOnClickListener {
            dialog.dismiss()
        }

        // Set transparent background
        if (dialog.window != null) {
            dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
        }

        dialog.show()
    }

    private fun showPropertyEditDialog(
        foodItem: FoodItem,
        property: String,
        displayName: String,
        currentValue: String,
        isNumber: Boolean
    ) {...}

    private fun confirmDelete(foodItem: FoodItem) {...}

    private fun updateFoodItem(foodItem: FoodItem) {
        val userId = FirebaseAuth.getInstance().currentUser?.uid
        if (userId != null) {
            val databaseRef = FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.europe-we
                .getReference("Information")
                .child(userId)
                .child("Food items")
                .child(foodItem.name)

            foodItem.convertto1gram() // Convert to 1g before saving
            databaseRef.setValue(foodItem)
                .addOnSuccessListener {
                    Toast.makeText(context, "Food item updated", Toast.LENGTH_SHORT).show()
                    notifyDataSetChanged()
                }
                .addOnFailureListener {
                    Toast.makeText(context, "Failed to update food item", Toast.LENGTH_SHORT).show()
                }
        }
    }

    private fun deleteFoodItem(foodItem: FoodItem) {
        val userId = FirebaseAuth.getInstance().currentUser?.uid
        if (userId != null) {
            val databaseRef = FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.europe-we
                .getReference("Information")
                .child(userId)
                .child("Food items")
                .child(foodItem.name)

            databaseRef.removeValue()
                .addOnSuccessListener {
                    Toast.makeText(context, "Food item deleted", Toast.LENGTH_SHORT).show()
                }
                .addOnFailureListener {
                    Toast.makeText(context, "Failed to delete food item", Toast.LENGTH_SHORT).show()
                }
        }
    }
```

```
class ItemFragment : Fragment() {

    private var _binding: FragmentItemBinding? = null
    private val binding get() = _binding!! // safe access to binding
    private lateinit var databaseReference: DatabaseReference
    private lateinit var firebaseAuth: FirebaseAuth
    private lateinit var searchView: SearchView

    private lateinit var adapter: FoodItemAdapter
    private var foodList = mutableListOf<FoodItem>()

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        // Inflate the layout for this fragment
        _binding = FragmentItemBinding.inflate(inflater, container, false)
        searchView = binding.search


        val recyclerView = binding.recyclerItemView // Assuming you have a RecyclerView in your Fragment layout
        recyclerView.layoutManager = LinearLayoutManager(context)
        adapter = FoodItemAdapter(requireContext(), emptyList()) // Initialize with an empty list to start now
        recyclerView.adapter = adapter

        databaseReference = FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.europe-west1.fi
        val userId = FirebaseAuth.getInstance().currentUser?.uid

        userId?.let {
            databaseReference.child(it).child("Food items").addValueEventListener(object : ValueEventListener {
                override fun onDataChange(snapshot: DataSnapshot) {
                    foodList.clear()
                    for (foodSnapshot in snapshot.children) {
                        val foodItem = foodSnapshot.getValue(FoodItem::class.java)
                        if (foodItem != null) {
                            foodItem.convertto100gram() //converts back to 100g for viewing ease
                            foodList.add(foodItem) // adds each item to a list
                        }
                    }
                    // Update adapter with fetched data
                    adapter.updateData(foodList)
                }

                override fun onCancelled(error: DatabaseError) {
                    Toast.makeText(requireContext(), "Failed to load data", Toast.LENGTH_SHORT).show()
                }
            })
        }

        searchView.setOnQueryTextListener(object : SearchView.OnQueryTextListener {
            override fun onQueryTextSubmit(query: String?): Boolean {
                return false
            }

            override fun onQueryTextChange(newText: String?): Boolean {
                searchList(newText ?: "")
                return true
            }
        })
        binding.itemUploadButton.setOnClickListener{...}
        return binding.root
    }


    private fun searchList(query: String) {
        if (foodList.isNotEmpty()){
            val filteredList = foodList.filter {
                it.name.toLowerCase().contains(query.toLowerCase()) // Filter food items by name
            }
            adapter.updateData(filteredList) // Update the adapter with filtered data
        }

    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null // Clean up binding reference
    }
}
```

Meal Creation Module

```kotlin
class CreateMealFragment : Fragment() {
    private var _binding: FragmentCreateMealBinding? = null
    private val binding get() = _binding!!
    private lateinit var databaseReference: DatabaseReference
    private lateinit var selectedFoodsAdapter: SelectedFoodsAdapter
    private var currentMeal = Meal()
    private var isEditing = false
    private var editingMealName = ""

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        // Get arguments passed from MealFragment
        // Retrieve edit mode parameters if they exist
        arguments?.let {
            isEditing = it.getBoolean("isEditing", false)
            editingMealName = it.getString("mealName", "")
        }
    }

    override fun onCreateView(inflater: LayoutInflater, container: ViewGroup?, savedInstanceState: Bundle?
    ): View {
        // Initialize the view and setup necessary components
        _binding = FragmentCreateMealBinding.inflate(inflater, container, false)

        setupRecyclerView()
        setupDatabase()


        binding.addFoodButton.setOnClickListener {
            // Show food selection dialog
            showFoodSelectionDialog()
        }
        binding.saveMealButton.setOnClickListener {
            saveMealToFirebase()
        }

        // Load existing meal data if in edit mode
        if (isEditing) {
            loadExistingMeal()
        }

        return binding.root
    }

    private fun setupRecyclerView() {
        // Initialize adapter for selected food items
        selectedFoodsAdapter = SelectedFoodsAdapter()
        // Setup click listener for removing items by tapping them
        selectedFoodsAdapter.setOnItemClickListener { foodItem ->
            // Remove the item from currentMeal
            currentMeal.foodItems.remove(foodItem.foodItem.name)
            currentMeal.updateTotals()
            // Update the display
            selectedFoodsAdapter.updateItems(currentMeal.foodItems.values.toList())
            updateMealDisplay()
            // Show confirmation toast
            Toast.makeText(context, "${foodItem.foodItem.name} removed", Toast.LENGTH_SHORT).show()
        }
        binding.selectedFoodsRecyclerView.apply {
            layoutManager = LinearLayoutManager(context)
            adapter = selectedFoodsAdapter
        }
    }

    private fun setupDatabase() {
        // Fetch food items from Firebase
        val userId = FirebaseAuth.getInstance().currentUser?.uid
        databaseReference = FirebaseDatabase.getInstance()
            .getReference("Information")
            .child(userId ?: "")
            .child("Meals")
    }

```

```
private fun showFoodSelectionDialog() {
    val userId = FirebaseAuth.getInstance().currentUser?.uid
    val foodsRef = FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.europe-west1.firebas
        .getReference("Information")
        .child(userId ?: "")
        .child("Food items")

    foodsRef.get().addOnSuccessListener { snapshot ->
        val foods = mutableListOf<FoodItem>()
        snapshot.children.forEach { foodSnapshot ->
            val food = foodSnapshot.getValue(FoodItem::class.java)
            food?.let {
                it.convertto100gram() // Convert to 100g for display
                foods.add(it)
            }
        }

        if (foods.isNotEmpty()) {

            // Setup dialog for food selection with search functionality
            val dialogView = LayoutInflater.from(requireContext()).inflate(R.layout.dialog_food_selection,
            val builder = AlertDialog.Builder(requireContext())
            builder.setView(dialogView)

            val dialog = builder.create()

            val searchView = dialogView.findViewById<SearchView>(R.id.foodSearchView)
            val foodListView = dialogView.findViewById<ListView>(R.id.foodListView)
            val cancelButton = dialogView.findViewById<Button>(R.id.cancelButton)

            val filteredFoods = mutableListOf<FoodItem>()
            filteredFoods.addAll(foods)

            // Simpler adapter implementation
            val adapter = object : BaseAdapter() {
                override fun getCount(): Int = filteredFoods.size
                override fun getItem(position: Int): FoodItem = filteredFoods[position]
                override fun getItemId(position: Int): Long = position.toLong()

                override fun getView(position: Int, convertView: View?, parent: ViewGroup): View {
                    val itemView = convertView ?: LayoutInflater.from(context).inflate(R.layout.recycler_it
                    val nameTextView = itemView.findViewById<TextView>(R.id.recTitle)
                    val caloriesTextView = itemView.findViewById<TextView>(R.id.recPriority)

                    val foodItem = filteredFoods[position]
                    nameTextView.text = foodItem.name
                    caloriesTextView.text = "${foodItem.calories} kcal"

                    // Add margin between items
                    val params = itemView.layoutParams as? ViewGroup.MarginLayoutParams ?: ViewGroup.Margin
                        ViewGroup.LayoutParams.MATCH_PARENT,
                        ViewGroup.LayoutParams.WRAP_CONTENT
                    )
                    params.setMargins(12, 8, 12, 8)
                    itemView.layoutParams = params

                    return itemView
                }
            }

            foodListView.adapter = adapter

            // Setup search functionality
            searchView.setOnQueryTextListener(object : SearchView.OnQueryTextListener {
                override fun onQueryTextSubmit(query: String?): Boolean = false

                override fun onQueryTextChange(newText: String?): Boolean {
                    filteredFoods.clear()
                    // Filter foods based on search text
                    if (newText.isNullOrEmpty()) {
                        filteredFoods.addAll(foods)
                    } else {
                        val filtered = foods.filter {
                            it.name.toLowerCase().contains(newText.toLowerCase())
                        }
                        filteredFoods.addAll(filtered)
                    }
                    adapter.notifyDataSetChanged()
                    return true
                }
            })
```

```
                foodListView.setOnItemClickListener { _, _, position, _ ->
                    val selectedFood = filteredFoods[position].copy()
                    selectedFood.convertto1gram() // Convert back to 1g before showing quantity dialog
                    showFoodQuantityDialog(selectedFood)
                    dialog.dismiss()
                }

                cancelButton.setOnClickListener {
                    dialog.dismiss()
                }

                if (dialog.window != null) {
                    dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
                }

                dialog.show()
            } else {
                Toast.makeText(context, "No food items available. Please add some first.", Toast.LENGTH_SHORT)
            }
        }.addOnFailureListener {
            Toast.makeText(context, "Failed to load food items", Toast.LENGTH_SHORT).show()
        }
    }

    private fun showFoodQuantityDialog(foodItem: FoodItem) {
        val builder = AlertDialog.Builder(requireContext())
        val view = layoutInflater.inflate(R.layout.dialog_quantity_input, null)

        val quantityInput = view.findViewById<EditText>(R.id.quantityInput)
        val confirmButton = view.findViewById<Button>(R.id.MealSaveBtn)
        val cancelButton = view.findViewById<Button>(R.id.MealCloseBtn)

        builder.setView(view)
        val dialog = builder.create()

        confirmButton.setOnClickListener {
            val quantity = quantityInput.text.toString().toDoubleOrNull()
            if (quantity != null && quantity > 0) {
                addFoodItemToMeal(foodItem, quantity)
                dialog.dismiss()
                updateMealDisplay()
            } else {
                Toast.makeText(context, "Please enter a valid quantity", Toast.LENGTH_SHORT).show()
            }
        }

        cancelButton.setOnClickListener { dialog.dismiss() }

        if (dialog.window != null){
            dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
        }

        dialog.show()
    }

    private fun addFoodItemToMeal(foodItem: FoodItem, grams: Double) {
        val foodItemWithQuantity = FoodItemWithQuantity(foodItem, grams)
        currentMeal.foodItems[foodItem.name] = foodItemWithQuantity
        currentMeal.updateTotals()
        selectedFoodsAdapter.updateItems(currentMeal.foodItems.values.toList())
    }

    private fun updateMealDisplay() {
        binding.mealTotalCalories.text = "Total Calories: ${currentMeal.totalCalories.roundToDecimal(1)}"
        binding.mealTotalNutrients.text =
            "Proteins: ${currentMeal.totalProteins.roundToDecimal(1)}g " +
                    "Fats: ${currentMeal.totalFats.roundToDecimal(1)}g " +
                    "Carbs: ${currentMeal.totalCarbs.roundToDecimal(1)}g"
    }
```

```
252
253     private fun loadExistingMeal() {
254         val userId = FirebaseAuth.getInstance().currentUser?.uid
255         if (userId != null && editingMealName.isNotEmpty()) {
256             // Set the meal name in the input field
257             binding.mealNameInput.setText(editingMealName)
258
259             val mealRef = FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.europe-west1.fire
260                 .getReference("Information")
261                 .child(userId)
262                 .child("Meals")
263                 .child(editingMealName)
264
265             mealRef.get().addOnSuccessListener { snapshot ->
266                 snapshot.getValue(Meal::class.java)?.let { meal ->
267                     currentMeal = meal
268                     selectedFoodsAdapter.updateItems(meal.foodItems.values.toList())
269                     updateMealDisplay()
270                 }
271             }.addOnFailureListener {
272                 Toast.makeText(context, "Failed to load meal data", Toast.LENGTH_SHORT).show()
273             }
274         }
275     }
276
277     private fun saveMealToFirebase() {
278         val mealName = binding.mealNameInput.text.toString()
279         if (mealName.isBlank()) {
280             Toast.makeText(context, "Please enter a meal name", Toast.LENGTH_SHORT).show()
281             return
282         }
283
284         if (isEditing && editingMealName != mealName) {
285             databaseReference.child(editingMealName).removeValue()
286         }
287
288         if (currentMeal.foodItems.isEmpty()) {
289             Toast.makeText(context, "Please add at least one food item", Toast.LENGTH_SHORT).show()
290             return
291         }
292
293         val userId = FirebaseAuth.getInstance().currentUser?.uid
294         databaseReference = FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.europe-west1.fi
295             .getReference("Information")
296             .child(userId ?: "")
297             .child("Meals")
298
299         currentMeal.name = mealName
300         currentMeal.updateTotals() // Make sure totals are up to date
301
302         databaseReference.child(mealName).setValue(currentMeal)
303             .addOnSuccessListener {
304                 Toast.makeText(context, "Meal saved successfully", Toast.LENGTH_SHORT).show()
305                 // Navigate back to MealFragment
306                 parentFragmentManager.popBackStack()
307             }
308             .addOnFailureListener {
309                 Toast.makeText(context, "Failed to save meal", Toast.LENGTH_SHORT).show()
310             }
311     }
312
313     private fun Double.roundToDecimal(decimals: Int): Double {...}
318
319     override fun onDestroyView() {
320         super.onDestroyView()
321         _binding = null
322     }
323 }
```

```
class MealAdapter : RecyclerView.Adapter<MealAdapter.MealViewHolder>() {
    interface MealClickListener {
        fun onMealClick(meal: Meal)
        fun onMealLongClick(meal: Meal)
    }

    private var meals = mutableListOf<Meal>()
    private var filteredMeals = mutableListOf<Meal>()

    init {
        filteredMeals = meals
    }



    fun setMealClickListener(listener: MealClickListener) {
        clickListener = listener
    }

    private var clickListener: MealClickListener? = null

    inner class MealViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) {
        val nameTextView: TextView = itemView.findViewById(R.id.recTitle)
        val detailsTextView: TextView = itemView.findViewById(R.id.recPriority)

        init {
            itemView.setOnClickListener {
                val position = adapterPosition
                if (position != RecyclerView.NO_POSITION) {
                    clickListener?.onMealClick(filteredMeals[position])
                }
            }

            itemView.setOnLongClickListener {
                val position = adapterPosition
                if (position != RecyclerView.NO_POSITION) {
                    clickListener?.onMealLongClick(filteredMeals[position])
                }
                true
            }
        }

    }


    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): MealViewHolder {
        val view = LayoutInflater.from(parent.context)
            .inflate(R.layout.recycler_item, parent, false)
        return MealViewHolder(view)
    }

    override fun onBindViewHolder(holder: MealViewHolder, position: Int) {
        val meal = filteredMeals[position]
        holder.nameTextView.text = meal.name
        holder.detailsTextView.text = "Calories: ${meal.totalCalories.roundToDecimal(1)}"
    }

    override fun getItemCount() = filteredMeals.size

    fun updateMeals(newMeals: List<Meal>) {
        meals.clear()
        meals.addAll(newMeals)
        filteredMeals = meals.toMutableList()
        notifyDataSetChanged()
    }

    fun searchMeals(query: String) {
        filteredMeals = if (query.isEmpty()) {
            meals.toMutableList()
        } else {
            meals.filter {
                it.name.toLowerCase().contains(query.toLowerCase())
            }.toMutableList()
        }
        notifyDataSetChanged()
    }
```

```
87
88      private fun Double.roundToDecimal(decimals: Int): Double {
89          return BigDecimal(this)
90              .setScale(decimals, RoundingMode.HALF_UP)
91              .toDouble()
92      }
93  }
94
95
96
97  class SelectedFoodsAdapter : RecyclerView.Adapter<SelectedFoodsAdapter.SelectedFoodViewHolder>() {
98      private var selectedFoods = mutableListOf<FoodItemWithQuantity>()
99      private var onItemClickListener: ((FoodItemWithQuantity) -> Unit)? = null
100
101     fun setOnItemClickListener(listener: (FoodItemWithQuantity) -> Unit) {
102         onItemClickListener = listener
103     }
104
105     inner class SelectedFoodViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) {
106         val nameTextView: TextView = itemView.findViewById(R.id.recTitle)
107         val detailsTextView: TextView = itemView.findViewById(R.id.recPriority)
108
109         init {
110             itemView.setOnClickListener {
111                 val position = adapterPosition
112                 if (position != RecyclerView.NO_POSITION) {
113                     onItemClickListener?.invoke(selectedFoods[position])
114                 }
115             }
116         }
117     }
118
119     override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): SelectedFoodViewHolder {
120         val view = LayoutInflater.from(parent.context).inflate(R.layout.recycler_item, parent, false)
121         return SelectedFoodViewHolder(view)
122     }
123
124     override fun onBindViewHolder(holder: SelectedFoodViewHolder, position: Int) {
125         val item = selectedFoods[position]
126         holder.nameTextView.text = "${item.foodItem.name} (${item.grams}g)"
127         holder.detailsTextView.text = "Cal: ${(item.calories * item.grams).roundToDecimal(1)}"
128     }
129
130     override fun getItemCount() = selectedFoods.size
131
132     fun updateItems(items: List<FoodItemWithQuantity>) {
133         selectedFoods.clear()
134         selectedFoods.addAll(items)
135         notifyDataSetChanged()
136     }
137
138     private fun Double.roundToDecimal(decimals: Int): Double {
139         return BigDecimal(this)
140             .setScale(decimals, RoundingMode.HALF_UP)
141             .toDouble()
142     }
143 }
```

```
class MealFragment : Fragment(), MealAdapter.MealClickListener {
    private var _binding: FragmentMealBinding? = null
    private val binding get() = _binding!!
    private lateinit var databaseReference: DatabaseReference
    private lateinit var mealAdapter: MealAdapter

    override fun onCreateView(inflater: LayoutInflater, container: ViewGroup?, savedInstanceState: Bundle?
    ): View {
        _binding = FragmentMealBinding.inflate(inflater, container, false)

        // Initialize Firebase first
        val userId = FirebaseAuth.getInstance().currentUser?.uid
        databaseReference = FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.europe-wes
            .getReference("Information")
            .child(userId ?: "")
            .child("Meals")


        setupRecyclerView()
        setupSearch()
        loadMealsFromFirebase()

        binding.createMealButton.setOnClickListener {
            val transaction = parentFragmentManager.beginTransaction()
            transaction.replace(R.id.fragment_container, CreateMealFragment())
            transaction.addToBackStack(null)
            transaction.commit()
        }

        return binding.root
    }

    override fun onMealClick(meal: Meal) {
        showMealDetailsDialog(meal)
    }

    override fun onMealLongClick(meal: Meal) {
        showEditDeleteDialog(meal)
    }

    private fun showMealDetailsDialog(meal: Meal) {...}

    private fun showEditDeleteDialog(meal: Meal) {
        val builder = AlertDialog.Builder(requireContext())
        val view = layoutInflater.inflate(R.layout.dialog_meal_options, null)

        val editButton = view.findViewById<Button>(R.id.editButton)
        val deleteButton = view.findViewById<Button>(R.id.deleteButton)
        val cancelButton = view.findViewById<Button>(R.id.cancelButton)

        builder.setView(view)
        val dialog = builder.create()

        // Set click listeners
        editButton.setOnClickListener {
            dialog.dismiss()
            editMeal(meal)
        }

        deleteButton.setOnClickListener {
            dialog.dismiss()
            showDeleteConfirmationDialog(meal)
        }

        cancelButton.setOnClickListener {
            dialog.dismiss()
        }

        // Set transparent background
        if (dialog.window != null) {
            dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
        }

        dialog.show()
    }
```

```
    private fun showDeleteConfirmationDialog(meal: Meal) {
        val builder = AlertDialog.Builder(requireContext())
        val view = layoutInflater.inflate(R.layout.dialog_meal_options, null)

        // Reuse the same layout but modify it for delete confirmation
        val titleText = view.findViewById<TextView>(R.id.dialogTitle)
        val confirmButton = view.findViewById<Button>(R.id.editButton)    // Reuse edit button for confirm
        val cancelButton = view.findViewById<Button>(R.id.deleteButton)   // Reuse delete button for cancel
        val bottomButton = view.findViewById<Button>(R.id.cancelButton)   // Hide the bottom button

        // Update UI elements
        titleText.text = "Delete Meal"
        confirmButton.text = "Delete"
        cancelButton.text = "Keep"
        confirmButton.backgroundTintList = ColorStateList.valueOf(Color.parseColor("#400a06"))
        cancelButton.backgroundTintList = ColorStateList.valueOf(Color.parseColor("#053411"))
        bottomButton.visibility = View.GONE

        builder.setView(view)
        val dialog = builder.create()

        // Set click listeners
        confirmButton.setOnClickListener {
            dialog.dismiss()
            deleteMeal(meal)
        }

        cancelButton.setOnClickListener {
            dialog.dismiss()
        }

        // Set transparent background
        if (dialog.window != null) {
            dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
        }

        dialog.show()
    }

    private fun editMeal(meal: Meal) {...}

    private fun confirmDelete(meal: Meal) {...}

    private fun deleteMeal(meal: Meal) {
        val userId = FirebaseAuth.getInstance().currentUser?.uid
        if (userId != null) {
            databaseReference.child(meal.name).removeValue()
                .addOnSuccessListener {
                    Toast.makeText(context, "Meal deleted", Toast.LENGTH_SHORT).show()
                }
                .addOnFailureListener {
                    Toast.makeText(context, "Failed to delete meal", Toast.LENGTH_SHORT).show()
                }
        }
    }

    private fun Double.roundToDecimal(decimals: Int): Double {
        return BigDecimal(this)
            .setScale(decimals, RoundingMode.HALF_UP)
            .toDouble()
    }


    private fun setupRecyclerView() {
        mealAdapter = MealAdapter()
        mealAdapter.setMealClickListener(this)
        binding.mealsRecyclerView.apply {
            layoutManager = LinearLayoutManager(context)
            adapter = mealAdapter
        }
    }
```

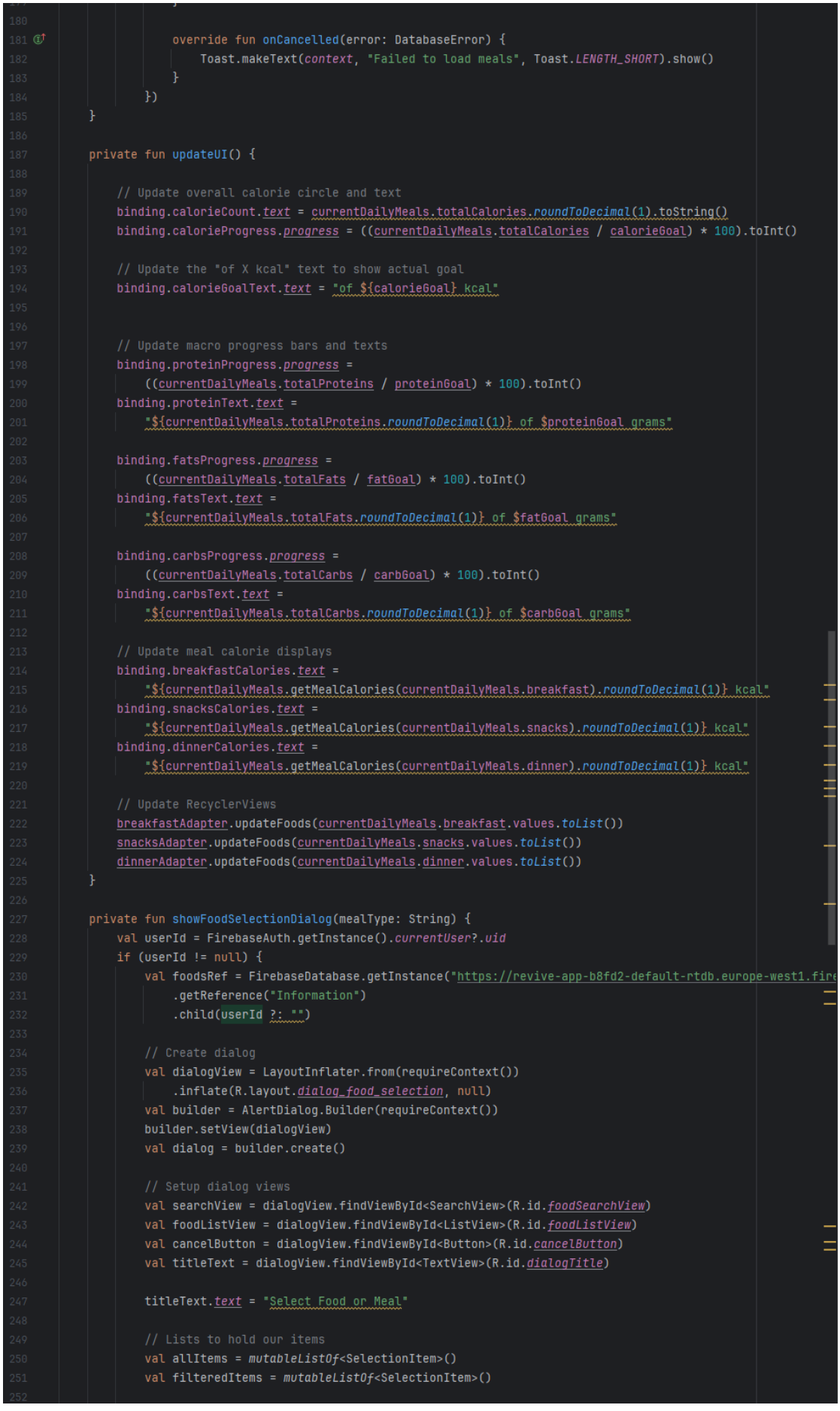
```
    private fun setupSearch() {
        binding.mealSearch.setOnQueryTextListener(object : SearchView.OnQueryTextListener {
            override fun onQueryTextSubmit(query: String?): Boolean {
                return false
            }

            override fun onQueryTextChange(newText: String?): Boolean {
                mealAdapter.searchMeals(newText ?: "")
                return true
            }
        })
    }

    private fun loadMealsFromFirebase() {
        try {
            databaseReference.addValueEventListener(object : ValueEventListener {
                override fun onDataChange(snapshot: DataSnapshot) {
                    val meals = mutableListOf<Meal>()
                    for (mealSnapshot in snapshot.children) {
                        val meal = mealSnapshot.getValue(Meal::class.java)
                        meal?.let { meals.add(it) }
                    }
                    mealAdapter.updateMeals(meals)
                }

                override fun onCancelled(error: DatabaseError) {
                    Toast.makeText(context, "Failed to load meals", Toast.LENGTH_SHORT).show()
                }
            })
        } catch (e: Exception) {
            Toast.makeText(context, "Error loading meals", Toast.LENGTH_SHORT).show()
        }
    }


    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

Calorie Tracking Module

```
class HomeMealAdapter(private val onItemClick: (FoodItemWithQuantity) -> Unit) :
    RecyclerView.Adapter<HomeMealAdapter.FoodViewHolder>() {

    private var foods = mutableListOf<FoodItemWithQuantity>()

    inner class FoodViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) {
        val nameTextView: TextView = itemView.findViewById(R.id.recTitle)
        val detailsTextView: TextView = itemView.findViewById(R.id.recPriority)

        init {
            itemView.setOnClickListener {
                val position = adapterPosition
                if (position != RecyclerView.NO_POSITION) {
                    onItemClick(foods[position])
                }
            }
        }
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): FoodViewHolder {
        val view = LayoutInflater.from(parent.context)
            .inflate(R.layout.recycler_item, parent, false)
        return FoodViewHolder(view)
    }

    override fun onBindViewHolder(holder: FoodViewHolder, position: Int) {
        val food = foods[position]
        holder.nameTextView.text = "${food.foodItem.name} (${food.grams.roundToDecimal(1)}g)"
        holder.detailsTextView.text = "${(food.calories * food.grams).roundToDecimal(1)} kcal"

    }

    override fun getItemCount() = foods.size

    fun updateFoods(newFoods: List<FoodItemWithQuantity>) {
        foods.clear()
        foods.addAll(newFoods)
        notifyDataSetChanged()
    }

    private fun Double.roundToDecimal(decimals: Int): Double {
        return BigDecimal(this)
            .setScale(decimals, RoundingMode.HALF_UP)
            .toDouble()
    }
}
```

```
class HomeFragment : Fragment() {
    private var _binding: FragmentHomeBinding? = null
    private val binding get() = _binding!!

    private lateinit var databaseReference: DatabaseReference
    private lateinit var auth: FirebaseAuth

    // Adapters for each meal section
    private lateinit var breakfastAdapter: HomeMealAdapter
    private lateinit var snacksAdapter: HomeMealAdapter
    private lateinit var dinnerAdapter: HomeMealAdapter

    // Current daily meals data
    private var currentDailyMeals = DailyMeals()

    // Initial Nutrient Goals
    private var calorieGoal = 2000.0
    private var proteinGoal = 150.0
    private var fatGoal = 65.0
    private var carbGoal = 275.0

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        _binding = FragmentHomeBinding.inflate(inflater, container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        loadUserGoals()
        setupFirebase()
        setupRecyclerViews()
        setupClickListeners()
    }

    private fun loadUserGoals() {
        val userId = FirebaseAuth.getInstance().currentUser?.uid
        if (userId != null) {
            FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.europe-west1.firebasedatabase.a
                .getReference("Information")
                .child(userId)
                .child("personal information")
                .get()
                .addOnSuccessListener { snapshot ->
                    calorieGoal = snapshot.child("deficitcalories").getValue(Double::class.java) ?: 0.0
                    proteinGoal = snapshot.child("protein").getValue(Double::class.java) ?: 0.0
                    fatGoal = snapshot.child("fats").getValue(Double::class.java) ?: 0.0
                    carbGoal = snapshot.child("carbs").getValue(Double::class.java) ?: 0.0
                    updateUI()
                    // Once we have the goals, load the meals
                    loadTodaysMeals()
                }
                .addOnFailureListener {
                    Toast.makeText(context, "Failed to load calorie goals", Toast.LENGTH_SHORT).show()
                }
        }
    }


    private fun setupFirebase() {
        auth = FirebaseAuth.getInstance()
        val userId = auth.currentUser?.uid
        databaseReference = FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.europe-west1.fi
            .getReference("Information")
            .child(userId ?: "")
            .child("DailyMeals")
    }

    private fun getCurrentDate(): String {
        val dateFormat = SimpleDateFormat("yyyy-MM-dd", Locale.getDefault())
        return dateFormat.format(Date())
    }
```

```
    private fun setupRecyclerViews() {
        // Initialize adapters with click handlers
        breakfastAdapter = HomeMealAdapter { foodItem ->
            showFoodItemDialog(foodItem, "Breakfast")
        }
        snacksAdapter = HomeMealAdapter { foodItem ->
            showFoodItemDialog(foodItem, "Snacks")
        }
        dinnerAdapter = HomeMealAdapter { foodItem ->
            showFoodItemDialog(foodItem, "Dinner")
        }

        // Setup RecyclerViews with adapters and layouts
        binding.breakfastRecyclerView.apply {
            layoutManager = LinearLayoutManager(context)
            adapter = breakfastAdapter
        }

        binding.snacksRecyclerView.apply {
            layoutManager = LinearLayoutManager(context)
            adapter = snacksAdapter
        }

        binding.dinnerRecyclerView.apply {
            layoutManager = LinearLayoutManager(context)
            adapter = dinnerAdapter
        }
    }

    private fun setupClickListeners() {
        // Add food buttons click listeners
        binding.addBreakfastButton.setOnClickListener {
            showFoodSelectionDialog("Breakfast")
        }

        binding.addSnacksButton.setOnClickListener {
            showFoodSelectionDialog("Snacks")
        }

        binding.addDinnerButton.setOnClickListener {
            showFoodSelectionDialog("Dinner")
        }
    }

    private fun loadTodaysMeals() {
        val currentDate = getCurrentDate()

        databaseReference.child(currentDate)
            .addValueEventListener(object : ValueEventListener {
                override fun onDataChange(snapshot: DataSnapshot) {
                    // Check if this is a new day
                    val savedDate = snapshot.child("date").getValue(String::class.java)
                    if (savedDate != currentDate) {
                        // It's a new day, reset the meals
                        currentDailyMeals = DailyMeals(date = currentDate)
                        // Save the empty day to Firebase
                        databaseReference.child(currentDate).setValue(currentDailyMeals)
                    } else {
                        // Load existing meals for today
                        currentDailyMeals = DailyMeals()
                        currentDailyMeals.date = currentDate

                        // Load meals from snapshot
                        listOf("breakfast", "snacks", "dinner").forEach { mealType ->
                            snapshot.child(mealType).children.forEach { foodSnapshot ->
                                val food = foodSnapshot.getValue(FoodItemWithQuantity::class.java)
                                food?.let {
                                    when (mealType) {
                                        "breakfast" -> currentDailyMeals.breakfast[it.foodItem.name] = it
                                        "snacks" -> currentDailyMeals.snacks[it.foodItem.name] = it
                                        "dinner" -> currentDailyMeals.dinner[it.foodItem.name] = it
                                    }
                                }
                            }
                        }

                        currentDailyMeals.updateTotals()
                        updateUI()
                    }
                }
```



```
            override fun onCancelled(error: DatabaseError) {
                Toast.makeText(context, "Failed to load meals", Toast.LENGTH_SHORT).show()
            }
        })
    }

    private fun updateUI() {

        // Update overall calorie circle and text
        binding.calorieCount.text = currentDailyMeals.totalCalories.roundToDecimal(1).toString()
        binding.calorieProgress.progress = ((currentDailyMeals.totalCalories / calorieGoal) * 100).toInt()

        // Update the "of X kcal" text to show actual goal
        binding.calorieGoalText.text = "of ${calorieGoal} kcal"


        // Update macro progress bars and texts
        binding.proteinProgress.progress =
            ((currentDailyMeals.totalProteins / proteinGoal) * 100).toInt()
        binding.proteinText.text =
            "${currentDailyMeals.totalProteins.roundToDecimal(1)} of $proteinGoal grams"

        binding.fatsProgress.progress =
            ((currentDailyMeals.totalFats / fatGoal) * 100).toInt()
        binding.fatsText.text =
            "${currentDailyMeals.totalFats.roundToDecimal(1)} of $fatGoal grams"

        binding.carbsProgress.progress =
            ((currentDailyMeals.totalCarbs / carbGoal) * 100).toInt()
        binding.carbsText.text =
            "${currentDailyMeals.totalCarbs.roundToDecimal(1)} of $carbGoal grams"

        // Update meal calorie displays
        binding.breakfastCalories.text =
            "${currentDailyMeals.getMealCalories(currentDailyMeals.breakfast).roundToDecimal(1)} kcal"
        binding.snacksCalories.text =
            "${currentDailyMeals.getMealCalories(currentDailyMeals.snacks).roundToDecimal(1)} kcal"
        binding.dinnerCalories.text =
            "${currentDailyMeals.getMealCalories(currentDailyMeals.dinner).roundToDecimal(1)} kcal"

        // Update RecyclerViews
        breakfastAdapter.updateFoods(currentDailyMeals.breakfast.values.toList())
        snacksAdapter.updateFoods(currentDailyMeals.snacks.values.toList())
        dinnerAdapter.updateFoods(currentDailyMeals.dinner.values.toList())
    }

    private fun showFoodSelectionDialog(mealType: String) {
        val userId = FirebaseAuth.getInstance().currentUser?.uid
        if (userId != null) {
            val foodsRef = FirebaseDatabase.getInstance("https://revive-app-b8fd2-default-rtdb.europe-west1.fire
                .getReference("Information")
                .child(userId ?: "")

            // Create dialog
            val dialogView = LayoutInflater.from(requireContext())
                .inflate(R.layout.dialog_food_selection, null)
            val builder = AlertDialog.Builder(requireContext())
            builder.setView(dialogView)
            val dialog = builder.create()

            // Setup dialog views
            val searchView = dialogView.findViewById<SearchView>(R.id.foodSearchView)
            val foodListView = dialogView.findViewById<ListView>(R.id.foodListView)
            val cancelButton = dialogView.findViewById<Button>(R.id.cancelButton)
            val titleText = dialogView.findViewById<TextView>(R.id.dialogTitle)

            titleText.text = "Select Food or Meal"

            // Lists to hold our items
            val allItems = mutableListOf<SelectionItem>()
            val filteredItems = mutableListOf<SelectionItem>()
```

```
253             // Custom adapter for our combined list
254             val adapter = object : BaseAdapter() {
255                 override fun getCount(): Int = filteredItems.size
256                 override fun getItem(position: Int): SelectionItem = filteredItems[position]
257                 override fun getItemId(position: Int): Long = position.toLong()
258
259                 override fun getView(position: Int, convertView: View?, parent: ViewGroup): View {
260                     val itemView = convertView ?: LayoutInflater.from(context)
261                         .inflate(R.layout.recycler_item, parent, false)
262
263                     val nameTextView = itemView.findViewById<TextView>(R.id.recTitle)
264                     val detailsTextView = itemView.findViewById<TextView>(R.id.recPriority)
265
266                     val item = filteredItems[position]
267                     nameTextView.text = item.name
268                     when (item) {
269                         is SelectionItem.FoodItem -> {
270                             detailsTextView.text = "${item.food.calories} kcal/100g"
271                         }
272                         is SelectionItem.MealItem -> {
273                             detailsTextView.text = "${item.meal.totalCalories.roundToDecimal(1)} kcal"
274                         }
275                     }
276
277
278                     return itemView
279                 }
280             }
281
282             // Load foods first
283             foodsRef.child("Food items").get().addOnSuccessListener { foodSnapshot ->
284                 Log.d("HomeFragment", "Food items snapshot: ${foodSnapshot.childrenCount}")
285                 foodSnapshot.children.forEach { foodSnap ->
286                     val food = foodSnap.getValue(FoodItem::class.java)
287                     Log.d("HomeFragment", "Found food: ${food?.name}")
288                     food?.let {
289                         it.convertto100gram() // Convert to 100g for display
290                         allItems.add(SelectionItem.FoodItem(it))
291                     }
292                 }
293
294                 // Then load meals
295                 foodsRef.child("Meals").get().addOnSuccessListener { mealSnapshot ->
296                     mealSnapshot.children.forEach { mealSnap ->
297                         val meal = mealSnap.getValue(Meal::class.java)
298                         meal?.let {
299                             allItems.add(SelectionItem.MealItem(it))
300                         }
301                     }
302
303                     if (allItems.isEmpty()) {
304                         Toast.makeText(context, "Please add food items first", Toast.LENGTH_SHORT).show()
305                         dialog.dismiss()
306                     } else {
307                         // Initial population of filtered list
308                         filteredItems.addAll(allItems)
309                         adapter.notifyDataSetChanged()
310                     }
311                 }.addOnFailureListener { error ->
312                     Toast.makeText(context, "Failed to load meals: ${error.message}", Toast.LENGTH_SHORT).show()
313                     dialog.dismiss()
314                 }
315             }.addOnFailureListener { error ->
316                 Toast.makeText(context, "Failed to load foods: ${error.message}", Toast.LENGTH_SHORT).show()
317                 dialog.dismiss()
318             }
319
320             foodListView.adapter = adapter
321
```

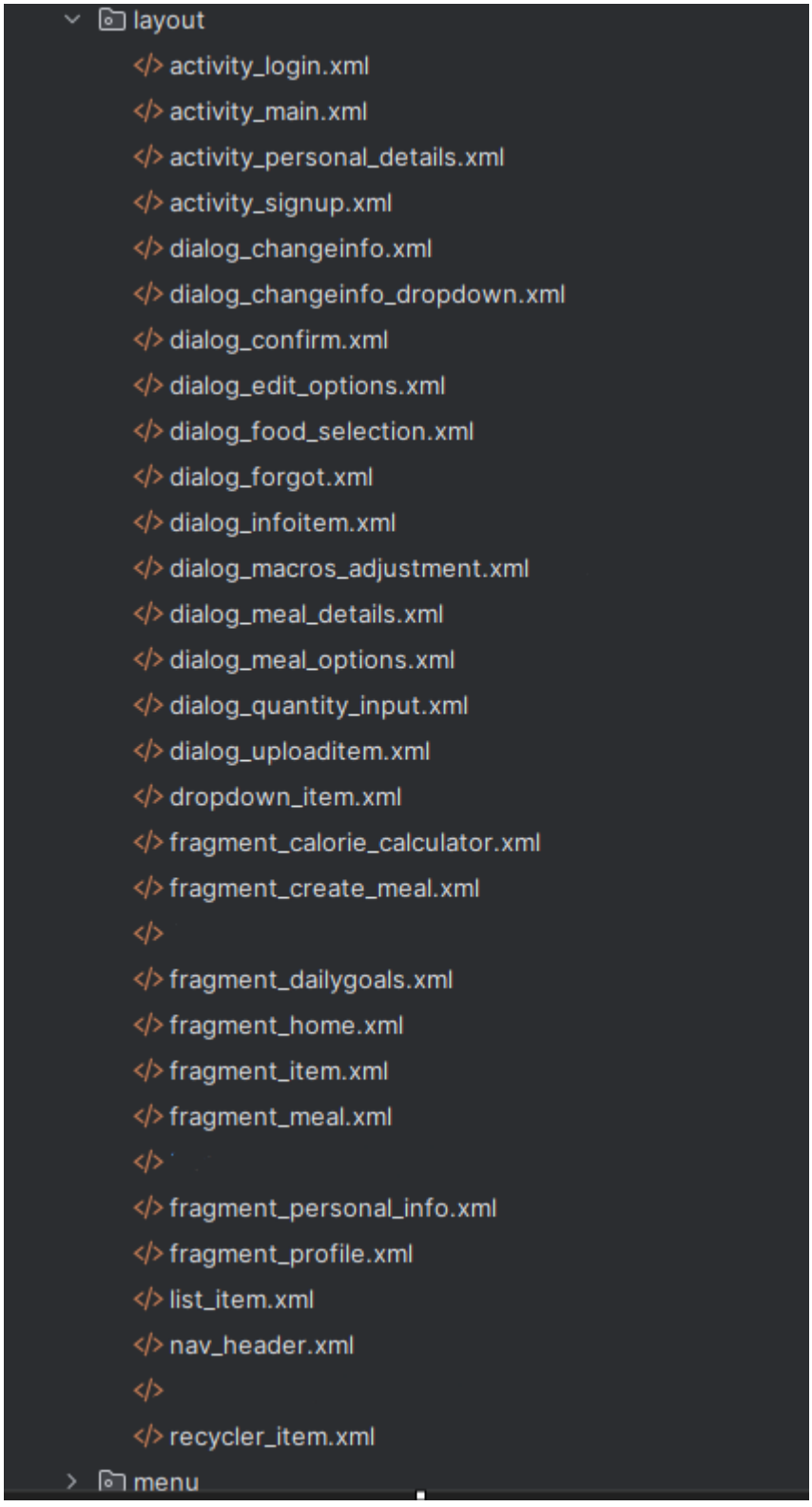
```
            foodListView.adapter = adapter

            // Setup search functionality
            searchView.setOnQueryTextListener(object : SearchView.OnQueryTextListener {
                override fun onQueryTextSubmit(query: String?): Boolean = false

                override fun onQueryTextChange(newText: String?): Boolean {
                    filteredItems.clear()
                    if (newText.isNullOrEmpty()) {
                        filteredItems.addAll(allItems)
                    } else {
                        val searchText = newText.toLowerCase()
                        filteredItems.addAll(allItems.filter { item ->
                            when (item) {
                                is SelectionItem.FoodItem ->
                                    item.food.name.toLowerCase().contains(searchText)
                                is SelectionItem.MealItem ->
                                    item.meal.name.toLowerCase().contains(searchText)
                            }
                        })
                    }
                    adapter.notifyDataSetChanged()
                    return true
                }
            })

            // Handle item selection
            foodListView.setOnItemClickListener { _, _, position, _ ->
                val selectedItem = filteredItems[position]
                when (selectedItem) {
                    is SelectionItem.FoodItem -> {
                        val selectedFood = selectedItem.food.copy()
                        selectedFood.convertto1gram() // Convert back to 1g before quantity input
                        showFoodQuantityDialog(selectedFood, mealType)
                    }
                    is SelectionItem.MealItem -> {
                        addMealToMeal(selectedItem.meal, mealType)
                    }
                }
                dialog.dismiss()
            }

            cancelButton.setOnClickListener {
                dialog.dismiss()
            }

            if (dialog.window != null) {
                dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
            }

            dialog.show()
        } else {
            Toast.makeText(context, "User not authenticated", Toast.LENGTH_SHORT).show()
        }
    }

    sealed class SelectionItem {...}

    // Function to add a meal's items to current meal
    private fun addMealToMeal(mealToAdd: Meal, mealType: String) {
        val currentDate = getCurrentDate()

        // Add all food items from the selected meal
        mealToAdd.foodItems.values.forEach { foodItem ->
            when (mealType) {
                "Breakfast" -> currentDailyMeals.breakfast[foodItem.foodItem.name] = foodItem
                "Snacks" -> currentDailyMeals.snacks[foodItem.foodItem.name] = foodItem
                "Dinner" -> currentDailyMeals.dinner[foodItem.foodItem.name] = foodItem
            }
        }
```

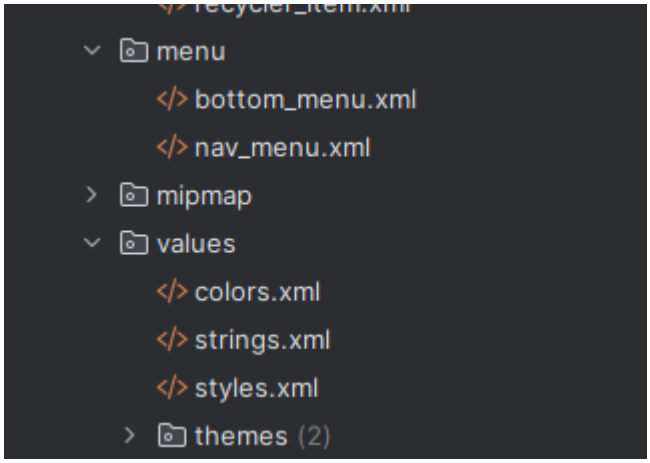
```
        databaseReference.child(currentDate)
            .setValue(currentDailyMeals)
            .addOnSuccessListener {
                currentDailyMeals.updateTotals()
                updateUI()
                Toast.makeText(context, "Meal added", Toast.LENGTH_SHORT).show()
            }
            .addOnFailureListener {
                Toast.makeText(context, "Failed to add meal", Toast.LENGTH_SHORT).show()
            }
    }

    private fun showFoodQuantityDialog(foodItem: FoodItem, mealType: String) {
        val builder = AlertDialog.Builder(requireContext())
        val view = layoutInflater.inflate(R.layout.dialog_quantity_input, null)

        val quantityInput = view.findViewById<EditText>(R.id.quantityInput)
        val confirmButton = view.findViewById<Button>(R.id.MealSaveBtn)
        val cancelButton = view.findViewById<Button>(R.id.MealCloseBtn)

        builder.setView(view)
        val dialog = builder.create()

        confirmButton.setOnClickListener {
            val quantity = quantityInput.text.toString().toDoubleOrNull()
            if (quantity != null && quantity > 0) {
                addFoodToMeal(foodItem, quantity, mealType)
                dialog.dismiss()
            } else {
                Toast.makeText(context, "Please enter a valid quantity", Toast.LENGTH_SHORT).show()
            }
        }

        cancelButton.setOnClickListener { dialog.dismiss() }

        if (dialog.window != null) {
            dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
        }

        dialog.show()
    }

    private fun addFoodToMeal(foodItem: FoodItem, grams: Double, mealType: String) {
        val foodItemWithQuantity = FoodItemWithQuantity(foodItem, grams)
        val currentDate = getCurrentDate()

        // Add to local data structure
        when (mealType) {
            "Breakfast" -> currentDailyMeals.breakfast[foodItem.name] = foodItemWithQuantity
            "Snacks" -> currentDailyMeals.snacks[foodItem.name] = foodItemWithQuantity
            "Dinner" -> currentDailyMeals.dinner[foodItem.name] = foodItemWithQuantity
        }
        currentDailyMeals.updateTotals()

        // Update Firebase
        databaseReference.child(currentDate)
            .setValue(currentDailyMeals)
            .addOnSuccessListener {
                currentDailyMeals.updateTotals()
                updateUI()
            }
            .addOnFailureListener {
                Toast.makeText(context, "Failed to save food item", Toast.LENGTH_SHORT).show()
            }
    }

    private fun showFoodItemDialog(foodItem: FoodItemWithQuantity, mealType: String) {
        val builder = AlertDialog.Builder(requireContext())
        val view = layoutInflater.inflate(R.layout.dialog_meal_options, null)

        // Set up dialog views
        val titleText = view.findViewById<TextView>(R.id.dialogTitle)
        val editButton = view.findViewById<Button>(R.id.editButton)
        val deleteButton = view.findViewById<Button>(R.id.deleteButton)
        val cancelButton = view.findViewById<Button>(R.id.cancelButton)

        // Customize dialog for meal items
        titleText.text = "${foodItem.foodItem.name} Details"
        editButton.text = "Edit Amount"
```

```
        editButton.text = "Edit Amount"

        builder.setView(view)
        val dialog = builder.create()

        // Handle button clicks
        editButton.setOnClickListener {
            dialog.dismiss()
            showQuantityEditDialog(foodItem, mealType)
        }

        deleteButton.setOnClickListener {
            dialog.dismiss()
            removeFoodFromMeal(foodItem, mealType)
        }

        cancelButton.setOnClickListener {
            dialog.dismiss()
        }

        if (dialog.window != null) {
            dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
        }

        dialog.show()
    }

    private fun showQuantityEditDialog(foodItem: FoodItemWithQuantity, mealType: String) {
        val builder = AlertDialog.Builder(requireContext())
        val view = layoutInflater.inflate(R.layout.dialog_quantity_input, null)

        val quantityInput = view.findViewById<EditText>(R.id.quantityInput)
        val confirmButton = view.findViewById<Button>(R.id.MealSaveBtn)
        val cancelButton = view.findViewById<Button>(R.id.MealCloseBtn)

        // Show current quantity
        quantityInput.setText(foodItem.grams.toString())

        builder.setView(view)
        val dialog = builder.create()

        confirmButton.setOnClickListener {
            val newQuantity = quantityInput.text.toString().toDoubleOrNull()
            if (newQuantity != null && newQuantity > 0) {
                updateFoodQuantity(foodItem, newQuantity, mealType)
                dialog.dismiss()
            } else {
                Toast.makeText(context, "Please enter a valid quantity", Toast.LENGTH_SHORT).show()
            }
        }

        cancelButton.setOnClickListener { dialog.dismiss() }

        if (dialog.window != null) {
            dialog.window!!.setBackgroundDrawable(ColorDrawable(0))
        }

        dialog.show()
    }

    private fun updateFoodQuantity(foodItem: FoodItemWithQuantity, newQuantity: Double, mealType: String) {
        val currentDate = getCurrentDate()
        val updatedFoodItem = foodItem.copy(grams = newQuantity)

        // Update local data
        when (mealType) {
            "Breakfast" -> currentDailyMeals.breakfast[foodItem.foodItem.name] = updatedFoodItem
            "Snacks" -> currentDailyMeals.snacks[foodItem.foodItem.name] = updatedFoodItem
            "Dinner" -> currentDailyMeals.dinner[foodItem.foodItem.name] = updatedFoodItem
        }

        // Update Firebase
        databaseReference.child(currentDate)
            .setValue(currentDailyMeals)
            .addOnSuccessListener {
                currentDailyMeals.updateTotals()
                updateUI()
                Toast.makeText(context, "Quantity updated", Toast.LENGTH_SHORT).show()
            }
            .addOnFailureListener {
```

```
563
564     private fun removeFoodFromMeal(foodItem: FoodItemWithQuantity, mealType: String) {
565         val currentDate = getCurrentDate()
566
567         // Remove from local data
568         when (mealType) {
569             "Breakfast" -> currentDailyMeals.breakfast.remove(foodItem.foodItem.name)
570             "Snacks" -> currentDailyMeals.snacks.remove(foodItem.foodItem.name)
571             "Dinner" -> currentDailyMeals.dinner.remove(foodItem.foodItem.name)
572         }
573
574         // Update Firebase
575         databaseReference.child(currentDate)
576             .setValue(currentDailyMeals)
577             .addOnSuccessListener {
578                 currentDailyMeals.updateTotals()
579                 updateUI()
580                 Toast.makeText(context, "Food item removed", Toast.LENGTH_SHORT).show()
581             }
582             .addOnFailureListener {
583                 Toast.makeText(context, "Failed to remove food item", Toast.LENGTH_SHORT).show()
584             }
585     }
586
587
588     private fun Double.roundToDecimal(decimals: Int): Double {
589         return BigDecimal(this)
590             .setScale(decimals, RoundingMode.HALF_UP)
591             .toDouble()
592     }
593 }
594
```

List of XMLS and Values


layout
activity_login.xml
activity_main.xml
activity_personal_details.xml
activity_signup.xml
dialog_changeinfo.xml
dialog_changeinfo_dropdown.xml
dialog_confirm.xml
dialog_edit_options.xml
dialog_food_selection.xml
dialog_forgot.xml
dialog_infoitem.xml
dialog_macros_adjustment.xml
dialog_meal_details.xml
dialog_meal_options.xml
dialog_quantity_input.xml
dialog_uploaditem.xml
dropdown_item.xml
fragment_calorie_calculator.xml
fragment_create_meal.xml
fragment_dailygoals.xml
fragment_home.xml
fragment_item.xml
fragment_meal.xml
fragment_personal_info.xml
fragment_profile.xml
list_item.xml
nav_header.xml
recycler_item.xml
menu




```xml
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>
    <color name="darkGreen">#053411</color>
    <color name="grey">#ff424242</color>
    <color name="grey_green">#A8E063</color>
    <color name="light_green">#56AB2F</color>
    <color name="red">#400a06</color>
</resources>
```

Side note:

Screenshots in the *appendix* have colour and backgrounds different from those in *development* as I have changed the theme of my android studio since the development.